Computational Intelligence Methods and Applications

Mansour Eddaly
Bassem Jarboui
Patrick Siarry *Editors*

Metaheuristics for Machine Learning

New Advances and Tools

Springer

Computational Intelligence Methods and Applications

Founding Editors

Sanghamitra Bandyopadhyay, Machine Intelligence Unit, Indian Statistical Institute, Kolkata, West Bengal, India

Ujjwal Maulik, Dept of Computer Science & Engineering, Jadavpur University, Kolkata, West Bengal, India

Patrick Siarry, LISSI, University of Paris-Est Créteil, Créteil, France

Series Editor

Patrick Siarry LiSSi, E.A. 3956, Université Paris-Est Créteil, Vitry-sur-Seine, France

The monographs and textbooks in this series explain methods developed in computational intelligence (including evolutionary computing, neural networks, and fuzzy systems), soft computing, statistics, and artificial intelligence, and their applications in domains such as heuristics and optimization; bioinformatics, computational biology, and biomedical engineering; image and signal processing, VLSI, and embedded system design; network design; process engineering; social networking; and data mining.

Mansour Eddaly • Bassem Jarboui • Patrick Siarry
Editors

Metaheuristics for Machine Learning

New Advances and Tools

Springer

Editors
Mansour Eddaly
Qassim University
Buraydah, Saudi Arabia

Patrick Siarry
Paris-Est Créteil University
Paris, France

Bassem Jarboui
Abu Dhabi Women Campus
Higher Colleges of Technology
Abu Dhabi, United Arab Emirates

ISSN 2510-1765 ISSN 2510-1773 (electronic)
Computational Intelligence Methods and Applications
ISBN 978-981-19-3887-0 ISBN 978-981-19-3888-7 (eBook)
https://doi.org/10.1007/978-981-19-3888-7

© The Editor(s) (if applicable) and The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023
This work is subject to copyright. All rights are solely and exclusively licensed by the Publisher, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilms or in any other physical way, and transmission or information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed.
The use of general descriptive names, registered names, trademarks, service marks, etc. in this publication does not imply, even in the absence of a specific statement, that such names are exempt from the relevant protective laws and regulations and therefore free for general use.
The publisher, the authors, and the editors are safe to assume that the advice and information in this book are believed to be true and accurate at the date of publication. Neither the publisher nor the authors or the editors give a warranty, expressed or implied, with respect to the material contained herein or for any errors or omissions that may have been made. The publisher remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

This Springer imprint is published by the registered company Springer Nature Singapore Pte Ltd.
The registered company address is: 152 Beach Road, #21-01/04 Gateway East, Singapore 189721, Singapore

Preface

Metaheuristics are well known as an efficient tool to solve hard optimization problems. The idea behind is to maintain the balance between diversification and intensification to provide high-quality solutions in a reasonable time. Research and development in employing metaheuristics to enhance machine learning techniques become trendy. Some successful applications are used for both supervised (classification and regression) and unsupervised (clustering and rule mining) problems. Furthermore, the automatic generation of programs through metaheuristics, such as evolutionary computation and swarm intelligence, has gained a significant widespread popularity. This success has been mostly promoted by the genetic programming paradigm developed by Koza in 1992.

In this book, we investigate different ways for using metaheuristics into machine learning techniques from both theoretical and practical points of view. This book reviews the latest innovations and applications of integrating metaheuristics into machine learning techniques, covering metaheuristic programming, meta-learning, etc. Moreover, some illustrations through real-world applications are given, including clustering, big data, machine health monitoring, underwater sonar targets, banking, etc.

Organization of the Book

This book is divided into two main parts: the first part "Metaheuristics for machine learning: theory and reviews" includes three chapters and deals with theoretical aspects, whereas the second part "Metaheuristics for machine learning: applications" includes five chapters and discusses some real-world applications.

Chapter 1 "From Metaheuristics to Automatic Programming" (S. Eleuch, B. Jarboui and P. Siarry) presents an overview of the main metaheuristics and explains their adaptation for evolving programs. Also, a detailed comparison between automatic programming metaheuristics and the traditional machine learning methods is provided in order to explain the advantages of using each of them.

Chapter 2 "Biclustering Algorithms Based on Metaheuristics: A Review" (A. José-García, J. Jacques, V. Sobanski and C. Dhaenens) presents a survey of metaheuristics approaches to address the biclustering problem. The review focuses on the underlying optimization methods and their main search. Moreover, a specific discussion on single versus multi-objective approaches is presented.

Chapter 3 "A Metaheuristic Perspective on Learning Classifier Systems" presents the Learning Classifier Systems, a family of rule-based learning systems. Furthermore, the similarities and differences of LCSs and the related machine learning techniques of genetic programming, decision trees, mixtures of experts, bagging and boosting are discussed.

Chapter 4 "Metaheuristic-Based Machine Learning Approach for Customer Segmentation" (P. Z. Lappas, S. Z. Xanthopoulos and A. N. Yannacopoulos) proposes an evolutionary clustering approach as a rule extractor mechanism that facilitates decision makers to recognize the most significant customer characteristics and profile them into segments. Therefore, a genetic algorithm is used in a hybrid synthesis with unsupervised machine learning algorithms (K-means algorithms) to solve data clustering problems.

Chapter 5 "Evolving Machine Learning-Based Classifiers by Metaheuristic Approach for Underwater Sonar Target Detection and Recognition" (M. Khishe, H. Javdanfar, M. Kazemirad and H. Mohammadi) investigates the performance of ten metaheuristic algorithms to be used in the support vector machines networks in order to have reliable and accurate underwater sonar target classifier.

Chapter 6 "Solving the Quadratic Knapsack Problem Using GRASP" (R. Jovanovic and S. VoSS) presents a greedy randomized adaptive search procedure (GRASP) approach for solving the Quadratic Knapsack Problem (QKP). In addition, a new local search has been developed that manages to explore a larger neighborhood of solutions, while maintaining the same asymptotical computational cost as the commonly used ones.

Chapter 7 "Algorithm vs Processing Manipulation to Scale Genetic Programming to Big Data Mining" (S. Ben Hamida and H. Hmida) proposes a taxonomy to classify the solutions of difficulties when training EA/GP on big data sets on three categories: processing manipulation, algorithm manipulation, and data manipulation. Two approaches are then presented and discussed. The first one, from the processing manipulation category, parallelizes genetic programming over spark. The second one, from the algorithm manipulation category, extends genetic programming with active learning, using dynamic and adaptive sampling.

Chapter 8 "Dynamic Assignment Problem of Parking Slots" (M. Ratli, A. Ait El Cadi, B. Jarboui and M. Eddaly) proposes a MIP formulation with a time partition and throws a set of decision points to handle the dynamic aspect. To solve this problem, a hybrid approach, using Munkres' assignment algorithm, a local search,

and an Estimation of Distribution Algorithm (EDA) with a reinforcement learning, is proposed. The obtained results show that the approach with the learning effect is efficient.

Qassim, Kingdom of Saudi Arabia — Mansour Eddaly
Abu Dhabi, United Arab Emirates — Bassem Jarboui
Créteil, France — Patrick Siarry
January, 2022

Contents

Part I Metaheuristics for Machine Learning: Theory and Reviews

1 From Metaheuristics to Automatic Programming 3
S. Elleuch, B. Jarboui and P. Siarry

2 Biclustering Algorithms Based on Metaheuristics: A Review 39
Adán José-García, Julie Jacques, Vincent Sobanski, and Clarisse Dhaenens

3 A Metaheuristic Perspective on Learning Classifier Systems 73
Michael Heider, David Pätzel, Helena Stegherr and Jörg Hähner

Part II Metaheuristics for Machine Learning: Applications

4 Metaheuristic-Based Machine Learning Approach for Customer Segmentation 101
P. Z. Lappas, S. Z. Xanthopoulos, and A. N. Yannacopoulos

5 Evolving Machine Learning-Based Classifiers by Metaheuristic Approach for Underwater Sonar Target Detection and Recognition 135
M. Khishe, H. Javdanfar, M. Kazemirad, and H. Mohammadi

6 Solving the Quadratic Knapsack Problem Using GRASP 157
Raka Jovanovic and Stefan Voß

7 Algorithm vs Processing Manipulation to Scale Genetic Programming to Big Data Mining 179
S. Ben Hamida and H. Hmida

8 Dynamic Assignment Problem of Parking Slots 201
M. Ratli, A. Ait El Cadi, B. Jarboui, and M. Eddaly

List of Contributors

Abdessamad Ait El Cadi University of Polytechnique Hauts-de-France, LAMIH, CNRS, Valenciennes, France
INSA Hauts-de-France Valenciennes, France

Sana Ben Hamida LAMSADE, Paris Dauphine University, PSL Research University, Paris, France

Clarisse Dhaenens University of Lille, CNRS, Centrale Lille, UMR 9189 CRIStAL, Lille, France

Mansour Eddaly Department of Management Information Systems and Production Management, College of Business and Economics, Qassim University, Buraydah, Kingdom of Saudi Arabia

Souhir Elleuch College of Business and Economics, Qassim University, Buraydah, Kingdom of Saudi Arabia

Jörg Hähner University of Augsburg, Augsburg, Germany

Michael Heider University of Augsburg, Augsburg, Germany

Hmida Hmida Institut Supérieur des Études Technologiques de Bizerte, Menzel Abderrahmane, Tunisia

Julie Jacques Univ. Lille, CNRS, Centrale Lille, UMR 9189 CRIStAL, Lille, France

Bassem Jarboui Higher Colleges of Technology, Abu Dhabi, United Arab Emirates

Hamed Javdanfar Department of Marine Electronics and Communication Engineering, Imam Khomeini Marine Science University, Nowshahr, Iran

Adán José-García University of Lille, CNRS, Lille, France

Raka Jovanovic Qatar Environment and Energy Research Institute (QEERI), Hamad bin Khalifa University, Doha, Qatar

Mohammad Kazemirad Department of Marine Electronics and Communication Engineering, Imam Khomeini Marine Science University, Nowshahr, Iran

Mohammad Khishe Department of Marine Electronics and Communication Engineering, Imam Khomeini Marine Science University, Nowshahr, Iran

P. Z. Lappas Dept. of Statistics and Actuarial-Financial Mathematics, University of the Aegean, Samos, Greece

EXUS AI Labs, EXUS, Athens, Greece

Hasan Mohammadi Department of Marine Electronics and Communication Engineering, Imam Khomeini Marine Science University, Nowshahr, Iran

David Pätzel University of Augsburg, Augsburg, Germany

Mustapha Ratli University of Polytechnique Hauts-de-France, LAMIH, CNRS, Valenciennes, France

Patrick Siarry LISSI, University Paris Est Créteil, Vitry-sur-Seine, France

Vincent Sobanski University of Lille, Inserm, CHU Lille, Institut Universitaire de France (IUF) U1286 - INFINITE - Institute for Translational Research in Inflammation, Lille, France

Helena Stegherr University of Augsburg, Augsburg, Germany

Stefan Voß Institute of Information Systems, University of Hamburg, Hamburg, Germany

S. Z. Xanthopoulos Deptartment of Statistics and Actuarial-Financial Mathematics, University of the Aegean, Samos, Greece

A. N. Yannacopoulos Deptartment of Statistics and Laboratory for Stochastic Modelling and Applications, Athens University of Economics and Business, Athens, Greece

Acronyms

ABC	Artificial Bee Colony
ABCP	Artificial Bee Colony Programming
ACF	Average Correlation Function
ACO	Ant Colony Optimization
ACP	Ant Colony Programming
ACV	Average Correlation Value
AHP	Analytic Hierarchy Process
AI	Artificial Intelligence
AIS	Artificial Immune Systems
ANN	Artificial Neural Network
AP	Automatic Programming
BBE	Binary Bicluster Encoding
BBC	Bayesian BiClustering
BEA	Bacterial Evolution Algorithm
BiBit	Bit-Pattern Biclustering Algorithm
Bimax	Binary Inclusion-Maximal Biclustering Algorithm
BSize	Bicluster Size
BUSS	Basic Under-Sampling Selection
CCA	Cheng and Church's Algorithm
CFG	Context-Free Grammar
CGP	Cartesian Genetic Programming
ChOA	Chimp Optimization Algorithm
CMA-ES	Covariance Matrix Adaptation Evolution Strategy
CoEA	Coevolutionary Algorithms
CTWC	Coupled Two-Way Clustering
CVF	Coefficient of Variation Function
DCA	Direct Clustering Algorithm
DCC	Double Conjugated Clustering
DE	Differential Evolution
DEAP	Distributed Evolutionary Algorithms in Python
DeBi	Differentially Expressed Biclusters

DSS	Dynamic Subset Selection
DT	Decision Tree
EA	Evolutionary Algorithms
EC	Evolutionary Computation
EDA	Estimation of Distribution Algorithm
EDP	Estimation of Distribution Programming
ES	Evolution Strategies
FABIA	Factor Analysis for Bicluster Acquisition
FIFO	First In First Out
GA	Genetic Algorithms
GBC	Grammatical Bee Colony
GE	Grammatical Evolution
GEP	Gene Expression Programming
GP	Genetic Programming
GPGPU	General Purpose Graphics Processing Units
GRASP	Greedy Randomized Adaptive Search Procedure
GS	Grammatical Swarm
GWO	Gray Wolf Optimizer
HDFS	Hadoop Distributed Files System
HGSO	Henry Gas Solubility Optimization
ILS	Iterated Local Search
ITWC	Interrelated Two-Way Clustering
KF	Kalman Filter
LCS	Learning Classifier Systems
LGP	Linear Genetic Programming
ML	Machine Learning
MSR	Mean Squared Residence
MPA	Marine Predator Algorithm
OPSM	Order-Preserving Submatrix
PC	Parking Coordinators
QKP	Quadratic Knapsack Problem
QUBIC	QUalitative BIClustering
RBML	Rule-Based Machine Learning
RDD	Resilient Distributed Datasets
RL	Reinforcement Learning
RSS	Random Subset Selection
SA	Simulated Annealing
SAMBA	Statistical-Algorithmic Method for Bicluster Analysis
SB	Spectral Biclustering
SBS	Static Balanced Sampling
SGP	Stack-based Genetic Programming
SI	Swarm Intelligence
SL	Supervised Learning
SMA	Slime Muld Algorithm
SMSR	Scaling Mean Squared Residence

SVC	Support Vector Classifier
SVM	Support Vector Machines
TAG	Tree-Adjunct Grammar
TBS	Topology Based Sampling
TS	Tabu Search
TSO	Tree Swarm Optimization
VAR	Bicluster Variance
VE	Virtual Error
VNS	Variable Neighborhood Search
WOA	Whale Optimization Algorithm

Part I
Metaheuristics for Machine Learning: Theory and Reviews

Chapter 1
From Metaheuristics to Automatic Programming



S. Elleuch, B. Jarboui, and P. Siarry

Abstract Metaheuristics are well known approaches to solve efficiently hard optimization problems. Furthermore, the automatic generation of computer programs using metaheuristics as search strategy becomes an active research area. Actually, this interest has been initially promoted by the genetic programming paradigm. Afterward, metaheuristic algorithms, especially population-based algorithms have been extended in order to produce adequate programs to a given problem. This paper introduces the intersection between metaheuristics and automatic programming algorithms. The most popular metaheuristics was described in this paper and the outline of the components and the steps that a researcher needs to follow in order to develop an automatic programming mechanism. To the best of our knowledge, we are writing the first investigation about automatic programming in general. We gathered both old and new contributions in this area trying to form a starting point for other researchers. Our objective is to stimulate the interest of the research community and keeping them updated on automatic programming using metaheuristics approaches and hoping to motivate the appearance of innovated contributions.

S. Elleuch (✉)
College of Business and Economics, Qassim University, Buraydah, Kingdom of Saudi Arabia

B. Jarboui
Higher Colleges of Technology, Abu Dhabi, United Arab Emirates
e-mail: bassem_jarboui@yahoo.fr

P. Siarry
LISSI, University Paris Est Créteil, Vitry-sur-Seine, France
e-mail: siarry@u-pec.fr

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

M. Eddaly et al. (eds.), *Metaheuristics for Machine Learning*,
Computational Intelligence Methods and Applications,
https://doi.org/10.1007/978-981-19-3888-7_1

1.1 Introduction

Before conducting an investigation of Metaheuristic Automatic Programming, it is important, first, to understand what we mean by the term Automatic Programming, the term Metaheuristic, and the relation between them.

Automatic programming is a type of code generation using some techniques to allow developers to easily write code with a higher level of abstraction. Originally, this term referred to methods using assemblers which are able to generate automatically machine code [3]. In fact, an assembler facilitated the task of a programmer by automatically transforming binary code to a machine code program. As time progressed, new compilers appeared for the languages of the new generations. Today, an assembler or a compiler is no longer called automatic programming. In fact, automatic programming gathers several approaches, one of the most famous among them is program synthesis [189]. Program synthesis produces executable programs according to the present specifications and without an intervention from an expert [188]. In this field, metaheuristic automatic programming are located [190].

In the other hand, metaheuristics are a wide class of algorithms designed to solve approximately optimization problems [192]. They use specific procedures to explore the search space and guide the optimization process. Generally, the goal of metaheuristics is to minimize or maximize an objective function which is defined based on the characteristics of the problem to resolve. In recent years, these algorithms are emerging as good alternatives to classical methods, depending on the mathematical and dynamic programming. In general, metaheuristics are applied to problems characterized by their difficulty and where no specific and satisfactory algorithm exists to solve them. They are flexible and do not need a deep adaptation to each problem. Theoretically, some of them converge to the optimal solution of some problems with an expected runtime.

The interest on metaheuristics has seen a big growth in the last thirty years. Despite the variety of algorithms in this area, we note that at every period we are witnessing the emergence of a new metaheuristic. Some of them are generalized to be automatic programming techniques. In a classic metaheuristic, the algorithm evolves a single solution or a population of strings. An automatic programming metaheuristic manipulates programs without any requirement to specify in advance the structure and the shape of the solution. This area started with the evolutionary programming algorithm [13] but it was more recognized when Koza proposed the genetic programming algorithm [14].

Genetic programming has been applied successfully to many machine learning problems [224]. While scientific researches on genetic programming continue other automatic programming metaheuristics are arising and providing similar results or even a performance better than the genetic programming algorithm for some problems. In 1998, the grammatical evolution was adapted to optimize program by Ryan et al. [17]. Two years after, Roux and Fonlupt introduced the first swarm-based automatic programming method called ant programming [16].

Immune programming was proposed in 2003, taking its inspiration from principles of the vertebrate immune system [18]. In 2004, particle swarm programming and bacterial programming were developed respectively by ONeill [20] and Cabrita [19]. Artificial bee colony was used later as a technique in automatic programming [21]. In 2014, the first automatic programming method based on iterated local search was proposed by Nguyen et al. [79] and it was applied on dynamic job shop scheduling. Recently, Elleuch et al. have put the first features of their innovative work 'variable neighborhood programming' in [80].

Artificial intelligence and machine learning are the main application areas of metaheuristic automatic programming. These algorithms are able to create an intelligent process adapted to a specific problem. Among the traditional machine methods, we cite, for example, inductive logic programming [180], artificial neural networks [181], reinforcement learning [191], etc.

To the best of our knowledge, we are writing the first investigation about automatic programming metaheuristics in general. This paper aims to present an overview of the main metaheuristics and explain their adaptation for evolving programs. We gathered old and new contributions in this area trying to form a starting point for other researchers. In addition to that we provide a detailed comparison between automatic programming metaheuristics and the traditional machine learning methods, explaining the advantages of using each of them.

Our work is organized as follows. In the second section, we shortly present metaheuristic algorithms. In Sect. 1.3, we described the field of automatic programming detailing the possible program presentations and the principle steps to implement an automatic programming algorithm. In the same section, we also summarize the different methods to create a program. Finally, in Sect. 1.4, we discuss the current research status and most interesting directions for future research. Some words of conclusions were given in Sect. 1.5.

1.2 Metaheuristics

The number of solutions in one iteration is the main criterion to classify metaheuristics [229]. The first class gathers population-based metaheuristics where the algorithm evolves a specified number of individuals. In the second class, the algorithms improve a single solution and they are called solution-based metaheuristics. In this section, we outline both classes of metaheuristics and we detail the well known existing algorithms in each of them.

1.2.1 Solution-Based Metaheuristics

Solution-based metaheuristics improve one current solution by applying some perturbations. The search process can be viewed as trajectories of walks through

neighborhoods [22]. The move from one solution to another is performed by an iterative process in the search space. The efficiency of this class of metaheuristics was proved in various optimization problems. Among the solution-based metaheuristics we will discuss simulated annealing, tabu search, variable neighborhood search and iterated local search.

1.2.1.1 Simulated Annealing

The Simulated Annealing method (SA) was inspired from the statistical mechanics of metropolis algorithms [23]. It was introduced by Kirkpatrick et al. [10]. SA is based on the annealing technique employed by the metallurgists to attain a thermal equilibrium at each temperature. This technique consists heating and slowly cooling a substance until converging to the steady frozen state (equilibrium state). The energy is represented by the objective function in an optimization problem and the solution is analog to the system state.

The SA algorithm starts from an initial solution S generated randomly or obtained using a heuristic. First, the temperature T is initialized. Then, at each iteration, a neighbor S_0 of S is randomly selected. The move from S to S_0 solution is accepted only if S_0 is better than S or with a probability p, in case S_0 is worse than S. Let f be the objective function to minimize the current problem, the probability follows the Boltzmann distribution presented by:

$$p(T, f(S_0), f(S)) = exp(-\frac{f(S_0) - f(S)}{T}) \tag{1.1}$$

By analogy with the annealing technique, SA decreases slowly the temperature T during its execution. Therefore the probability of the deteriorating moves acceptance was reduced at the end of the search.

Many researchers chose to apply SA to solve discrete or continuous optimization problems. For more details a wide bibliography is available in [24–27]. In the literature, several variants of SA have been suggested [31] as the Noising method [28], Microcanonic annealing [29], and Threshold accepting method [30].

1.2.1.2 Tabu Search

Tabu Search (TS) is a metaheuristic formalized in 1986 [11] that uses the history of a local heuristic search to escape from local minima and to explore the search space with a predefined strategy. In fact, the main principle of TS is the use of adaptive memory inspired by the human memory. This memory provides a more flexible search behavior.

A tabu list is a short-term memory used in the TS algorithm to avoid cycles. It memorizes properties of the trajectory recently undertaken by the algorithm. Thus, this memory prevents last visited solutions from being selected again. Tabu list

is updated at each iteration. TS algorithm uses other types of memory which are the medium-term memory and the long-term memory. The medium term memory records the best-found solutions during the execution of the algorithm. The idea is to give priority to explore the search space around solutions having the same features as the stored solutions. This step is called intensification. The long-term memory is defined in TS algorithm to ensure diversification on the search space. In fact, it aims to force the search for unvisited solutions by discouraging moves based on best solutions characteristics. Then, the set of rules, called aspiration criteria, in TS overrides tabu restrictions and may allow forbidden moves. An extensive survey of TS and its principles is available in [32, 33].

1.2.1.3 Hill Climbing

Hill-Climbing is one of the earliest mathematical optimization algorithms [187]. It is based on an iterative local search starting with an initial solution to find local optimums. In fact, this algorithm examines the neighborhood and accepts only candidate solutions which improve the fitness function.

Actually, when the present problem is a bit difficult, Hill-Climbing does not produce good results. Therefore, several techniques in literature are proposed to improve its research process. We mention, for example, the late acceptance strategy [203] which consists of comparing the current solution with the solution produced several iterations before.

1.2.1.4 Variable Neighborhood Search

As we mentioned in the introduction, Variable Neighborhood Search (VNS) was founded by Hansen and Mladenovic [1]. Its strategy is based on the systemic change within neighborhood structures. In the beginning of each problem resolution, a set of neighborhood structures $\{\mathcal{N}_1, \mathcal{N}_2 \ldots \mathcal{N}_P\}$ of cardinality P must be defined. Then, from a starting solution S the algorithm increasingly uses complex moves to reach local optima on all selected neighborhood structures.

The main steps in VNS algorithm are: shaking, local search and neighborhood change (move). In the shaking step, a solution S_0 is randomly selected in the nth neighborhood of S. The set of neighborhood structures for shaking phase can be different from the neighborhood structures used in local search. The two well-known search strategies employed as local searches are called first improvement and best improvement. First improvement local search selects the first detected solution S' in $\mathcal{N}_i(S)$ where S' is better than the current solution S. The best improvement method consists in selecting all improving solutions in $\mathcal{N}_i(S)$, then the value of S' will be the best among them (if any) [34].

The purpose of a neighborhood change step is to guide the variable neighborhood to precise, in the next step, which neighborhood will be explored and if S' will be accepted as a new current solution. The most commonly used neighborhood change

procedures are sequential neighborhood change, cyclic neighborhood change step, pipe neighborhood change step, and skewed neighborhood change step. The interested reader can consult [34–36] for a detailed description of them.

Many variants are derived from the basic VNS schemes [34]. The well known are fixed neighborhood search, basic VNS, general VNS, skewed VNS, cyclic VNS, nested VNS, and two-level VNS. These variants indicate that VNS heuristics can be successfully applied to various types of NP-hard optimization problems.

1.2.1.5 Iterated Local Search

Iterated Local Search (ILS) framework was defined by Stutzle [37] in 1998. Three years later, Stützle and his colleagues highlighted the main schema of ILS and they applied it to the single machine total weighted tardiness problem [139]. Then the generalization was designed by Lourenço [38]. In fact, instead of applying local search to a new randomly generated starting solution in each iteration (multistart local search), ILS perturbs the local optimum of the incumbent iteration to get a new starting solution. The principle of perturbation has a big influence on the process of the ILS algorithm. In fact, if the perturbation is too weak, possibly, the algorithm may not avoid the convergence to the same local optimum. Furthermore, a strong perturbation would change the algorithm to a local search with multistarting solutions. An ILS review, its variants, and its applications are detailed in [39].

1.2.2 Population-Based Metaheuristics

The main difference between population-based metaheuristics and solution-based metaheuristic is the use of a set of solutions (population) rather than a single solution. The common concept of population-based metaheuristics is the iterative improvement of a population. In the beginning, a first population of solutions is initialized. Secondly, new solutions are generated based on some operators. Finally, the new population is formed from the current one and new solutions by using selection strategies.

The well known population-based metaheuristics can be divided into two classes called Evolutionary Computation (EC) and Swarm Intelligence (SI). The first algorithms developed in EC class are based on the evolutionary theory of Darwin (the exploitation of recombination and mutation operators). The second set uses simple analogs of social interaction in order to produce computational intelligence.

1.2.2.1 Evolutionary Computation

Evolutionary Computation class is also known by Evolutionary Algorithms (EA). Genetic Algorithms (GA) [40] and Evolution Strategies (ES) [41] belong to EC and

they are inspired by the natural evolution of Darwin. However, some EC methods such as Estimation of Distribution Algorithm (EDA) [5], Differential Evolution (DE) [6], and Coevolutionary algorithms (CoEA) [42] are known by non Darwin Evolutionary algorithms. They evolve according to a defined distribution. In the next two sections, we give details about GA, ES and DE.

Genetic Algorithm

GA is arguably the most famous and the most applied evolutionary computation method. This algorithm appeared in the early 1970s [12]. Originally, GA was associated with a binary presentation of the individual. Then this presentation was improved with other types. GA is implemented based on selection, crossover and mutation which are inspired from the human evolution phenomena.

Crossover is seen as the main variation operator. Generally, it combines two selected chromosomes also called parents, by taking a part from each of them to create new individuals. The mutation is applied to one (or many) chromosome(s) from the current population by changing it randomly. This operator ensures the genetic diversity from one generation to another. The role of the fitness function in GA is to estimate the quality of each individual. Generally, the next iteration keeps chromosomes having the highest fitness. Individual selection is an important step in GA algorithm. Some of the well known selection procedures are tournament selection, ranking selection, roulette-wheel selection, etc. A comparative study of selection methods used in GA was done by Goldberg [45] and Blum [46].

GA evolution process depends on a number of parameters such as crossover probability, mutation probability, population size, number of generations, etc. These parameters are determined and adjusted by the user. In the literature, most of the researchers in GA fixed the crossover probability in the range [0.6,1.0] [47] and the mutation rate is usually less than 1%. In fact the appropriate probability values are still an open research issue. Recent reviews are available in [43, 44, 75] for interested readers.

Evolution Strategies

ES were originally introduced by Rechenberg and Schewefel at the Technical University of Berlin in 1964 [41, 48]. ES algorithms are usually applied to solve continuous optimization. The first ES application was in the experimental parameter optimization field. The process is formed by a simple mutation selection operator known by two-membered ES.

More recently, Covariance Matrix Adaptation Evolution Strategy (CMA-ES) was developed by Hansen et al. as an extension of ES [7]. It's an effective method which is currently the most widely applied. Hansen affirmed that CMA-ES is highly competitive when applied to local optimization [49] and on global optimization [50].

In recent years, evolutionary strategies application on a variety of problems are studied and summarized on numerous of theoretical surveys [51–54].

Differential Evolution

As we mention previously, the Differential Evolution (DE) algorithm was introduced by Storn and K. Price [6] in 1996. DE is, indeed, a vector-based evolutionary metaheuristic, which has some similarity to genetic algorithms due to its inclusion of genetic operators.

In fact, DE starts from a population of solutions also called agents. Its strategy is based on three steps which are evaluation, selection, and recombination. In the recombination process, two population members are chosen randomly, then the new solution is created based on the difference between the two selected solutions. Many extension of the DE algorithm are developed to solve numerous real world problems. The recent advances in DE are available in [4].

1.2.2.2 Swarm Intelligence

This class includes several metaheuristics inspired by the intelligence of swarms such as Ant Colony Optimization, Particle swarm optimization, Artificial bee colony algorithm, etc.

Ant Colony Optimization

(ACO) was proposed in 1992 by Dorigo [9]. Their basic idea was based on the imitation of the foraging behavior of ant colonies. In fact, real ants sightedness is low. Therefore, these ants perform a randomized walk to explore the surrounding areas and to search food. Along their trips, they deposit a chemical trail on the ground. The pheromone (trail) allows ants to mark good paths and to guide the others to follow it [55]. An ant is attracted by the path having the highest concentration of pheromone.

The main steps in ACO framework are:

- Initialization: All ACO parameters are adjusted and pheromones are initialized.
- Ant solution construction: To build starting-solutions, artificial ants begin from an empty partial solution which is extended by adding a possible component from a subset of the possible components that can be added, while keeping feasibility. This extension is done according to a probabilistic choice.
- Pheromones update includes two steps. The first step is pheromone evaporation. It consists in decreasing pheromone concentration on the components over time. This process aims to avoid the convergence to suboptimal solutions. The second

mechanism is pheromone deposit. It is applied to make high quality solutions components more attractive [56].
- Daemon action: This action must be done by more than one ant. The application of a local search is the most used daemon action.

ACO have been applied successfully to several optimization problems such as routing, scheduling, and assignment [56].

Particle Swarm Optimization

Particle Swarm Optimization (PSO) is another SI algorithm introduced by Kennedy et al. [8]. It's inspired by the social behavior of a bird flocking searching for a place of food.

A population in PSO is composed of N candidate solutions called particles. Swarm particles coordinate together with local movements without a central control. Each particle is characterized by a velocity vector, a position in the search space and a memory storing its best previous position. Particles with the best fitness influence the behavior of the swarm. In fact, the update of the velocity vector is based on the previous velocity of the particle, its best position and the best position of the population. Note that, at each iteration, particles move from their current location according to their velocity vector.

The general schema of PSO can be summarized as follows:

- Initialize location and velocity vectors of each particle in the population by random values.
- Calculate the fitness of each particle and update the best particle position if the current fitness value is greater.
- Update the global best position, if necessary, by the best particle location in the swarm of the current iteration.
- Calculate the velocity of each particle and update its location.

The most known disadvantage of the standard PSO procedure is the possible premature convergence to local optima. To overcome this drawback, researchers have employed the stability analysis of the trajectories [58, 59]. In the PSO literature background, we find several approaches based on the hybridization of PSO with other methods [60–62].

Artificial Bee Colony Algorithm

Artificial Bee Colony (ABC) algorithm is a computational intelligence metaheuristic inspired by the honeybee intelligent behaviors. Several ABC algorithms have been implemented based on different bee features such as waggle dance and communication, mating, food foraging, task allocation, navigation of collective

decision making, marriage, floral and pheromones laying, reproduction and nest site selection [63, 64].

A bee colony is constituted by two categories: the employed bees and the onlookers. Employed bees number and food sources number are equal. When a food source is exhausted, its corresponding employed bee becomes a scout for discovering randomly new food sources. Indeed the information about food place is carried by an employed bee and it is shared with the other bees through waggle dance. This information is given with a certain probability. Then an onlooker bee observes all dances and selects source food (where the nectar is more concentrated) based on probabilities.

To write a code of the ABC algorithm, the developer should initialize the first population. Then repeat the next steps until the stopping condition is met:

- Each employed is placed in its food source and the corresponding nectar amount is determined.
- The probability value of each source is calculated.
- Each onlooker bee is placed on a selected source food according to nectar amount.
- When a source food is exhausted by onlookers it is excluded from the exploitation process.
- Scout bees discover randomly new source food from the search space.
- The best food source is memorized.

Interested readers can refer to [65] to find more explanations on ABC paradigm and to discover the application areas.

Artificial Immune Systems

It doesn't exist a common classification for the artificial Immune Systems (AIS). AIS algorithm is a population-based algorithm and some authers have said that it also belongs to SI systems. In this study, we decided to include it in the SI section since it has self-organizing properties [57, 74]. The artificial immune systems principles are derived from the biological immune systems. The natural immune system is a network including organs, cells, and tissues that work together. This coordination aims to protect the body and to combat against attacks by dysfunction from its own cells (tumors and cancerous cells) and by foreign invaders. In fact, this system is self-organized, inherently parallel, and highly robust. It's characterized by a powerful learning and a potential storage capacity [66].

Although Jerne was the first who has defined immune system theory in 1974 [67], the work of Farmer et al. in 1986, where he proposed a variety of immune algorithms, was considered the pioneer [66]. Several surveys and books of AIS research have been written to cover this area [68–70]. AIS algorithms have been recently applied to solve computer security and machine learning problems [71, 72].

Four popular AIS approaches have succeeded in attracting researchers: the danger theory based algorithms, negative selection algorithms, artificial immune

networks, and clonal selection algorithms. Detailed descriptions of the basic features of these approaches were published in [73].

1.3 Automatic Programming

In the previous section, we present the well known and widely used metaheuristics. Despite the large number and the variety of metaheuristic algorithms, we still witness the emergence of new contributions [76–78]. We believe that a single metaheuristic method cannot be a solution to all optimization problems. On the other hand, we notice that few techniques among them have been adapted to Automatic Programming (AP) field.

In the past three decades, the automatic generation of programs through metaheuristics as evolutionary computation and swarm intelligence, has gained a significant widespread popularity. This success is mainly due to the research work of Koza and the development of the Genetic Programming (GP) algorithm in 1992 [14]. In fact, an AP metaheuristics developer should pay attention when treating and defining the components and the structure of his program. Algorithm 1 provides a general template adequate to any automatic programming process.

Algorithm 1 AP algorithm template

```
Initialization: choose the program representation and define the fitness function
Solution or population generation using an initialization method
Programs execution
Fitness evaluation
while Termination criterion is not met do
    Application AP metaheuristic operators
    Programs execution
    Fitness evaluation
end while
Return best program
```

In this section, we give, first, structures of a program in automatic programming t. Then we describe the different AP metaheuristics.

1.3.1 Program Representation

The encoding schema of a program is defined based on the present problem. Two general structures are described in this survey: tree structure and linear structure. In addition, it exists through many ways to represent a program with a predefined language in an AP system. Initially, the Lisp S-expression was employed in GP tree-individuals as a target language, then it's widely adopted in automatic programming

algorithms. Furthermore, many research works searched to produce other languages more specific to their problems. Grammar has also enjoyed much popularity as an effective language to express a computer program. Various AP techniques have been built using different forms of grammar. They can be represented, for example, in Backus Naur Form (BNF) [187] which is a language in the form of production rules.

1.3.1.1 Tree Structure Representations

The representation based on a tree is usually used in AP applications. Several types of tree were adopted in the literature. According to Koza, a program is, generally, a hierarchical combination of functions and terminals of undefined shape and dynamically varying in size [14, 116]. Typically, a syntax tree is expressed in a prefix notation (Lisp) and it's composed of internal nodes including functions ($+, *, ADD, Sin, max, if \ldots$) and external nodes, also called terminals, containing problem variables and constants [14].

According to the handled problem, the author decides the solution structure. For a classification purpose, one can use decision tree. As well, program tree is a good pattern to control robots. In fact, this type of tree is simple to manipulate, however one of its limitations is that all variables and return values must be of the same type of data. Montana overcame this drawback by changing some basics in this representation [179]. He proposed a Strongly Typed Genetic Programming algorithm, where each variable and constant in terminal nodes has a specific type and functional nodes return a value of the type required by arguments.

1.3.1.2 Linear Representations

Popular linear schema used in literature are linear arrays storing information such as linear GP, grammatical presentation, gene expression chromosomes, tree adjoining grammars, and cellular Automata rules.

Linear GP form was introduced by Brameier [201] to solve a classification problem. A Linear GP program is a sequence of C language instructions of a variable length. These sequences are recorded in structures called registers. The main advantage of this presentation is its ability to evolve programs quickly in a low-level language and run them directly on the processor.

In 1998, Ryan defined programs in the grammatical evolution algorithm using the context-free grammar (CFG) language and he encoded them by a variable lineal structure [17]. Each solution is binary string with variable length that includes the binary information for selecting a production rule from a BNF grammar. The main advantage of modeling a program grammatically is the possibility to include multiple types of data in one solution, in contrast to the traditional GP modeling, where a program (solution) is restricted to a single type. Nevertheless, GE program

decoding process is complex and some individuals can be translated to wrong expressions.

Gene expression is another encoding design, where traditional parse tree are represented by a linear chromosome of gene with a fixed length [150]. The structural organization of genes is influenced by the open reading frames (coding sequence of a gene in biology). Genes expression are built by a head and a tail. The head can represent functions and terminals whereas the trail includes only terminals. Ferreira's gene language takes its principle from the Karva language and she calls it K-expressions [150]. The separation of head and tail in gene expression gives an effective way of encoding programs. However, in the case where the target expression is short and the head is large, the chromosome will not be totally used and the memory will be wasted. Linear chromosome can be decoded to the expression tree by a process called translation.

Many researchers extended and/or adapted the tree structure defined by Koza to find the adequate shape to their AP paradigm. We cite, for example, the work of Hoai proposing an individual representation called Tree-Adjunct Grammar (TAG) [81]. It has been employed with success in natural language processing. One of the advantages of this representation is that it is a linear genome supporting many types of data and it has tree shape. Abbass et al. [82] have also employed TAG in their AP algorithm based on the ant colony optimization method .

Cellular Automata rules is another linear form which is less commonly used as a program structure [202]. Indeed, one encoding method can be more suitable than another for a given problem according to its type and its complexity. In the literature, one can find more structures and grammar which are less used for presenting a computer program.

1.3.1.3 Other Representations

- Cartesian genetic programming represents programs as graphs encoded by a set of integers [186]. A graph is, in fact, a two-dimensional grid of nodes. Only some of these nodes are used to calculate the output and the majority is ignored. This phenomenon is called neutrality and it has a benefice impact on the performance of the evolutionary process [186, 208, 209, 211]. The main advantage of this structure is its ability to represent several kinds of computational models, such as digital circuit, mathematical equations, and pictures, etc.
- Program using a memory is an important structure for a solution using more complex data types. The most popular memories are scalar memory, indexed memory, and memory implemented via recurrency. Their goal is to allow access to variables independently of the program input. Adding a memory in a program structure is a good way to extend the space over which AP algorithms can search [210, 212].
- Push is a strongly typed language which is based on trees [204]. It uses the automatic programming metaheuristic GP to build programs in the push programming language. Each type of push has its own stack which can be object

of program or integer, float, etc. In fact, the use of stacks allows programs to perform their own genetic operation and to push themselves onto the stack for subsequent manipulation [205]. This program representation has recently been used to solve a variety of general program synthesis problems [206, 207]. Since a metaheuristic as a search algorithm can easily evolve a classifier, solution in an iteration can be a decision tree, a classification rule, an artificial neural network [232], etc [200]. The detailed description of these representations are summarized in [199]

1.3.2 Genetic Programming

GP is considered the pioneer work that adapted a metaheuristic for searching solutions in the space of computer programs [14] and since then, many other AP metaheuristics have been proposed. GP adopts an analog search strategy as the genetic algorithm and respects automatic programming concepts [218, 220, 225].

Selection, crossover, and mutation are the main features in a genetic algorithm [94]. Programs, in GP, are probabilistically selected according to their fitness from the current population. Generally, individuals with best fitness are more likely to have descendant programs and tournament is the most commonly-used strategy for selecting programs in GP. Crossover is, usually, done between two parents and results in two offspring composed of parents sub-tree [87]. Other types of crossover have been described in literature, such as one-point crossover [88], size-fair and homologous crossover [89], uniform crossover [90], context-preserving crossover, and subtree semantic geometric crossover [86].

Mutation is also a crucial operator in GP. Typically, mutation consists in selecting randomly a mutation point in a program and substitutes the corresponding subtree with another randomly generated. Other styles of mutation can be found in [91] as single node mutation. A study about its effects on programs is available in [92]. A recent extended version of single node mutations was discussed in [93] and compared to the old versions on five symbolic regression benchmarks.

After crossover and mutation steps, a selection phase is needed to produce the new generation from both parent and offspring populations. Generally, two alternatives are used: the first is called a generational method; where the overall parent population is replaced by child population. The second alternative called steady-state method, consists of replacing some individuals from the parent population according to predefined rules [94]. Recently, a subset selection method was proposed to solve regression and classification problems [228]

As we have already said, the evolution from a generation to another in GP is based on the modification of individuals. This evolution can induce a loss of useful information [82]. Indeed, the impact on the phenotype of a little change in the genotype can be great. Another major problem of GP is the fast development of tree structures which become increasingly deep and unbalanced over the evolution process [95]. This phenomenon is called bloat and it was much-

discussed by many researchers [94, 96–98]. A huge tree (model) will not only be hard to translate and execute but also may over-fit the training data in many cases. GP has been implemented to solve a large number of hard problems such as symbolic regression [220, 223, 233], time series forecasting [221, 225, 226], pattern recognition [99, 100], robotic control [101–104], data mining [105, 106], the design of artificial neural network architectures [107, 108, 231, 232], electrical engineering [84], software engineering [85, 182, 227]... etc. Furthermore, GP is the most common algorithm used to evolve heuristics. In this case, GP is considered as a Hyper-heuristic method. Hyper-heuristic approaches aim to design automatically heuristics for solving a given problem effectively. The applications in this area have been arisen in the few last years [183–185, 222].

Several variants of GP have been developed in literature, the most popular of them are: linear GP (LGP) [193, 194], Grammatical Evolution(GE) [17, 195], Cartesian GP (CGP) [186, 196, 208, 209, 211], and stack-based GP (SGP) [197, 206, 207].

In fact, artificial intelligence tools-users are always looking for improving their techniques and extending their boundaries by searching solutions of hard problems. In fact, to find a solution to hard problems, many strategies to parallelise, speed up and distribute GP have been proposed. For example, Teller and Andre proposed an improved selection procedure to gain the running time [215]. A program is selected if it works well on a fraction of the training data compared to the remaining of the population. In the other case, if the fitness of a program is poor when being run on only subset of the training data, then it's not likely to be selected as a parent. In addition to the computational cost, using all samples of the training data set can lead to a static fitness function. Thus good programs which preform well on most fitness cases are omitted because they have a fitness value lower than some other programs which fits only few samples of the training data. To solve this problem, a dynamic selection strategy was developed and was tested on large data mining applications [216, 217, 219].

1.3.3 Immune Programming

Immune programming is an automatic programming algorithm based on Artificial Immune Systems. As we described in Sect. 2.2.2, AIS follow the learning way of natural immune systems in order to respond to attacks. Johnson [18] has described an AIS programming proposal in 2003. He used the clonal selection algorithm described in [110] and he applied it to simple Lisp parse-tree to solve symbolic regression problems.

In 2006, Musilek developed a new IP system which presents programs as Stack-based machines [15]. His algorithm was implemented based on the clonal selection principles and replacement of antibodies characterized by low affinities. To prove the effectiveness of his IP system, the author has adapted the algorithm to solve symbolic regression problems. Three years earlier Lau and Musilek have extended the IP algorithm and they have obtained results better than previously

found in the literature [111] when it was applied to implement a model of disinfection of Cryptosporidium parvum. Gan has proposed a Clonal Selection Programming algorithm [114] different from IP by the use of an encoding scheme similar to the encoding proposed by Ferreira in her gene expression programming algorithm [113]. This work was also evaluated for symbolic regression problems and compared with the immune programming and gene expression programming algorithms [150]. The Immune Programming and its extension were also applied to medical image segmentation [198] and electrical circuit design optimization [112].

The principles of the artificial immune programming was also employed in Grammar-based frameworks. Bernardino and Barbosa [213] represented the candidate program as in O'Neill and Ryan [115] but a new decoding method was proposed. In 2015, Bernardino tried to ameliorate the performance of the GIP algorithm [109]. His goal, in this research work, is to search for a solution to functional equations.

The IP algorithm may be superior to the GP in some fields, but since 2003, we haven't noted a significant number of applications. It is still not tested to solve a wide range of automatic programming problems.

1.3.4 Gene Expression Programming

Gene Expression Programming (GEP) was proposed by Ferreira. In fact, GEP looks like the GP algorithm [113, 150]. The basic difference between them resides in the kind of the individuals [150]. According to Ferreira, the individuals in GP are encoded as nonlinear entities (parse trees) of different sizes and shapes. However, in GEP, the individuals are presented by linear strings of fixed length called the genomes. GEP individual is characterized by the simplicity, the linearity and the small size.

The reproduction in GEP includes genetic recombination and a new operator called replication. Replication consists of copying the genome and transmitting it to the next generation. Except mutation, an operator cannot do an action upon a chromosome more than once. GEP is tested not only on simple problems as symbolic regression and Boolean concept learning and planning, but also on a complex problem called cellular automata rule for the density classification task. Results demonstrated that the innovative system behind the linear genes enabled GEP to significantly outperform GP.

Over the past decades, gene expression programming has attracted the attention of authors from several research fields, leading to a number of enhanced GEP algorithms. In 2005, Li proposed an extended GEP adding a prefix notation to represent programs, and this has significantly improved the effectiveness of the evolutionary search [162]. Many attempts to decrease the computational time of GEP have been proposed in the literature [164–166]. The majority of these research

works are based on parallel computing technologies. Recently, Zhong developed a new method called Self-Learning GEP (SL-GEP) [163]. In SL-GEP, the individual is represented using Automatically Defined Functions (ADFs). The main advantage of this variant is that it has little control parameters and it also has both the simplicity and the generality. A detailed investigation about GEP and its variants is available in [167]

Nowadays, GEP algorithm has proved its effectiveness in searching for concise and accurate programs [155, 159]. It has been implemented and tested on several real world issues with much success reported, including symbolic regression problems [159, 160], recognition [152], time series predictions [153, 154, 161], classification problems [151, 155], data mining and knowledge discovery [157, 158], network optimization [156], etc.

1.3.5 Particle Swarm Programming

The automated evolution of programs by the use of the particle swarm optimization algorithm was introduced in 2004 by O'Neill and Brabazon [20]. Solutions are encoded using a CFG expressed in BNF. Therefore they called their algorithm grammatical swarm (GS). In this GS version, a program is presented on computers using a fixed-length string. The main difference between the GS and GE is the use of PSO instead of GA in the search process. The authors applied GS to four famous data sets and they showed that GS outperformed GE in two of the studied automatic programming problems. The new algorithm was adapted to be applied for solving classification problems [83]. An extended version analyzing parameter adjustment was written by the same authors in 2006 [122]. Ramstein and his colleagues were also interested by the GS. They developed a new variant of the algorithm and they used it for the classification of protein [118] and the detection of remote protein homologs [117]. To ensure diversity and explore a larger research space, Si proposed a method to update velocity equations for each particle using GS [119]. The application of GS to design and train the artificial neural network was performed initially by DeMingoLopez et al. [120] then by Si et al. [121]. All mentioned GS algorithms encode programs as a fixed or variable linear structure.

There are also other types of research works adapting PSO in automatic programming, where they represent a program as a parse tree. Veenhuis was the first who employed this structure in the PSO [123]. He called his algorithm Tree Swarm Optimization (TSO) and he tested it in symbolic regression and classification areas. Results showed that TSO needs less evaluations than the GP and Ant colony programming algorithm using Tree Adjoining Grammar (AntTAG) to find best trees. Another variant of PSO named geometric PSO (GPSO) [124] was implemented and tested in automatic programming.

1.3.6 Artificial Bee Colony Programming

The aspect of programming in the ABC algorithm was outlined by Karaboga under the name of Artificial Bee Colony programming (ABCP) [21]. As we mentioned in Sect. 2.2.2, the ABCP search method is based on the simulation of the intelligent foraging behavior of bee swarms. Karaboga encoded each individual by an expression tree which corresponds to the position of a given food source. To build a new program, this algorithm employs a sharing mechanism. The idea is to produce a new solution from the candidate one and its neighbor. First, one node is selected from the current tree and another node is selected from one of its neighbors. Hence, we obtain a new tree from the structure of the current solution and from replacing subtree of the chosen node by subtree corresponding to the neighbor node previously selected. The authors described the application of their new method on symbolic regression which represents the first application area of the most automatic programming algorithms. In 2013, Si introduced the Grammatical adaptation of the ABC algorithm under the name of Grammatical Bee Colony (GBC) [125]. Using the context free grammar BNF, GBC builds programs through genotype-to-phenotype mapping. The genotype is presented by a food source's position. GBC was evaluated on symbolic regression, Santa Fe Ant Trail, even-3 parity, and multiplexer benchmarks. The same work was recently improved and applied to classification of medical data showing the ability of GBC algorithm to generate effective computer programs [126].

1.3.7 Ant Colony Programming

Swarm programming has appeared thanks to the development of Ant Colony Programming (ACP) algorithm in 2000 [16]. Roux has outlined the first features of the algorithm combining the automatic generation of programs with the ant paradigm. The main difference between the Ant Programming and GP is the use of a global memory presented with a pheromone matrix instead of a local memory. Hence, each node of a given parse tree is characterized by a pheromone table stored to keep track of the pheromone amount associated with all possible terminals and functions. As in ACO, Ant programming individuals are evaluated and the pheromone tables are updated using evaporation and reinforcement processes according to the fitness of each program. The application of this algorithm to symbolic regression and multiplexer problems showed a slight improvement of results compared to GP.

Unlike other new automatic programming metaheuristics, ACP was widely used and applied to many problems as solving approximation problems [128], evolving flexible neural network architecture [127], resolving differential equations [129] and fuzzy differential equations [130] etc....

A second type of algorithms employing grammar and ACO as search strategy was developed firstly by Abbas and his colleagues [82]. This proposition is named AntTAG. TAG is the abbreviation of Tree Adjoining Grammar which is a compact context-sensitive grammar. The pheromone level provided by ants while building their solution is recorded in a matrix. The performance of AntTAG algorithm was proven when applied to the symbolic regression area and compared to G3P. Another attempt to use AntTAG as program presentation is described in Ref [131], extending the previous research work and testing it in more difficult symbolic regression problems. The main disadvantage of AntTAG encoding solution is the complexity of the fixed grammar structure. In fact, the useful information may be excluded which inhibits the convergence of the algorithm.

Other interesting grammar based ACO algorithms have appeared recently and were applied to association rule mining [133], classification [132, 134, 135, 137], solving grammatical inference tasks [136].

1.3.8 Other Automatic Programming Algorithms

In the previous subsections, we described the most popular AP algorithms. This section is focused to present the other AP metaheuristics which are used less frequently, as bacterial programming, iterated local search programming, variable neighborhood programming, Firefly Programming, Herd Programming, Artificial Fish Swarm Programming and Tree Based Differential Evolution.

1.3.8.1 Bacterial Programming

In bacterial programming, programs are encoded by expression trees [19]. The search process is based on the Bacterial Evolution Algorithm (BEA). BEA mimics a biological phenomenon of microbial evolution. For evolving population, BEA is characterized by two special operations, which are the bacterial mutation and the gene transfer operation. Bacteroids (individuals) have their specific fitness function calculated based on energy replenishment rates and collision avoidance [138]. In fact, BEA is different from the bacterial foraging optimization (BFO). The latter evolves according to a team foraging behavior named chemotaxis [2]. However, up to now, BFO is not yet implemented as an automatic programming algorithm. In BP algorithm, while one individual is mutated, the gene transfer operator is employed for the whole population allowing the bacteria to transfer information to the other programs. BP was initially applied to the B-spline neural network design and compared to GP. Although results are slightly better than GP, the advantage of BP is the ease of parameter adjustment.

1.3.8.2 Iterated Local Search Programming

Iterated local search is considered as the first solution-based metaheuristic implemented to automatic programming under the name of APRILS (Automatic PRogramming via Iterated Local Search). APRILS has been proposed by Nguyen [79]. The algorithm starts from a program presented by dispatching rules and performs a set of local searches. A rule is encoded by a tree structure. The main step in the iterated local search algorithm is the perturbation operation. Its objective is to avoid the premature convergence to a local optimum. In APRILS, authors defined two types of perturbation methods which are RSM and subtree extraction [79]. This research work was applied to dynamic job shop scheduling problem. Different simulation scenarios were performed showing the efficiency of APRILS to generate programs compared to GP and GEP. The advantage is the ability to use less lines of code to develop the algorithm and to produce shorter programs (solution).

1.3.8.3 Tree Based Differential Evolution

The first investigation describing the automatic construction of programs using the differential evolution algorithm was published by O'Neill under the name of Grammatical Differential Evolution (GDE) [142]. In GDE each program is encoded by construction rules specified in the BNF grammar. GDE was tested on multiplexer, quartic symbolic regression and Santa Fe Ant trail problems. The results demonstrated that GDE is able to generate effective programs.

Indeed, the main advantage of DE algorithm is that it requires only few evaluation steps to optimize a function. Researchers benefited from this characteristic and developed DE variants to solve other automatic programming problems. Veenhuis proposed a successful algorithm called Tree based Differential Evolution (TreeDE) [140]. In TreeDE, trees are converted to vectors and discrete symbols are modeled by points in a real-valued vector space. Another variant for program evolution based on DE was introduced by Moraglio [143] and it was called Geometric Differential Evolution. In the automatic programming version, the same geometric interpretation of the search dynamic was kept. Although the results on standard benchmarks are promising, Geometric Differential Evolution has some limitations; for example Koza's search space based on subtree crossover style cannot be modeled.

In 2012, Fonlupt proposed a TreeDE method quite different from Veenhuis system [141]. To represent a solution, his algorithm is based on Banzhaf's Linear GP. Individuals are linear imperative programs, storing the continuous representation. Hence, Fonlupt' system includes the use of float constants which is not the case for Veenhuis mechanism. Results in symbolic regression problems demonstrated that the TreeDE behavior was improved.

TreeDE algorithm was also applied to solve some types of problems such as EvoLisat challenge (consisting in the approximation of an image using artistic elements) [144] and optimization of trees and interactive evolutionary computation [145]. Despite the early apparition of DE in the automatic programming area (since

2006), we have not seen a large number of variants in comparison with other techniques.

1.3.8.4 Estimation of Distribution Programming

The automated construction of computer programs by using estimation of distribution algorithm was explored initially by Yanai in 2003 under the name of Estimation of Distribution Programming (EDP) [168]. Individuals are expressed as parse trees. The dependency relationship of tree nodes is explicit in EDP. In each generation, the probability distribution of the population is estimated based on the Boolean function problem then, programs are generated based on the results. The experimental results on max and Boolean function problems demonstrated that EDP is an effective automatic programming algorithm compared to GP [168, 169]. In the literature, we find only a very limited amount of work using EDA as research method to solve automatic programming problems [170, 171]. However, more research studies are focused on probabilistic model building GP [172–175].

1.3.8.5 Grammatical Fireworks Algorithm

Grammatical firework is an automatic programming algorithm expressing individuals using BNF of CFG [146]. It follows the evolution search strategy of algorithms inspired by the firework explosion phenomenon. Firework algorithm is a recently proposed metaheuristic [147]. In fact, two different types of explosion processes are developed. The Grammatical fireworks algorithm was applied to symbolic regression, Santa Fe ant trail and 3-input multiplexer problems. The comparative study with Grammatical Evolution, Grammatical Swarm, Grammatical Differential Evolution and Grammatical Artificial Bee Colony demonstrated that the proposed algorithm is able to generate good programs.

1.3.8.6 Artificial Fish Swarm Programming

Artificial fish swarm optimization was initially proposed by Li et al. [149]. The algorithm is inspired by the intelligent fish swarm behavior in the water to discover and locate nutritious areas. Five main activities namely, preying, following, swarming, moving (randomly), and leaping are performed by fishes.

Artificial fish swarm programming was proposed by Liu and his colleagues, [148], where an individual is encoded based on gene expression programming schema. The authors decided to include four behaviors inspired by artificial fish swarm optimization which are moving (randomly), preying, following, and avoiding. To evaluate a program, a new fitness function considering the number of nodes in parse tree was defined. The performance of the proposed algorithm was

tested on symbolic regression problem. Results showed the high precision of the artificial fish programming algorithm compared to gene expression programming.

1.3.8.7 Variable Neighborhood Programming

There had been recently growing interest in the use of metaheuristics based entirely on local searches. However, to our knowledge, only two approaches are proposed and presented in automatic programming. The first is the previously described method (APRILS) and the second is called Variable Neighborhood Programming (VNP). The Basic features of VNP were recently introduced by Elleuch et al. [80]. Two main ideas are developed in this work. In fact, the most commonly program encoding is a parse tree. According to the authors, the simple structure of a tree suffers from the inability to model complicated problems. Moreover, it seems difficult for this structure to be a representative of several real life issues. To overcome this drawback, a new set called coefficient set was added. Each coefficient is associated with each terminal in order to avoid giving the same weight to all terminals. The second contribution was the description of the elementary tree transformation local search. The method allows exploring more solutions horizontally. The algorithm was developed and tested to solve forecasting problems and it gave good experimental results.

1.4 Discussion and Challenges

Automatic programming is an active research field with applications in many domains. Table 1.1 gives an overview of first AP metaheuristic algorithms, encoding schema, program initialization and their application fields. In fact, the majority of these methods were initially tested in symbolic regression area and they were generally compared to genetic programming. The table summarized also the most recent references corresponding to the use of each AP technique. While GP algorithm and its variants have been widely used by experts and scientists, other AP metaheuristics are less applied mainly in the wider domains of science and engineering. To get a fair comparison between GP and new methods, it would be interesting to test them on large scale real world applications. Therefore, we decide to give a short comparison between GP and the Artificial Neural Network (ANN) which is also a very popular machine learning method [214, 230, 231].

An ANN is a computational tool that has found extensive utilization in solving many complex problems [214]. It's a distributed and parallel information processing model which is constituted by several layers of neurons. From inputs, an ANN produces a non linear function as an output.

ANN and GP seem to have the same behavior when solving problems. However, GP is more flexible since it can build several type of programs; it can even evolve an ANN structure. On the other hand, it's known that GP suffers from the bloating

Table 1.1 Literature overview table of first AP metaheuristic contributions

Metaheuristic	First AP method	Reference	Encoding	Program generation	First applications	Recent references
GA	GP	[14]	Parse tree	Ramped half-and-half	Optimal control, robotic planning, symbolic regression and Boolean 11-multiplexer	[86, 93, 97, 98]
AIS	IP	[18]	Parse tree	Ramped half-and-half	Symbolic regression	[109]
–	GEP	[150]	Gene expression tree	–	Symbolic regression, Block stacking and classification	[155, 157, 163, 165]
PSO	GS	[20]	CFG expressed on BNF	–	Classification	[119–121]
ABC	ABCP	[21]	Parse tree	Ramped half and half	Classification	[125, 126]
ACO	ACP	[16]	Parse tree + Pheromone table	Ramped half-and-half	Symbolic regression and multiplexer	[129, 130, 134, 136]
BEA	BP	[19]	Expression tree	Grow	B-spline neural networks design	–
ILS	APRILS	[79]	Expression tree	Grow	Dynamic job shop scheduling	–
DE	GDE	[142]	Expression tree	Grow	Multiplexer, quartic symbolic regression and Santa Fe Ant trail	[144, 145]
EDA	EDP	[168]	Parse tree	Distribution-based generation	Max and Boolean function	–
FA	FP	[146]	Expression tree	–	Santa Fe ant trail, symbolic regression and 3-input multiplexer	–
AFSA	AFSP	[148]	Gene expression tree	–	Symbolic regression	–
VNS	VNP	[80]	Parse tree	–	Time series forecasting	–

GA: genetic algorithm; GP: Genetic programming; AIS: artificial immune system; IP: immune programming; EA: evolutionary algorithm; GEP: gene expression programming; PSO: particle swarm optimization; GS: grammatical swarm; ABC: artificial bee colony; ABCP: artificial bee colony programming; ACO: ant colony optimization; ACP: ant colony optimization; BEA: bacterial evolution algorithm; BEP: bacterial evolution programming; ILS: iterated local search; APRILS: automatic programming via iterated local search; DE: differential evolution; GDE:grammatical differential evolution; EDA: estimation of distribution algorithm ; EDP: estimation of distribution programming ; FA: firework algorithm; FP: firework programming; AFSA: artificial fish swarm algorithm;ADSP: artificial fish swarm programming; VNS:variable neighborhood search; VNP: variable neighborhood programming

problem: the built of complex models sometimes for minimal predictive ability gain. In addition, when we see the coding side, we can say that ANN are generally easy to implement and work fine. However, they have a blackbox nature which makes them non-user friendly. On the other hand, GP output is always human friendly, but implementing such an algorithm can be painstaking especially in the phase of parameters setting. Notwithstanding, we advise readers interested by this topic to take a look at the research study of Wolpert which states that GP and ANN methods provide equivalent results when their performance is averaged across all possible problems [178]. To conclude we can say that for your problem, to define which approach works better you should to answer tow main questions: "What are the restrictions on topology of your program and your ANN and what error metric is ultimately important? Moreover, there are several additional issues must be considered such as computational complexity, training time, number of hyperparameters which are not obtained by training, and scaling of the complexity taking into account the number of variables.

Despite the success of metaheuristics implemented in AP, several algorithms still not adapted to this area. As we mentioned previously, the No Free Lunch theorems, developed by Wolpert [178], demonstrated that for the entire set of problems all optimization algorithms have the same average of performance. However, on a particular class of problems, some algorithms perform better than others and we think that it's the case of machine learning problems.

We notice that GP and population-based AP metaheuristics have shown their effectiveness to solve complicated problems. However, when the target computer program is based on many features, the search strategy employed by population-based AP metaheuristics can suffer from a trouble balancing on its exploitation and exploration abilities. Actually, population-based algorithms explore at early generations a large number of computer programs. Nevertheless, some potential computer programs could not be considered because of the lack of the early exploitation. Previous studies demonstrated that applying local search to these algorithms is a good idea to improve the exploitation behavior unless the computational cost will significantly increased. Therefore, there is no doubt that the automatic construction of programs using more solution-based methods, such as simulated annealing and tabu search, etc, will emerge in the near future.

Another important research area in AP, as for any metaheuristic technique, is parameter tuning. Any AP algorithm has parameters dependent to the problem and to the algorithm itself. Indeed, experiments have shown that the selected parameter values have a great influence in the performance of the algorithm. Non expert users need self adaptive proposals to apply these methods with no prior knowledge. In addition, as for GP [176], the automatic design of other AP algorithms should be explored.

Furthermore, the 'parallelization' of automatic programming algorithms is still an unexplored issue. Indeed, a parallel implementation of a GP system has been proposed since 1997 to solve a compute-intensive financial application [177]. A parallelization approach of a multi-objective ACP was also developed for classification tasks, using GPUs [134]. These studies have shown that parallel computing

could improve the performance of the AP technique and the computational time. Rarely, a research work on automatic programming indicates the complexity or the computational time required to execute the algorithm.

1.5 Conclusions

In this work, we provide an investigation showing the passage from a simple metaheuristic to an automatic programming one. Metaheuristics are classified to population-based and solution-based. However, the automatic construction of program using solution-based metaheuristics is described only for two algorithms namely, iterated local search and variable neighborhood search. Therefore, in AP section, well known AP metaheuristics are initially detailed while the other ones are briefly described.

Common concepts for automatic programming systems and encoding schema have been illustrated for providing a starting reference for researchers in the field. The need of the automatic generation of computer programs is growing with the development of smart systems. On the other hand, the research communities, the number of workshops, conferences, and sessions dealing with AP algorithms are also increasing significantly. Well known conferences in this field include European Conference on Genetic Programming (EUROGP), IEEE Congress on Evolutionary Computation (CEC), IEEE/ACM International Conference on Automated Software Engineering, The Genetic and Evolutionary Computation Conference (GECCO) etc.

References

1. Mladenović, N. and Hansen, P. Variable neighborhood search. *Computers & Operations Research*. **24**, 1097–1100 (1997,11)
2. Passino, K. Biomimicry of bacterial foraging for distributed optimization and control. *IEEE Control Systems Magazine*. **22**, 52–67 (2002,6)
3. Lirov, Y. Computer-aided software engineering of expert systems. *Expert Systems With Applications*. **2**, 333–343 (1991,1)
4. Das, S., Mullick, S. andSuganthan, P. Recent advances in differential evolution – An updated survey. *Swarm And Evolutionary Computation*. **27** pp. 1–30 (2016,4)
5. Mühlenbein, H. and Paaß, G. From recombination of genes to the estimation of distributions I. Binary parameters. (Springer, Berlin, Heidelberg,1996)
6. Storn, R. and Price, K. Differential Evolution – A Simple and Efficient Heuristic for global Optimization over Continuous Spaces. *Journal Of Global Optimization*. **11**, 341–359 (1997)
7. Hansen, N., Ostermeier, A. and Gawelczyk, A. On the Adaptation of Arbitrary Normal Mutation Distributions in Evolution Strategies: The Generating Set Adaptation. *Proceedings Of The 6th International Conference On Genetic Algorithms*. pp. 57–64 (1995)
8. Kennedy, J. and Eberhart, R. Particle Swarm Optimization. *International Conference On Neural Network*. pp. 1942–1948 (1995)
9. Dorigo, M. Optimization, Learning and Natural Algorithms. (Politecnico di Milano,1992)

10. Kirkpatrick, S., Gelatt, C. and Vecchi, M. Optimization by Simulated Annealing. *Science*. **220** pp. 671–680 (1983)
11. Glover, F. Future paths for integer programming and links to artificial intelligence. *Computers And Operations Research*. **13**, 533–549 (1986)
12. Goldberg, D. Genetic algorithms in search, optimization, and machine learning. (Addison-Wesley Longman Publishing Co., Inc.,1989)
13. Fogel, L. Toward Inductive Inference Automata.. *IFIP Congress*. pp. 395–400 (1962)
14. Koza, J. Genetic programming: on the programming of computers by means of natural selection. (MIT Press Cambridge, MA, USA,1992,12)
15. Musilek, P., Lau, A., Reformat, M. and Wyardscott, L. Immune programming. *Information Sciences*. **176**, 972–1002 (2006,4)
16. Roux, O. and Fonlupt Cyril Ant Programming: or how to use ants for automatic programming. *International Conference On Swarm Intelligence*. pp. 121–129 (2000)
17. Ryan, C., Collins, J. and Neill, M. Grammatical evolution: Evolving programs for an arbitrary language. (Springer, Berlin, Heidelberg,1998)
18. Johnson, C. Artificial Immune Systems Programming for Symbolic Regression. *LNCS 2610, 2610. Springer*. pp. 345–353 (2003)
19. Cabrita, C., Botzheim, J., Ruano, A. and Koczy, L. Design of B-spline neural networks using a bacterial programming approach. *2004 IEEE International Joint Conference On Neural Networks (IEEE Cat. No.04CH37541)*. **3** pp. 2313–2318 (2004)
20. O'Neill, M. and Brabazon, A. Grammatical Swarm. *Genetic And Evolutionary Computation Conference (GECCO)*. pp. 163–174 (2004)
21. Karaboga, D., Ozturk, C., Karaboga, N. and Gorkemli, B. Artificial bee colony programming for symbolic regression. *Information Sciences*. **209** pp. 1–15 (2012)
22. Crainic, T. and Toulouse, M. Parallel Strategies for Meta-Heuristics. *Handbook Of Metaheuristics*. pp. 475–513 (2003)
23. Metropolis, N., Rosenbluth, A., Rosenbluth, M., Teller, A. and Teller, E. Equation of State Calculations by Fast Computing Machines. *The Journal Of Chemical Physics*. **21**, 1087–1092 (1953,6)
24. Hajek, B. A tutorial survey of theory and applications of simulated annealing. *1985 24th IEEE Conference On Decision And Control*. pp. 755–760 (1985,12)
25. Suman, B. and Kumar, P. A survey of simulated annealing as a tool for single and multiobjective optimization. *Journal Of The Operational Research Society*. **57**, 1143–1160 (2006,10)
26. Koulamass, C., Antony, S. and Jaen, R. A survey of simulated annealing applications to operations research problems. *Omega*. **22**, 41–56 (1994,1)
27. Nara, K. Simulated Annealing Applications. *Modern Optimisation Techniques In Power Systems*. pp. 15–38 (1999)
28. Charon, I. and Hudry, O. The noising method: a new method for combinatorial optimization. *Operations Research Letters*. **14**, 133–137 (1993,10)
29. Creutz, M. Microcanonical Monte Carlo Simulation. *Physical Review Letters*. **50**, 1411–1414 (1983,5)
30. Dueck, G. and Scheuer, T. Threshold accepting: A general purpose optimization algorithm appearing superior to simulated annealing. *Journal Of Computational Physics*. **90**, 161–175 (1990,9)
31. Siddique, N. and Adeli, H. Simulated Annealing, Its Variants and Engineering Applications. *International Journal On Artificial Intelligence Tools*. **25** (2016,12)
32. Glover, F. and Laguna, M. Tabu search. (Kluwer Academic Publishers,1997)
33. Gendreau, M. An Introduction to Tabu Search. *Handbook Of Metaheuristics*. pp. 37–54 (2003)
34. Hansen, P., Mladenović, N., Todosijević, R. and Hanafi, S. Variable neighborhood search: basics and variants. *EURO Journal On Computational Optimization*. pp. 1–32 (2016,8)

35. Todosijević, R., Mladenović, M., Hanafi, S., Mladenović, N. and Crévits, I. Adaptive general variable neighborhood search heuristics for solving the unit commitment problem. *International Journal Of Electrical Power & Energy Systems*. **78** pp. 873–883 (2016,6)
36. Brimberg, J., Mladenović, N. and Urošević, D. Solving the maximally diverse grouping problem by skewed general variable neighborhood search. *Information Sciences*. **295**, 650–675 (2015,2)
37. Stutzle, T. Local search algorithms for combinatorial problems : analysis, improvements, and new applications. (Infix,1998)
38. Lourenço, H., Lourenço, H., Martin, O. and Stützle, T. Iterated local search. *Handbook Of Metaheuristics, International Series In Operations Research And Management Science*. **57** pp. 321–353 (2002)
39. Lourenço, H., Martin, O. and Stützle, T. Iterated Local Search: Framework and Applications. (Springer US,2010)
40. Holland, J. Adaptation in natural and artificial systems : an introductory analysis with applications to biology, control, and artificial intelligence. (MIT Press,1992)
41. Rechenberg, I. Evolutionsstrategie; Optimierung technischer Systeme nach Prinzipien der biologischen Evolution.. (Frommann-Holzboog,1973)
42. Hillis, W. Co-evolving parasites improve simulated evolution as an optimization procedure. *Physica D: Nonlinear Phenomena*. **42**, 228–234 (1990,6)
43. Sindhiya, S. and Gunasundari, S. A survey on genetic algorithm based feature selection for disease diagnosis system. *Proceedings Of IEEE International Conference On Computer Communication And Systems ICCCS14*. pp. 164–169 (2014,2)
44. Li, S., Kang, L. and Zhao, X. A survey on evolutionary algorithm based hybrid intelligence in bioinformatics.. *BioMed Research International*. **2014** pp. 362–370 (2014)
45. Goldberg, D., Goldberg, D. and Deb, K. A comparative analysis of selection schemes used in genetic algorithms. *Foundations Of Genetic Algorithms*. pp. 69–93 (1991)
46. Blum, C. and Roli, A. Metaheuristics in combinatorial optimization. *ACM Computing Surveys*. **35**, 268–308 (2003,9)
47. Vekaria, K. and Clack, C. Selective crossover in genetic algorithms: An empirical study. *Lecture Notes In Computer Science*. **1498** pp. 438–447 (1998)
48. Rechenberg, I. Cybernetic Solution Path of an Experimental Problem. *Library Translation 1122, Farnborough.*. (1965)
49. Hansen, N. and Ostermeier, A. Completely Derandomized Self-Adaptation in Evolution Strategies. *Evolutionary Computation*. **9**, 159–195 (2001,6)
50. Hansen, N. The CMA Evolution Strategy: A Comparing Review. *Towards A New Evolutionary Computation*. pp. 75–102 (2006)
51. Bäck, T., Hoffmeister, F. and Schwefel, H. A Survey of Evolution Strategies. *Proceedings Of The Fourth International Conference On Genetic Algorithms*. pp. 2–9 (1991)
52. Arnold, D. and Beyer, H. Performance analysis of evolution strategies with multi-recombination in high-dimensional RN-search spaces disturbed by noise. *Theoretical Computer Science*. **289**, 629–647 (2002,10)
53. Beyer, H. and Schwefel, H. Evolution strategies – A comprehensive introduction. *Natural Computing*. **1**, 3–52 (2002)
54. Hansen, N., Arnold, D., Auger, A., Auger Evolution Strategies Janusz Kacprzyk, A. and Pedrycz, W. Evolution Strategies. (Janusz Kacprzyk; Witold Pedrycz. Handbook of Computational Intelligence, Springer,2015)
55. Dorigo, M. and Blum, C. Ant colony optimization theory: A survey. *Theoretical Computer Science*. **344**, 243–278 (2005,11)
56. Dorigo, M. and Stützle, T. Ant Colony Optimization: Overview and Recent Advances. (Springer US,2010)
57. Boussaıd, I., Lepagnot, J. and Siarry, P. A survey on optimization metaheuristics. *Information Sciences*. **237** pp. 82–117 (2013)

58. Ozcan, E. and Mohan, C. Particle swarm optimization: surfing the waves. *Proceedings Of The Congress On Evolutionary Computation-CEC99 (Cat. No. 99TH8406)*. pp. 1939–1944 (1999)
59. Clerc, M. and Kennedy, J. The particle swarm - explosion, stability, and convergence in a multidimensional complex space. *IEEE Transactions On Evolutionary Computation*. **6**, 58–73 (2002)
60. Thangaraj, R., Pant, M., Abraham, A. and Bouvry, P. Particle swarm optimization: Hybridization perspectives and experimental illustrations. *Applied Mathematics And Computation*. **217**, 5208–5226 (2011)
61. Masrom, S., Moser, I., Montgomery, J., Abidin, S. and Omar, N. Hybridization of Particle Swarm Optimization with adaptive genetic algorithm operators. *13th International Conference On Intellient Systems Design And Applications*. pp. 153–158 (2013,12)
62. Arasomwan, A. and Adewumi, A. On the Hybridization of Particle Swarm Optimization Technique for Continuous Optimization Problems. *Lecture Notes In Computer Science*. pp. 358–366 (2016,6)
63. Karaboga, D. and Akay, B. A survey: algorithms simulating bee swarm intelligence. *Artificial Intelligence Review*. **31**, 61–85 (2009,6)
64. Tuyls, K., Guessoum, Z., Kudenko, D. and Nowe, A. Adaptive Agents and Multi-Agent Systems III. Adaptation and Multi-Agent Learning 5th, 6th, and 7th European Symposium, ALAMAS 2005–2007 on Adaptive and Learning Agents and Multi-Agent Systems, Revised Selected Papers. (Springer-Verlag Berlin Heidelberg,2008)
65. Karaboga, D., Gorkemli, B., Ozturk, C. and Karaboga, N. A comprehensive survey: artificial bee colony (ABC) algorithm and applications. *Artificial Intelligence Review*. **42**, 21–57 (2014,6)
66. Farmer, J., Packard, N. and Perelson, A. The immune system, adaptation, and machine learning. *Physica D: Nonlinear Phenomena*. **22**, 187–204 (1986,10)
67. Jerne, N. Towards a network theory of the immune system.. *Annales D'immunologie*. **125C**, 373–89 (1974,1)
68. Hosseinpour, F., Bakar, K., Hardoroudi, A. and Kazazi, N. Survey on Artificial Immune System as a Bio-inspired Technique for Anomaly Based Intrusion Detection Systems. *2010 International Conference On Intelligent Networking And Collaborative Systems*. pp. 323–324 (2010,11)
69. Li, C., Peng, H., Xu, A. and Wang, S. Immune System and Artificial Immune System Application. *World Congress On Medical Physics And Biomedical Engineering 2006*. pp. 477–480 (2007)
70. Yang, H., Li, T., Hu, X., Wang, F. and Zou, Y. A Survey of Artificial Immune System Based Intrusion Detection. *The Scientific World Journal*. **2014** pp. 1–11 (2014)
71. Tan, Y. Artificial immune system : applications in computer security. (Wiley-IEEE Computer Society Press,2016)
72. Sotiropoulos, D. and Tsihrintzis, G. Machine Learning Paradigms : Artificial Immune Systems and their Applications in Software Personalization. (Springer International Publishing,2017)
73. Timmis, J., Andrews, P., Owens, N. and Clark, E. An interdisciplinary perspective on artificial immune systems. *Evolutionary Intelligence*. **1**, 5–26 (2008,3)
74. Timmis, J., Andrews, P. and Hart, E. On artificial immune systems and swarm intelligence. *Swarm Intelligence*. **4**, 247–273 (2010,12)
75. Rozenberg, G., Back, T. and Kok, J. Handbook of Natural Computing. (Springer Berlin Heidelberg,2012)
76. Kaedi, M. Fractal-based Algorithm : A New Metaheuristic Method for Continuous Optimization. *International Journal Of Artificial Intelligence*. **15**, 76–92 (2017)
77. Kaveh, A. and Bakhshpoori, T. A new metaheuristic for continuous structural optimization: water evaporation optimization. *Structural And Multidisciplinary Optimization*. **54**, 23–43 (2016,7)

78. Wu, X., Zhou, Y. and Lu, Y. Elite Opposition-Based Water Wave Optimization Algorithm for Global Optimization. *Mathematical Problems In Engineering*. **2017** pp. 1–25 (2017)
79. Su Nguyen, Mengjie Zhang, Johnston, M. and Kay Chen Tan Automatic Programming via Iterated Local Search for Dynamic Job Shop Scheduling. *IEEE Transactions On Cybernetics*. **45**, 1–14 (2015,1)
80. Elleuch, S., Hansen, P., Jarboui, B. and Mladenović, N. New VNP for automatic programming. *Electronic Notes In Discrete Mathematics*. **58** pp. 191–198 (2017)
81. Hoai, N. and McKay, R. A framework for tree adjunct grammar guided genetic programming. *Proceedings Of The Post-Graduate ADFA Conference On Computer Science (PACCS'01)*. pp. 93–99 (2001)
82. Abbass, H., Xuan Hoai and McKay, R. AntTAG: a new method to compose computer programs using colonies of ants. *Proceedings Of The 2002 Congress On Evolutionary Computation. CEC'02 (Cat. No.02TH8600)*. **2** pp. 1654–1659 (2002)
83. O'Neill, M., Brabazon, A. and Adley, C. The automatic generation of programs for classification problems with grammatical swarm. *Proceedings Of The 2004 Congress On Evolutionary Computation (IEEE Cat. No.04TH8753)*. pp. 104–110 (2004)
84. Hosseini, S. and Nemati, A. Application of Genetic Programming for Electrical Engineering Predictive Modeling: A Review. *Handbook Of Genetic Programming Applications*. pp. 141–154 (2015)
85. Afzal, W. and Torkar, R. On the application of genetic programming for software engineering predictive modeling: A systematic review. *Expert Systems With Applications*. **38**, 11984–11997 (2011)
86. Nguyen, Q., Pham, T., Nguyen, X. and McDermott, J. Subtree semantic geometric crossover for genetic programming. *Genetic Programming And Evolvable Machines*. **17**, 25–53 (2016,3)
87. Spears, W. and Anand, V. A study of crossover operators in genetic programming. (Springer, Berlin, Heidelberg,1991)
88. Poli, R. and Langdon, W. Genetic Programming with One-Point Crossover. *Soft Computing In Engineering Design And Manufacturing*. pp. 180–189 (1998)
89. Langdon, W. Size Fair and Homologous Tree Crossovers for Tree Genetic Programming. *Genetic Programming And Evolvable Machines*. **1**, 95–119 (2000)
90. Schaffer, J. and Gilbert Proceedings of the Third International Conference on Genetic Algorithms. *Proceedings Of The 3rd International Conference On Genetic Algorithms*. pp. 445 (1989)
91. Piszcz, A. and Soule, T. A survey of mutation techniques in genetic programming. *Proceedings Of The 8th Annual Conference On Genetic And Evolutionary Computation - GECCO '06*. pp. 951–952 (2006)
92. Quan, W. and Soule, T. A Study of the Role of Single Node Mutation in Genetic Programming. (Springer, Berlin, Heidelberg,2004)
93. Kubalık, J., Alibekov, E., Žegklitz, J. and Babuška, R. Hybrid Single Node Genetic Programming for Symbolic Regression. *Transactions On Computational Collective Intelligence XXIV. Lecture Notes In Computer Science*. **9770** pp. 61–82 (2016)
94. Poli, R., Langdon, W. and McPhee, N. A Field Guide to Genetic Programming. (Lulu Enterprises, UK Ltd,2008)
95. Vanneschi, L., Castelli, M. and Silva, S. Measuring bloat, overfitting and functional complexity in genetic programming. *Proceedings Of The 12th Annual Conference On Genetic And Evolutionary Computation - GECCO '10*. pp. 877 (2010)
96. Vega, F., Gil, G., Gómez Pulido, J. and Guisado, J. Control of Bloat in Genetic Programming by Means of the Island Model. (Springer, Berlin, Heidelberg,2004)
97. Whigham, P. and Dick, G. Implicitly Controlling Bloat in Genetic Programming. *IEEE Transactions On Evolutionary Computation*. **14**, 173–190 (2010,4)
98. Trujillo, L., Muñoz, L., Galván-López, E. and Silva, S. neat Genetic Programming: Controlling bloat naturally. *Information Sciences*. **333** pp. 21–43 (2016,3)

99. Lopes, H. and S., H. Genetic programming for epileptic pattern recognition in electroencephalographic signals. *Applied Soft Computing*. **7**, 343–352 (2007,1)
100. Escalante, H., Mendoza, K., Graff, M. and Morales-Reyes, A. Genetic Programming of Prototypes for Pattern Classification. (Springer, Berlin, Heidelberg,2013)
101. Martin, M. Genetic programming for real world robot vision. *IEEE/RSJ International Conference On Intelligent Robots And System*. **1** pp. 67–72 (2002)
102. Foster, J., Ziegler, J., Aue, C., Ross, A., Sawitzki, D. and Banzhaf, W. Genetic programming : 5th European Conference, EuroGP 2002, Kinsale, Ireland, April 3–5, 2002 : proceedings. *Proceedings Of The 5th European Conference On Genetic Programming*. pp. 335 (2002)
103. Diveev, A., Ibadulla, S., Konyrbaev, N. and Shmalko, E. Variational Genetic Programming for Optimal Control System Synthesis of Mobile Robots. *IFAC-PapersOnLine*. **48**, 106–111 (2015)
104. Macedo, J., Marques, L. and Costa, E. Robotic odour search: Evolving a robot's brain with Genetic Programming. *2017 IEEE International Conference On Autonomous Robot Systems And Competitions (ICARSC)*. pp. 91–97 (2017,4)
105. Otero, F., Silva, M., Freitas, A. and Nievola, J. Genetic Programming for Attribute Construction in Data Mining. (Springer, Berlin, Heidelberg,2003)
106. Gandomi, A., Sajedi, S., Kiani, B. and Huang, Q. Genetic programming for experimental big data mining: A case study on concrete creep formulation. *Automation In Construction*. **70** pp. 89–97 (2016)
107. Ritchie, M., White, B., Parker, J., Hahn, L., Moore, J., Parl, F. and Moore, J. Optimization of neural network architecture using genetic programming improvesdetection and modeling of gene-gene interactions in studies of humandiseases. *BMC Bioinformatics 2003 4:1*. **105**, 60–61 (2003)
108. Rivero, D., Dorado, J., Rabuñal, J. and Pazos, A. Modifying genetic programming for artificial neural network development for data mining. *Soft Computing*. **13**, 291–305 (2009,2)
109. Bernardino, H. and Barbosa, H. Grammar-based immune programming to assist in the solution of functional equations. *2015 IEEE Congress On Evolutionary Computation (CEC)*. pp. 1167–1174 (2015,5)
110. Castro, L. and Von Zuben, F. Learning and optimization using the clonal selection principle. *IEEE Transactions On Evolutionary Computation*. **6**, 239–251 (2002,6)
111. Lau, A. and Musilek, P. Immune programming models of Cryptosporidium parvum inactivation by ozone and chlorine dioxide. *Information Sciences*. **179**, 1469–1482 (2009,4)
112. Ciccazzo, A., Conca, P., Nicosia, G. and Stracquadanio, G. An Advanced Clonal Selection Algorithm with Ad-Hoc Network-Based Hypermutation Operators for Synthesis of Topology and Sizing of Analog Electrical Circuits. *Artificial Immune Systems*. pp. 60–70 (2008)
113. Ferreira, C. and Cândida Gene expression programming : mathematical modeling by an artificial intelligence. (Springer-Verlag,2006)
114. Gan, Z., Chow, T. and Chau, W. Clone selection programming and its application to symbolic regression. *Expert Systems With Applications*. **36**, 3996–4005 (2009)
115. O'Neill, M. and Ryan, C. Grammatical evolution. *IEEE Transactions On Evolutionary Computation*. **5**, 349–358 (2001)
116. Koza, J. Genetic programming as a means for programming computers by natural selection. *Statistics And Computing*. **4**, 87–112 (1994,6)
117. Ramstein, G., Beaume, N. and Jacques, Y. Detection of Remote Protein Homologs Using Social Programming. (Springer, Berlin, Heidelberg,2009)
118. Ramstein, G., Beaume, N. and Jacques, Y. A Grammatical Swarm for protein classification. *2008 IEEE Congress On Evolutionary Computation (IEEE World Congress On Computational Intelligence)*. pp. 2561–2568 (2008,6)
119. Si, T., De, A. and Bhattacharjee, A. Grammatical Swarm Based-Adaptable Velocity Update Equations in Particle Swarm Optimizer. (Springer, Cham,2014)
120. De Mingo López, L., Gómez Blas, N. and Arteta, A. The optimal combination: Grammatical swarm, particle swarm optimization and neural networks. *Journal Of Computational Science*. **3**, 46–55 (2012,1)

121. Si, T., De, A. and Bhattacharjee, A. Grammatical swarm for Artificial Neural Network training. *2014 International Conference On Circuits, Power And Computing Technologies [ICCPCT-2014]*. pp. 1657–1661 (2014,3)
122. O'Neill, M. and Brabazon, A. Grammatical Swarm: The generation of programs by social programming. *Natural Computing*. **5**, 443–462 (2006,11)
123. Veenhuis, C., Koppen, M., Kruger, J. and Nickolay, B. Tree Swarm Optimization: An Approach to PSO-based Tree Discovery. *2005 IEEE Congress On Evolutionary Computation*. **2** pp. 1238–1245 (2005)
124. Togelius, J., De Nardi, R. and Moraglio, A. Geometric PSO + GP = Particle Swarm Programming. *2008 IEEE Congress On Evolutionary Computation (IEEE World Congress On Computational Intelligence)*. pp. 3594–3600 (2008,6)
125. Si, T., De, A. and Bhattacharjee, A. Grammatical Bee Colony. (Springer, Cham,2013)
126. Si, T. and Sujauddin, S. A Comparison of Grammatical Bee Colony and Neural Networks in Medical Data Mining. *International Journal Of Computer Applications*. **134**, 1–4 (2016,1)
127. Chen, Y., Yang, B. and Dong, J. Evolving Flexible Neural Networks Using Ant Programming and PSO Algorithm. (Springer, Berlin, Heidelberg,2004)
128. Boryczka, M., Czech, Z. and Wieczorek, W. Ant Colony Programming for Approximation Problems. *Lecture Notes In Computer Science*. pp. 142–143 (2003)
129. Kamali, M., Kumaresan, N. and Ratnavelu, K. Solving differential equations with ant colony programming. *Applied Mathematical Modelling*. **39**, 3150–3163 (2015)
130. Kamali, M., Kumaresan, N. and Ratnavelu, K. Takagi–Sugeno fuzzy modelling of some nonlinear problems using ant colony programming. *Applied Mathematical Modelling*. **48** pp. 635–654 (2017)
131. Shan, Y., Shan, Y., Abbass, H., Mckay, R. and Essam, D. AntTAG: a further study. *Proceedings Of The Sixth Australia-Japan Joint Workshop On Intelligent And Evolutionary Systems, Australian National University*. **30** pp. 93–99 (2002)
132. Olmo, J., Romero, J. and Ventura, S. Classification rule mining using ant programming guided by grammar with multiple Pareto fronts. *Soft Computing*. **16**, 2143–2163 (2012,12)
133. Olmo, J., Luna, J., Romero, J. and Ventura, S. Association rule mining using a multi-objective grammar-based ant programming algorithm. *2011 11th International Conference On Intelligent Systems Design And Applications*. pp. 971–977 (2011,11)
134. Cano, A., Olmo, J. and Ventura, S. Parallel multi-objective Ant Programming for classification using GPUs. *Journal Of Parallel And Distributed Computing*. **73**, 713–728 (2013,6)
135. Olmo, J., Romero, J. and Ventura, S. Using Ant Programming Guided by Grammar for Building Rule-Based Classifiers. *IEEE Transactions On Systems, Man, And Cybernetics, Part B (Cybernetics)*. **41**, 1585–1599 (2011,12)
136. Wieczorek, W. Inductive Synthesis of Cover-Grammars with the Help of Ant Colony Optimization. *Foundations Of Computing And Decision Sciences*. **41**, 297–315 (2016,1)
137. Hara, A., Watanabe, M. and Takahama, T. Cartesian Ant Programming. *2011 IEEE International Conference On Systems, Man, And Cybernetics*. pp. 3161–3166 (2011,10)
138. Numaoka, C. Bacterial Evolution Algorithm for rapid adaptation. (Springer, Berlin, Heidelberg,1996)
139. Besten, M., Stützle, T. and Dorigo, M. Design of Iterated Local Search Algorithms. (Springer, Berlin, Heidelberg,2001)
140. Veenhuis, C. Tree Based Differential Evolution. *Lecture Notes In Computer Science*. **5481** pp. 208–219 (2009)
141. Fonlupt, C., Robilliard, D. and Marion-Poty, V. Continuous Schemes for Program Evolution. *Genetic Programming - New Approaches And Successful Applications*. (2012,10)
142. O'Neill, M. and Brabazon, A. Grammatical Differential Evolution.. *International Conference On Artificial Intelligence*. pp. 231–236 (2006)
143. Moraglio, A. and Silva, S. Geometric Differential Evolution on the Space of Genetic Programs. (Springer, Berlin, Heidelberg,2010)

144. Zamuda, A. and Mlakar, U. Tiled EvoLisa image evolution with blending triangle brushstrokes and gene compression DE. *2016 IEEE Congress On Evolutionary Computation (CEC)*. pp. 2618–2625 (2016,7)
145. Funaki, R., Takano, H. and Murata, J. Tree structure based differential evolution for optimization of trees and interactive evolutionary computation. *2015 54th Annual Conference Of The Society Of Instrument And Control Engineers Of Japan (SICE)*. pp. 331–336 (2015,7)
146. Tapas Si, T. Grammatical Evolution Using Fireworks Algorithm. (Springer, Singapore,2016)
147. Tan, Y. and Zhu, Y. Fireworks Algorithm for Optimization. (Springer, Berlin, Heidelberg,2010)
148. Liu, Q., Odaka, T., Kuroiwa, J. and Ogura, H. Application of an Artificial Fish Swarm Algorithm in Symbolic Regression. *IEICE Transactions On Information And Systems*. **E96.D**, 872–885 (2013)
149. L. Li, X., J. Shao, Z. and X. Qian, J. An optimizing method based on autonomous animate: Fish swarm algorithm. *System Engineering Theory And Practice*. **22** pp. 32–38 (2002,11)
150. Ferreira, C. Gene Expression Programming: A New Adaptive Algorithm for Solving Problems. *Complex Syst*. **13** pp. 87–129 (2001,3)
151. Guerrero-Enamorado, A., Morell, C., Noaman, A. and Ventura, S. An Algorithm Evaluation for Discovering Classification Rules with Gene Expression Programming. *International Journal Of Computational Intelligence Systems*. **9**, 263–280 (2016,3)
152. Laskar, B., Ashutosh and Majumder, S. Artificial Neural Networks and Gene Expression Programing based age estimation using facial features. *Journal Of King Saud University - Computer And Information Sciences*. **27**, 458–467 (2015)
153. Zhang, Y., Pu, Y., Zhang, H., Su, Y., Zhang, L. and Zhou, J. Using gene expression programming to infer gene regulatory networks from time-series data. *Computational Biology And Chemistry*. **47**, 198–206 (2013)
154. Alghieth, M., Yang, Y. and Chiclana, F. Development of 2D curve-fitting genetic/gene-expression programming technique for efficient time-series financial forecasting. *2015 International Symposium On Innovations In Intelligent SysTems And Applications (INISTA)*. pp. 1–8 (2015,9)
155. Xu, L., Huang, Y., Shen, X. and Liu, Y. Parallelizing Gene Expression Programming Algorithm in Enabling Large-Scale Classification. *Scientific Programming*. **2017** pp. 1–10 (2017)
156. Wang, H., Liu, S., Meng, F. and Li, M. Gene Expression Programming Algorithms for Optimization of Water Distribution Networks. *Procedia Engineering*. **37**, 359–364 (2012), The Second SREE Conference on Engineering Modelling and Simulation (CEMS 2012)
157. Yang, L., Qin, Z., Wang, K. and Deng, S. Hybrid gene expression programming-based sensor data correlation mining. *China Communications*. **14**, 34–49 (2017,1)
158. Wang, C., Zhang, J., Wu, S. and Ma, C. An improved gene expression programming algorithm based on hybrid strategy. *2015 8th International Conference On Biomedical Engineering And Informatics (BMEI)*. pp. 639–643 (2015,10)
159. Diveev, A., Konyrbaev, N. and Sofronova, E. Method of Binary Analytic Programming to Look for Optimal Mathematical Expression. *Procedia Computer Science*. **103**, 597–604 (2017), XII International Symposium Intelligent Systems 2016, INTELS 2016, 5–7 October 2016, Moscow, Russia
160. Mwaura, J., Keedwell, E. and Engelbrecht, A. Evolved Linker Gene Expression Programming: A New Technique for Symbolic Regression. *2013 BRICS Congress On Computational Intelligence And 11th Brazilian Congress On Computational Intelligence*. pp. 67–74 (2013,9)
161. Sermpinis, G., Fountouli, A., Theofilatos, K. and Karathanasopoulos, A. Gene Expression Programming and Trading Strategies. (Springer, Berlin, Heidelberg,2013)
162. Li, X., Zhou, C., Xiao, W. and Nelson, P. Prefix Gene Expression Programming. *Genetic And Evolutionary Computation Conf*. pp. 25–31 (2005)
163. Zhong, J., Ong, Y. and Cai, W. Self-Learning Gene Expression Programming. *IEEE Transactions On Evolutionary Computation*. **20**, 65–80 (2016,2)

164. Park, H., Grings, A., Santos, M. and Soares, A. Parallel hybrid evolutionary computation: Automatic tuning of parameters for parallel gene expression programming. *Applied Mathematics And Computation*. **201**, 108–120 (2008)
165. Deng, S., Yue, D., Yang, L., Fu, X. and Feng, Y. Distributed Function Mining for Gene Expression Programming Based on Fast Reduction. *PLOS ONE*. **11**, e0146698 (2016,1)
166. Mwaura, J. and Keedwell, E. Adaptive Gene Expression Programming Using a Simple Feedback Heuristic. *14th Annu. Conf. Genetic And Evolutionary Computation*. pp. 999–1006 (2012)
167. Zhong, J., Feng, L. and Ong, Y. Gene Expression Programming: A Survey [Review Article]. *IEEE Computational Intelligence Magazine*. **12**, 54–72 (2017,8)
168. Yanai, K. and Iba, H. Estimation of distribution programming based on Bayesian network. *The 2003 Congress On Evolutionary Computation, 2003. CEC '03.*. **3** pp. 1618–1625 (2003)
169. Yanai, K. and Iba, H. Estimation of Distribution Programming: EDA-based Approach to Program Generation. *Towards A New Evolutionary Computation*. pp. 103–122 (2006)
170. Hasegawa, Y. and Iba, H. Latent Variable Model for Estimation of Distribution Algorithm Based on a Probabilistic Context-Free Grammar. *IEEE Transactions On Evolutionary Computation*. **13**, 858–878 (2009,8)
171. Yoshihiko Hasegawa and Hitoshi Iba Estimation of distribution algorithm based on probabilistic grammar with latent annotations. *2007 IEEE Congress On Evolutionary Computation*. pp. 1043–1050 (2007,9)
172. Salustowicz and Schmidhuber Probabilistic incremental program evolution. *Evolutionary Computation*. **5**, 123–41 (1997)
173. Sastry, K. and Goldberg, D. Probabilistic Model Building and Competent Genetic Programming. *Genetic Programming Theory And Practice*. pp. 205–220 (2003)
174. Looks, M., Goertzel, B. and Pennachin, C. Learning computer programs with the bayesian optimization algorithm. *Proceedings Of The 2005 Conference On Genetic And Evolutionary Computation - GECCO '05*. pp. 747 (2005)
175. Hasegawa, Y. and Iba, H. A Bayesian Network Approach to Program Generation. *IEEE Transactions On Evolutionary Computation*. **12**, 750–764 (2008,12)
176. Nyathi, T. and Pillay, N. Automated Design of Genetic Programming Classification Algorithms Using a Genetic Algorithm. (Springer, Cham,2017)
177. Oussaidène, M., Chopard, B., Pictet, O. and Tomassini, M. Parallel genetic programming and its application to trading model induction. *Parallel Computing*. **23**, 1183–1198 (1997)
178. Wolpert, D. and Macready, W. No free lunch theorems for optimization. *IEEE Transactions On Evolutionary Computation*. **1**, 67–82 (1997,4)
179. Montana, D. Strongly Typed Genetic Programming. *Evolutionary Computation*. **3**, 199–230 (1995,6)
180. Muggleton, S. and Raedt, L. Inductive Logic Programming: Theory and methods. *The Journal Of Logic Programming*. **19–20** pp. 629–679 (1994,5)
181. Graupe, D. Principles of artificial neural networks. (World Scientific,2007)
182. Langdon, W., Lam, B., Modat, M., Petke, J. and Harman, M. Genetic improvement of GPU software. *Genetic Programming And Evolvable Machines*. **18**, 5–44 (2017,3)
183. Burke, E., Hyde, M., Kendall, G. and Woodward, J. A Genetic Programming Hyper-Heuristic Approach for Evolving 2-D Strip Packing Heuristics. *IEEE Transactions On Evolutionary Computation*. **14**, 942–958 (2010,12)
184. Branke, J., Nguyen, S., Pickardt, C. and Zhang, M. Automated Design of Production Scheduling Heuristics: A Review. *IEEE Transactions On Evolutionary Computation*. **20**, 110–124 (2016,2)
185. Burke, E., Gendreau, M., Hyde, M., Kendall, G., Ochoa, G., Özcan, E. and Qu, R. Hyper-heuristics: a survey of the state of the art. *Journal Of The Operational Research Society*. **64**, 1695–1724 (2013,12)
186. Miller, J. and Thomson, P. Cartesian Genetic Programming. *Proceedings Of The European Conference On Genetic Programming*. pp. 121–132 (2000)

187. Backus, J., Wegstein, J., Wijngaarden, A., Woodger, M., Nauer, P., Bauer, F., Green, J., Katz, C., McCarthy, J., Perlis, A., Rutishauser, H., Samelson, K. and Vauquois, B. Revised report on the algorithm language ALGOL 60. *Communications Of The ACM*. **6**, 1–17 (1963,1)
188. Bodik, R. and Jobstmann, B. Algorithmic program synthesis: introduction. *International Journal On Software Tools For Technology Transfer*. **15**, 397–411 (2013,10)
189. Czarnecki, K. and Eisenecker, U. Generative and Component-Based Software Engineering. (Springer Berlin Heidelberg,2000)
190. Pillay, N. and Chalmers, C. A hybrid approach to automatic programming for the object-oriented programming paradigm. *Proceedings Of The 2007 Annual Research Conference Of The South African Institute Of Computer Scientists And Information Technologists On IT Research In Developing Countries - SAICSIT '07*. pp. 116–124 (2007)
191. Mahadevan, S. and Connell, J. Automatic programming of behavior-based robots using reinforcement learning. *Artificial Intelligence*. **55**, 311–365 (1992,6)
192. Gendreau, M. and Potvin, J. Handbook of metaheuristics. (Springer,2010)
193. Ravansalar, M., Rajaee, T. and Kisi, O. Wavelet-linear genetic programming: A new approach for modeling monthly streamflow. *Journal Of Hydrology*. **549** pp. 461–475 (2017,6)
194. Song, D., Heywood, M. and Zincir-Heywood, A. A Linear Genetic Programming Approach to Intrusion Detection. (Springer, Berlin, Heidelberg,2003)
195. Ryan, C., O'Neill, M. and Collins, J. Handbook of Grammatical Evolution. (Springer International Publishing,2018)
196. Manazir, A. and Raza, K. Recent Developments in Cartesian Genetic Programming and its Variants. *ACM Computing Surveys*. **51**, 1–29 (2019,1)
197. Chitty, D. Faster GPU-based genetic programming using a two-dimensional stack. *Soft Computing*. **21**, 3859–3878 (2017,7)
198. Mabrouk, E., Ayman, A., Raslan, Y. and Hedar, A. Immune system programming for medical image segmentation. *Journal Of Computational Science*. **31** pp. 111–125 (2019,2)
199. Espejo, P., Ventura, S. and Herrera, F. A Survey on the Application of Genetic Programming to Classification. *IEEE Transactions On Systems, Man, And Cybernetics, Part C (Applications And Reviews)*. **40**, 121–144 (2010,3)
200. Nag, K. and Pal, N. Genetic Programming for Classification and Feature Selection. (Springer, Cham,2019)
201. Brameier, M. and Banzhaf, W. A comparison of linear genetic programming and neural networks in medical data mining. *IEEE Transactions On Evolutionary Computation*. **5**, 17–26 (2001)
202. Bandini, S., Manzoni, S. and Vanneschi, L. Evolving robust cellular automata rules with genetic programming.. (2008,1)
203. Burke, E. and Bykov, Y. The late acceptance Hill-Climbing heuristic. *European Journal Of Operational Research*. **258**, 70–78 (2017,4)
204. Spector, L. and Robinson, A. Genetic Programming and Autoconstructive Evolution with the Push Programming Language. *Genetic Programming And Evolvable Machines*. **3**, 7–40 (2002)
205. Spector, L. Automatic quantum computer programming : a genetic programming approach. (Springer,2007)
206. Helmuth, T. and Spector, L. Evolving a digital multiplier with the pushgp genetic programming system. *Proceeding Of The Fifteenth Annual Conference Companion On Genetic And Evolutionary Computation Conference Companion - GECCO '13 Companion*. pp. 1627 (2013)
207. Spector, L., Martin, B., Harrington, K. and Helmuth, T. Tag-based modules in genetic programming. *Proceedings Of The 13th Annual Conference On Genetic And Evolutionary Computation - GECCO '11*. pp. 1419 (2011)
208. Miller, J. and Smith, S. Redundancy and computational efficiency in Cartesian genetic programming. *IEEE Transactions On Evolutionary Computation*. **10**, 167–174 (2006,4)
209. Vassilev, V. and Miller, J. The Advantages of Landscape Neutrality in Digital Circuit Evolution. *Lecture Notes In Computer Science* . pp. 252–263 (2000)

210. Teller, A. Turing completeness in the language of genetic programming with indexed memory. *Proceedings Of The First IEEE Conference On Evolutionary Computation. IEEE World Congress On Computational Intelligence*. pp. 136–141 (1994)
211. Turner, A. and Miller, J. Recurrent Cartesian Genetic Programming. (Springer, Cham,2014)
212. Alexandros, A., Anthony, B. and Michael, O. Genetic Programming with Memory For Financial Trading. *19th European Conference On The Applications Of Evolutionary Computation*. **9597** pp. 19–34 (2016)
213. Bernardino, H. and Barbosa, H. Grammar-Based Immune Programming for Symbolic Regression. *Lecture Notes In Computer Science*. pp. 274–287 (2009)
214. Vanneschi, L. and Castelli, M. Multilayer Perceptrons. *Encyclopedia Of Bioinformatics And Computational Biology*. pp. 612–620 (2019,1)
215. Teller, A., Teller, A. and Andre, D. Automatically Choosing the Number of Fitness Cases: The Rational Allocation of Trials. *GENETIC PROGRAMMING 1997: PROCEEDINGS OF THE SECOND ANNUAL CONFERENCE*. pp. 321–328 (1997)
216. Curry, R., Lichodzijewski, P. and Heywood, M. Scaling Genetic Programming to Large Datasets Using Hierarchical Dynamic Subset Selection. *IEEE Transactions On Systems, Man And Cybernetics, Part B (Cybernetics)*. **37**, 1065–1073 (2007,8)
217. Hmida, H., Hamida, S., Borgi, A. and Rukoz, M. Scale Genetic Programming for large Data Sets: Case of Higgs Bosons Classification. *Procedia Computer Science*. **126** pp. 302–311 (2018,1)
218. Wolfgang, B., Lee, S. and Leigh, S. Genetic Programming Theory and Practice XVI. (Springer International Publishing,2019)
219. Ragalo, A. and Pillay, N. Evolving dynamic fitness measures for genetic programming. *Expert Systems With Applications*. **109** pp. 162–187 (2018,11)
220. Mousavi Astarabadi, S. and Ebadzadeh, M. Genetic programming performance prediction and its application for symbolic regression problems. *Information Sciences*. **502** pp. 418–433 (2019,10)
221. Liu, S. and Shi, H. Correction to: A Recursive Approach to Long-Term Prediction of Monthly Precipitation Using Genetic Programming. *Water Resources Management*. **33**, 2973–2973 (2019,6)
222. Lin, J., Zhu, L. and Gao, K. A genetic programming hyper-heuristic approach for the multi-skill resource constrained project scheduling problem. *Expert Systems With Applications*. **140** pp. 112915 (2020,2)
223. Gomes, F., Pereira, F., Silva, A. and Silva, M. Multiple response optimization: Analysis of genetic programming for symbolic regression and assessment of desirability functions. *Knowledge-Based Systems*. **179** pp. 21–33 (2019,9)
224. Bruns, R., Dunkel, J. and Offel, N. Learning of complex event processing rules with genetic programming. *Expert Systems With Applications*. **129** pp. 186–199 (2019,9)
225. Hamida, S., Abdelmalek, W. and Abid, F. Applying Dynamic Training-Subset Selection Methods Using Genetic Programming for Forecasting Implied Volatility. *Computational Intelligence*. **32**, 369–390 (2016,8)
226. Hamida, S., Abdelmalek, W. and Abid, F. Applying Dynamic Training-Subset Selection Methods Using Genetic Programming for Forecasting Implied Volatility. *Computational Intelligence*. **32**, 369–390 (2016,8)
227. Estébanez, C., Saez, Y., Recio, G. and Isasi, P. AUTOMATIC DESIGN OF NONCRYPTOGRAPHIC HASH FUNCTIONS USING GENETIC PROGRAMMING. *Computational Intelligence*. **30**, 798–831 (2014,11)
228. Rivero, D., Fernandez-Blanco, E., Fernandez-Lozano, C. and Pazos, A. Population subset selection for the use of a validation dataset for overfitting control in genetic programming. *Journal Of Experimental & Theoretical Artificial Intelligence*. pp. 1–29 (2019,7)
229. Gogna, A. and Tayal, A. Metaheuristics: review and application. *Journal Of Experimental & Theoretical Artificial Intelligence*. **25**, 503–526 (2013,12)
230. Zhuang, B., Shen, C. and Reid, I. Training Compact Neural Networks with Binary Weights and Low Precision Activations. *Journal Of Machibe Learning Research*. **18** pp. 1–30 (2018)

231. Wistuba, M., Rawat, A. and Pedapati, T. A Survey on Neural Architecture Search. *Journal Of Machibe Learning Research*. **20** pp. 1–21 (2019), http://arxiv.org/abs/1905.01392
232. Gomez, F., Schmidhuber, J. and Miikkulainen, R. Accelerated neural evolution through cooperatively coevolved synapses. *Journal Of Machine Learning Research*. **9** pp. 937–965 (2008)
233. Martínez, Y., Naredo, E., Trujillo, L., Legrand, P. and López, U. A comparison of fitness-case sampling methods for genetic programming. *Journal Of Experimental & Theoretical Artificial Intelligence*. **29**, 1203–1224 (2017,11)

Chapter 2
Biclustering Algorithms Based on Metaheuristics: A Review



Adán José-García, Julie Jacques, Vincent Sobanski, and Clarisse Dhaenens

Abstract Biclustering is an unsupervised machine learning technique that simultaneously clusters rows and columns in a data matrix. Biclustering has emerged as an important approach and plays an essential role in various applications such as bioinformatics, text mining, and pattern recognition. However, finding significant biclusters is an NP-hard problem that can be formulated as an optimization problem. Therefore, different metaheuristics have been applied to biclustering problems because of their exploratory capability of solving complex optimization problems in reasonable computation time. Although various surveys on biclustering have been proposed, there is a lack of a comprehensive survey on the biclustering problem using metaheuristics. This chapter will present a survey of metaheuristics approaches to address the biclustering problem. The review focuses on the underlying optimization methods and their main search components: representation, objective function, and variation operators. A specific discussion on single versus multi-objective approaches is presented. Finally, some emerging research directions are presented.

2.1 Introduction

Biclustering is an unsupervised learning task to simultaneously cluster the rows and columns of a data matrix to obtain coherent and homogeneous biclusters (sub-matrices). For instance, in biological data, a subset of rows (or genes) is

A. José-García (✉) · J. Jacques · C. Dhaenens
Univ. Lille, CNRS, Centrale Lille, UMR 9189 CRIStAL, Lille, France
e-mail: adan.josegarcia@univ-lille.fr; julie.jacques@univ-lille.fr; clarisse.dhaenens@univ-lille.fr

V. Sobanski
Univ. Lille, Inserm, CHU Lille, Institut Universitaire de France (IUF), U1286 - INFINITE - Institute for Translational Research in Inflammation, Lille, France
e-mail: vincent.sobanski@univ-lille.fr

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

M. Eddaly et al. (eds.), *Metaheuristics for Machine Learning*,
Computational Intelligence Methods and Applications,
https://doi.org/10.1007/978-981-19-3888-7_2

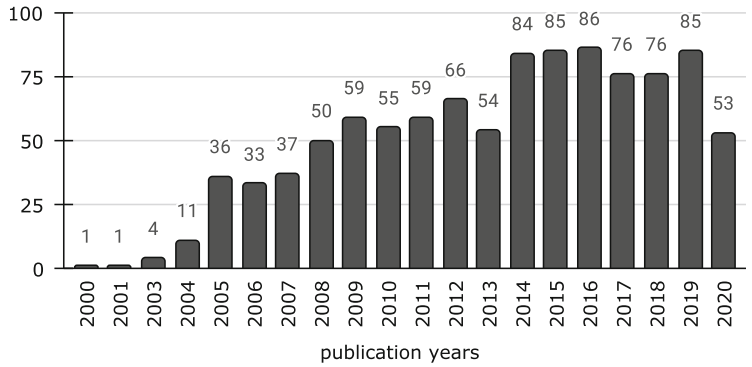


Fig. 2.1 Number of studies on biclustering throughout the last 20 years. This bibliometric data was extracted from the Web of Science (WoS) database using the keyword "Biclustering"

often correlated over a small subset of columns (or conditions). Biclustering[1] is a helpful data mining technique that has gradually become a widely used technique in different applications such as bioinformatics, information retrieval, text mining, dimensionality reduction, recommendation systems, disease identification, and many more [75, 76]. In order to show the volume of studies currently been published in the field of biclustering, Fig. 2.1 presents a year-count of research publications about biclustering in the last 20 years. This plot shows an increasing trend in the number of publications since the first works in 2000.

The biclustering task has been proven to be an NP-complete[2] problem [81], leading to propose optimization methods as heuristics to solve this combinatorial problem. Thus, developing an effective heuristic method and a suitable cost function are critical factors for discovering meaningful biclusters. In this regard, Pontes et al. [76] analyzed a large number of approaches to biclustering and classified them depending on whether they use evaluation metrics within the search heuristic method. In particular, nature-inspired metaheuristics have gradually been applied successfully to biclustering problems because of their excellent exploratory capability [4, 8, 76]. These approaches model the behavior of natural phenomena, which exhibit an ability to learn or adapt to new situations to solve problems in complex and changing environments [50].

Several reviews on biclustering have been conducted, emphasizing different aspects and perspectives of the problem [75, 76, 85]. However, despite the relevance of these review articles, to the best of our knowledge, no review paper on metaheuristics to biclustering has been published. Therefore, we present an up-to-date overview of metaheuristic-based biclustering approaches that have been reported in

[1] Biclustering is also referred to as co-clustering, subspace clustering, bi-dimensional or two-way clustering in the specialized literature.

[2] Nondeterministic Polynomial-time Complete: Any of a class of computational problems for which no efficient solution algorithm has been found.

the last 15 years. This chapter contributes in the following three main aspects: (1) it provides a review of important aspects of the biclustering task when addressed as an optimization problem such as encoding schemes, variation operators, and metric functions; (2) it presents an in-depth review on single-objective and multi-objective metaheuristics applied to biclustering; and (3) it gives some research opportunities and future tendencies in the biclustering field.

The outline of this chapter is as follows. Section 2.2 describes the basic terms and concepts related to biclustering analysis. Section 2.3 outlines the main components involved in metaheuristic-based biclustering algorithms. Section 2.4 reviews single-objective biclustering algorithms in which a unique cost function is optimized. Section 2.5 presents multi-objective biclustering metaheuristics, which optimize distinct cost functions simultaneously. Section 7.6 presents a discussion and future tendencies in biclustering. Finally, the conclusions are given in Sect. 7.7.

2.2 Basic Preliminaries

In this section we define the problem of biclustering, present classical approaches and expose how this problem may be modeled as a combinatorial optimization one.

2.2.1 Biclustering Definition

Given a data matrix $\mathbf{X} \in \mathbb{R}^{N \cdot M}$ where N denotes the number of patterns (rows) and M denotes the number of attributes (columns). Let us define a set of patterns as R and the set of attributes as C; therefore, the matrix $\mathbf{X}_{R,C} = (R, C)$ denotes the full dataset $\mathbf{X}$. Thus, a bicluster is a subset of rows that exhibit similar behavior across a subset of columns, which is denoted as $B_{I,J} = (I, J)$ such that $I \subseteq R$ and $J \subseteq C$.

Depending on the application and nature of data, several *types of biclusters* have been described in the literature [85]. In the following, let $B_{I,J} = (I, J)$ be a bicluster in which b_{ij} refers to the value of the i-th pattern under the j-th attribute. We can identify four major types of biclusters:

- Biclusters with constant values on all rows and columns: $b_{ij} = \pi$
- Biclusters with constant values on rows or columns.
  - Constant rows: $b_{ij} = \pi + \alpha_i$ or $b_{ij} = \pi \times \alpha_i$
  - Constant columns: $b_{ij} = \pi + \beta_j$ or $b_{ij} = \pi \times \beta_j$

- Biclusters with coherent values on both rows and columns [84].
  - Shifting model (additive): $b_{ij} = \pi + \alpha_i + \alpha_j$
  - Scaling model (multiplicative): $b_{ij} = \pi \times \beta_i \times \beta_j$
- Bicluster with coherent evolutions: A subset of patterns (rows) is up-regulated or down-regulated coherently across subsets of attributes (columns) irrespective of their actual values; that is, in the same directions but with varying magnitude. In these scenarios, coherent-evolution of biclusters are difficult to model using a mathematical equation.

In the previous bicluster definitions, π represents any constant value for B, α_i $(1 \leqslant i \leqslant |I|)$ and α_j $(1 \leqslant j \leqslant |J|)$ refers to the constant values used in the additive models for each pattern and attribute; and β_i $(1 \leqslant i \leqslant |I|)$ and β_j $(1 \leqslant j \leqslant |J|)$ corresponds to the constant values used in the multiplicative models.

In most real-world problems, the cluster analysis involves the extraction of several biclusters, where the relations between the biclusters are defined by two criteria, *exclusivity* and *exhaustivity*. The exclusivity criterion indicates that an element must belong to a single bicluster, whereas the exhaustivity criterion specifies that every element must be part of one or more biclusters. Commonly, exclusivity refers to the covering of the input matrix, while exhaustivity is associated with overlapping among biclusters.

Considering these two criteria, several *bicluster structures* can be obtained [85]. Figure 2.2 illustrates some examples of bicluster structures, for instance, biclusters with: exclusive rows or exclusive columns (no overlapping and partial covering), exclusive rows and exhaustive columns (columns overlapping and partial covering), exhaustive rows and exhaustive columns (full overlapping and full covering).

Consequently, the selection of biclusters types and structures to be discovered will depend on both the problem being solved and the type of data involved. Therefore, the biclustering algorithm should be able to identify the desired biclusters.

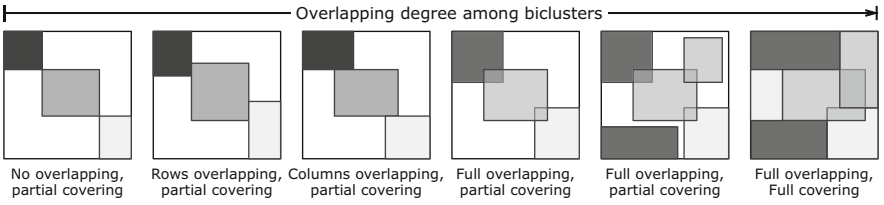


Fig. 2.2 Different bicluster structures based on exclusivity (matrix covering) and exhaustivity criteria (bicluster overlapping)

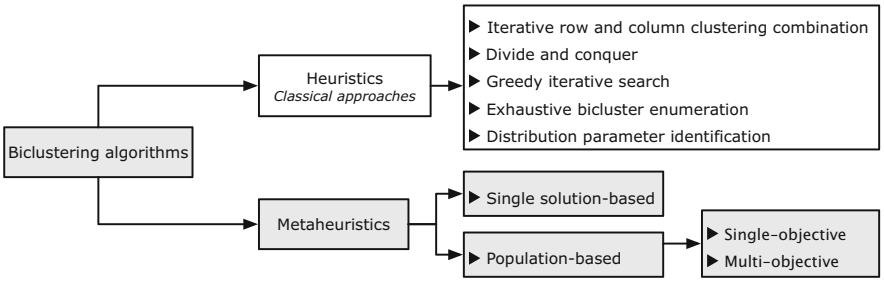


Fig. 2.3 Biclustering algorithms taxonomy

2.2.2 Classical Biclustering Approaches

Biclustering has gained much interest since the seminal work of Cheng and Church on biclustering to analyze gene expression data [81]. There are several biclustering algorithms in the literature, in particular, to deal with biological data, including detection of gene expression [81, 82], protein interaction [83], and microarray data analysis [47].

To study such enriched literature, we categorized the biclustering algorithms into two main classes: *heuristic-based* and *metaheuristic-based* approaches. This categorization builds upon previous taxonomies proposed by Madeira and Oliveira [85] and by Seridi et al. [8]. Figure 2.3 illustrates the employed taxonomy of biclustering algorithms, where the metaheuristic-based approaches are the primary focus of this review work and is detailed in the following sections. Regarding the heuristic-based algorithms, Madeira and Oliveira proposed to divide these approaches into five classes [85]:

2.2.2.1 Iterative Row and Column Clustering Combination

To identify biclusters, classical clustering methods are applied to the rows and columns of the data matrix separately; then, the resulting clusters are combined using an iterative procedure to obtain biclusters of good quality. Among the algorithms adopting this approach are the Coupled Two-Way Clustering (CTWC) [38], the Interrelated Two-Way Clustering (ITWC) [39], and the Double Conjugated Clustering (DCC) algorithm [85].

2.2.2.2 Divide-and-Conquer

These approaches split the problem into several smaller sub-problems of the same type, solving each one recursively. Then, the solutions of the sub-problems are combined to obtain a single solution to the original problem. Divide-and-conquer

biclustering algorithms start with the entire data matrix as the initial bicluster. Then, this bicluster is split into several biclusters iteratively until satisfying a certain termination criterion. These approaches are quite fast, but good biclusters may be split before they can be identified. The main algorithms in this class are the Direct Clustering Algorithm (DCA) [36] and the Binary Inclusion-Maximal Biclustering Algorithm (Bimax) [40].

2.2.2.3 Greedy Iterative Search

These methods create biclusters by adding or removing rows and columns using a quality criterion that maximizes the local gain. Therefore, although these approaches may make wrong decisions and miss good biclusters, they usually tend to be very fast. The most representative work in this category is the Cheng and Church's Algorithm (CCA) [81]. Other approaches are the Order-Preserving Submatrix (OPSM) [42], the QUalitative BIClustering (QUBIC) [41], and the Large Average Submatrices (LAS) algorithm [43].

2.2.2.4 Exhaustive Bicluster Enumeration

These algorithms consider that the best submatrices can only be identified by generating all the possible row and column combinations of the data matrix. Therefore, an exhaustive enumeration of all possible biclusters in the matrix is performed. These approaches find the best biclusters (if they exist), but they are suitable only for very small datasets. The main drawback is their high complexity, requiring restrictions on the size of the biclusters when performing the exhaustive enumeration. The main algorithms in this category are the Statistical-Algorithmic Method for Bicluster Analysis (SAMBA) [44], Bit-Pattern Biclustering Algorithm (BiBit) [45], and the Differentially Expressed Biclusters (DeBi) [46].

2.2.2.5 Distribution Parameter Identification

These approaches assume a given statistical model and try to identify the distribution parameters used to generate the data by optimizing a particular quality criterion. Some of the algorithms adopting this approach are Spectral Biclustering (BS) [47], the Bayesian BiClustering (BBC) [48], and the Factor Analysis for Bicluster Acquisition (FABIA) algorithm [49].

2.2.3 *Biclustering as an Optimization Problem*

As mentioned previously, a bicluster $B(I, J)$ associated with a data matrix $\mathbf{X}(R, C)$ is a submatrix such that $I \subseteq R$ and $J \subseteq C$, where R is a set of patterns (rows) and C is a set of attributes (columns). The biclustering problem aims to extract biclusters of a maximal size that satisfy a coherence constraint. This task of extracting bicluster from a data matrix can be seen as a combinatorial optimization problem [78, 79].

Designing a bicluster is equivalent to jointly selecting a subset of rows and a subset of columns from an input data matrix $\mathbf{X} \in \mathbb{R}^{N \cdot M}$, where N denotes the number of rows and M denotes the number of columns. Let us assume that there is no restriction on the number of rows and columns and no constraints about the nature of the biclusters (i.e., bicluster type and structure). Then, by nature, the biclustering task is a combinatorial problem with a search space size of $\mathcal{O}\left(2^N * 2^M\right) = \mathcal{O}\left(2^{N+M}\right)$.

However, identifying interesting biclusters is a complex task that requires defining some quality criteria to be optimized. Such criteria can measure the similarity within a bicluster, coherence, and dissimilarity (when a set of biclusters is searched). These biclustering quality criteria can be used as an objective function in a combinatorial optimization context, either alone or multiple in a multi-objective perspective. In the second scenario, several complementary quality criteria are optimized simultaneously.

In the general case, Cheng and Church showed that the problem of finding significant biclusters is NP-hard [81], giving rise to a large number of heuristic and metaheuristic approaches. These approaches do not guarantee the optimality of their solutions; however, their exploratory capabilities allow them to find suitable solutions in reasonable computation time. The following section is devoted to the review of metaheuristics designed to deal with biclustering problems.

2.3 Main Components of Metaheuristics for Biclustering

This section presents the main components involved when designing a metaheuristic to address the biclustering problem. First, a metaheuristic requires a suitable representation of biclusters, which is directly related to the objective function (i.e., bicluster quality measure) to be optimized. In general, the main steps in population-based metaheuristics are initializing the population, evaluating individuals, applying variation operators, and selecting the best biclustering solutions. This iterative process is repeated until a particular termination criterion is satisfied. Figure 2.4 illustrates these main components, where the main types of bicluster encodings, the variation operators to generate new biclusters, and different quality measures (evaluation step) are described below in this section. The other specific components, including the initialization procedures and selection strategies, are presented in

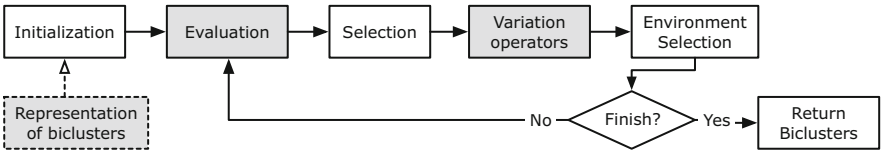


Fig. 2.4 Main components of biclustering algorithms based on metaheuristics

detail when describing the single- or multi-objective biclustering algorithms in Sects. 2.4 and 2.5.

2.3.1 Bicluster Encoding

Metaheuristics require a representation or an encoding of potential solutions to the optimization problem. The encoding scheme is directly related to the objective function to be optimized and the recombination operators to generate new solutions. Therefore, the encoding schemes play a relevant role in the efficiency and effectiveness of any metaheuristic and constitute an essential step in its design.

In the literature, different encoding schemes are commonly used in metaheuristics to represent biclusters: *binary encoding* and *integer encoding*. These bicluster encodings are exemplified in Fig. 2.5 by considering a 4×3 didactic data matrix and are separately described below.

2.3.1.1 Binary Bicluster Encoding (BBE)

A bicluster $B(I, J)$ is represented as a binary vector of fixed-length size $(N + M)$: $x = \{p_1, \ldots, p_N, s_1, \ldots, s_M\}$, where the first N positions are related to the number

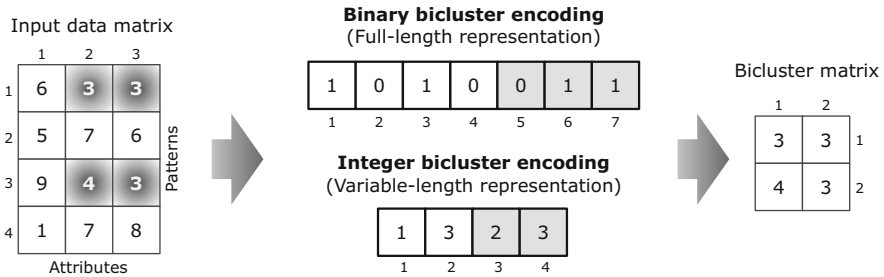


Fig. 2.5 Exemplification of two types of bicluster encodings in metaheuristic-based biclustering algorithms. The bicluster elements are shaded in the input data matrix, whereas the pattern and attribute indices are represented with white and gray boxes, respectively

of patterns (rows) and the remaining M positions to the number of attributes (columns) from the data matrix $\mathbf{X}$. If the i-th pattern or the j-th attribute in $\mathbf{X}$ belongs to the bicluster $B(I, J)$, then $p_i = 1$ or $s_j = 1$ for $1 \leqslant i \leqslant N$ and $1 \leqslant j \leqslant M$; otherwise, $p_i = 0$ or $s_j = 0$. Figure 2.5 illustrates the binary encoding for $N = 4$ and $M = 3$ when $x = \{1, 0, 1, 0, 0, 1, 1\}$.

2.3.1.2 Integer Bicluster Encoding (IBE)

A bicluster $B(I, J)$ is represented as an integer vector of variable-length size $N_p + M_s$: $x = \{p_1, \ldots, p_{N_p}, s_1, \ldots, s_{M_s}\}$, where the first N_p positions are ordered pattern indices, whereas the last M_s positions correspond to ordered attribute indices from the data matrix $\mathbf{X}$. Each i-th pattern position takes an integer value in the set $\{1, \ldots, N\}$, and each j-th attribute position takes a value in $\{1, \ldots, M\}$, where N and M are the number of patterns and attributes in the data matrix, respectively [53]. Figure 2.5 illustrates the integer encoding for $N_p = 2$ and $M_s = 2$ when $x = \{1, 3, 2, 3\}$.

Overall, the binary bicluster encoding is a practical representation, but it requires exploring all patterns and attributes of each bicluster. On the contrary, the integer encoding requires less computation time and memory space as it depends on the number of ordered patterns and attributes for a particular bicluster. Therefore, the integer bicluster encoding is more efficient in terms of time and space; however, it is more impractical when dealing with variable-length solutions in population-based metaheuristics.

2.3.2 Variation Operators

In metaheuristic algorithms, variation operators play an important role in providing a good compromise between exploration and diversification to search solutions of high quality. In the following, we describe some crossover and mutation operators that are commonly used in biclustering algorithms.

2.3.3 Crossover Operators

The crossover is a basic component in metaheuristic biclustering methods, and in general, in evolutionary genetic algorithms. It tries to combine pairs of individuals to produce new offsprings that inherit parent's features.

Let us consider two parent individuals $P_1 = \{r_1, \ldots, r_n, c_1, \ldots, c_m\}$ and $P_2 = \{r'_1, \ldots, r'_l, c'_1, \ldots, c'_k\}$, where $r_n \leqslant r'_l$.

2.3.3.1 Single Crossover Operator [3, 8]

The crossover is performed in each part of the individual (rows part and columns part). The crossover point λ_1 in P_1 is generated randomly in the range $r_1 < \lambda_i \leqslant r_n$. The random point in P_2, $\lambda_2 = r'_j$ where $r'_j \geqslant \lambda_1$ and $r'_{j-1} \leqslant \lambda_1$. The crossover in the columns part is performed similarly to the rows part.

2.3.3.2 Bicluster Crossover Operator [27]

The crossover is based on the following four steps: (1) creation of the merge bicluster B_{merge} from two parent biclusters (individuals); (2) discretization of the merge bicluster, B_{discrete}; (3) construction of the variation matrix, $M_{\mathbf{var}}$; (4) extraction of child biclusters.

2.3.4 Mutation Operators

Mutation operators aim to modify an individual, either randomly or using a specific strategy.

2.3.4.1 Random Mutation Operator [27]

Let us consider an individual in the population $B = \{r_1, \ldots, r_n, c_1, \ldots, c_m\}$, two mutation points r_i and c_j are generated corresponding to the rows and columns parts, respectively, such that $r_1 < r_i \leqslant r_n$ and $c_1 < c_j \leqslant c_m$. Then, a random row r'_i and column c'_j values are generated to replace the chosen positions r_i and c_j, respectively.

2.3.4.2 CC-Based Mutation Operator [3, 8]

This mutation strategy is based on the CC algorithm [81], which aims to generate k biclusters. Thus, given an individual, its irrelevant rows or columns having mean squared residue (MSR) values above (or below) a certain threshold are eliminated (or added) using the following conditions:[3] (1) multiple node deletion, (2) single node deletion, and (3) multiple node addition.

[3] A "node" refers to a row or a column.

2.3.5 Objective Functions

Because of the complexity of the biclustering problem, several evaluation functions have been proposed to measure the quality of biclusters. This section presents some of the most well-known evaluation measures that have been used as objective functions in different metaheuristics.

For convenience, consider the following common notation for interpretation of the different objective functions. Given a bicluster $B(I, J)$ with $|I|$ patterns (rows), $|J|$ attributes (columns), the average b_{iJ} of the i-th row, the average b_{Ij} of j-th column, and the average value b_{IJ} of the bicluster $B(I, J)$ are represented respectively as follows:

$$b_{iJ} = \frac{1}{|J|}\sum_{j\in J} b_{ij},\ \ b_{Ij} = \frac{1}{|I|}\sum_{i\in I} b_{ij},\ \ b_{IJ} = \frac{1}{|I|\cdot|J|}\sum_{i\in I, j\in J} b_{ij}\,. \tag{2.1}$$

Additionally, the following notation is used, an abbreviation of the bicluster evaluation measure followed by an arrow to denote if the function is maximized (↑) or minimized (↓), indicating the bicluster solution's quality.

2.3.5.1 Bicluster Size (BSize)

This function represents the size of a bicluster, where α is a constant representing a preference towards the maximization of the number of rows or columns:

$$\text{BSize}(B)_{\uparrow} = \alpha\frac{|I|}{|I'|} + (1-\alpha)\,\frac{|J|}{|J'|}\,. \tag{2.2}$$

Sometimes this measure is referred as the *Volume* of a bicluster, i.e., as the number of elements b_{ij} in B, $|I|\cdot|J|$.

2.3.5.2 Bicluster Variance (VAR)

Hartigan [36] proposed the variance measure to identify biclusters with constant values:

$$\text{VAR}(B)_{\uparrow} = \sum_{i=1}^{|I|}\sum_{j=1}^{|J|}\left(b_{ij} - b_{IJ}\right)^2\,. \tag{2.3}$$

Existing biclustering approaches deal with biclustering variance in different ways. For instance, Pontes et al. [16] used the average variance to avoid obtaining trivial biclusters. In practice, the row variance (rVAR) is used to avoid trivial or constant-value biclusters (i.e., biclusters with significant rows variances). The rVAR

measure is defined as:

$$\text{rVAR}(B)_{\uparrow} = \frac{1}{|I| \cdot |J|} \sum_{i=1}^{|I|} \sum_{j=1}^{|J|} \left(b_{ij} - b_{iJ}\right)^2 . \tag{2.4}$$

2.3.5.3 Mean Squared Residence (MSR)

Cheng and Chung [81] proposed the MSR to measure the correlation (or coherence) of a bicluster. It is defined as follows:

$$\text{MSR}(B)_{\downarrow} = \frac{1}{|I| \cdot |J|} \sum_{i=1}^{|I|} \sum_{j=1}^{|J|} \left(b_{ij} - b_{iJ} - b_{Ij} + b_{IJ}\right)^2 . \tag{2.5}$$

The lower the MSR, the stronger the coherence exhibited by the bicluster, and the better its quality. If a bicluster has an MSR value lower than a given threshold δ, then it is called a δ-bicluster.

2.3.5.4 Scaling Mean Squared Residence (SMSR)

Mukhopadhyay et al. [37] developed this measure to recognize scaling patterns in biclusters. The SMSR measure is defined as:

$$\text{SMSR}(B)_{\downarrow} = \frac{1}{|I| \cdot |J|} \sum_{i=1}^{|I|} \sum_{j=1}^{|J|} \frac{\left(b_{ij} - b_{iJ} - b_{Ij} + b_{IJ}\right)^2}{b_{iJ}^2 \cdot b_{Ij}^2} . \tag{2.6}$$

2.3.5.5 Average Correlation Function (ACF)

Nepomuceno et al. [5] proposed the ACF to evaluate the correlation between patterns in a bicluster. It is defined as:

$$\text{ACF}(B)_{\uparrow} = \frac{2}{|I| \left(|I| - 1\right)} \sum_{i=1}^{|I|} \sum_{j=i+1}^{|I|} \left| \frac{cov\left(p_i, p_j\right)}{\sigma_{p_i} \sigma_{p_j}} \right| , \tag{2.7}$$

where $cov\left(p_i, p_j\right) = {}^{1}/_{|J|} \sum_{k=1}^{|J|} \left(b_{ik} - b_{iJ}\right) \left(b_{jk} - b_{jJ}\right)$ represents the covariance of the rows corresponding to patterns p_i and p_j, and σ_{p_i} (respectively, σ_{p_j}) represents the standard deviations of the rows corresponding to patterns p_i and p_j. The ACF measure generates values in the range $[-1, 1]$, where values close to the unity represents that the patterns in B are highly correlated.

2.3.5.6 Average Correlation Value (ACV)

Teng and Chan [35] proposed this measure to evaluate the correlation homogeneity of a bicluster. The ACV is defined as:

$$\mathrm{ACV}(B)_{\uparrow} = \max \left\{ \frac{1}{|I|\,(|I|-1)} \sum_{i,j \in I, i \neq j} r_{ij}, \frac{1}{|J|\,(|J|-1)} \sum_{i,j \in J, i \neq j} r'_{ij} \right\}, \tag{2.8}$$

where r_{ij} or r'_{ij} is a Pearson correlation coefficient between the i-th row and j-th column. ACV generates values in the interval [0, 1], where a value closer to 1 indicates that the rows or columns in the bicluster are highly co-expressed, whereas a low ACV values means the opposite.

2.3.5.7 Virtual Error (VE)

Divina et al. [33] proposed the VE function to identify shifting or scaling patterns in biclusters. It is defined as follows:

$$\mathrm{VE}(B)_{\downarrow} = \frac{1}{|I| \cdot |J|} \sum_{i=1}^{|I|} \sum_{j=i}^{|J|} \left| \hat{b}_{ij} - \hat{\rho}_i \right|, \tag{2.9}$$

where $\rho_i = {}^1/_j \sum_{j=1}^{|J|} b_{ij}$ represents a *virtual pattern* from B, and $\hat{b}_{ij}$ (respectively, $\hat{\rho}_i$) is the *standardized value* of the element b_{ij}. The standardized values of a vector $V = \{v_1, \ldots, v_n\}$, denoted as $\hat{V}$, is the set $\hat{V} = \left\{\hat{v}_1, \ldots, \hat{v}_n\right\}$ with $\hat{v}_k = (v_k - \mu_V) \big/ \sigma_V$ for $1 \leqslant k \leqslant n$, where μ_V and σ_V are the mean and standard deviation of V. A small value of VE represents a high similarity among the patterns in the biclusters.

2.3.5.8 Coefficient of Variation Function (CVF)

Maatouk et al. [12] presented the CVF to characterize the variability of a bicluster. The function is defined as follows:

$$CVF(B)_{\uparrow} = \frac{\sigma_B}{b_{IJ}}, \tag{2.10}$$

where σ_B represents the standard deviation of the bicluster and b_{IJ} denotes the average of all the values in the bicluster B. A high value of CVF indicates that the bicluster presents a high level of dispersion.

2.4 Single-Objective Biclustering

This section presents a review of single-objective metaheuristics for the biclustering problem. These approaches are mostly population-based metaheuristics that iteratively attempt to improve a population of biclustering solutions. First, the population is usually initialized randomly. Then, a new population of potential solutions is generated, which could be integrated into the current one by using some selection criteria. Finally, the search process stops when a given condition is satisfied (see Fig. 2.4).

Table 2.1 summarizes relevant details of 23 single-objective biclustering algorithms based on metaheuristics. These biclustering algorithms, including hybrid

Table 2.1 List of single-objective biclustering algorithms based on metaheuristics. "Metaheuristic" indicates the type of nature-inspired metaheuristic, which can be simulated annealing (SA), genetic algorithm (GA), particle swarm optimization (PSO), scatter search (SS), and cuckoo search (CS). "Objective Function" refers to the bicluster evaluation metric as defined in Sect. 2.3.5 (when several are indicated a linear combination of them is used); and "Encoding" indicates the type of bicluster encoding binary (BBE) or integer (IBE) as described in Sect. 2.3.1

Year	Algorithm	Metaheuristic	Objective function	Encoding	Ref.
2004	`HEA`	GA	MSR	BBE	[25]
2006	`SAB`	SA	MSR	Other	[23]
2006	`SEBI`	GA	MSR, BSize, Var	BBE	[19]
2009	`BiHEA`	GA	MSR	BBE	[18]
2009	`SS&GA`	Hybrid: GA + SS	MSR	BBE	[7]
2010	`HEAB`	Hybrid: GA + SS	ACF	BBE	[5]
2011	`SSB`	SS	ACF	BBE	[4]
2011	`BPSO`	PSO	ACV	BBE	[1]
2012	`CBEB`	GA	MSR	BBE	[17]
2012	`EvoBic`	GA	BSize, MSR, ACF	IBE	[10]
2012	`PSO-SA-BIC`	Hybrid: PSO + SA	ACV	BBE	[2]
2013	`Evo-Bexpa`	GA	VE, Vol, Overlap, Var	BBE	[16]
2014	`EBACross`	GA	BSize, MSR, ACF, CVF	BBE	[12]
2014	`TriGen`	GA	MSR_{time}, LSL_{time}	IBE	[28]
2015	`COCSB`	CS	MSR	BBE	[14]
2015	`SSB-Bio`	SS	BSize, ACF, ACF_{bio}	BBE	[6]
2015	`BISS`	SS	ACF	BBE	[9]
2018	`BISS-go`	SS	ACF_{go}	BBE	[57]
2018	`GACSB`	Hybrid: GA + CS	MRS, VE, ACV	BBE	[13]
2018	`EBA`	GA	BSize, MSR, ACF, CVF	BBE	[27]
2019	`BP-EBA`	GA	MSR, SMSR, BSize	BBE	[29]
2021	`HPSO-TriC`	Hybrid: PSO + SA	ACF_{time}	BBE	[26]
2021	`ELSA`	GA	ACF_{stat}, ACF_{bio}	BBE	[15]

The approaches that list more than one objective function indicate that the algorithms combine the information from these metrics into a single objective function

biclustering metaheuristic approaches, are described in more detail below in this section.

2.4.1 Simulated Annealing

Simulated annealing (SA) is a probabilistic method proposed by Kirkpatrick et al. [51] to find the global minimum of a cost function. SA emulates the physical process whereby a melted solid material (initial state) is gradually cooled until the minimum energy state is reached, that is when the material structure is "frozen". SA is a single-solution-based metaheuristic that improves a single point solution, evaluated by a single criterion function, and could be viewed as search trajectories through the search space [50].

Simulated annealing has been used to address the biclustering problem. Brayan et al. [23] proposed an SA-based biclustering approach called **SAB**. In SAB, each solution's fitness function is computed using the MSR criterion, and the algorithm is run k times to obtain k biclusters. In order to avoid overlap among biclusters, in the SAB algorithm, the discovered biclusters are masked in the original data. This strategy is similar to the Cheng and Church (CC) [81], where the original values are replaced with random ones to prevent them from being part of any further bicluster.

2.4.2 Genetic Algorithms

Genetic algorithm (GA), developed by Holland in the early 1970s [52], emulates the principle of evolution by natural selection stated by Charles Darwin. Several biclustering approaches have been proposed based on GAs, which sometimes are referred to as evolutionary algorithms (EAs). Next, we summarize these approaches.

Bleuler et al. [25] proposed the first evolutionary biclustering algorithm in 2004, namely, **HEA**. This algorithm uses a binary encoding of fixed length, and the MSR criterion is used as the fitness function. In addition, a bit mutation and uniform crossover are used as variation operators to generate new biclustering solutions during the evolutionary process. In HEA, a diversity maintenance strategy is considered, which decreases the overlapping level among bicluster, and the CC algorithm [81] is also applied as a local search method to increase the size of the biclusters. In the end, the entire population of individuals is returned as the set of resulting biclusters. Gallo et al. [18] proposed the **BiHEA** algorithm, similar to Bleuler's approach as both perform a local search based on the CC algorithm and use the same objective function. However, the approaches differ in the crossover operators (BiHEA uses a two-point crossover). Additionally, BiHEA considers an external archive to keep the best-generated biclusters through the evolutionary process.

Divina and Aguilar-Ruiz [19] presented a sequential evolutionary biclustering algorithm (**SEBI**). The term sequential refers to that the evolutionary algorithm generates only one bicluster per run; thus, in order to generate several biclusters, SEBI needs to be invoked iteratively. Furthermore, a general matrix of weights is considered to control the overlapping among biclusters. In SEBI, three crossover and mutation strategies are used with equal probability of reproduction: one-point, two-points and uniform crossovers; and mutations that add a row or a column to the bicluster, or the standard mutation. The evaluation of individuals is carried out by an objective function that involves three different criteria: MSR, bicluster size, and row variance.

Huang et al. [17] proposed a biclustering approach based on genetic algorithms and hierarchical clustering, called **CBEB**. First, the rows of the data matrix (conditions) are separated into a number of condition subsets (subspaces). Next, the genetic algorithm is applied to each subspace in parallel. Then, an expanding-merging strategy is employed to combine the subspaces results into output biclusters. In CBEB, the MSR metric is used as an objective function, whereas a simple crossover and a binary mutation are used to reproduce new solutions. Although this approach outperforms several traditional biclustering algorithms, it requires a longer computation time than the other methods. This disadvantage of CBEB is mainly due to its utilization process and separation method for creating and evaluating several subspaces.

An evolutionary biclustering algorithm (**EvoBic**) with a variable-length representation was proposed by Ayadi et al. in [10]. This integer encoding represents the individuals as a string composed of ordered genes and conditions indices, reducing time and the memory space [53]. EvoBic algorithm considers three different biclustering metrics (BSize, MSR, ACF), a single-point crossover, and a standard mutation to generate new biclusters.

Another evolutionary biclustering algorithm, called **Evo-Bexpa**, was presented by Pontes et al. [16]. This algorithm allows identifying types of biclusters in terms of different objectives. These objectives have been put together by considering a single aggregative objective function. Evo-Bexpa bases the bicluster evaluation on the use of expression patterns, recognizing both shifting and scaling patterns by considering the VE, Vol, Overlap, and Var quality metrics.

Maatouk et al. [12] proposed an evolutionary biclustering algorithm named **EBACross**. First, the initialization of the initial population is based on the CC algorithm [81]. Then, the evaluation and selection of the individuals are based on four complementary biclusters metrics: bicluster size (BSize), MSR metric, average correlation (ACF), and the coefficient of variation function (CVF). In EBACross, a binary encoding of fixed length, a crossover method based on the standard deviation of the biclusters, and a mutation strategy based on the biclusters' coherence are considered. Later, the same authors proposed a generic evolutionary biclustering algorithm (**EBA**) [27]. In this work, the authors analyzed the EBA's performance by varying its genetic components. In the study, they considered three different biclustering metrics (BSize, MSR, and ACF), two selection operators (parallel and aggregation methods), two crossover methods (random-order and biclustering

methods), and two mutation operators (random and biclustering strategies). Hence, several versions of the EBA algorithm were introduced. In terms of statistical and biological significance, the clustering performance showed that the EBA configuration based on a selection with aggregation and biclustering crossover and mutation operators performed better for several microarray data. Recently, also Maatouk et al. [15] proposed the **ELSA** algorithm, an evolutionary algorithm based on a local search method that integrates biological information in the search process. The authors stated that statistical criteria are reflected by the size of the biclusters and the correlation between their genes, while the biological criterion is based on their biological relevance and functional enrichment degree. Thus, the ELSA algorithm evaluates the statistical and biological quality of the biclusters separately by using two objective functions based on the average correlation metric (ECF). Furthermore, in order to preserve the best biclusters over the different generations, an archiving strategy is used in ELSA.

Huang et al. [29] proposed a bi-phase evolutionary biclustering algorithm (**BP-EBA**). The first phase is dedicated to the evolution of rows and columns, and the other is for the identification of biclusters. The interaction of the two phases guides the algorithm toward feasible search directions and accelerates its convergence. BP-EBA uses a binary encoding, while the population is initialized using a hierarchical clustering strategy to discover bicluster seeds. The following biclustering metrics were employed to evaluate the individuals in the population: MSR, SMSR, BSize. Finally, the performance of this approach was compared with other biclustering algorithms using microarray datasets.

Gutierrez-Aviles [28] proposed an evolutionary algorithm, **TriGen**, to find biclusters in temporal gene expression data (known as *triclusters*). Thus, the aim is to find triclusters of gene expression that simultaneously take into account the experimental conditions and time points. Here, an individual is composed of three structures: a sequence of genes, a sequence of conditions, and a sequence of time points. Furthermore, the authors proposed specific genetic operators to generate new triclusters. Two different metrics were taken into account to evaluate the individual: MSR_{time} (modification of the MSR metric) and LSL_{time} (least-squares approximation for the points in a 3D-space representing a tricluster). As a result, TriGen could extract groups of genes with similar patterns in subsets of conditions and times, and these groups have shown to be related in terms of their functional annotations extracted from the Gene Ontology.

2.4.3 Scatter Search

Scatter search (SS) is a population-based evolutionary metaheuristic that emphasizes systematic processes against random procedures [56]. The optimization process consists of evolving a set called *reference*, which iteratively is updated by using *combination* and *improvement* methods that exploit context knowledge. In contrast to other evolutionary approaches, SS is founded on the premise that

systematic designs and methods for creating new solutions afford significant benefits beyond those derived randomly.

Nepomuceno and his collaborators have developed a series of SS-based biclustering algorithms for gene expression data [4, 6, 9, 57]. In [4], the authors presented the **SSB** algorithm to find shifting and scaling patterns biclusters, which are interesting and relevant patterns from a biological point of view. For this purpose, the average correlation function (ACF) was modified and used as the objective function. The SSB algorithm uses a binary encoding to represent biclustering solutions and includes a local search method to improve biclusters with positively correlated genes. The same authors proposed another SS-based biclustering algorithm that integrates prior biological knowledge [6]. This algorithm (herein referred to as **SSB-bio**) requires as input, in addition to the gene expression data matrix, an annotation file that relates each gene to a set of terms from the repository *Gene Ontology* (GO). Thus, two biological measures, *FracGO* and *SimNTO*, were proposed and integrated as part of the objective function to be optimized. This fitness function is a weighted-sum function composed of three factors: the bicluster size (BSize), the bicluster correlation (ACF metric), and the bicluster biological relevance based on FracGO or SimNTO measures (ACF_{bio} metric).

Nepomuceno et al. [9] proposed the **BISS** algorithm to find biclusters with both shifting and scaling patterns and negatively correlated patterns. This algorithm, similar to SSB-Bio, is based on a priori biological information from the GO repository, particularly the categories: biological process, cellular components, and molecular function. In BISS, a fitness function involving the AFC and BSize metrics was used to evaluate the quality of the biclusters. Furthermore, in another recent study, Nepomuceno et al. [57] used the BISS algorithm for biclustering of high-dimensional expression data. This algorithm, referred here as **BISS-go**, also considers the biological knowledge available in the GO repository to find biclusters composed of groups of genes functionally coherent. This task is achieved by defining two GO semantic similarity measures integrated into the fitness function optimized by the BISS-go algorithm. The reported results showed that the inclusion of biological information improves the performance of the biclustering process.

2.4.4 Cuckoo Search

Cuckoo search (CS), developed by Xin-She Yang and Suash Deb [55], is a nature-inspired metaheuristic based on the brood parasitism of some cuckoo species. In this algorithm, the exploration of the search space is enhanced using Lévy flights (Lévy distribution) instead of using simple isotropic random walks.

Yin Lu et al. [14] introduced a CS-based biclustering algorithm for gene expression data named **COCSB**. The authors incorporated different strategies in COCSB to improve its diversity performance and convergence rate, such as the searching- and abandoning-nest operations. In CBEB, the MSR metric is used as the objective function, whereas a binary bicluster representation of the solutions is considered.

This approach was compared to several classical biclustering algorithms, including CC and SEBI, obtaining a good biological significance and time computation performance.

2.4.5 Particle Swarm Optimization

Particle swarm optimization (PSO), introduced by Kennedy and Eberhart [54], is a population-based search method in which the individuals (referred to as particles) are grouped into a swarm. The particles explore the search space by adjusting their trajectories iteratively according to self-experience and neighboring particles [56].

Rathipriya et al. [1] proposed a PSO-based biclustering algorithm called **BPSO**. BPSO was applied web data to find biclustering that contains relationships between web users and webpages, useful for E-Commerce applications like web advertising and marketing. The individuals were encoded using the traditional binary bicluster representation, and the average correlation value (ACV) metric was used as the fitness function. This PSO-based algorithm outperformed two traditional biclustering algorithms based on greedy search. Furthermore, the identified biclusters by BPSO covered a more considerable percentage of users and webpages, capturing the global browsing patterns from web usage data.

2.4.6 Hybrid Metaheuristic Approaches

It is well-known that nature-inspired metaheuristics are effective strategies for solving optimization problems. However, sometimes it is difficult to choose a metaheuristic for a particular instance problem. In these scenarios, hybrid approaches provide flexible tools that can help to cope with this problem. In line with this, different hybrid nature-inspired metaheuristics for the biclustering problem are described below.

Nepomuceno et al. [5, 7] proposed a couple of hybrid metaheuristics for biclustering based on *scatter search* (SS) and *genetic algorithms* (GAs). In [7], the authors proposed the **SS&GA** biclustering algorithm, in which the general scheme is based on SS but incorporates some GA's features such as the mutation and crossover operators to generate new biclustering solutions. This algorithm uses a binary encoding to represent solutions and considers the MSR metric to evaluate the quality of the biclusters. Later, the authors proposed a similar hybrid evolutionary algorithm (herein referred to as **HEAB**) for biclustering gene expression data [5]. In HEAB, the ACF metric is used as the fitness function based on a correlation measure. First, HEAB searches for biclusters with groups of highly correlated genes; then, new biclusters with shifting and scaling patterns are created by analyzing the correlation matrix. The experimental results using the *Lymphoma dataset* indicated that the correlation-based metric outperformed the well-known MSR metric.

Recently, Lu Yin et al. [13] proposed a hybrid biclustering approach based on *cuckoo search* (CS) and *genetic algorithms* (GAs), **GACSB**. This approach considers the CS algorithm as the main framework and uses the tournament strategy and the elite-retention strategy based on the GA to generate the next generation of solutions. In addition, GACSB uses as objective functions different metrics, namely ACV, MSR, and VE. The experimental results obtained by GACSB were compared with several classic biclustering algorithms, such as the CC algorithm and SEBI, where GACSB outperformed these algorithms when considering various gene expression datasets.

Furthermore, hybrid biclustering approaches that combine *particle swarm optimization* (PSO) features and *simulated annealing* (SA) have been proposed in the literature [2, 26]. First, Thangavel et al. [2] proposed a PSO-SA biclustering algorithm (**PSO-SA-BIC**) to extract biclusters of gene expression data. In this approach, SA is used as a local search procedure to improve the position of the particles with low performance. A modified version of the ACV metric is used to identify biclusters with shifting and scaling patterns. The experimental results showed that the PSO-SA-BIC algorithm outperformed some classical algorithms by providing statistically significant biclusters. Recently, Narmadha and Rathipriya [26] developed a hybrid approach combining PSO and SA to extract triclusters from a 3D-gene expression dataset (*Yeast Cell Cycle* data). This algorithm named **HPSO-TriC** uses a fitness function based on the ACF metric, which aims to identify tricluster with a high correlation degree among genes over samples and time points (this function is referred to as ACF_{time}). The HPSO-TriC algorithm was compared with a PSO-based biclustering algorithm, performing better as the extracted tricluster was more biologically significant.

2.4.7 Summary

We described 23 single-objective biclustering methods based on nature-inspired metaheuristics, including simulated annealing (SA), genetic algorithm (GA), particle swarm optimization (PSO), scatter search (SS), and cuckoo search (CS). These algorithms are summarized in Table 2.1, where GA-based biclustering algorithms represent 61% of the surveyed methods. Moreover, some hybrid approaches (combination of different metaheuristics) have been proposed due to the complexity of the biclustering problem; indeed, several of the reviewed approaches often incorporate a local search strategy to better exploit the search space. Regarding the objective function, many biclustering algorithms (43%) combine information from multiple bicluster metrics in such a way that these optimization functions usually consider the homogeneity (e.g., the MSR metric) and the bicluster size (BSize). For measuring the bicluster homogeneity, the mean squared residence (MSR) and the average correlation function (ACF) are commonly used with percentages of 52% and 43%, respectively. Finally, we noticed that the binary bicluster encoding (BBE) is the most

used among the biclustering algorithms (87%), even though it is less efficient than the integer encoding (IBE) in terms of computation time and memory space.

2.5 Multi-Objective Biclustering

Several real-world optimization problems naturally involve multiple objectives. As the name suggests, a multi-objective optimization problem (MOOP) has a number of objective functions to be minimized or maximized. Therefore, the optimal solution for MOOPs is not a single solution but a set of solutions denoted as Pareto-optimal solutions. In this sense, a solution is Pareto-optimal if it is impossible to improve a given objective without deteriorating another. Generally, such a set of solutions represents the compromise solutions between different conflicting objectives [58].

The biclustering problem can be formulated as a combinatorial optimization problem, such that multiple bicluster quality criteria can be optimized simultaneously [8, 85]. Indeed, in gene expression data analysis, the quality of a bicluster can be defined by its size and its intra-cluster variance (coherence). However, these criteria are independent and notably in conflict as the bicluster's coherence can constantly be improved by removing a row or a column, i.e., by reducing the bicluster's size.

This section presents different multi-objective nature-inspired metaheuristics that have been proposed for the biclustering problem. Table 2.2 summarizes some relevant details about this type of algorithms that are introduced in this section.

2.5.1 *Multi-Objective Evolutionary Algorithms Based on* `NSGA-II`

One of the most representative multi-objective algorithms is the non-dominated sorting genetic algorithm (NSGA-II) [58]. This Pareto-dominance algorithm is characterized by incorporating an explicit diversity-preservation mechanism. As NSGA-II has widely been used to address the biclustering problem, we will outline how the algorithm operates. In NSGA-II, the offspring population Q_t is first created by using the parent population P_t; then, the two populations are combined to form the population R_t. Next, a non-dominated sorting method is used to classify the entire population R_t. Then, the new population P_{t+1} is filled by solutions of different non-dominated fronts, starting with the best non-dominated front, followed by the second front, and so on. Finally, when the last allowed front is being considered, there may exist more solutions than the remaining slots in the new population; in this case, a niching strategy based on the crowding distance is used to choose the members of the last front. For a more detailed description and understanding of

Table 2.2 List of multi-objective biclustering algorithms based on nature-inspired metaheuristics. "MOEA" indicates the type of multi-objective evolutionary algorithm that is used as the underlying optimization strategy. "Objective Functions" refers to the bicluster evaluation metrics as defined in Sect. 2.3.5; and "Encoding" indicates the type of bicluster encoding binary (BBE) or integer (IBE) as described in Sect. 2.3.1

Year	Algorithm	MOEA	Objective functions	Encoding	Ref.
2006	MOEAB	NSGA-II	BSize, MSR	BBE	[24]
2007	SMOB	NSGA-II	BSize, MSR, rVAR	BBE	[22]
2008	MOFB	NSGA-II	BSize, MSR, rVAR	IBE	[59]
2008	MOPSOB	MOPSO	BSize, MSR, rVAR	BBE	[61]
2008	CMOPSOB	MOPSO	BSize, MSR, rVAR	BBE	[21]
2009	HMOPSOB	MOPSO	BSize, MSR, rVAR	BBE	[60]
2009	MOACOB	MOACO	BSize, MSR	BBE	[62]
2009	MOM-aiNet	MOAIS	BSize, MSR	IBE	[20]
2009	MOGAB	NSGA-II	MSR, rVAR	IBE	[34]
2009	SPEA2B	SPEA2	BSize, MSR, rVAR	BBE	[11]
2011	MOBI	NSGA-II	BSize, MSR, rVAR	IBE	[3]
2012	SMOB-VE	NSGA-II	BSize, VE, rVAR	BBE	[33]
2015	HMOBI	IBEA	MSR, rVAR	IBE	[8]
2015	SPEA2B-δ	SPEA2	BSize, MSR	BBE	[63]
2016	AMOSAB	AMOSA	BSize, MSR	IBE	[67]
2017	PBD-SPEA2	SPEA2	BSize, MSR, rVAR	IBE*	[31]
2019	BP-NSGA2	NSGA-II	BSize, MSR	BBE	[32]
2019	AMOSAB	AMOSA	BSize, MSR	IBE	[66]
2020	MMCo-Clus	NSGA-II	MSR, rVAR, AI	BBE*	[30]

$*$ Modification to the original encoding described in Sect. 2.3.1.

NSGA-II, the reader is referred to [58]. Next, we describe different biclustering algorithms that use NSGA-II as the underlying optimization method.

Several multi-objective biclustering approaches have been proposed based on the well-known NSGA-II algorithm [3, 22, 24, 30, 32–34, 59]. Research in multi-objective biclustering became popular after the work by Mitra and Banka [24] entitled "Multi-objective evolutionary biclustering of gene expression data," which was published in 2006. This algorithm (referred to as **MOEAB**) uses the CC method [81] during the population's initialization and after applying the variation operators. MOEAb uses a binary encoding representation (BBE), a uniform single-point crossover, and a single-bit mutation. Concerning the objective functions, the size of the bicluster (BSize) and the MSR metric were considered.

Divina et al. [22, 33] have proposed some multi-objective biclustering approaches based on NSGA-II for microarray data. First, Divina and Aguilar-Ruiz [22] presented the sequential multi-objective biclustering (**SMOB**) algorithm for finding biclusters of high quality with large variation. SMOB adopts a sequential strategy such that the algorithm is invoked several times, each time returning a bicluster stored in a temporal list. This algorithm considered a binary encoding, three

different crossover operators (one-point, two-point, and uniform), and tree mutation strategies (single-bit, add-row, and add-column). The objectives considered in SMOB were: the MSR metric, the bicluster size (BSize), and the row variance (rVAR). Later, Divina et al. [33] presented the virtual error (VE) metric, which measures how well the genes in a bicluster follow the general tendency. Then, the VE metric was used as an additional objective function in SNOB. This modified algorithm, referred to as **SMOB-VE**, aims to find biclusters with shifting and scaling patterns using VE instead of the MSR metric.

Maulik et al. [34, 59] also presented biclustering algorithms based on the NSGA-II algorithm. In [59], the authors presented a multi-objective fuzzy biclustering (**MOFB**) algorithm for discovering overlapping biclusters. MOFB simultaneously optimizes fuzzy versions of the metrics MSR, BSize, and rVAR. Furthermore, MOFB uses an integer encoding of variable string length, a single-point crossover, and a uniform mutation strategy. Subsequently, the authors proposed the **MOGAB** algorithm [34], which optimizes two objective functions, MSR and rVAR. Similar to MOFB, the MOGAB algorithm uses a variable string length encoding, a single-point crossover, and a uniform mutation strategy. Additionally, the authors presented the bicluster index (BI) to validate the obtained biclusters from microarray data.

Seridi et al. [3] proposed a multi-objective biclustering algorithm that simultaneously optimizes three conflicting objectives, the bicluster size (BSize), the MSR metric, and the row variance (rVAR). The presented evolutionary framework **MOBI** can integrate any evolutionary algorithm, such as NSGA-II in this case. MOBI uses an integer bicluster encoding, a single-point crossover, and the CC local-search heuristic [81] replaces the mutation operator.

Kong et al. [32] presented an interesting biclustering algorithm incorporating a bi-phase evolutionary architecture and the NSGA-II algorithm. In this algorithm, referred to as **BP-NSGA2**, the first phase consists of evolving the population of rows and columns and then the population of biclusters. The two populations are initialized using a hierarchical clustering method, and then they are evolved independently. Next, during the evolutionary process, the NSGA-II algorithm optimizes the MSR metric and bicluster size simultaneously. This multi-objective approach outperformed two traditional biclustering approaches. The same authors revisited this idea of incorporating a bi-phase evolutionary strategy for the proposal of the BP-EBA algorithm [29].

Recently, Cui et al. [30] proposed a multi-objective optimization-based multi-view co-clustering algorithm (named **MMCo-Clus**) for feature selection of gene expression data. First, two data views are constructed using information from two different biological data sources. Next, the MMCo-Clus algorithm identifies biclusters (co-clustering solutions) considering the constructed views. Finally, a small number of non-redundant features are selected from the original feature space using consensus clustering. MMCo-Clus uses two well-known bicluster measures, the MSR metric and the bicluster size (BSize), and the agreement index. Although this approach focuses on feature selection, it applies an intrinsic biclustering strategy to select relevant features.

2.5.2 Multi-Objective Evolutionary Algorithms Based on SPEA2 and IBEA

Similar to the NSGA-II algorithm, the strength Pareto evolutionary algorithm (SPEA2) [64] is a well-known multi-objective evolutionary algorithm (MOEA) in the specialized literature. This algorithm is characterized by maintaining an external population, which is used to introduce elitism. This population stores a fixed number of non-dominated solutions found during the entire evolutionary process. Additionally, SPEA2 uses these elite solutions to participate in the genetic operations along with the current population to improve the convergence and diversity of the algorithm. Below we describe some biclustering approaches that use SPEA2 as the primary multi-objective optimization method.

Gallo et al. [11] addressed the microarray biclustering problem using different MOEAs, where the SPEA2 algorithm performed better. This approach (**SPEA2B**) considered a binary encoding, a probabilistic-based mutation, and a two-point crossover operator. Four different objectives were considered in SPEA2B: the number of genes, number of conditions, row variance (rVAR), and the MSR metric. Additionally, a greedy method was implemented based on the CC algorithm to maintain large size and low homogeneity biclusters in the population.

Golchin et al. [31, 63] have proposed biclustering approaches based on the SPEA2 algorithm. First, Golchin et al. [63] presented a multi-objective biclustering algorithm (herein referred to as **SPEA2B-δ**), which optimizes the MSR metric and the size of the bicluster simultaneously. This algorithm used a binary bicluster encoding, a single-point crossover, and a single-bit mutation operator. SPEA2B-δ also incorporated a search heuristic strategy similar to the CC algorithm to remove unwanted genes and conditions. As SPEA2B-δ generates a set of biclustering solutions (Pareto front approximation), a fitness selection function based on the coherence and size of the biclusters is considered to choose the best solutions. Later on, Golchin and Liew [31] proposed a SPEA2-based biclustering algorithm for gene expression data named **PBD-SPEA2**. This algorithm considers three objective functions, namely MSR, BSize, and rVAR. An interesting aspect in PBD-SPEA2 is that each individual in the population represents multiple bicluster, instead of only one bicluster as in other approaches. Thus, given a user-defined number of biclusters, k, an integer-based encoding of fixed length but extended to k biclusters is considered. Regarding the variation operators for generating new solutions, PBD-SPEA2 uses the CC heuristic as a mutation operator and a similarity-based crossover. Finally, a sequential selection technique is used to choose the final solution from the Pareto from approximations obtained by PBD-SPEA2.

Similar to NSGA-II and SPEA2 algorithms, the indicator-based multi-objective algorithm (IBEA) [70] is another representative MOEA in the evolutionary computation literature. In this regard, Seridi et al. [8] proposed a biclustering approach based on the IBEA algorithm named **HMOBI** [8]. This algorithm used an integer-based representation, a single-point crossover, and a mutation operator based on the CC algorithm. In HMOBI, three biclustering metrics are considered as objective

functions, the MSR metric and the row variance, and the bicluster size. The obtained results were compared in terms of the bicluster quality and their biological relevance.

2.5.3 Multi-Objective Particle Swarm Optimization

The particle swarm optimization (PSO) algorithm has been extended to solve multi-objective optimization problems (MOPs). Most of the existing multi-objective PSO (MOPSO) algorithms involve developments from the evolutionary computation field to address MOPs. For a review of different MOPSO algorithms, the reader is referred to the survey by Reyes-Sierra [77]. Below we present different MOPSO algorithms for the biclustering problem.

Two multi-objective approaches to microarray biclustering based on MOPSO algorithms were presented by Junwan Liu et al. [21, 61]. Both approaches use a binary bicluster encoding and optimize simultaneously three objective functions: the bicluster size (BSize), the MSR metric, and the row variance (rVAR). The first algorithm, named **MOPSOB** [61], considers a relaxed form of the Pareto dominance ($\in$-dominance), whereas the second approach (**CMOPSOB** [21]) uses a Pareto-based dominance as in the NSGA-II algorithm. Additionally, in the CMOPSOB algorithm, the information of nearest neighbors between particles is considered when updating the particles' velocity, aiming to accelerate the algorithm's convergence. In the comparative analysis, the biological relevance of the biclusters obtained by CMOPSOB was analyzed considering the information of the GO repository, showing that this approach was able to find biologically meaningful clusters.

Another gene expression biclustering algorithm based on a MOPSO was proposed by Lashkargir et al. [60]. The authors proposed a hybrid MOPSO for biclustering algorithm (**HMOPSOB**), which uses a binary encoding and optimizes four objective functions: bicluster size, row variance, and the MSR metric. The HMOPSOB algorithm includes a local search method based on the CC algorithm and, in addition to the PSO steps, three mutation operators: standard, add-row, and add-column. Additionally, this approach can find biclusters with a low level of overlap among biclusters by considering an external archive.

2.5.4 Other Multi-Objective Approaches

This section describes a number of multi-objective approaches to biclustering that do not fit into the previous classification. The approaches described below are based on ant colony optimization (ACO), artificial immune systems (AISs), and simulated annealing (SA).

The ACO algorithm [65] is a probabilistic technique inspired by the behavior of ants in finding paths from their colony to a food source. In this regard, a multi-objective ACO algorithm for microarray biclustering (**MOACOB**) was introduced by Junwan Liu et al. [62]. MOACOB algorithm uses ACO concepts for biclustering microarray data, where the bicluster size and the MSR metric are optimized simultaneously. Furthermore, this algorithm uses a relaxed form of the Pareto dominance ($\in$-dominance) and considers a binary encoding to represent biclusters. In MOACOB, a number of ants probabilistically construct solutions using a given pheromone model; then, a local search procedure is applied to the constructed solutions. In general, this multi-objective approach based on ACO outperformed another three biclustering algorithms in terms of the size of the obtained biclusters.

AISs are inspired by the principles of immunology and the observed immune process of vertebrates [68]. Additionally, AIS is highly robust, adaptive, inherently parallel, and self-organized. In line with these, Coelho et al. [20] proposed a multi-objective biclustering algorithm based on AIS to analyze texts, named **BIC-aiNet**. In the text mining problem, the input data matrix is composed of rows (texts) and columns (attributes of the corresponding texts), and the aim is to find bipartitions of the whole dataset. The BIC-aiNet algorithm uses an integer bicluster representation, a mutation strategy to insert or remove rows and columns, and a suppression procedure to eliminate entire biclusters if a particular condition is satisfied. This approach was compared with the k-means algorithm, showing that BIC-aiNet discovered more meaningful text biclusters.

The simulated annealing-based multi-objective optimization algorithm (AMOSA) [69] has been used as the underlying optimization strategy for finding bicluster in gene expression data [66, 67]. First, Sahoo et al. [67] presented an AMOSA-based biclustering algorithm (**AMOSAB**) that optimized the MSR metric and the row variance (rVAR) simultaneously. AMOSAB used a real-based encoding of biclusters and a decodification method based on the Euclidean distance to obtain the final biclusters. Then, the same authors, Acharya et al. [66], proposed modifications to the AMOSAB algorithm where the decodification method considered three different distance functions: Euclidean, Point Symmetry (PS), and Line Symmetry (LS). The results showed that the AMOSAB algorithm using the PS and LS distance performed better than the Euclidean version.

2.5.5 Summary

We analyzed 19 biclustering approaches that use different nature-inspired multi-objective metaheuristics such as NSGA-II, SPEA2, MOPSO, and AMOSA. The main characteristics of these methods are summarized in Table 2.2, where multi-objective evolutionary algorithms are the most widely used with 63%, the NSGA-II algorithm being the most common of this group with 42%. Regarding the objective functions used to guide the search of the multi-objective algorithms, we noticed that the bicluster size (BSize), the bicluster coherence (MSR metric), and the

row variance (rVAR) are commonly optimized simultaneously as these criteria are independent and usually in conflict. Finally, regarding the types of bicluster representations, it was noted that both techniques are used almost equally, the binary representation (BBE) with 58%, while the integer representation (IBE) with 42%. However, in multi-objective algorithms, there is a tendency in recent years to use the IBE representation, which is more efficient than the binary representation for biclustering problems.

2.6 Discussion and Future Directions

Metaheuristic-based biclustering algorithms have gained much relevance in recent decades, mainly because biclustering remains an important problem in practice and, computationally, it is a highly combinatorial problem. In this sense, Fig. 2.6 illustrates the 42 algorithms, surveyed in this book chapter, that have been proposed within the last 15 years.

In this chapter, we introduced the fundamental concepts and principles of biclustering optimization methods, which could be classified into two main categories depending on their underlying search strategy, namely single-objective and multi-objective biclustering approaches. Indeed, single-objective metaheuristics represent 55% of the surveyed algorithms whereas multi-objective algorithms represent 45%. Overall, biclustering approaches based on the evolutionary computation paradigm represent 71% of the reviewed works; the remaining 29% corresponds to other types of nature-inspired metaheuristics. Regarding the representation of biclusters, the binary encoding is the most used with 73%, but there is a tendency to use the integer encoding, especially in multi-objective approaches.

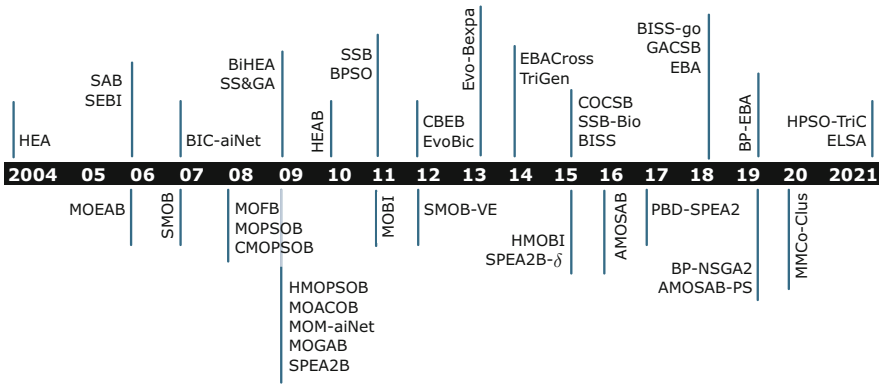


Fig. 2.6 Timeline of biclustering algorithms based on nature-inspired metaheuristics. Above the timeline, the 23 single-objective approaches are presented, whereas below the 19 multi-objective algorithms are illustrated

Based on the revised works, it is notable that a large number of single-objective approaches (44%) consider as objective function a combination of multiple biclustering criteria, suggesting that the biclustering problem is inherently a multi-objective optimization problem. Besides, multi-objective biclustering algorithms optimize multiple biclustering metrics simultaneously, having the advantage of discovering several types of biclusters (see Sect. 2.2.1). However, these approaches generate a set of solutions requiring an additional mechanism to filter and select the best biclustering solution. Therefore, the optimization technique's selection depends on the complexity of the biclustering problem in terms of the type of biclusters and the bicluster structures to be discovered (overlapping among bicluster and matrix covering matrix).

2.6.1 Future Directions

Biclustering is an open field with several research directions, opportunities, and challenges that involve the following issues:

- To solve complex biclustering problems to discover different bicluster types and bicluster structures, it is necessary to study and analyze the current objective function and bicluster representations that will help to select the appropriate optimization scheme and components according to the biclustering scenario. For instance, it is clear that an integer based-representation is preferable over a binary representation [3, 66]. Most of these approaches using an integer representation encodes a single bicluster; however, it is possible to codify multiple biclusters in a single solution, as demonstrated recently by Golchin [31].
- Novel nature-inspired metaheuristics are continuously proposed in the literature as potential approaches to solve the biclustering problem. Particularly, many-objective optimization evolutionary algorithms such as MOEA/D [71], NSGA-III [72] can be used to cope with multiple biclustering criteria (more than three objective functions). It is important to mention that selecting the best biclustering solution is an added challenge when using these multi-objective biclustering approaches.
- Although many synthetic and real-life datasets have been used systematically in the literature, there is no recognized benchmark that the research community could use to evaluate and compare biclustering approaches. Such a benchmark should include different types of bicluster, diverse bicluster structures, noise, overlapping, etc. Furthermore, when comparing the performance of biclustering approaches, it is crucial to consider their statistical and biological significance (i.e., to consider the available biological information [15]).

- Recently, the biclustering problem is referred to as triclustering when the time dimension is considered in addition to rows and columns information. Finding tricluster when considering temporal data brings up new research challenges as it will require the adaptation of current algorithms, bicluster metrics, evaluation measures, etc. Indeed, the triclustering problem has been addressed recently using single-objective metaheuristics [26, 28]; however, there are opportunities to address this problem as multi-objective optimization.
- Most of the proposed metaheuristic-based biclustering algorithms have been designed to work on biological data (mainly gene expression and microarray data). Thus, the application of biclustering to other data types, such as heterogeneous data, is very limited as it brings up additional challenges. In [80], the authors proposed a greedy procedure to extract biclusters from heterogeneous, temporal, and large-scale data. This procedure has been applied successfully on Electronic Health Records (EHR) thanks to the sparsity of data in this scenario, optimizing the enumeration. It will be interesting to study the potential of nature-inspired metaheuristics to discover heterogeneous-like bicluster in EHR applications.
- There are many application domains where multiple pieces of information are available for each individual subject. For instance, multiple data matrices might be available for the same set of genes and conditions. In this regard, multi-view data clustering algorithms can integrate these information pieces to find consistent clusters across different data views [73, 74]. This same multi-view clustering concept can be extended to biclustering, where the aim is to discover biclusters across multiple data matrices (i.e., data views).

2.7 Conclusion

Biclustering has emerged as an important approach and currently plays an essential role in various applications ranging from bioinformatics to text mining. Different nature-inspired metaheuristics have been applied to address the biclustering problems as, from the computational point of view, this is a NP-hard optimization problem.

In this regard, this chapter presented a detailed survey of metaheuristics approaches to address the biclustering problem. The review focused on the underlying optimization methods and their main search components: biclustering encoding, variation operators and bicluster objective functions. This review focused on single versus multi-objective approaches. Additionally, we presented a discussion and some emerging research directions.

Acknowledgments This work has been partially supported by the I-Site ULNE (Université Lille-Nord Europe) and the Lille European Metropolis (MEL).

References

1. Rathipriya, R., Thangavel, K. & Bagyamani, J. Binary particle swarm optimization based biclustering of web usage data. *International Journal Of Computer Applications*. **25**, 43–49 (2011)
2. Thangavel, K., Bagyamani, J. & Rathipriya, R. Novel hybrid PSO-SA model for biclustering of expression data. *Procedia Engineering*. **30** pp. 1048–1055 (2012)
3. Seridi, K., Jourdan, L. & Talbi, E. Multi-objective evolutionary algorithm for biclustering in microarrays data. *IEEE Congress Of Evolutionary Computation*. pp. 2593–2599 (2011)
4. Nepomuceno, J., Troncoso, A. & Aguilar-Ruiz, J. Biclustering of gene expression data by correlation-based scatter search. *BioData Mining*. **4**, 3 (2011)
5. Nepomuceno, J., Troncos, A. & Aguilar-Ruiz, J. Evolutionary metaheuristic for biclustering based on linear correlations among genes. *ACM Symposium On Applied Computing*. pp. 1143 (2010)
6. Nepomuceno, J., Troncoso, A., Nepomuceno-Chamorro, I. & Aguilar-Ruiz, J. Integrating biological knowledge based on functional annotations for biclustering of gene expression data. *Computer Methods And Programs In Biomedicine*. **119**, 163–180 (2015)
7. Nepomuceno, J., Troncoso, A. & Aguilar-Ruiz, J. A hybrid metaheuristic for biclustering based on scatter search and genetic algorithms. *International Conference On Pattern Recognition In Bioinformatics*. pp. 199–210 (2009)
8. Seridi, K., Jourdan, L. & Talbi, E. Using multiobjective optimization for biclustering microarray data. *Applied Soft Computing*. **33** pp. 239–249 (2015)
9. Nepomuceno, J., Troncoso, A. & Aguilar-Ruiz, J. Scatter search-based identification of local patterns with positive and negative correlations in gene expression data. *Applied Soft Computing*. **35** pp. 637–651 (2015)
10. Ayadi, W., Maatouk, O. & Bouziri, H. Evolutionary biclustering algorithm of gene expression data. *International Workshop On Database And Expert Systems Applications*. pp. 206–210 (2012)
11. Gallo, C., Carballido, J. & Ponzoni, I. Microarray biclustering: A novel memetic approach based on the PISA platform. *European Conference On Evolutionary Computation, Machine Learning And Data Mining In Bioinformatics*. pp. 44–55 (2009)
12. Maâtouk, O., Ayadi, W., Bouziri, H. & Duval, B. Evolutionary algorithm based on new crossover for the biclustering of gene expression data. *International Conference On Pattern Recognition In Bioinformatics*. pp. 48–59 (2014)
13. Yin, L., Qiu, J. & Gao, S. Biclustering of gene expression data using cuckoo search and genetic algorithm. *International Journal Of Pattern Recognition And Artificial Intelligence*. **32**, 1850039 (2018)
14. Lu, Y. & Liu, Y. Biclustering of the gene expression data by coevolution cuckoo search. *International Journal Bioautomation*. **19**, 161–176 (2015)
15. Maâtouk, O., Ayadi, W., Bouziri, H. & Duval, B. Evolutionary local search algorithm for the biclustering of gene expression data based on biological knowledge. *Applied Soft Computing*. **104** pp. 107177 (2021)
16. Pontes, B., Giráldez, R. & Aguilar-Ruiz, J. Configurable pattern-based evolutionary biclustering of gene expression data. *Algorithms For Molecular Biology*. **8**, 4 (2013)
17. Qinghua Huang, Dacheng Tao, Xuelong Li & Liew, A. Parallelized evolutionary learning for detection of biclusters in gene expression data. *IEEE/ACM Transactions On Computational Biology And Bioinformatics*. **9**, 560–570 (2012)
18. Gallo, C., Carballido, J. & Ponzoni, I. BiHEA: A hybrid evolutionary approach for microarray biclustering. *Brazilian Symposium On Bioinformatics*. pp. 36–47 (2009)
19. Divina, F. & Aguilar-Ruiz, J. Biclustering of expression data with evolutionary computation. *IEEE Transactions On Knowledge And Data Engineering*. **18**, 590–602 (2006)

20. Coelho, G., França, F. & Von Zuben, F. Multi-objective biclustering: when non-dominated solutions are not enough. *Journal Of Mathematical Modelling And Algorithms*. **8**, 175–202 (2009)
21. Liu, J., Li, Z., Hu, X. & Chen, Y. Biclustering of microarray data with MOSPO based on crowding distance. *BMC Bioinformatics*. **10**, S9 (2009)
22. Divina, F. & Aguilar-Ruiz, J. A multi-objective approach to discover biclusters in microarray data. *Genetic And Evolutionary Computation Conference - GECCO '07*. pp. 385 (2007)
23. Bryan, K., Cunningham, P. & Bolshakova, N. Application of simulated annealing to the biclustering of gene expression data. *IEEE Transactions On Information Technology In Biomedicine*. **10**, 519–525 (2006)
24. Mitra, S. & Banka, H. Multi-objective evolutionary biclustering of gene expression data. *Pattern Recognition*. **39**, 2464–2477 (2006)
25. Bleuler, S., Prelic, A. & Zitzler, E. An EA framework for biclustering of gene expression data. *Congress On Evolutionary Computation (CEC'2004)*. pp. 166–173 (2004)
26. Narmadha, N. & Rathipriya, R. Gene ontology analysis of gene expression data using hybridized PSO triclustering. *Machine Learning And Big Data Analytics Paradigms: Analysis, Applications And Challenges*. pp. 437–466 (2021)
27. Maâtouk, O., Ayadi, W., Bouziri, H. & Duval, B. Evolutionary biclustering algorithms: an experimental study on microarray data. *Soft Computing*. **23**, 7671–7697 (2019)
28. Gutiérrez-Avilés, D., Rubio-Escudero, C., Martínez-Álvarez, F. & Riquelme, J. TriGen: A genetic algorithm to mine triclusters in temporal gene expression data. *Neurocomputing*. **132** pp. 42–53 (2014)
29. Huang, Q., Huang, X., Kong, Z., Li, X. & Tao, D. Bi-phase evolutionary searching for biclusters in gene expression data. *IEEE Transactions On Evolutionary Computation*. **23**, 803–814 (2019)
30. Cui, L., Acharya, S., Mishra, S., Pan, Y. & Huang, J. MMCO-Clus – an evolutionary co-clustering algorithm for gene selection. *IEEE Transactions On Knowledge And Data Engineering*. pp. 1–1 (2020)
31. Golchin, M. & Liew, A. Parallel biclustering detection using strength Pareto front evolutionary algorithm. *Information Sciences*. **415–416** pp. 283–297 (2017)
32. Kong, Z., Huang, Q. & Li, X. Bi-Phase evolutionary biclustering algorithm with the NSGA-II algorithm. *IEEE International Conference On Advanced Robotics And Mechatronics (ICARM)*. pp. 146–149 (2019)
33. Divina, F., Pontes, B., Giráldez, R. & Aguilar-Ruiz, J. An effective measure for assessing the quality of biclusters. *Computers In Biology And Medicine*. **42**, 245–256 (2012)
34. Maulik, U., Mukhopadhyay, A. & Bandyopadhyay, S. Finding multiple coherent biclusters in microarray data using variable string length multiobjective genetic algorithm. *IEEE Transactions On Information Technology In Biomedicine*. **13**, 969–975 (2009)
35. Teng, L. & Chan, L. Discovering biclusters by iteratively sorting with weighted correlation coefficient in gene expression data. *Journal Of Signal Processing Systems*. **50** pp. 267–280 (2008)
36. Hartigan, J. Direct clustering of a data matrix. *Journal Of The American Statistical Association*. **67**, 123 (1972)
37. Mukhopadhyay, A., Maulik, U. & Bandyopadhyay, S. A novel coherence measure for discovering scaling biclusters from gene expression data. *Journal Of Bioinformatics And Computational Biology*. **7**, 853–868 (2009)
38. Getz, G., Levine, E. & Domany, E. Coupled two-way clustering analysis of gene microarray data. *Proceedings Of The National Academy Of Sciences*. **97**, 12079–12084 (2000)
39. Chun Tang, Li Zhang, Aidong Zhang & Ramanathan, M. Interrelated two-way clustering: an unsupervised approach for gene expression data analysis. *IEEE International Symposium On Bioinformatics And Bioengineering*. pp. 41–48 (2001)
40. Prelić, A., Bleuler, S., Zimmermann, P., Wille, A., Bühlmann, P., Gruissem, W., Hennig, L., Thiele, L. & Zitzler, E. A systematic comparison and evaluation of biclustering methods for gene expression data. *Bioinformatics*. **22**, 1122–1129 (2006,5)

41. Li, G., Ma, Q., Tang, H., Paterson, A. & Xu, Y. QUBIC: a qualitative biclustering algorithm for analyses of gene expression data. *Nucleic Acids Research*. **37**, e101-e101 (2009,8)
42. Ben-Dor, A., Chor, B., Karp, R. & Yakhini, Z. Discovering local structure in gene expression data: the order-preserving submatrix problem. *Journal Of Computational Biology*. **10**, 373–384 (2003,6)
43. Shabalin, A., Weigman, V., Perou, C. & Nobel, A. Finding large average submatrices in high dimensional data. *The Annals Of Applied Statistics*. **3** (2009)
44. Tanay, A., Sharan, R. & Shamir, R. Discovering statistically significant biclusters in gene expression data. *Bioinformatics*. **18**, S136-S144 (2002)
45. Rodriguez-Baena, D., Perez-Pulido, A. & Aguilar-Ruiz, J. A biclustering algorithm for extracting bit-patterns from binary datasets. *Bioinformatics*. **27**, 2738–2745 (2011)
46. Serin, A. & Vingron, M. DeBi: Discovering differentially expressed biclusters using a frequent itemset approach. *Algorithms For Molecular Biology*. **6**, 18 (2011)
47. Kluger, Y. Spectral biclustering of microarray data: coclustering genes and conditions. *Genome Research*. **13**, 703–716 (2003)
48. Gu, J. & Liu, J. Bayesian biclustering of gene expression data. *BMC Genomics*. **9**, S4 (2008)
49. Hochreiter, S., Bodenhofer, U., Heusel, M., Mayr, A., Mitterecker, A., Kasim, A., Khamiakova, T., Van Sanden, S., Lin, D., Talloen, W., Bijnens, L., Göhlmann, H., Shkedy, Z. & Clevert, D. FABIA: factor analysis for bicluster acquisition. *Bioinformatics*. **26**, 1520–1527 (2010)
50. José-García, A. & Gómez-Flores, W. Automatic clustering using nature-inspired metaheuristics: A survey. *Applied Soft Computing*. **41** pp. 192–213 (2016)
51. Kirkpatrick, S., Gelatt, C. & Vecchi, M. Optimization by Simulated Annealing. *Science*. **220**, 671–680 (1983)
52. Holland, J. Adaptation in natural and artificial systems. (University of Michigan Press,1975)
53. Castro, P., França, F., Ferreira, H. & Von Zuben, F. Applying biclustering to text mining: an immune-inspired approach. *International Conference On Artificial Immune Systems*. pp. 83–94 (2007)
54. Kennedy, J. & Eberhart, R. Particle Swarm Optimization. *International Conference On Neural Networks*. **4** pp. 1942–1948 (1995)
55. Yang, X. & Deb, S. Cuckoo search: recent advances and applications. *Neural Computing And Applications*. **24**, 169–174 (2014)
56. Laguna, M. & Martí, R. Scatter Search. *Metaheuristic Procedures For Training Neutral Networks*. pp. 139–152 (2006)
57. Nepomuceno, J., Troncoso, A., Nepomuceno-Chamorro, I. & Aguilar-Ruiz, J. Pairwise gene GO-based measures for biclustering of high-dimensional expression data. *BioData Mining*. **11**, 4 (2018)
58. Deb, K., Pratap, A., Agarwal, S. & Meyarivan, T. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions On Evolutionary Computation*. **6**, 182–197 (2002)
59. Maulik, U., Mukhopadhyay, A., Bandyopadhyay, S., Zhang, M. & Zhang, X. Multiobjective fuzzy biclustering in microarray data: Method and a new performance measure. *IEEE Congress On Evolutionary Computation*. pp. 1536–1543 (2008)
60. Lashkargir, M., Monadjemi, S. & Dastjerdi, A. A new biclustering method for gene expression data based on adaptive multiobjective particle swarm optimization. *International Conference On Computer And Electrical Engineering*. pp. 559–563 (2009)
61. Liu, J., Li, Z., Liu, F. & Chen, Y. Multi-objective particle swarm optimization biclustering of microarray data. *EEE International Conference On Bioinformatics And Biomedicine*. pp. 363–366 (2008)
62. Liu, J., Li, Z., Hu, X. & Chen, Y. Multi-objective ant colony optimization biclustering of microarray data. *IEEE International Conference On Granular Computing*. pp. 424–429 (2009,8)
63. Golchin, M., Davarpanah, S. & Liew, A. Biclustering analysis of gene expression data using multi-objective evolutionary algorithms. *International Conference On Machine Learning And Cybernetics*. pp. 505–510 (2015)

64. Zitzler, E., Laumanns, M. & Thiele, L. SPEA2: Improving the strength Pareto evolutionary algorithm. (Swiss Federal Institute Technology,2001)
65. Dorigo, M. & Blum, C. Ant colony optimization theory: A survey. *Theoretical Computer Science*. **344**, 243–278 (2005)
66. Acharya, S., Saha, S. & Sahoo, P. Bi-clustering of microarray data using a symmetry-based multi-objective optimization framework. *Soft Computing*. **23**, 5693–5714 (2019)
67. Sahoo, P., Acharya, S. & Saha, S. Automatic generation of biclusters from gene expression data using multi-objective simulated annealing approach. *International Conference On Pattern Recognition*. pp. 2174–2179 (2016)
68. Talbi, E. Metaheuristics from design to implementation. (John Wiley,2009)
69. Bandyopadhyay, S., Saha, S., Maulik, U. & Deb, K. A simulated annealing-based multiobjective optimization algorithm: AMOSA. *IEEE Transactions On Evolutionary Computation*. **12**, 269–283 (2008)
70. Zitzler, E. & Künzli, S. Indicator-based selection in multiobjective search. *Parallel Problem Solving From Nature - PPSN VIII*. **3242** pp. 832–842 (2004)
71. Qingfu Zhang & Hui Li MOEA/D: A multiobjective evolutionary algorithm based on decomposition. *IEEE Transactions On Evolutionary Computation*. **11**, 712–731 (2007,12)
72. Deb, K. & Jain, H. An evolutionary many-objective optimization algorithm using reference-point-based nondominated sorting approach, Part I: Solving problems with box constraints. *IEEE Transactions On Evolutionary Computation*. **18**, 577–601 (2014)
73. José-García, A., Handl, J., Gómez-Flores, W. & Garza-Fabre, M. Many-view clustering: An illustration using multiple dissimilarity measures. *Genetic And Evolutionary Computation Conference - GECCO '19*. pp. 213–214 (2019)
74. José-García, A., Handl, J., Gómez-Flores, W. & Garza-Fabre, M. An evolutionary many-objective approach to multiview clustering using feature and relational data. *Applied Soft Computing*. **108** (2021)
75. Xie, J., Ma, A., Fennell, A., Ma, Q. & Zhao, J. It is time to apply biclustering: a comprehensive review of biclustering applications in biological and biomedical data. *Briefings In Bioinformatics*. **20**, 1450–1465 (2019,7)
76. Pontes, B., Giráldez, R. & Aguilar-Ruiz, J. Biclustering on expression data: A review. *Journal Of Biomedical Informatics*. **57** pp. 163–180 (2015,10)
77. Reyes-Sierra, M., Coello, C. & Others Multi-objective particle swarm optimizers: A survey of the state-of-the-art. *International Journal Of Computational Intelligence Research*. **2**, 287–308 (2006)
78. Dhaenens, C. & Jourdan, L. Metaheuristics for data mining. *4OR*. **17**, 115–139 (2019)
79. Dhaenens, C. & Jourdan, L. Metaheuristics for big data. (John Wiley & Sons,2016)
80. Vandromme, M., Jacques, J., Taillard, J., Jourdan, L. & Dhaenens, C. A biclustering method for heterogeneous and temporal medical data. *IEEE Transactions On Knowledge And Data Engineering*. (2020)
81. Cheng, Y. & Church, G. Biclustering of expression data. *International Conference On Intelligent Systems For Molecular Biology*. **8**, 93–103 (2000)
82. Pontes, B., Giráldez, R. & Aguilar-Ruiz, J. Biclustering on expression data: A review. *Journal Of Biomedical Informatics*. **57** pp. 163–180 (2015)
83. Ding, C., Zhang, Y., Li, T. & Holbrook, S. Biclustering protein complex interactions with a biclique finding algorithm. *Sixth International Conference On Data Mining (ICDM'06)*. pp. 178–187 (2006)
84. Aguilar-Ruiz, J. Shifting and scaling patterns from gene expression data. *Bioinformatics*. **21**, 3840–3845 (2005)
85. Madeira, S. & Oliveira, A. Biclustering algorithms for biological data analysis: a survey. *IEEE/ACM Transactions On Computational Biology And Bioinformatics*. **1**, 24–45 (2004)

Chapter 3
A Metaheuristic Perspective on Learning Classifier Systems



Michael Heider, David Pätzel, Helena Stegherr, and Jörg Hähner

Abstract Within this book chapter we summarize Learning Classifier Systems (LCSs), a family of rule-based learning systems with a more than forty-year-long research history, and differentiate LCSs from related approaches like Genetic Programming, Decision Trees, Mixture of Experts as well as Bagging and Boosting. LCS training constructs a finite collection of if-then rules. While the conclusion (the then-part) of each rule is based on a problem-dependent local submodel (e. g.linear regression), the individual conditions and, by extension, the global model structure, are optimized using a—typically evolutionary—metaheuristic. This makes the employed metaheuristic a central part of learning and indispensable for successful training. While most traditional LCSs solely rely on Genetic Algorithms, in this chapter, we also explore systems that employ different metaheuristics while still being similar to LCSs. Furthermore, we discuss the different problems that metaheuristics are solving when applied in this context, for example, discrete or real valued input domains, optimization of individual rule conditions or entire sets of rule conditions, and fitness functions that support nicheing or control bloating. We ascertain that, despite the optimizer being essential, it has been investigated less directly than the learning components so far. Thus, overall, we provide an analysis of LCSs with a focus on the used metaheuristics and present existing solutions as well as current challenges to other practioners in the fields of metaheuristic ML and rule-based learning.

3.1 Motivation and Structure

In a book about metaheuristics and machine learning (ML), Learning Classifier Systems (LCSs) have to be mentioned as they were among the first metaheuristics-based ML approaches.

M. Heider (✉) · D. Pätzel · H. Stegherr · J. Hähner
University of Augsburg, Augsburg, Germany
e-mail: michael.heider@uni-a.de

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

M. Eddaly et al. (eds.), *Metaheuristics for Machine Learning*,
Computational Intelligence Methods and Applications,
https://doi.org/10.1007/978-981-19-3888-7_3

Originally, LCSs stem from approaches to utilize genetic algorithms (GAs) for interactive tasks [39, 82]. While they were developed somewhat independently of reinforcement learning (RL) methods in the beginning, the two fields merged later. LCSs build sets of if-then rules and there are variants for all of the major ML tasks. Although some researchers refer to LCSs as *evolutionary rule-based machine learning (RBML)* methods, we would like to stress that—from our perspective—evolutionary optimization is not required but actually any metaheuristic may be employed.

In this chapter, we provide an in-depth view at LCSs from the metaheuristics side. To our knowledge, this has not been done yet, despite the employed metaheuristic playing a central role in these methods. Indeed, neither the metaheuristics used in LCSs nor their operators have received a notable amount of research attention so far. With this chapter, we want to raise awareness of this fact and, by organizing existing research, provide inspiration for new developments in both areas, metaheuristics and LCSs, and thus lay the ground work that researchers can then build upon to more easily remedy this deficiency.

To this end, we start with a short introduction to the field of LCSs, explaining the overall approach and discussing the learning and optimization tasks that these systems try to solve (Sect. 3.2). We then describe the relation of LCSs to other similar ML approaches to highlight similarities and differences and thus deepen the reader's understanding (Sect. 3.3). Next, we elaborate on the role of metaheuristics in LCSs, the types of systems that different design choices result in (in the process proposing a new classification instead of the known Michigan/Pittsburgh one), and approaches to solution representation, metaheuristic operators and fitness functions (Sect. 3.4). Finally, we look at recent ML approaches that employ metaheuristics and that were developed somewhat independently of LCSs but that rather closely resemble what we would call an LCS (Sect. 3.5); some of the techniques from these fields may be relevant for LCS as well.

3.2 What Are Learning Classifier Systems?

While there is no official definition of what constitutes an LCS, the consensus [26, 76] seems to be that an LCS is an ML algorithm that creates an ensemble model consisting of a set of simpler *submodels*, each of which models a certain part of the problem space. At that, the problem space is not necessarily split into disjoint sets—submodel responsibilities may overlap. This is achieved by pairing each submodel with a *matching function* that, for each possible input, states whether that input is modelled by the submodel. Although overlapping responsibilities introduce some additional complexity, they are considered a feature (e. g. the resulting overall model can be expected to be more smooth than one based on disjoint splits). In areas of overlap, the predictions of the submodels that model the input are *mixed*, usually by employing a weighted average. The combination of a matching function and a submodel can be seen as an *if-then rule* (or just *rule*, in short) which is also

the technical term in the general field of RBML. Out of historic reasons, in LCS literature, a rule together with its corresponding mixing weight (and possibly further bookkeeping parameters) is called *a classifier*—hence the name *Learning Classifier Systems*. In this chapter, we try to avoid the slightly overloaded term *classifier* and rely more on the term *rule* which we use to denote a tuple consisting of a matching function, a submodel as well as a mixing weight and bookkeeping parameters such as how often the rule was correct. Just like existing LCS literature, we further use the term *condition* to refer to the genotypic representation of a matching function (i. e. a certain matching function is the phenotype of a certain condition).

Most of the LCSs currently popular in the field were developed in an ad-hoc manner [26]. This means that the systems' mechanisms were not derived from a set of central, formally specified, assumptions about the learning task (e. g. the available data); instead, in particular in very early LCSs and presumably due to their role as proof of concept for the ideas regarding artificial evolution and genetics, methods that were to be used were selected first and then combined so certain kinds of inputs could be dealt with [39]. Later LCSs then built on those early systems: For example, XCS [82] is a simplification (and improvement) of ZCS, which on the other hand is a simplification of Holland's original framework [81].

3.2.1 Learning Tasks

LCSs are a family of ML methods. At the time of this writing and aside from pre- and postprocessing, task primitives found in ML can be very roughly divided into *supervised learning* (SL), *unsupervised learning* and *reinforcement learning* (which is sometimes also called *sequential decision making*) [12]. While there are LCSs that perform unsupervised learning [see e. g. 90], these play a rather minor role in recent years. Most currently researched LCSs are meant for supervised learning (SL) or RL [59, 60].

The very first LCSs focused solely on RL tasks [39, 81, 82], even though that term was not being used in the beginning (RL and early LCS frameworks were developed somewhat independently). However, over the years it has been seen that those LCSs, while performing competitively on SL tasks such as regression and classification (discussed in the next paragraph), do *not* on non-trivial RL tasks [9, 69]. In hindsight it can be said that this was probably the cause of more and more researchers falling back to analyzing what are called in LCS literature *single-step problems*, which are RL tasks with an episode length of one. Since, in many of the considered tasks, there is a finite set of reward levels, these kinds of studies often actually investigate classification tasks. Besides, many of the tasks considered are deterministic which means that the corresponding classification tasks are additionally noise-free. Nevertheless, several existing LCSs *are* capable of solving more or less difficult RL tasks; for example, anticipatory classifier systems [19, 55] whose rules also model the environment's behaviour.

The larger part of LCS research is concerned with SL and, in particular, *classification* and *regression* tasks. At the time of this writing, the probably most commonly used (and certainly most investigated) LCSs for these kinds of tasks are almost exclusively XCS derivatives [59, 60, 76] with a notable exception being *BioHEL* [see e. g. 7, 29]. XCS [82] is one of those aforementioned early systems that originally were meant for RL but that, over the years, have been used more and more exclusively for classification [59, 60]. In spite of that, XCS still has several RL-specific mechanisms that are not directly useful or even detrimental for performing classification; this gave rise to the *UCS* derivative [11, 78] which explicitely focuses on supervised classification tasks.

In 2002, Wilson developed *XCSF* [84], an XCS derivative for function approximation. XCSF has been further improved up to quite recently, resulting in competitiveness, at least for input spaces with low dimensionality [68, 70]. Besides XCSF, there are other lesser known LCS-based regressors as welldrugowitsch2008b. Note that one application of regression is approximating action-value functions in RL problems [72]—which circles back to the early goals of LCS research.

Overall, it can be said that LCSs are a versatile framework that can be used for all different kinds of learning problems.

3.2.2 LCS Models

Most models built by LCSs are of a form similar to

$$\sum_{k=1}^{K} m_k(x)\gamma_k \hat{f}_k(\theta_k, x) \tag{3.1}$$

where

- x is the input for which a prediction is to be made,
- K is the overall number of rules in the model,
- $m_k(x)$ is the output of the matching function of rule k for input x; typically, $m_k(x) = 1$ iff rule k is responsible for input x (i. e. if its submodel models that input), and $m_k(x) = 0$ otherwise,
- γ_k is the mixing weight of rule k; mixing weights usually fulfill $\sum_{k=1}^{K} m_k(x)\gamma_k = 1$ and $0 \leqslant \gamma_k \leqslant 1$ (this can easily be achieved using normalization, which is standard for almost all LCSs),
- $\hat{f}_k(\theta_k, x)$ is the output of the submodel of rule k (with parameters θ_k) for input x.

Note that although the sum goes over all rules, by multiplying with $m_k(x)$ only the rules that match the considered input x contribute to the result (for non-matching rules, the respective summand is 0). Further, instead of binary matching, that is, $m_k(x)$ being a function with codomain $\{0, 1\}$, *matching by degree* can be an option: $m_k(x)$ then has codomain $[0, 1]$ and may correspondingly take on any value between

0 and 1. However, since all of the more prominent LCSs use binary matching, we assume the same for the remainder of this chapter—unless we explicitly write otherwise.

3.2.3 Overall Algorithmic Structure of an LCS

In order to build such an ensemble model, an LCS performs two main tasks, each of which can be seen as having two subtasks:

Model selection consists of

(I) choosing an appropriate number of rules K for the task at hand and
(II) assigning each rule a part of the input space (i. e. choosing matching functions $\{m_k(\cdot)\}_{k=1}^{K}$).

It is important to note once more, that, in LCSs, the input space is not split into disjoint partitions but that instead an input may be modelled by several submodels. The set of matching functions is also called the model's *model structure*.

Model fitting is concerned with selecting the best-performing submodels and combining them optimally, given a fixed model structure. It usually consists of

(III) fitting each submodel $\hat{f}_k(\theta_k, \cdot)$ to the data its rule is responsible for by adjusting its parameters θ_k. It is often beneficial to do this such that each submodel is trained independently of the others, since, that way, upon getting new training examples or changing the model structure (model selection is typically done iteratively and alternatingly with model fitting), only partial retraining is necessary [26].
(IV) fitting the mixing weights $\{\gamma_k\}_{k=1}^{K}$ (see the introduction to Sect. 3.2) such that areas where the responsibilities of several rules overlap are dealt with optimally.

We now shortly discuss the four subtasks with a focus on whether it is common that metaheuristics come into play.

The number of rules K is the main indicator for the overall model's complexity and with that, its expressiveness. A larger number of rules increases the expressiveness at the cost of an increased training effort and a decrease in interpretability. Also, if the number of rules is too large, then overfitting can be expected (extreme case: more submodels than there are data points in the training data) resulting in worse generalization capabilities. On the other hand, if the number of rules is too small, the overall model may not be able to sufficiently capture the patterns in the available

data. Choosing an appropriate number of rules (subtask (I)) is thus an optimization problem.

Based on the chosen number of rules, the next open task is to decide which submodel is responsible for modelling which parts of the problem space (subtask (II)). *This is the task that all existing LCSs use metaheuristics for*. Since the quality of a solution candidate to this subtask highly depends on the number of rules (for example, if there are fewer rules, then, in order to properly cover the problem space, at least one of them has to be responsible for a larger part of it than if there were more rules), subtasks (I) and (II) are often solved together or using a combination of a metaheuristic for (II) and a simple heuristic for (I). For instance, the latter approach has been taken for XCS [82] and its main derivatives, where the set of matching functions is optimized using GAs [11, 82].

How exactly the individual submodels are fit to the data that their respective rule's matching function matches (subtask (III)) highly depends on the type of submodel. For example, for linear submodels, methods such as least squares may be employed [26]. Several well-known LCSs (most prominently, most XCS derivatives) perform online learning which means that training data is seen instance-by-instance and that predictions can be performed during learning. This demands that the submodels support this kind of learning as well; for instance, in the original XCS, the submodels' parameters are updated incrementally using a gradient-based update [82]. Overall, fitting a submodel is also an optimization problem (e. g. minimization of expected risk on the data matched by the corresponding rule) that is typically not approached with metaheuristics.

Given fixed sets of matching functions and (fitted) submodels, the final open question (subtask (IV)) is how submodel predictions should be combined into a single prediction. As mentioned before, an LCS's submodels are allowed to overlap, and for areas where that is the case, predictions of several submodels have to be combined sensibly. A straightforward approach which is used by several popular LCSs is to compute a quality metric for each rule (e. g. an accuracy score on the training data that the rule matches) and use a weight proportional to that metric's output in a weighted average. This is, for example, how XCS does it [82]. While there has been some research regarding this task [26], the larger part of LCS research more or less neglected it and never really explored different methods [26]. In general, computing mixing weights given fixed matching functions and submodels is another optimization problem; just like for each individual submodel in subtask (III), here, for the whole model, the expected risk is to be minimized. While there are cases where provably optimal solutions can be found, their computation is often rather expensive and cheap heuristics based on rule properties or metrics (such as the one of XCS) can be a better choice [26]. To our knowledge, there have not yet been any approaches of using metaheuristics for subtask (IV).

This concludes this first, rather high-level discussion of the four subtasks. Section 3.4 discusses in more detail how subtasks (I) and (II) are approached metaheuristically by existing LCSs as well as the challenges encountered in doing so.

3.2.4 LCSs in Metaheuristics Literature

After introducing LCSs from the ML perspective, we now shortly discuss their presentation in existing metaheuristics literature. Overall, it can be said, that, if mentioned at all, they are only discussed from a rather limited viewpoint with a focus on older systems. In addition, recent work often does not include LCSs at all. In the following, we summarize the main aspects we found—note how this contrasts the more general description of these systems we give in this chapter.

LCSs are considered as a special kind of production system [34, 80] or as methods for genetics-based machine learning (GBML) [62], evolutionary reinforcement learning (ERL) or evolutionary algorithms for reinforcement learnings (EARLs) [80], or policy optimization [48]. It is typically assumed that LCSs utilize a GA and build a set of if-then rules. Fitness is calculated based on individual rule statistics such as accuracy or strength [8, 62]. While it is unclear whether evolutionary algorithms (EAs) are the only approach qualified to be used in an LCS [62, 80], the importance of the solution representation and the applied metaheuristic operators are quite clear. In particular, the operators for the generational replacement and the mutation need to be adapted for usage in LCSs [34]. Still, GAs in LCSs are said to be similar to the ones applied to search and optimization problems [34].

3.3 ML Systems Similar to LCSs

LCS models, as presented in the previous section, have some similarities with other well-known ML frameworks. We now shortly review them informally[1] and the differences to LCSs in order to better highlight the LCS approach—especially in light of the rather loose definition of what constitutes an LCS and the many existing variants. To this end, we now differentiate LCSs from decision trees (DTs), mixture of experts (MoE) systems, genetic programming (GP) as well as the ensemble learning techniques *bagging* and *boosting*.

3.3.1 Decision Trees

Decision trees are amongst the most widely known rule-based systems. They are often applied to both regression and classification tasks and can be either induced from data automatedly or manually constructed by domain experts. A typical DT divides the input space into a set of disjoint regions; these are created by repeatedly splitting the input space axis-parallelly. The resulting hierarchy defines

[1] At the time of this writing, a more formal treatment of the differences is under active investigation.

a tree structure of which each path from the root to a leaf node corresponds to one region. For each such region, a submodel is fitted (and, by convention, stored in the corresponding leaf); for example, DTs for classification often simply use a constant function corresponding to the majority class of the training data lying in that region. Where to split the input space (i. e. in which dimension and at what value and at what level of the tree) poses an optimization problem that can be solved in many ways. Famous (and widely used) traditional algorithms to construct DTs include CART [15] and C4.5 [61]. Besides these and other related methods, metaheuristics for tree building have been investigated over the years as well: For example, [87] use ant colony optimization (ACO) to construct trees similarly to CART, [86] propose an EA-based hyperheuristic to design heuristics that then construct DTs and [89] introduce a GA that optimizes a population of DTs based on both their accuracy and size.

There is a direct correspondence between LCSs and DTs: Just like a path from the DT root to a leaf, a rule in an LCS specifies a region and a submodel. The only conceptual difference is that in LCSs, regions are allowed to overlap whereas they are not in classical DT. A DT can thus be transformed into an LCS model straightforwardly without losing information. The other direction can be done as well by extending an LCS model by a new region for each overlap of regions and then splitting regions further such that a proper hierachy among them is achieved. Since regions in LCS models may be placed arbitrarily, it can be expected that a rather large number of regions (and correspondingly, tree nodes) has to be added in order to get a proper DT out of an LCS model. Finally, note, that there are fuzzy DT approaches [10, 42] whose predictive models are very close if not the same as LCS models that use matching by degree—although the way that the model is fitted is fundamentally different.

3.3.2 Mixture of Experts

Mixture of experts is a research direction that developed independently of LCSs [41, 44, 85]. Nevertheless, the resulting models share a lot of similarity: The only differences are that, in MoE, usually,

1. a probabilistic view is taken which provides prediction distributions instead of the point estimates that LCSs models return,
2. submodels are not trained independently,
3. submodels are not localized using matching functions,
4. mixing weights are not constant but depend on the input.

This means that an MoE is more expressive than an LCS[2] since in the latter, localization (matching) essentially decides whether the submodel's output is multiplied by 0 or by a constant mixing weight whereas in MoE more than these two values can occur when mixing. On the other hand, LCSs are inherently more interpretable since binary decisions as the one mentioned before are much more comprehensible than decisions on somewhat arbitrary values. Note that MoE-like models whose mixing weights do not depend at all on the input are typically called *unconditional mixture models* [12]. Therefore, typical LCSs can be seen as in-between unconditional mixtures and MoE models in terms of both interpretability and expressiveness. Finally, independent submodel training inherently results in a slightly worse model performance in areas of submodel overlap [26]; thus an MoE can be expected to outperform an equivalent LCS. However, independent submodel training has two major advantages that may actually lead to an improved performance given the same amount of compute: Model structure search is much more efficient (e. g. when changing a single matching function, only the corresponding rule and no others have to be refitted) and, for some forms of submodels, no local optima during submodel fitting arise [26].

There are currently two MoE-inspired formulations of LCSs, one by Drugowitsch [26] and one by [88]. Drugowitsch [26] creates an LCS for regression and classification by adding matching to the standard MoE model. The resulting fully Bayesian probabilistic model is fitted using variational Bayesian inference and agnostic of the employed model structure search (Drugowitsch explores GAs as well as markov chain monte carlo (MCMC) methods). Other than in typical MoEs, the submodels are trained independently (like in an LCS), resulting in the dis-/advantages discussed in the previous paragraph. The resulting model is probabilistic, that is, it provides for any possible input a probability distribution over all possible outputs; to our knowledge, no other LCS is currently able to do this.

[88] take a less general approach and closely model UCS, an LCS for classification, using an MoE. They are able to provide a simpler training routine than Drugowitsch since their system is only capable of dealing with binary inputs (just like the original UCS) and they always model all possible rules. The latter makes training infeasible in high-dimensional space which is why, in a follow-up paper [27], they extend their system with a GA for performing model selection. They also add iterative learning to their model [27] which is not supported by Drugowitsch's system as of yet.

[2] At least as long as the LCS uses the typical binary matching functions. A matching-by-degree LCS can actually be more expressive than a comparable MoE.

3.3.3 Bagging and Boosting

The two most well-known ensemble learning techniques are bagging [14] and boosting [32].

Bagging [14] means generating several bootstrap data sets from the training data and then training on each of them one (weak) learner. In order to perform a prediction for a certain input, the predictions of all the available weak learners for that input are averaged (regression) or combined using majority voting (classification). This procedure has been shown to decrease prediction error if the weak learners have instabilities; for example, this is the case for DTs, neural networks (NNs) and RBML [24]. On first glance, the set of weak learners is somewhat akin to the set of submodels in LCSs. However, there are several major differences: Other than in bagging, in LCSs,

- no bootstrapping is performed to assign data points to the submodels; instead a good split of the data set is *learned* by performing model selection.
- the learned splits do not only influence training but also prediction.
- submodel predictions are mixed based on some quality measure and not using an unweighted average.

Boosting (e. g. the prominent AdaBoost algorithm [32]) trains a set of weak learners (submodels) sequentially and changes the error function used in training after each trained submodel based on the performance of the previous submodels. To perform predictions, the submodels are combined using weighted averaging (regression) or weighted majority voting (classification). LCSs differ from boosting by

- submodels being localized, they do not model all inputs.[3]
- submodels being trained independently of each other.
- directly optimizing the function that combines submodels. While Boosting's repeatedly changed error function is somewhat reminiscent to the interaction of matching and mixing in LCSs, matching and mixing are optimized more explicitly.

3.3.4 Genetic Programming

Another well-known symbolic method applicable to many different ML tasks is genetic programming [45], the evolution of syntax trees that represent functions or programs. When compared to LCSs, GP approaches typically have more degrees of

[3] If matching by degree is used (which is not the case for any of the promiment LCSs), they are softly localized. This means that inputs are somewhat weighted—which goes more in the same direction as boosting.

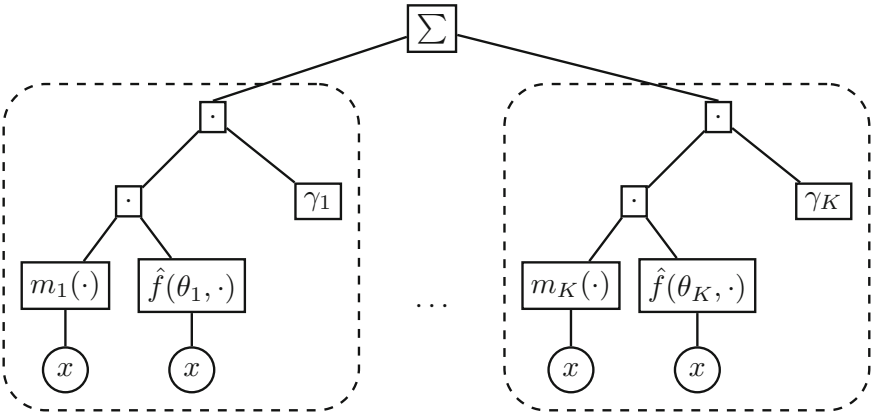


Fig. 3.1 Direct transformation of an LCS model (cf. Eq. (3.1)) to a GP tree. We summarized the nested summation of the submodel outputs as a single large sum. Input nodes are circular

freedom: While an LCS model can be transformed directly into a GP syntax tree (e. g. see Fig. 3.1), the other direction is not that straightforward since GP trees in general do not need to adhere to the general form of an LCS model. Other than in LCSs, there is no explicit generation of submodels in GP.

There are several approaches that use GP techniques in LCSs. For example, [40] use small GP trees (so-called *code fragments*) as conditions to more efficiently explore model structure space by reusing building blocks from previous training on subproblems.

3.4 The Role of Metaheuristics in LCSs

This section discusses the role of metaheuristics in LCS learning more in depth. Section 3.2 already established that metaheuristics are almost always used to solve the task of model selection, that is, subtasks (II) (choosing matching functions) and (I) (choosing an appropriate number of rules), where subtask (I) is sometimes approached heuristically. We now first introduce the options for the general structure of the metaheuristic process and then continue with analysing the used representations, operators and fitness functions.

3.4.1 Four Types of LCSs

In the early years of GBML, there emerged two "schools" with which many of the investigated systems have later been associated: the Michigan and Pittsburgh

approaches [22]. The classical definitions of these two terms are [22, 30]:

Pittsburgh-style systems are methods using a population-based metaheuristic (e. g. a GA) at the level of complete solutions to the learning tasks. They thus hold a *population of rule sets* whose conditions they diversify and intensify; the operators of the metaheuristic operate at the level of entire sets of conditions.

Michigan-style systems on the other hand consider a single solution (a single rule set) as a population on whose conditions a metaheuristic operates. The operators of the employed metaheuristic *operate on individual conditions*.

We deem this differentiation as slightly problematic. For one, there are many systems that cannot quite be sorted into one of the two classes (see, for example, the systems marked as "hybrid" in the list in [77]). Secondly, the two terms focus on population-based approaches. However, what about an approach that uses a metaheuristic such as simulated annealing (SA) for model structure search and gradient-based local optimization for model fitting? Since that metaheuristic works at the level of complete solutions to the problem, this may be seen as a Pittsburgh-style system. On the other hand, only a single solution is considered at any time and individual rules of that solution are changed by the metaheuristic—therefore it could also be called a Michigan-style system.

We thus propose a different distinction based on two major design decisions that greatly influence how exactly a certain LCS, or, more general, an RBML algorithm works:

Training data processing *Online* algorithms update their model after each data point whereas *batch* algorithms process the entire available data at once [12, p. 143].

Model structure search LCSs differ greatly if a single model structure is considered at a time or more than one. We thus introduce the differentiation between *single-model* and *multi-model* LCSs.

These two dimensions define four base types of RBML algorithms (online single-model, online multi-model, batch single-model and batch multi-model systems). The proposed terms also allow for gradual classification (e. g. mini-batch approaches lie somewhere between batch and online methods) and thus resolve the issue of having a category of "hybrid" systems that hold everything that does not fall into one of the categories. Finally, the terms also have a meaning outside of the RBML community which increases transparency and eases entry to the field.

Table 3.1 gives a quick overview of several well-known LCSs and how they relate to the two design decisions. Note that according to our definitions, most systems that were said to be Michigan-style belong to the class of *online single-solution systems* whereas most systems that were said to be Pittsburgh-style belong to the class of *batch multi-solution systems*. XCS and its derivatives [11, 82, 84] are online learners using a typical Michigan-style metaheuristic; they therefore fall into the category of online single-model methods. BioHEL [7] on the other hand performs batch learning but does only ever consider a single set of rules. GAssist [6, 29] is a classical Pittsburgh-style system that considers more than one set of rules at any

Table 3.1 Examples for the different types of LCS algorithms

	Online	Batch
Single-model	XCS(F) [82, 84], UCS [11]	BioHEL [7]
Multi-model		GAssist [6]

time and performs batch learning. Note that there currently is, to our knowledge, no well-known online multi-model LCSs.

Each of the classes has some unique advantages and disadvantages. Dividing model selection and model fitting is easier to achieve for batch algorithms; this makes these systems often simpler and more straightforward to analyse formally than online systems [26]. Particularly, existing population-based single-model systems (e. g. XCS, where the GA operates on individual rules) are often hard to analyse formally due to the rules both competing for a place in the population but also cooperating to form a good global model.

It can be argued that the overall training structure of batch multi-solution systems has a comparably high computational cost since for each model structure considered, the submodels are fitted until convergence—whereas online single-solution systems can get away with fitting the submodels iteratively until convergence only once. However, online single-solution systems perform fitting steps alternatingly with model selection steps which makes the comparison slightly unfair as this can be expected to amount to significantly more fitting steps required until convergence than if the model structure were held fixed. Indeed, given the enormous amount of training examples provided regularly to many state-of-the-art Michigan-style LCSs [47, 53, 68], the comparison may actually not be as unfavourable as it looks at first glance. However, this hypothesis has to our knowledge not yet been investigated in depth.

3.4.2 Metaheuristic Solution Representation

Metaheuristics optimize the set of matching functions and, at that, operate on the set of conditions (useful representations of matching functions, cf. Sect. 3.2).

Early LCSs were designed mostly for binary input domains and these systems are still among the most-used and researched ones [77]. Matching functions for these domains are typically represented by ternary strings, that is, binary strings extended by an additional symbol #, the so-called *wild card*, that represents any of the other two options and thus enables generalization. An example for the representation of a matching function $m : \{0, 1\}^5 \to \{0, 1\}$ is

$$(1, 1, 0, 1, \#). \tag{3.2}$$

It assigns 1 to (matches) the inputs $(1, 1, 0, 1, 0)$ and $(1, 1, 0, 1, 1)$ and 0 to any other inputs [76, 82].

For real-valued or mixed integer problem domains, many different representations have been proposed. Among the simplest that are commonly applied are hypercuboid [83] and hyperellipsoid [18] conditions. An example for a hypercuboid condition for a matching function $m : \mathbb{R}^3 \rightarrow \{0, 1\}$ is the 3-dimensional interval $[l, u)$ (with $l, u \in \mathbb{R}^3$) which can be seen as a tuple

$$(l_1, u_1, l_2, u_2, l_3, u_3). \tag{3.3}$$

It matches all $x = (x_1, x_2, x_3) \in \mathbb{R}^3$ that fulfill[4] $l \leqslant x < u$:

$$m(x) = \begin{cases} 1, & l \leqslant x < u \\ 0, & \text{otherwise} \end{cases} \tag{3.4}$$

Aside from the mentioned ones, several more complex function families have been proposed, e. g. neural networks [17] and GP-like code fragment graphs [40] (also referred to in Sect. 3.3.4).

In general, any representation is possible including composites or combinations of other representations as long as appropriate operators can be defined. It has to be kept in mind, however, that more complex representations often lead to more complex operators being required as well as that both the size and the topology of the search space is directly influenced by the representation (e. g. there are functions $[0, 1]^5 \rightarrow \{0, 1\}$ that cannot be represented by above-introduced representation of ternary strings of length 5). Furthermore, too simplified or restricted encodings of the feature space can result in parts of the matching functions space being inacessible. Variable length representations may alleviate some of these problems but can render the operator design more difficult [76].

3.4.3 Metaheuristic Operators

Metaheuristic operators need to not only be compatible with the chosen form of conditions but also perform well in optimizing the corresponding sets of matching functions. This can be a challenging task for ML practitioners without a strong metaheuristic background, in turn causing the same operators to be used repeatedly, regardless of whether they may actually be suboptimal.

An important aspect in LCSs is *when* the metaheuristic is invoked and whether it operates on the entire set of conditions or on a subset. For instance, in XCS [82], for each input provided to the system, the GA applies its operators only to the conditions of a subset of rules; namely to the ones that matched the input seen last and of those only the ones that also proposed the action taken. Due to the dependence on the

[4] This is equivalent to $(l_1 \leqslant x_1 < u_1) \wedge (l_2 \leqslant x_2 < u_2) \wedge (l_3 \leqslant x_3 < u_3)$.

input introduced by only considering matching and used rules, the GA may operate on a different subset in the very next iteration.

The initial set of conditions is usually created at random, possibly slightly directed by requiring the corresponding matching functions to match certain inputs (matching functions that do not match any of the training data may not be that useful since their merit cannot be estimated properly) [76, 82].

For the generation of new individuals from existing ones, existing LCSs use both recombination as well as mutation operators [76, 77]. Recombination operators in single-solution systems may exchange condition attributes (e. g. hypercuboid boundaries) whereas, in multi-solution systems, they probably should also include an option to recombine sets of conditions in a meaningful way (e. g. exchanging entire conditions between sets of condition).

Optimizing matching functions poses a difficult problem if the training data is sparse, or, more generally, if there are sparsely sampled parts of the input space: On the one hand, changing a condition only leads to a detectable difference in accuracy-based fitness if that change alters the subset of the training data that is matched by the corresponding matching function. On the other hand, these differences, if occurring, can be very large (e. g. if a rule now matches three training examples while, before, it only had matched two). This means that, depending on the training data and initialization, the operators may have a *low locality* if fitness computation only takes into account accuracy statistics on the training data. As a result, areas between training data points may not be covered by solution candidates because there is no fitness signal when exploring having some rules match them. Choosing a combination of suitable operators and a fitness measure that present a consistent answer to this issue is an open problem; the common workaround is to simply rely on comparably large amounts of training data.

While multi-solution systems can explore different solution sizes rather naturally, single-solution systems require explicit mechanisms to control and optimize condition/rule set size. A popular option for population-based single-solutions (used, e. g., in XCS [82]) is a simple heuristic: There is a maximum number of rules (a hyperparameter) that, when violated by rule generation mechanisms, gets enforced by deleting rules based on roulette wheel or tournament selection. Aside from that, the *numerosity* mechanism is typically employed (also used, e. g., in XCS [82]): If there is a well-performing rule r_1 that is responsible for more inputs than another rule r_2 but the inputs matched by r_2 are already covered by r_1, then r_2 may be replaced by a copy of r_1 (typically, no real copies are used but instead a count associated with each rule in the set). The set of conditions thus contains one less unique condition.

In existing online single-solution systems, metaheuristic operators gradually change parts of conditions of the rule set in a steady-state fashion. At that, the central challenges are maintaining a healthy diversity and identifying the required niches—and keeping them in the rule set [76]. Selection is usually based on either roulette wheel or tournament selection from either the whole set of rules (esp. in earlier LCSs [77]) or subsets (e. g. in XCS [82]), the latter promoting niching. Niching is also promoted when performing rule discovery for examples that are not yet

matched by any rule in the set. The mutation operator modifies a single condition; how this occurs primarily depends on the specific representation but is typically stochastic while balancing generalization and specialization pressures. Commonly used operators include bitflip [82] and Gaussian mutation [26]. Recombination also works at the condition level and is usually a single-point, two-point or uniform crossover with encoding-dependent crossover points. The replacement of rules usually employs elitism operators. [76]

Batch multi-solution systems are more similar to other well known optimization approaches [76]. Most existing systems of this category are generational rather than steady-state. Unlike in existing online single-solution systems, parents are condition sets and not individual conditions and are selected from a population of condition sets, typically using roulette wheel or (primarily in later systems) tournament selection. The mutation operator mutates at two levels: at the level of condition sets by adding and removing rules, as well as at the level of individual conditions using a method appropriate for the rule representation, for example, Gaussian mutations of all bounds when using an interval-based condition. The recombination operator mostly exchanges rules between rule sets but can also be extended to additionally exchange parts of individual conditions [6].

3.4.4 Typical Fitness Functions

As already noted in Sect. 3.2.3, the problem that an LCS's metaheuristic tries to solve is model structure selection, that is, choosing the size of the rule set (subtask (I)) and proper matching functions (subtask (II)). The goal of this optimization task is to enable the model to be optimal after it has been fitted. At that, optimality of the model, and with it, optimality of the model selection, is typically defined slightly handwavy based on the fitness measure used for model selection. That fitness measure commonly weighs a high accuracy of overall system predictions against a low model structure complexity (i. e. number of rules) [6, 82]. However, there are also lesser-known, but more principled approaches to what an LCS is meant to learn that we cannot expand on here for the sake of brevity; for example, the one by Drugowitsch [26] based on Bayesian model selection.

The need for high accuracy and a low number of rules induces a multi-objective optimization problem with conflicting goals (the highest accuracy can consistently be achieved with a very high number of rules, e. g., one rule per training example). However, the utilized fitness functions are not always modelled explicitly multi-objectively but often use a (weighted) sum of the objectives—or even focus on only one of them. The exact fitness computation within the system strongly influences the metaheuristic to be used; therefore, we will shortly describe the different options and their implications.

Online single-model systems usually incorporate niching and thus require mechanisms for fitness sharing. In earlier LCSs for RL there was an *explicit fitness sharing* mechanism that split the reward among all rules in the same, activated

niche [39, 81]. The more generally applicable technique is *implicit fitness sharing* which is based on computing the fitness relative to the rules in the same niche and applying metaheuristic operators only within that niche [82]. The fitness functions are usually to be maximized and are often based on either rule strength [81] or rule accuracy [82]. Strength-based fitness is often used in earlier LCSs built for RL settings; it builds on the sum of RL rewards after applying the rule. Accuracy-based fitness, on the other hand, is based on the frequency of the rule's correct predictions; and, due to its increased stability much more common as of now.

In batch multi-model systems, purely accuracy-based fitness functions can result in bloating [26], that is, a significant amount of additional rules being included in the rule set that do not improve the model. To resolve this issue, multi-objective fitness functions with the second objective being the reduction of rule set size are used. They are often modelled as weighted sums of the individual objectives and thus still remaining a single-objective problem. One example for this is the utilization of the minimum description length (MDL) principle for the fitness function in BioHEL [7] and GAssist [6]. MDL is also a common strategy in optimization for feature selection problems.

3.5 Other Metaheuristics-Centric Rule-Based Machine Learning Systems

We next discuss the similarity of LCSs to other metaheuristics-focussed RBML systems, that is, other systems utilizing GAs, ACO, other metaheuristics or hybrids, and artificial immune systems (AISs). These were developed independently of LCSs but are often based on the same ideas (e. g. [39]) or on the work of [30] which summarizes genetic approaches for data mining. Furthermore, they are used to construct if-then rules, mostly applied to classification tasks, and often divided into Michigan and Pittsburgh approaches [30]. The main difference between these metaheuristics-focussed RBML systems and LCSs is that most of them do not utilize any additional bookkeeping parameters.

The list of learning systems in this section is not exhaustive, but aims at providing a broader view with some short examples. It is important to note that we restrict ourselves to systems

- that are more or less close to the definition of an LCS given in Sect. 3.2 but whose authors do not relate them to existing LCS research (or do not explicitly call them LCSs)
- that somewhat lie at the very border of the field of LCS research and may thus not be known well.

While LCSs are commonly associated with evolutionary (or more specifically genetic) algorithms, any metaheuristic can be used [62, 76]. This makes the comparison of these other metaheuristics-focussed RBML systems even more

relevant, as algorithms from both fields can profit from the respective other field's research.

3.5.1 Approaches Based on Evolutionary Algorithms

The degree of resemblance to LCSs is most obvious for approaches which also utilize EAs and of which a non-exhaustive overview is given in this section. Most of these approaches, for example the general description of GAs for data mining and rule discovery by [30], have been proposed with—if at all—only little differentiation or comparison to LCSs.

Approaches utilizing GAs for the discovery and optimization of classification rules are for example described in [1, 21, 50, 51, 79]. These are mostly subsumed under the definition of Michigan-style systems, though many of them are actually batch single-model systems. They differ from LCSs in terms of the operators used in the GA, the fitness computation, or the use of additional strategies. For example, [50] extended their RBML system to perform multi-label classification, while [21] utilize a parallel GA in their approach. A classification rule mining system using a multi-objective GA (MOGA) is presented by [35]. There are also approaches using evolutionary techniques such as co-evolution, which is applied to sets of examples, the rules being induced at the end [43]. This presents an inverse order of the process compared to the traditional LCS approach. Furthermore, other EAs can be used for rule discovery, for example a quantum-inspired differential evolution algorithm [71].

While there often is no direct relation provided between other evolutionary RBML systems and LCSs, at least some summaries describe a few of the different approaches [31] or provide experimental comparisons [74]. An exception is the combination of DTs and a GA by [64], which includes two rule inducing phases (a decision tree produces rules, the GA refines these rules) and which is simultaneously described as a Michigan-style LCS with three phases.

3.5.2 Approaches Based on the Ant System

Another branch of approaches for (classification) rule discovery is based on the ant system, or Ant Colony Optimization (ACO), with the Ant-Miner as the most prominent representative. Their similarity to LCSs is strongly dependent on the variant of LCS and on how much the utilized metaheuristic is seen as a defining component. For example, the Ant-Miner [58] is a batch single-solution system with an overall concept similar to BioHEL [7]. Its pheromone table is similar to the attribute tracking concept in ExSTraCS [76]. Furthermore, it uses the same rule representation strategies as LCSs in general, with the exception that continuous variables are more often discretized in the Ant-Miner. Nevertheless, the ACO

algorithm [25] is quite dissimilar from GAs and the resulting RBML systems can exhibit further differences.

The Ant-Miner develops if-then rules whose condition consists of a concatenation of terms (i. e. *attribute*, *operator*, and *value*). Rules are constructed by probabilistically adding terms to an empty rule under utilization of a problem-dependent heuristic function and the ACO-typical pheromone table. Afterwards, the rule is pruned and the pheromone table is updated and the next rule is constructed. This process is repeated until the maximum population size (number of ants) is reached or the same rule has been constructed more than once. Then, the best rule is selected and the data points it matches and correctly classifies are removed from the training set. The overall algorithm is repeated until enough cases in the training set are covered by the aggregated rule set [58].

There are several extensions and variants for the Ant-Miner [3, 13], for example, different pheromone update functions or heuristic functions and adaptations to cope with continuous data [46, 56]. Furthermore, there also exists a regression rule miner based on Ant-Miner [16] and a batch multi-solution Ant-Miner variant [57]. Also, other ACO-based classifier systems have been developed simultaneously to Ant-Miner [65].

3.5.3 Approaches Based on Other Metaheuristics or Hybrids

Next to GAs and ACO, there are many more metaheuristics and hybrid algorithms that can be utilized in RBML, especially for classification rule mining [23]. They share roughly the same basic view on rules as well as a classification into Michigan- and Pittsburgh-style approaches, although the term Michigan-style often subsumes both online and batch single-model systems. Again, their similarity to LCSs depends strongly on the respective variants and the underlying definitions but a direct integration into existing LCS research is often not provided. While this section can not present these approaches exhaustively, it showcases further insights on how metaheuristics can be applied to RBML.

Particle swarm optimizations (PSOs), for example, has been used for classification rule discovery [66, 67] as well as for a regression rule miner [49]. Furthermore, the artificial chemical reaction optimization algorithm (ACROA) was used to optimize classification rules as well [2]. While these approaches all use population-based metaheuristics, it is not impossible (or infeasible) to use single-solution based optimizers, as was demonstrated in [52] where SA determines fuzzy rules for classification. This SA variant was also extensively compared to the LCSs GAssist and XCSTS.

Hybrid approaches, that is, algorithms combining two different metaheuristics to combine their benefits, are common to classification rule discovery as well. They are, again, often presented as Michigan-style systems; however, many of them perform batch single-model learning. There exist, for example, hybrids of PSO and ACO [37, 38], SA and tabu search (TS) [20], and ACO and SA [63]. Some of these

hybrids explicitly divide their rule discovery process into two phases; this is the case, for example, for the HColonies algorithm, a combination of AntMiner+ and artificial bee colony (ABC) optimization [4]. Additionally, batch multi-model hybrids are possible, as presented by an Ant-Miner-SA combination [54].

Another type of hybrid systems for classification rule mining combines not only two metaheuristics, but utilizes them in what the authors call a Michigan-style phase and a subsequent Pittsburgh-style phase. In our new classification system, these would simply fall into the batch multi-model category. For example, [73] use a combination of a GA and GP, depending on the types of attributes, in the Michigan phase to generate a pool of rules and then perform a Pittsburgh-style optimization with a GA in the second phase to evolve the best rule set from the pool. Similarly, [5] use a hybrid of ACO and a GA. AntMiner+ is used in the first phase to construct several models based on different subsets of the training data, while the GA uses these models as an initial population for optimization. This approach utilizes the smart crossover developed for Pittsburgh-style LCSs, which is an indication for at least some overlap between the two research communities.

3.5.4 Artificial Immune Systems

Artificial immune systems (AISs) are another class of algorithms inspired by biological processes and suitable for ML and optimization tasks. AISs are differentiated by the general strategies they employ, that is, clonal selection theory, immune network theory, negative selection and danger theory. [36, 62, 75]

First of all, the similarity of AIS algorithms and LCSs depends strongly on the strategy. Clonal selection– and negative selection–based AISs are more similar to evolutionary RBML systems than immune network or danger theory AISs. Furthermore, both AISs and LCSs entail many variants, depending on the learning task and implementation choices. At that, for example, solution encoding and operator choice further increases or decreases the similarity between these approaches. Finally, note that AISs research often acknowledges the similarities and differences to LCSs [28, 33].

3.6 Future Work

As demonstrated in the previous sections, LCS research has mostly focussed on the ML side and neglected its metaheuristics to some extent. Nevertheless, there still are shortcomings in terms of the learning aspect: For instance, LCSs continue to show inferior performance to many other systems when it comes to non-trivial RL, although they were originally designed for exactly this kind of task [26, 69]. Their learning performance often depends on a large number of semantically very different and non-trivially interacting system parameters [26]; this complicates

utilization of these systems and often necessitates parameter tuning. Furthermore, there are close to no formal guarantees for most of the popular LCSs, mainly due to the ad-hoc approach taken for their development [26]. As a result, the assumptions these algorithms make about the data are slighly obscure, as are the algorithms' objectives resulting in uncertainty about which learning tasks LCSs are suited for best.

To remedy these shortcomings is one important direction for future research on LCSs. However, questions relating to the metaheuristics side should not be neglected any longer either. This includes starting to utilize state-of-the-art metaheuristic knowledge and techniques for optimization, but also and probably even more understanding the relationship between the problems of LCS model fitting and LCS model selection. An open question is, for example, how the nature of the model fitting problem in LCSs relates to the nature of the corresponding model selection problem and therefore what metaheuristic to use. This begins at picking representations and operators that perform well, and translates to the usage of matching functions and submodels.

3.7 Conclusion

We presented Learning Classifier Systems (LCSs), a family of rule-based machine learning systems. They construct models of overlapping if-then rules from data. Construction of an LCS's model involves two main optimization tasks that each consist of two subtasks. To perform *model selection* LCSs employ metaheuristics to optimize the resulting model's *rules' conditions* (i. e. when a certain rule applies) and often also to determine an *appropriate number of rules* to be used. The task of *model fitting* entails optimizing each of the *rules' submodels* so it fits the data that fulfills the rule's condition; this is typically done non-metaheuristically. In situations where multiple rules' conditions match a certain input a *mixing strategy* is used to determine the model's prediction based on each rule's prediction.

Furthermore, we discussed the similarities and differences of LCSs and the related machine learning techniques of Genetic Programming, Decision Trees, Mixtures of Experts, Bagging and Boosting. We found that these techniques are in fact unique but can often be combined with LCSs or transformed into LCS models by making or dropping assumptions. However, bidirectional transformation is non-trivial.

We introduced a new nomenclature to differentiate LCSs into four distinct types that supersedes the earlier classification into Michigan-style and Pittsburgh-style systems. We categorize them based on their *training data processing* scheme into *online* and *batch* learning and based on whether *model structure search* involves a *single-model* or a *multi-model* approach.

We then focussed on how metaheuristics for model selection are commonly designed: Firstly, discussing various rule condition *representations* in relation to input domains and operators. Secondly, giving an overview about how and when

operators are usually applied, stressing issues such as locality and niching, and which operators are typically used in existing literature, especially with regards to the distinct types of LCSs. Lastly, detailing *fitness function* requirements and typical representatives.

Additionally, we summarized *other metaheuristics-centric rule-based machine learning systems*. Although these systems are not referred to by their authors as LCSs, according to the definitions presented in this chapter, they may actually qualify as such. The most well-known of these systems is probably Ant-Miner which optimizes rule sets using Ant Colony Optimization and whose research community even uses some of the terminology typical to the field of LCS such as distinguishing between Michigan-style and Pittsburgh-style systems.

Overall, we find that both the general metaheuristics community, as well as the LCS community do overlap and are researching similar issues but do not interact enough. Metaheuristics researchers rarely consider the optimization task found in an LCS as an application of their algorithms and LCS researchers often stick to more basic metaheuristics and—especially in the case of those systems that are referred to by their authors as LCSs—solely consider the application of genetic or evolutionary algorithms. Contrastingly, we are certain that *interaction and cooperation* can only benefit the advancement of both fields and that many exciting research questions remain to be answered.

References

1. Basheer M. Al-Maqaleh and Hamid Shahbazkia. A Genetic Algorithm for Discovering Classification Rules in Data Mining. *International Journal of Computer Applications*, 41(18):40–44, mar 2012.
2. Bilal Alatas. A novel chemistry based metaheuristic optimization method for mining of classification rules. *Expert Systems with Applications*, 39(12):11080–11088, sep 2012.
3. Zulfiqar Ali and Waseem Shahzad. Comparative Analysis and Survey of Ant Colony Optimization based Rule Miners. *International Journal of Advanced Computer Science and Applications*, 8(1), 2017.
4. Sarab AlMuhaideb and Mohamed El Bachir Menai. HColonies: a new hybrid metaheuristic for medical data classification. *Applied Intelligence*, 41(1):282–298, feb 2014.
5. Sarab AlMuhaideb and Mohamed El Bachir Menai. A new hybrid metaheuristic for medical data classification. *International Journal of Metaheuristics*, 3(1):59, 2014.
6. Jaume Bacardit. *Pittsburgh genetics-based machine learning in the data mining era: representations, generalization, and run-time*. PhD thesis, PhD thesis, Ramon Llull University, Barcelona, 2004.
7. Jaume Bacardit and Natalio Krasnogor. *BioHEL: Bioinformatics-oriented Hierarchical Evolutionary Learning*. 2006.
8. Thomas Bäck, D. B. Fogel, and Z. Michalewicz, editors. *Handbook of Evolutionary Computation*. CRC Press, jan 1997.
9. Alwyn Barry. The stability of long action chains in XCS. *Soft Comput.*, 6(3–4):183–199, 2002.
10. Marco Barsacchi, Alessio Bechini, and Francesco Marcelloni. An analysis of boosted ensembles of binary fuzzy decision trees. *Expert Systems with Applications*, 154, 2020.

11. Ester Bernadó-Mansilla and Josep M. Garrell-Guiu. Accuracy-Based Learning Classifier Systems: Models, Analysis and Applications to Classification Tasks. *Evolutionary Computation*, 11(3):209–238, 09 2003.
12. Christopher M. Bishop. *Pattern recognition and machine learning, 8th Edition*. Information science and statistics. Springer, 2009.
13. Urszula Boryczka and Jan Kozak. New Algorithms for Generation Decision Trees—Ant-Miner and Its Modifications. In *Studies in Computational Intelligence*, pages 229–262. Springer Berlin Heidelberg, 2009.
14. Leo Breiman. Bagging predictors. *Mach. Learn.*, 24(2):123–140, 1996.
15. Leo Breiman, Jerome H. Friedman, Richard A. Olshen, and Charles J. Stone. *Classification And Regression Trees*. Routledge, October 1984.
16. James Brookhouse and Fernando E. B. Otero. Discovering Regression Rules with Ant Colony Optimization. In *Proceedings of the Companion Publication of the 2015 Annual Conference on Genetic and Evolutionary Computation*. ACM, jul 2015.
17. Larry Bull and Jacob Hurst. A neural learning classifier system with self-adaptive constructivism. In *The 2003 Congress on Evolutionary Computation, 2003. CEC '03.*, volume 2, pages 991–997, 2003.
18. Martin V. Butz, Pier Luca Lanzi, and Stewart W. Wilson. Function approximation with XCS: Hyperellipsoidal conditions, recursive least squares, and compaction. *IEEE Transactions on Evolutionary Computation*, 12(3):355–376, 2008.
19. Martin V. Butz and Wolfgang Stolzmann. An algorithmic description of ACS2. In Pier Luca Lanzi, Wolfgang Stolzmann, and Stewart W. Wilson, editors, *Advances in Learning Classifier Systems*, pages 211–229, Berlin, Heidelberg, 2002. Springer Berlin Heidelberg.
20. Ivan Chorbev, Boban Joksimoski, and Dragan Mihajlov. SA Tabu Miner: A hybrid heuristic algorithm for rule induction. *Intelligent Decision Technologies*, 6:265–271, 2012.
21. Dieferson L. Alves de Araujo, Heitor S. Lopes, and Alex A. Freitas. A parallel genetic algorithm for rule discovery in large databases. In *IEEE SMC'99 Conference Proceedings. 1999 IEEE International Conference on Systems, Man, and Cybernetics (Cat. No.99CH37028)*. IEEE, 1999.
22. Kenneth DeJong. Learning with genetic algorithms: An overview. *Mach. Learn.*, 3:121–138, 1988.
23. Clarisse Dhaenens and Laetitia Jourdan. Metaheuristics for data mining. *4OR*, 17(2):115–139, apr 2019.
24. Thomas G. Dietterich. Ensemble methods in machine learning. In *Multiple Classifier Systems*, pages 1–15, Berlin, Heidelberg, 2000. Springer Berlin Heidelberg.
25. Marco Dorigo and Thomas Stützle. *Ant colony optimization*. MIT Press, Cambridge, Mass, 2004.
26. Jan Drugowitsch. *Design and Analysis of Learning Classifier Systems - A Probabilistic Approach*, volume 139 of *Studies in Computational Intelligence*. Springer Berlin Heidelberg, Berlin, Heidelberg, 2008.
27. Narayanan Unny Edakunni, Gavin Brown, and Tim Kovacs. Online, GA based mixture of experts: a probabilistic model of UCS. In Natalio Krasnogor and Pier Luca Lanzi, editors, *13th Annual Genetic and Evolutionary Computation Conference, GECCO 2011, Proceedings, Dublin, Ireland, July 12–16, 2011*, pages 1267–1274, New York, NY, USA, 2011. ACM.
28. J. Doyne Farmer, Norman H. Packard, and Alan S. Perelson. The immune system, adaptation, and machine learning. *Physica D: Nonlinear Phenomena*, 22(1–3):187–204, oct 1986.
29. María A. Franco, Natalio Krasnogor, and Jaume Bacardit. GAssist vs. BioHEL: critical assessment of two paradigms of genetics-based machine learning. *Soft Comput.*, 17(6):953–981, 2013.
30. Alex A. Freitas. *Data Mining and Knowledge Discovery with Evolutionary Algorithms*. Springer Berlin Heidelberg, 2002.
31. Alex A. Freitas. A Survey of Evolutionary Algorithms for Data Mining and Knowledge Discovery. In *Natural Computing Series*, pages 819–845. Springer Berlin Heidelberg, 2003.

32. Yoav Freund and Robert E. Schapire. Experiments with a new boosting algorithm. In Lorenza Saitta, editor, *Machine Learning, Proceedings of the Thirteenth International Conference (ICML '96), Bari, Italy, July 3–6, 1996*, pages 148–156. Morgan Kaufmann, 1996.
33. Simon M. Garrett. How Do We Evaluate Artificial Immune Systems? *Evolutionary Computation*, 13(2):145–177, jun 2005.
34. David Goldberg. *Genetic algorithms in search, optimization, and machine learning*. Addison-Wesley Publishing Company, Reading, Mass, 1989.
35. Preeti Gupta, Tarun Kumar Sharma, Deepti Mehrotra, and Ajith Abraham. Knowledge building through optimized classification rule set generation using genetic based elitist multi objective approach. *Neural Computing and Applications*, 31(S2):845–855, may 2017.
36. Emma Hart and Jon Timmis. Application areas of AIS: The past, the present and the future. *Applied Soft Computing*, 8(1):191–201, jan 2008.
37. Nicholas Holden and Alex A. Freitas. A hybrid particle swarm/ant colony algorithm for the classification of hierarchical biological data. In *Proceedings 2005 IEEE Swarm Intelligence Symposium, 2005. SIS 2005.* IEEE, 2005.
38. Nicholas Holden and Alex A. Freitas. A hybrid PSO/ACO algorithm for discovering classification rules in data mining. *Journal of Artificial Evolution and Applications*, 2008:1–11, may 2008.
39. John H. Holland. Adaptation. *Progress in theoretical biology*, 4:263–293, 1976.
40. Muhammad Iqbal, Will N. Browne, and Mengjie Zhang. Reusing building blocks of extracted knowledge to solve complex, large-scale boolean problems. *IEEE Transactions on Evolutionary Computation*, 18(4):465–480, 2014.
41. Robert A. Jacobs, Michael I. Jordan, Steven J. Nowlan, and Geoffrey E. Hinton. Adaptive mixtures of local experts. *Neural Computation*, 3(1):79–87, 1991.
42. Cezary Z. Janikow. Fuzzy decision trees: issues and methods. *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)*, 28(1):1–14, 1998.
43. Licheng Jiao, Jing Liu, and Weicai Zhong. An organizational coevolutionary algorithm for classification. *IEEE Transactions on Evolutionary Computation*, 10(1):67–80, feb 2006.
44. Michael I. Jordan and Robert A. Jacobs. Hierarchical mixtures of experts and the EM algorithm. *Neural Computation*, 6(2):181–214, 1994.
45. John R. Koza. *Genetic programming - on the programming of computers by means of natural selection*. Complex adaptive systems. MIT Press, 1993.
46. Bo Liu, H. A. Abbas, and B. McKay. Classification rule discovery with ant colony optimization. In *IEEE/WIC International Conference on Intelligent Agent Technology, 2003. IAT 2003.* IEEE Comput. Soc, 2003.
47. Yi Liu, Will N. Browne, and Bing Xue. Absumption and subsumption based learning classifier systems. In *Proceedings of the 2020 Genetic and Evolutionary Computation Conference*, GECCO '20, pages 368–376, New York, NY, USA, 2020. Association for Computing Machinery.
48. Sean Luke. *Essentials of metaheuristics: a set of undergraduate lecture notes*. Lulu Com, 2013.
49. Bart Minnaert and David Martens. Towards a Particle Swarm Optimization-Based Regression Rule Miner. In *2012 IEEE 12th International Conference on Data Mining Workshops*. IEEE, dec 2012.
50. Thiago Zafalon Miranda, Diorge Brognara Sardinha, Mrcio Porto Basgalupp, Yaochu Jin, and Ricardo Cerri. Generation of Consistent Sets of Multi-Label Classification Rules with a Multi-Objective Evolutionary Algorithm, March 2020. https://arXiv.org/abs/2003.12526.
51. Thiago Zafalon Miranda, Diorge Brognara Sardinha, and Ricardo Cerri. Preventing the Generation of Inconsistent Sets of Classification Rules, August 2019. https://arXiv.org/abs/1908.09652.
52. Hamid Mohamadi, Jafar Habibi, Mohammad Saniee Abadeh, and Hamid Saadi. Data mining with a simulated annealing based fuzzy classification system. *Pattern Recognition*, 41(5):1824–1833, may 2008.

53. Masaya Nakata, Will N. Browne, Tomoki Hamagami, and Keiki Takadama. Theoretical XCS parameter settings of learning accurate classifiers. In Peter A. N. Bosman, editor, *Proceedings of the Genetic and Evolutionary Computation Conference 2017*, GECCO '17, pages 473–480, New York, NY, USA, 2017. ACM.
54. Bijaya Kumar Nanda and Satchidananda Dehuri. Ant Miner: A Hybrid Pittsburgh Style Classification Rule Mining Algorithm. *International Journal of Artificial Intelligence and Machine Learning*, 10(1):45–59, jan 2020.
55. Romain Orhand, Anne Jeannin-Girardon, Pierre Parrend, and Pierre Collet. PEPACS Integrating probability-enhanced predictions to acs2. GECCO '20, New York, NY, USA, 2020. Association for Computing Machinery.
56. Fernando E. B. Otero, Alex A. Freitas, and Colin G. Johnson. cAnt-Miner: An Ant Colony Classification Algorithm to Cope with Continuous Attributes. In *Ant Colony Optimization and Swarm Intelligence*, pages 48–59. Springer Berlin Heidelberg, 2008.
57. Fernando E. B. Otero, Alex A. Freitas, and Colin G. Johnson. A New Sequential Covering Strategy for Inducing Classification Rules With Ant Colony Algorithms. *IEEE Transactions on Evolutionary Computation*, 17(1):64–76, feb 2013.
58. Rafael S. Parpinelli, Heitor S. Lopes, and Alex A. Freitas. An Ant Colony Algorithm for Classification Rule Discovery. In *Data Mining*, pages 191–208. IGI Global, 2002.
59. David Pätzel, Michael Heider, and Alexander R. M. Wagner. An overview of LCS research from 2020 to 2021. In *Proceedings of the Genetic and Evolutionary Computation Conference Companion*, GECCO '21, pages 1648–1656, New York, NY, USA, 2021. Association for Computing Machinery.
60. David Pätzel, Anthony Stein, and Masaya Nakata. An overview of LCS research from IWLCS 2019 to 2020. In *Proceedings of the 2020 Genetic and Evolutionary Computation Conference Companion*, GECCO '20, pages 1782–1788, New York, NY, USA, 2020. Association for Computing Machinery.
61. J. Ross Quinlan. *C4.5: Programs for Machine Learning*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 1993.
62. Grzegorz Rozenberg, Thomas Bäck, and Joost N. Kok, editors. *Handbook of Natural Computing*. Springer Berlin Heidelberg, 2012.
63. Rizauddin Saian and Ku Ruhana Ku-Mahamud. Hybrid Ant Colony Optimization and Simulated Annealing for Rule Induction. In *2011 UKSim 5th European Symposium on Computer Modeling and Simulation*. IEEE, nov 2011.
64. Bikash Kanti Sarkar, Shib Sankar Sana, and Kripasindhu Chaudhuri. A genetic algorithm-based rule extraction system. *Applied Soft Computing*, 12(1):238–254, jan 2012.
65. P. S. Shelokar, V. K. Jayaraman, and B. D. Kulkarni. An ant colony classifier system: application to some process engineering problems. *Computers & Chemical Engineering*, 28(9):1577–1584, aug 2004.
66. Tiago Sousa, Arlindo Silva, and Ana Neves. A Particle Swarm Data Miner. In *Progress in Artificial Intelligence*, pages 43–53. Springer Berlin Heidelberg, 2003.
67. Tiago Sousa, Arlindo Silva, and Ana Neves. Particle swarm based data mining algorithms for classification tasks. *Parallel Computing*, 30(5–6):767–783, may 2004.
68. Anthony Stein. *Interpolation-Assisted Evolutionary Rule-Based Machine Learning - Strategies to Counter Knowledge Gaps in XCS-Based Self-Learning Adaptive Systems*. doctoralthesis, Universität Augsburg, 2019.
69. Anthony Stein, Roland Maier, Lukas Rosenbauer, and Jörg Hähner. XCS classifier system with experience replay. In *Proceedings of the 2020 Genetic and Evolutionary Computation Conference*, GECCO '20, pages 404–413, New York, NY, USA, 2020. Association for Computing Machinery.
70. Anthony Stein, Simon Menssen, and Jörg Hähner. What about interpolation? a radial basis function approach to classifier prediction modeling in XCSF. In *Proceedings of the Genetic and Evolutionary Computation Conference*, GECCO '18, pages 537–544, New York, NY, USA, 2018. Association for Computing Machinery.

71. Haijun Su, Yupu Yang, and Liang Zhao. Classification rule discovery with DE/QDE algorithm. *Expert Systems with Applications*, 37(2):1216–1222, mar 2010.
72. Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction. Second Edition*. MIT Press, Cambridge, MA, 2018.
73. K. C. Tan, Q. Yu, C. M. Heng, and T. H. Lee. Evolutionary computing for knowledge discovery in medical diagnosis. *Artificial Intelligence in Medicine*, 27(2):129–154, feb 2003.
74. Ajay Kumar Tanwani and Muddassar Farooq. Classification Potential vs. Classification Accuracy: A Comprehensive Study of Evolutionary Algorithms with Biomedical Datasets. In *Lecture Notes in Computer Science*, pages 127–144. Springer Berlin Heidelberg, 2010.
75. Jon Timmis, Paul Andrews, Nick Owens, and Ed Clark. An interdisciplinary perspective on artificial immune systems. *Evolutionary Intelligence*, 1(1):5–26, jan 2008.
76. Ryan J. Urbanowicz and Will N. Browne. *Introduction to Learning Classifier Systems*. Springer Briefs in Intelligent Systems. Springer, 2017.
77. Ryan J. Urbanowicz and Jason H. Moore. Learning classifier systems: A complete introduction, review, and roadmap. *Journal of Artificial Evolution and Applications*, 2009, 2009.
78. Ryan J. Urbanowicz and Jason H. Moore. ExSTraCS 2.0: description and evaluation of a scalable learning classifier system. *Evolutionary Intelligence*, 8(2):89–116, Sep 2015.
79. A. H. C. van Kampen, Z. Ramadan, M. Mulholland, D. B. Hibbert, and L. M. C. Buydens. Learning classification rules from an ion chromatography database using a genetic based classifier system. *Analytica Chimica Acta*, 344(1–2):1–15, may 1997.
80. Thomas Weise. *Global optimization algorithms-theory and application*. Self-Published Thomas Weise, 2009.
81. Stewart W. Wilson. ZCS: A zeroth level classifier system. *Evolutionary Computation*, 2(1):1–18, 1994.
82. Stewart W. Wilson. Classifier fitness based on accuracy. *Evolutionary Computation*, 3(2):149–175, 1995.
83. Stewart W. Wilson. Get real! xcs with continuous-valued inputs. In Pier Luca Lanzi, Wolfgang Stolzmann, and Stewart W. Wilson, editors, *Learning Classifier Systems*, pages 209–219, Berlin, Heidelberg, 2000. Springer Berlin Heidelberg.
84. Stewart W. Wilson. Classifiers that approximate functions. *Natural Computing*, 1(2):211–234, 6 2002.
85. Seniha Esen Yuksel, Joseph N. Wilson, and Paul D. Gader. Twenty years of mixture of experts. *IEEE Transactions on Neural Networks and Learning Systems*, 23(8):1177–1193, 2012.
86. Rodrigo C. Barros, Márcio P. Basgalupp, André C.P.L.F. de Carvalho, and Alex A. Freitas. A hyper-heuristic evolutionary algorithm for automatically designing decision tree algorithms. In *Proceedings of the Fourteenth International Conference on Genetic and Evolutionary Computation - GECCO '12*. ACM Press, 2012.
87. Urszula Boryczka and Jan Kozak. Ant Colony Decision Trees – A New Method for Constructing Decision Trees Based on Ant Colony Optimization. In *Computational Collective Intelligence. Technologies and Applications*, pages 373–382. Springer Berlin Heidelberg, 2010.
88. Narayanan Unny Edakunni, Tim Kovacs, Gavin Brown, and James A. R. Marshall. Modeling ucs as a mixture of experts. In *Proceedings of the 11th Annual Conference on Genetic and Evolutionary Computation*, GECCO '09, pages 1187–1194, New York, NY, USA, 2009. ACM.
89. Vili Podgorelec, Matej Šprogar, and Sandi Pohorec. Evolutionary design of decision trees. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, 3(2):63–82, 2012.
90. Kreangsak Tamee, Larry Bull, and Ouen Pinngern. Towards clustering with XCS. In *Proceedings of the 9th Annual Conference on Genetic and Evolutionary Computation*, GECCO '07, pages 1854–1860, New York, NY, USA, 2007. Association for Computing Machinery.

Part II
Metaheuristics for Machine Learning: Applications

Chapter 4
Metaheuristic-Based Machine Learning Approach for Customer Segmentation



P. Z. Lappas, S. Z. Xanthopoulos, and A. N. Yannacopoulos

Abstract In the globalized knowledge economy, the challenge of translating best available evidence from customer profiling and experience into policy and practice is universal. Customers are diverse in nature and require personalized services from financial institutions, whereas financial institutions need to predict their wants and needs to understand them on a deeper level. Customer segmentation is a very crucial process for a financial institution to profile new customers into specific segments and find patterns from existing customers. Usually, rule-based techniques focusing on specific customer characteristics, according to expert knowledge, are applied to segment them. However, these techniques highlight the fact that traditional classifications in the big data era are becoming increasingly irrelevant and agree to the claim of financial institutions not knowing their customers well enough. The main objective of this work is to propose an evolutionary clustering approach as a rule extractor mechanism that facilitates decision makers to recognize the most significant customer characteristics and profile them into segments. Particularly, a population-based metaheuristic algorithm (Genetic Algorithm) is used in a hybrid synthesis with unsupervised machine learning algorithms (K-means Algorithms) to solve data clustering problems. Based on the clustering result, labels are added for every data point in the dataset. This dataset is used to train supervised

P. Z. Lappas (✉)
Dept. of Statistics and Actuarial-Financial Mathematics, University of the Aegean, Samos, Greece

EXUS AI Labs, EXUS, Athens, Greece
e-mail: pzlappas@aegean.gr; p.lappas@exus.co.uk

S. Z. Xanthopoulos
Department of Statistics and Actuarial-Financial Mathematics, University of the Aegean, Samos, Greece
e-mail: xanthos@aegean.gr

A. N. Yannacopoulos
Department of Statistics and Laboratory for Stochastic Modelling and Applications, Athens University of Economics and Business, Athens, Greece
e-mail: ayannaco@aueb.gr

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

M. Eddaly et al. (eds.), *Metaheuristics for Machine Learning*,
Computational Intelligence Methods and Applications,
https://doi.org/10.1007/978-981-19-3888-7_4

ML algorithms such as deep learning and random forests to predict in which cluster a new customer can be mapped. A cluster analysis conducted on behalf of the EXUS financial solutions company that provides financial institutions with financial software that can deliver debt collection services effectively, meeting both academic requirements and practical needs. Two real-world datasets collected from financial institutions in Greece, explored and analyzed for segmentation purposes. To demonstrate the effectiveness of the proposed method, well-known benchmark datasets from UCI machine learning repository were also used.

4.1 Introduction

Nowadays, customer segmentation has become a mainstream in a large amount of business domains. Concurrently, innovations in Artificial Intelligence (AI) and Machine Learning (ML) create new possibilities for gleaning intelligence from data and translating that into business advantage. Especially in the banking sector, customer segmentation has received a great deal of attention from academics, consultants and practitioners [1–3]. Traditionally, the customer segmentation is a pre-requisite step before the development of any credit or/and behavioral scorecard [4–6]. In particular, credit scoring per segment aims at predicting the probability of default for a new customer [7, 8], while behavioral scoring per segment intends to predict the payment behavior of an existing customer [9–11]. Inferentially, customer segmentation can be seen as a process of dividing customers into different groups, with the main purpose of providing decision-makers with the opportunity to develop appropriate strategies for more effective management based on in-depth knowledge of customer groups.

Cluster analysis (or data clustering) based on unsupervised ML algorithms has become a key enabling technology for the process of customer segmentation. Data clustering is an essential tool in data mining [12, 13] that aims at partitioning a given dataset (e.g., categorical data, text data, multimedia data, time-series data, network data, discrete sequences, mixed data, etc.) into homogeneous groups based on some similarity/dissimilarity metrics (e.g., distance based on binary variables for nominal/categorical variables, Spearman or Footrule distance for ordinal variables, Mahalanobis or Euclidean distance for quantitative variables, etc.) [14]. Many algorithms have been proposed in the literature using different techniques such as (1) partitional clustering (e.g., K-means, K-medoid, Fuzzy C-Means, etc.), (2) probabilistic model or distribution-based clustering (e.g., Gaussian mixture model, Bernoulli mixture model, Expectation-Maximization (EM) algorithm, etc.), (3) hierarchical clustering (e.g., agglomerative clustering, divisive clustering, etc.), (4) density-based clustering (e.g., DBSCAN, OPTICS algorithms, etc.), (5) grid-based clustering (e.g., WaveCluster, NSGC, ADCC, GDILC, CLIQUE algorithms, etc.), as well as (6) spectral clustering (e.g., normalized or unnormalized spectral clustering, etc.) [15].

There are a number of studies carried out on customer segmentation in the financial domain. In 2010, Bizhani and Tarokh [16] combined RFM (Recency, Frequency

and Monetary) analysis with ULVQ (unsupervised learning vector quantization) algorithm to apply behavioral segmentation of bank's point-of-sales. Additionally, Rezaeinia et al. [17] proposed a clustering method based on Analytic Hierarchy Process (AHP), RFM and K-means algorithms to recognize valuable customers in a banking environment. Particularly, the AHP method used for computing the weights of recency, frequency and monetary values with respect to which K-means algorithm grouped customers. Self-organizing maps and knowledge-based constrained clustering approaches have also been proposed in the literature for grouping business and individual banking customers using real-world datasets (e.g., data from banks in Croatia and Portugal) [18–20]. Furthermore, clustering ensemble algorithms have been introduced in the literature. The basic idea is to evaluate customer segmentation by various clustering approaches and build prediction algorithms (i.e., classification based on clustering). In this direction, Wang et al. [21] used K-means, EM, DBSCAN and OPTICS algorithm for grouping customers in personal financial market, Sivasankar and Vijaya [22] focused on unsupervised ML algorithms such as Fuzzy C-Means, Possibilistic Fuzzy C-Means and K-Means to build an effective hybrid learning system on churn prediction dataset, whereas Barman et al. [23] proposed X-means clustering algorithm which tested on real-world marketing campaign datasets and developed prediction algorithms based on Naïve Bayes, Decision Trees and Support Vector Machines.

It is worth mentioning that nowadays data generation and storage have been exploded as a result of the rapid development of the advanced technology. In addition to this, it is commonly accepted that big data accompanies with difficulties and challenges (e.g., data imperfection, data inconsistency, data confliction, data correlation, data type heterogeneity, etc.) in a data-driven service provision due to its 5Vs (i.e., volume, velocity, variety, veracity and value) [25]. As the volume of data generated increases significantly, new data challenges arise to understand such data and discover knowledge. Feature weighting and selection of reliable and accurate information in big data are two of the most significant research topics nowadays [24].

Large-scale data-driven customer segmentation is a typical example of high-dimensional data clustering. An alternative term for high-dimensional data clustering is "Big Data clustering" indicating that the process of extracting useful information and reaching specific data by traditional data clustering algorithms is challenging due to the time and computational power needed to analyze them. Recently, some researchers mentioned the necessity for improving traditional clustering algorithms (e.g., K-means, Fuzzy-means, etc.) and using them in parallel environments such as Hadoop, MapReduce, Graphical Processing Units and multi-core systems [26–28]. Large-scale data-driven customer segmentation has not been extensively researched in the literature. Only few works founded highlighting the necessity of developing a big-data framework for explainable customer segmentation in the banking domain. For instance, Hossain et al. [29] presented a large-scale data-driven customer segmentation based on K-means, K-medoids, K-shape, association rules mining and anomaly detection, whereas Motevali et al. [30] introduced a new metaheuristic algorithm called WHO (Wildebeests Herd

Optimization), that is inspired from the glorious life of wildebeests, and tested it on a case study on banking customer segmentation.

Considering the literature and the importance of addressing big data challenges, metaheuristics seem to be good candidate to solve large-scale problems induced by the big data context [31]. In particular, metaheuristics are associated with a class of techniques and methods designed to numerically find a near-optimal solution for large-scale optimization problems in a reasonable amount of time [32]. Generally, metaheuristics can be divided into two categories: single-point search metaheuristics (i.e., improve upon a specific solution by exploring its neighborhood with a set of moves for escaping local optima) and population-based search metaheuristics (i.e., combine a number of solutions in an effort to generate new solutions). Metaheuristics have mostly been used within the data analysis step for solving data mining problems. For instance, nature-inspired and bio-inspired metaheuristics have been introduced to give insights into the automatic clustering problem where the best estimate of the number of clusters should be determined [33–35], whereas several metaheuristic schemas have been proposed for improving time-series [36], partitional [37–40], fuzzy [41–43], categorical [44] and hierarchical [45] clustering problems. Table 4.1 presents various single-point search and population-based search metaheuristic algorithms introduced in the literature for the data clustering problem. As it can be observed the majority of works put a special emphasis on population-based metaheuristic algorithms.

In addition to this, research into combining ML with metaheuristics has received considerable attention from researchers in solving feature selection problems [71, 72]. Feature selection aims at finding an optimal subset of features to (1) avoid overfitting and improve ML model performance, (2) provide faster and more cost-effective models (i.e., data size reduction, redundant noise elimination from the dataset), and (3) gain a deeper insight into the underlying data generation processes. In general, feature selection techniques are divided into three categories: (1) filter (statistical ranking) methods (e.g., Mutual Information, etc.), (2) wrapper methods (e.g., sequential forward/backward feature selection, etc.) and (3) hybrid approaches (e.g., Mutual Information and Genetic Algorithms, etc.) [73]. The feature selection problem can be mathematically formulated as a constrained optimization problem depicting thus a NP-hard problem. Metaheuristics such as the Greedy Randomized Adaptive Search Procedure (GRASP) [74], Particle Swarm Optimization (PSO) [75, 76] and Genetic Algorithms (GA) [77] were often used to solve the aforementioned problem. It is worth noting that feature selection approaches in data clustering have not been extensively researched in the literature. In 2019, Kumar and Kumar [78], as well as Prakash and Singh [79] mentioned the necessity for simultaneous feature selection and data clustering. Furthermore, to the best of our knowledge, applications associated with customer segmentation in a feature selection context has not been investigated.

Most of the research papers related to customer segmentation in financial domain propose machine learning algorithms that allow the development of increasingly robust and autonomous ML applications. However, many of these ML-based systems are not able to explain their autonomous decisions and actions to human users.

Table 4.1 Metaheuristics and data clustering

Authors (Year)	Single-point search metaheuristics	Population-based search metaheuristics
Pacheco (2005) [46]	Scatter search, Greedy Randomized Adaptive Search Procedure, Tabu Search	–
Nayak et al. (2019) [47]	–	Differential Evolution
Hu et al. (2020) [48]	Multiple-search Multi-start Single-solution-based Metaheuristic	–
Belacel, et al. (2002) [49]	Variable Neighborhood Search Algorithm	–
Senthilnath et al. [50]	–	Flower Pollination Algorithm
Hansen and Mladenović (2001) [51]	Variable Neighborhood Search Algorithm	–
Kumar et al. (2017) [52]	–	Grey Wolf Algorithm
Bonab et al. (2019) [53]	–	Swarm-based Simulated Annealing Hyper-Heuristic Algorithm
González-Almagro et al. (2020) [54]	Dual Iterative Local Search	–
Liu et al. (2005) [55]	Tabu Search	–
Dowlatshahi and Nezamabadi-pour (2014) [56]	–	Gravitational Search Algorithm
Gyamfi et al. (2017) [57]	Tabu Search	–
Boushaki et al. (2018) [58]	–	Quantum Cuckoo Search Algorithm
Kuo and Zulvia (2020) [59]	–	Gradient Evolution Algorithm
Liu and Shen (2010) [60]	–	Cat Swarm Optimization Algorithm
Kamel and Boucheta (2014) [61]	–	Chameleon Army Strategy

(continued)

Table 4.1 (continued)

Authors (Year)	Single-point search metaheuristics	Population-based search metaheuristics
Harifi et al. (2020) [62]	–	Giza Pyramids Construction Algorithm
Kaur and Kumar (2021) [63]	–	Water Wave Optimization Algorithm
Irsalinda et al. (2017) [64]	–	Chicken Swarm Optimization Algorithm
Komarasamy and Wahi (2012) [65]	–	Bat Algorithm
Alshamiri et al. (2016) [66]	–	Artificial Bee Colony Algorithm
Kumar et al. (2015) [67]	–	Black Hole Algorithm
Mageshkumar et al. (2019) [68]	–	Ant Colony Optimization Algorithm, Ant Lion Optimization Algorithm
Silva-Filho et al. (2015) [69]	–	Particle Swarm Optimization Algorithm
Sharma and Chhabra (2019) [70]	–	Particle Swarm Optimization

Understanding the reason why a decision has been made by a machine is crucial to grant trust to a human decision-maker [80]. Furthermore, the high-dimensional datasets may consist of irrelevant and redundant features which deteriorate the data clustering result. Handling general classes of bias in ML algorithms is a major step towards an explainable and trustworthy AI [81]. It should not pass unnoticed that the main objective of the customer segmentation is to group existing customers into homogeneous groups. However, new customers should also be mapped to a known segment. Traditionally, a rule-based approach with respect to the clustering result is followed for grouping new customers. As the clustering result is associated with a labeled dataset it would be very interesting to use it as training dataset and create a prediction mechanism for classifying new customers. Considering the aforementioned issues, the contribution of this work is fourfold:

1. To assist financial institutions in the task of customer segmentation when high-dimensional data are available.
2. To extract data-driven rules for the segmentation and explain clustering result with non-technical terms in order to make it trusted and easily understandable by experts.
3. To verify the effectiveness of the proposed methodology by applying it to well-known datasets from UCI ML repository. Table 4.2 presents related works using UCI datasets for the data clustering problem.
4. To train supervised ML algorithms using labeled datasets obtained from the clustering task.

As to the main contribution and novelty of this work, we introduce a feature-selection-based evolutionary clustering ensemble algorithm for customer segmentation. The proposed method aims at selecting the most appropriate subset of relevant features that can improve the discrimination ability of the original feature space and the clustering result. The ability of interpretation of the discrimination power of each feature in the original dataset is strengthened by the usage of the Genetic Algorithm in a hybrid synthesis with unsupervised machine learning algorithm such as the K-means (distance-based clustering) algorithm. Based on the clustering result, labels are added for every data point in the dataset. This dataset is used to train supervised ML algorithms such as deep learning and random forests. Therefore, trained models can be used for prediction purposes (i.e., predict in which cluster a new customer can be mapped). A cluster analysis conducted on behalf of the EXUS financial solutions company that delivers debt collection software services. Two real-world datasets collected from financial institutions in Greece and explored so as to (1) segment customers into groups and (2) illustrate significant features of each group.

The remainder of this chapter is structured as follows. The second section introduces the proposed methodology for customer segmentation and prediction based on metaheuristic, unsupervised and supervised ML algorithms. The third section summarizes the analysis of data and results. Conclusions and future directions are given in the fourth section.

Table 4.2 Related works using UCI dataset

Authors (Year)	Metaheuristics	UCI datasets
Gaikwad et al. (2020) [82]	Artificial Bee Colony	Wine, Seeds
Marinakis et al. (2008) [83]	Particle Swarm Optimization, Greedy Randomized Adaptive Search Procedure	Iris, Wine, Breast Cancer, Ionosphere, Heart, Hepatitis, Australian Credit Approval
Kowalski et al. (2019) [84]	Krill Herd Algorithm, Flower Pollination Algorithm	Iris, Wine, Glass, Breast Cancer, Ionosphere, Image Segmentation, Seeds, Sonar, Thyroid, Automobile Data, Pima Indians Diabetes
Marinakis et al. (2007) [85]	Particle Swarm Optimization, Greedy Randomized Adaptive Search Procedure	Iris, Wine, Breast Cancer, Ionosphere, Spambase, Heart, Hepatitis, Australian Credit Approval
Marinakis et al. (2008) [86]	Particle Swarm Optimization, Ant Colony Optimization	Iris, Wine, Ionosphere, Spambase, Hepatitis
Marinakis et al. (2008) [87]	Particle Swarm Optimization, Greedy Randomized Adaptive Search Procedure	Iris, Wine, Breast Cancer, Ionosphere, Spambase, Heart, Hepatitis, Australian Credit Approval
Marinakis et al. (2009) [88]	Bumble Bees Mating Optimization, Greedy Randomized Adaptive Search Procedure	Iris, Wine, Breast Cancer, Ionosphere, Spambase, Heart, Hepatitis, Australian Credit Approval
Marinakis et al. (2011) [89]	Ant Colony Optimization, Greedy Randomized Adaptive Search Procedure	Iris, Wine, Breast Cancer, Ionosphere, Spambase, Image Segmentation, Heart, Zoo, Hepatitis, Australian Credit Approval, Automobile Data
Saida et al. (2014) [90]	Cuckoo Search Optimization	Iris, Wine, Breast Cancer, Vowel
Singh et al. (2021) [91]	Moth-flame Optimization	Iris, Wine, Glass, Yeast
Tian et al. (2016) [92]	Elephant Search Algorithm	Iris, Haberman, Mice Protein, Gesture
Cho and Nyunt (2020) [93]	Differential Evolution	Iris, Wine, Glass, Breast Cancer, Thyroid
Kuo et al. (2020) [94]	Genetic Algorithm, Particle Swarm Optimization, Sine Cosine Algorithm	Iris, Wine, Haberman, Glass, Heart, Credit Approval, German, Dermatology, Adult, Zoo, Flags, Hepatitis, Pima Indians Diabetes

(continued)

Table 4.2 (continued)

Authors (Year)	Metaheuristics	UCI datasets
Eskandari and Javidi (2019) [95]	Bat Algorithm	Wine, Ionosphere, Monk3, Tic-tac-toe, Sybean-small, Heart, Credit Approval, Dermatology, Abalone
Agbaje et al. (2019) [96]	Firefly Algorithm, Particle Swarm Optimization	Iris, Wine, Glass, Breast Cancer, Yeast, Heart, Thyroid
Wu et al. (2019) [97]	Whale Optimization Algorithm	Iris, Wine, Glass, Breast Cancer, Yeast, Car Evaluation, Heart, CMC
Hatamlou and Hatamlou (2013) [98]	Particle Swarm Optimization	Iris, Wine, Glass, Breast Cancer, Vowel, CMC

4.2 Proposed Method

This section provides detailed discussions on the methodology of the proposed feature-selection-based evolutionary clustering ensemble algorithmic (FS-ECEA) design concept. The proposed solution approach is composed of three main stages. The first stage is the Genetic Algorithm (GA) that is associated with the feature selection part, whereas the second stage is an unsupervised ML algorithm (K-means) that is related to the clustering problem. The third stage is a training pipeline into which the clustering result is used for training supervised ML algorithms such as Deep Learning and Random Forests. The main idea is to reduce the dimensionality of data by selecting the most appropriate subset of features that can improve the clustering result and better explain the clusters. Then, the labeled dataset (i.e., clustering result) is used as a training dataset for prediction purposes.

4.2.1 Feature Selection

The main goal of the feature selection stage is to find an optimal subset of features that result in the best clustering outcome. This is a crucial step in high-dimensional clustering problems due to feature quality (redundant features), time and computational power constraints. The GA is a population-based metaheuristic optimization algorithm inspired by the natural behavior of the population evolution [99]. Actually, GAs work throughout populations of possible solutions (called "generations") where each candidate solution (called "individual") is represented as a finite length of string (called "chromosome") over some finite set of symbols. Each symbol of the finite length string (called "gene") represents the value of the decision variable on a continuous (real-coded chromosome) or on a binary domain (chromosome as string of 0's and 1's).

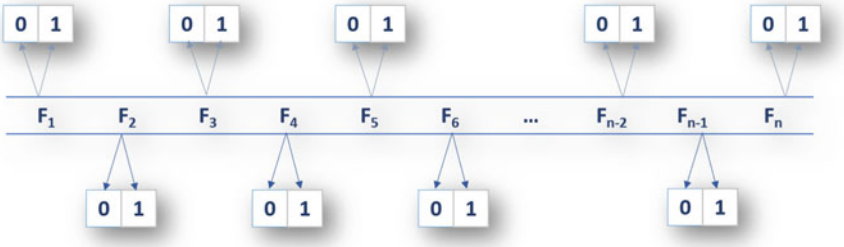


Fig. 4.1 Chromosome representation

The main idea of using GAs is to take a set of candidate solutions and iteratively refine them by alternating and selecting good solutions for the next generation. The selection depends upon the Darwinian survival theory, where the fittest individuals will survive, while the rest will be discarded. Individuals (i.e., solutions) are evaluated on the basis of a fitness function which is associated with the evaluation/objective function of the problem. A new generation of possible solutions is obtained from the current generation by applying suitable genetic operators. In general, GAs employ three basic operators over a limited number of individuals to find the global near-optimal solutions: (1) selection, (2) crossover, and (3) mutation operators.

In the context of the feature selection problem, each chromosome represents the feature space as a bit string of 0's (a given feature is not chosen) and 1's (a given feature is chosen) values. Figure 4.1 illustrates the structure of a chromosome (the value of each feature $F_i, i = 1, \ldots, n$, can be 0 or 1).

Upon choosing a pair of high performing individuals (called "parents") new individuals are generated by applying (a) a crossover operator for exploring new individuals (called "offspring) and exchanging information (i.e., genes) and (b) a mutation operator for maintaining and introducing diversity in the population (called "mutants"). Figure 4.2 illustrates an example of selecting two parent individuals (i.e., p1 and p2) from a population, their chromosome representation, as well as the operators of crossover (e.g., a single-point crossover to produce two offspring by exchanging their genes, i.e., o1 and o2) and mutation (e.g.,a bit string mutation to produce one mutant (i.e., m5) after selecting a parent (i.e., p5) and changing a gene of its chromosome).

A fitness-proportional selection technique, in which each individual's probability of being selected is proportional to its fitness, is a common parent selection method. As a result, the higher the fitness value of an individual, the more likely it is to be chosen. In this work, the fitness value is associated with the performance metric of the selected clustering algorithm (see Sect. 4.2.2). The selection criterion is the tournament selection which involves running tournaments among a number of h (h > 2) individuals chosen at random from the current population, where h is the tournament size (i.e., the size of the tournament group). For each tournament the

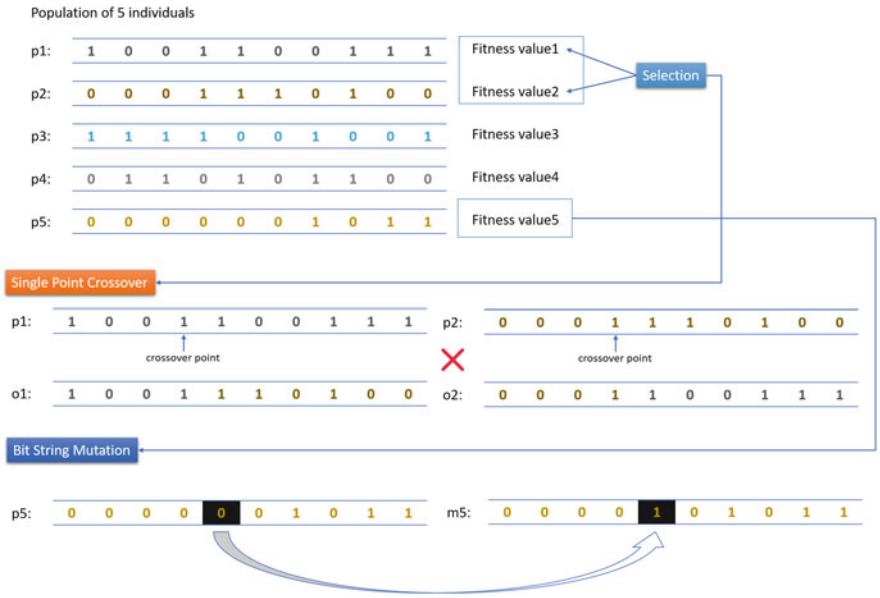


Fig. 4.2 Genetic operators

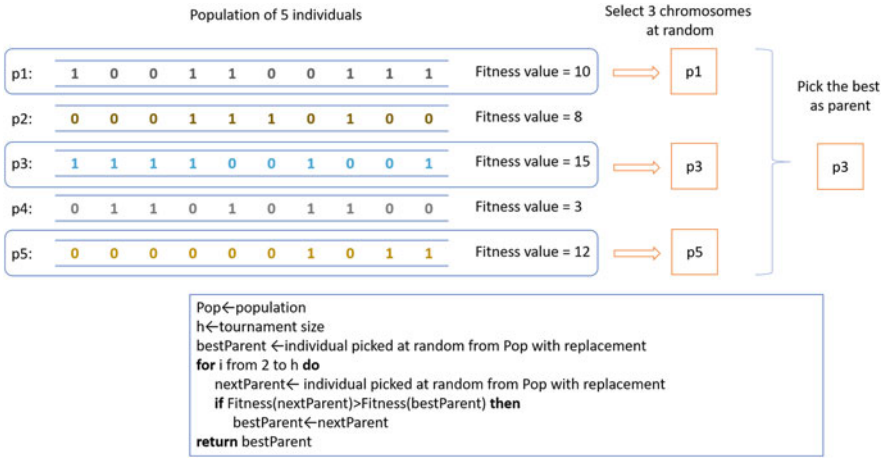


Fig. 4.3 Tournament selection

one with the best fitness is selected for crossover. As a result, to select 4 parents, the tournament procedure is carried out 4 times. A good rule of thump is to include roughly 20% of the population as a tournament size. Figure 4.3 illustrates a tournament selection example and presents the tournament selection algorithm in a pseudo-code form.

Since two parents are selected, a crossover operator can be applied. In the example illustrated in Fig. 4.2, a crossover point after the fourth position of a chromosome was selected. For instance, the first offspring (i.e., o1) receives some genetic information (i.e., genes) from one parent (the first four genes of p1), and the other genetic information from the second parent (i.e., 5th–10th genes of p2). After a crossover process, an individual is subjected to mutation. For instance, in the aforementioned example, p5 was selected to produce a mutant (i.e., m5). Usually, a randomly selected gene of a parent is modified (e.g., the fifth feature is not selected in p5, but is selected in m5). It is worth mentioning that the number of offspring and mutants are based on predefined crossover (e.g., apply crossover to 70% of the parents) and mutation (e.g., apply mutation to 10% of the parents) percentages, while the number of genes that could be modified in a mutation process is based on the mutation rate.

In this work, we use two crossover operators and one mutation operator. As far as the crossover operators are concerned, single-point crossover and uniform crossover are considered. Regarding the mutation operator, bit string mutation is used. Suppose that we have a population of N individuals $\{p_1, p_2, p_3, \ldots, p_N\}$. Each individual is associated with n genes, and we denote the $k-th$ gene of the $i-th$ individual as $p_i(k)$ for $k \in [1, n]$. So, we can represent p_i as the row-vector $p_i = [p_i(1), p_i(2), p_i(3), \ldots, p_i(n)], i = 1, \ldots, N$. Additionally, we denote an offspring obtained after a crossover, as o, and the $k-th$ gene of o as $o(k)$, hence: $o = [o(1), o(2), o(3), \ldots, o(n)]$.

Suppose now that we have two parents, p_a, p_b where $a, b \in [1, N]$. The result of the single-point crossover is represented as follows: $o(k) = [p_a(1), \ldots, p_a(m), p_b(m+1), \ldots, p_b(n)]$ where m is a randomly selected crossover point; that is, $m \sim U[0, n]$. If $m = 0$ then o is a clone of p_b. If $m = n$, then o is a clone of p_a. Single-point crossover aims at obtaining two offspring from a pair of parents. This is done by selecting each gene of the second offspring o_2 from the opposite parent than the one from which offspring o_1 obtained its gene: $o_1(k) = [p_a(1), \ldots, p_a(m), p_b(m+1), \ldots, p_b(n)]$ and $o_2(k) = [p_b(1), \ldots, p_b(m), p_a(m+1), \ldots, p_a(n)]$. Uniform crossover results in the offspring o, where the $k-th$ feature of o is: $o(k) = p_{i(k)}(k)$ for each $k \in [1, N]$, where we randomly choose $i(k)$ from the set $\{0, 1\}$. That is, we randomly choose each offspring feature from one of its two parents, each with a probability of 50%. Figure 4.4 illustrates an example and the uniform crossover algorithm in a pseudo-code form.

Mutation is a straightforward operation in binary evolutionary algorithms. If we have a population of N individuals, where each individual has n bits, and our mutation rate is θ, then at the end of each generation we flip each bit in each individual with a probability of $r \longleftarrow U[0, 1]$, as follows: $p_i(k) \longleftarrow p_i(k)$ if $r \geqslant \theta$; $p_i(k) \longleftarrow 0$ if $r < \theta$ and $p_i(k) = 1$; $p_i(k) \longleftarrow 1$ if $r < \theta$ and $p_i(k) = 0$; for $i \in [1, N]$ and $k \in [1, n]$ where $U[0, 1]$ is a random number that is uniformly distributed on [0, 1]. The mutation of bit strings ensue through bit flips at random positions. The probability of a mutation of a bit is $1/n$, where n is the length of the binary vector. Hence, in average only one variable is mutated.

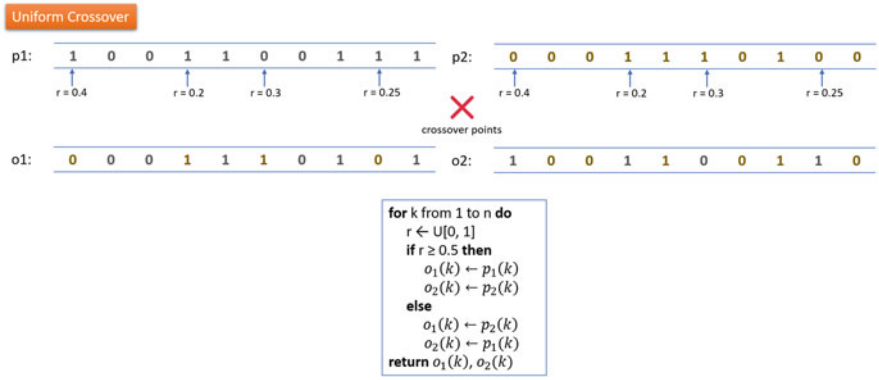


Fig. 4.4 Uniform crossover

4.2.2 Clustering

In this work, GA is used as a wrapper method in which the data size is reduced with respect to a previously selected subset of features and then data is fitted to a clustering algorithm and evaluated with a chosen performance metric. A distance-based unsupervised machine learning algorithm is embedded in the wrapper approach of GA in particular, the K-means algorithm. The K-means algorithm is a partitional clustering method aims at forming a partition such that the squared error between the empirical mean of a cluster and the data points in the cluster is minimized.

Defining a target number K, which refers to the number of centroids you need in the dataset (i.e., desired number of clusters), every data point is allocated to each of the clusters through reducing the in-cluster sum of squares. Namely, a data point is considered to be in a particular cluster if it is closer to that cluster's centroid than any other centroid. K-means process starts with identifying data $X = \{x_{i,j}\}, i = 1, \ldots, m; j = 1, \ldots, n$ where m is the number of data points to be carried out in the clustering and n is the number of independent variables (i.e., features) characterizing each data point. To process the learning data, the K-means algorithm starts with a first group of randomly selected centroids, which are used as the beginning points for every cluster, and then it performs repetitive calculations to optimize the positions of the centroids. In particular, the Euclidean distance between each data point, x_{ij}, with each cluster center $c_{k,j}, k = 1, \ldots, K; j = 1, \ldots, n$ is calculated as follows: $d_{i,k} = \sqrt{\sum_{j=1}^{n} \left(x_{i,j} - c_{k,j}\right)^2}$ where $d_{i,k}$ is the distance of data point i and centroid k, n represents data dimensions, $x_{i,j}$ are the coordinates of data point i in dimension j, and $c_{k,j}$ are the coordinates of data point k in dimension j. A data point will be a member of the cluster k if the distance of that data point to centroid k has the smallest value when compared to the distance to another centroid, that is min $\sum_{k=1}^{K} d_{i,k}$. The "means" in the K-means refers to averaging of the data

(i.e., finding the centroid), indicating that centroids are recomputed by taking the mean of all data points assigned to that centroid's cluster. The above procedure is repeated until a stopping criterion is satisfied (e.g., data points do not change clusters, while the sum of distances is minimized).

Since the K-means is a minimization problem, the fitness function for each chromosome is defined as follows: $1/\min \sum_{k=1}^{K} d_{i,k}$. As a result, each individual has a probability that is proportional to its fitness. The higher the individual's fitness is, the more likely it is to be selected. In this direction, the best K-means result in terms of minimizing the total distance within a cluster, increases the fitness value of an individual with respect to selected subset of features. The flow diagram for the K-means algorithm is as given in Fig. 4.5. It is worth pointing out that the K-means process is triggered whenever a fitness value of an individual in a specific population and in a specific generation should be calculated, facilitating thus the evolution process defined in the feature selection stage of the proposed method.

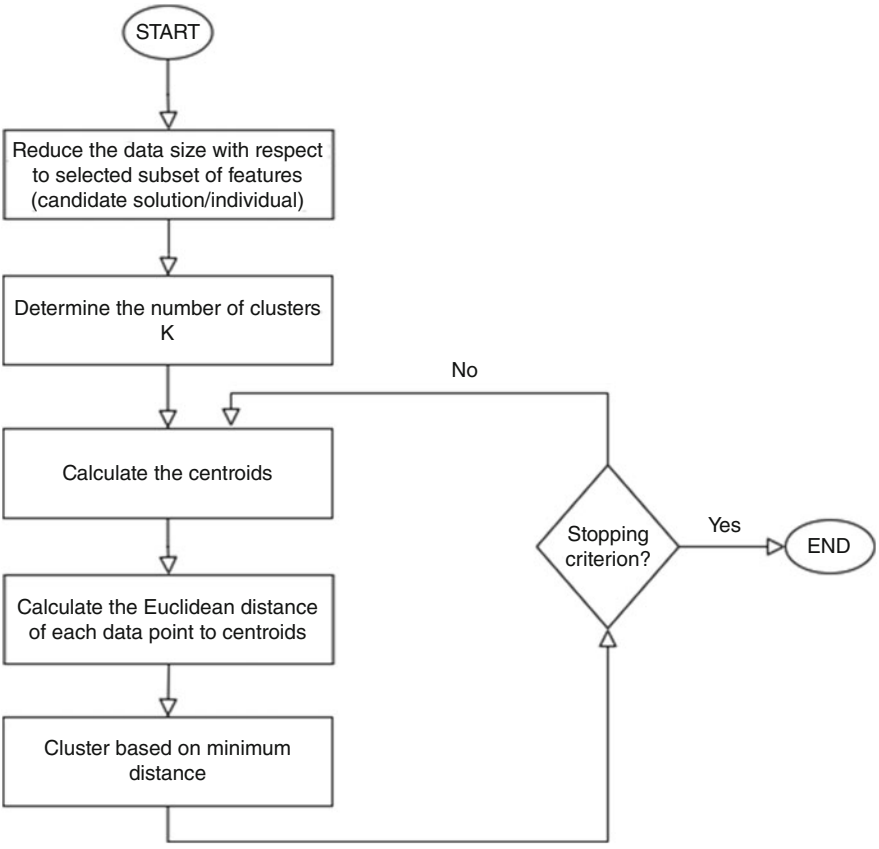


Fig. 4.5 K-means algorithm

4.2.3 General Structure of the Proposed Hybrid Scheme

Considering the basic concepts presented in Sects. 4.2.1 and 4.2.2, this sub-section gives the flow diagram for the feature-selection-based evolutionary clustering algorithm (see Fig. 4.6). The process starts by initializing the population (symbolized as "pop") with random candidate solutions. Each individual is evaluated via K-means clustering algorithm (i.e., find "K" clusters) that contributes to the calculation of each individual's fitness value. Since the "pop" is evaluated and sorted, the best chromosome is placed as the first individual in the population. With respect to the population size (symbolized as "popSize"), as well as predefined crossover (symbolized as "crossPerc") and mutation (symbolized as "mutPerc") percentages,

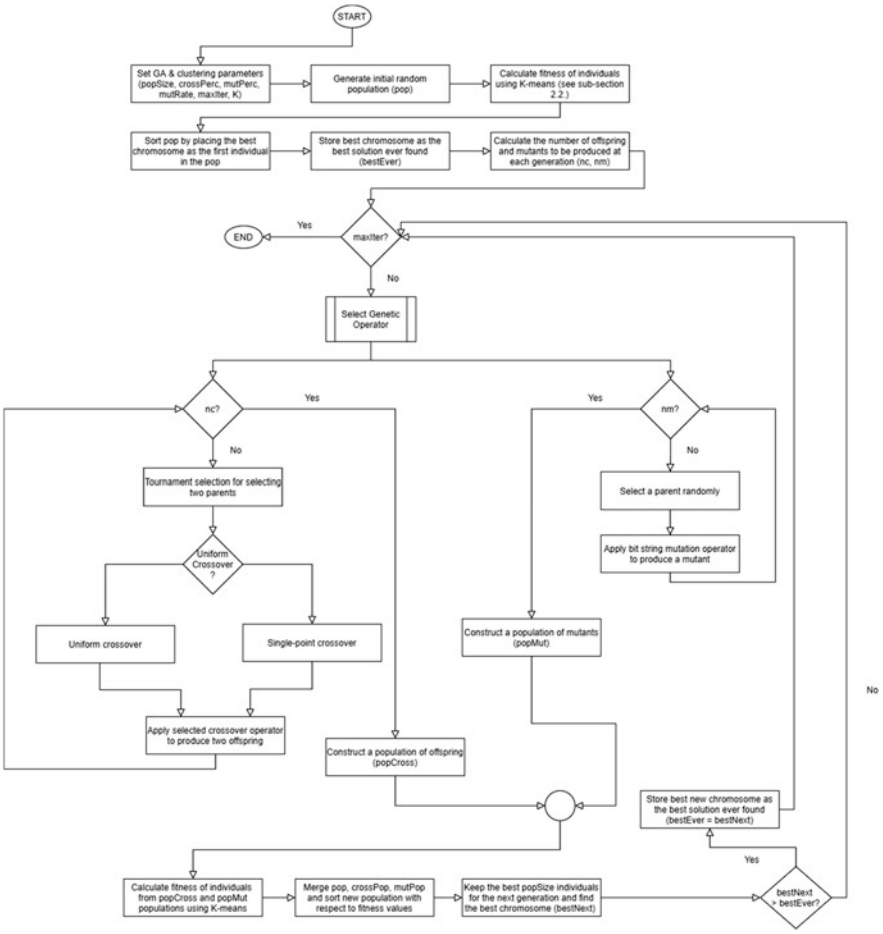


Fig. 4.6 Evolutionary clustering approach

the number of offspring (symbolized as “nc”) and mutants (symbolized as “nm”) are calculated, respectively. Each iteration of the GA main loop consists of three main steps. The first step chooses two parents using a tournament selection method and applies a selected crossover operator (single-point crossover or uniform crossover based on user preferences) to generate two offspring. This procedure is repeated until “nc” offspring are generated, constructing thus a population of offspring (symbolized as “popCross”).

In the second step, a parent is randomly selected for applying a bit string mutation operator with respect to the predefined mutation rate (symbolized as “mutRate”). The mutation process is repeated until “nm” mutants are generated (symbolized as “popMut”). Then “popCross” and “popMut” are evaluated and merged with “pop” in third step. The new population is sorted and based on the “popSize” value, only the best “popSize” individuals are selected to proceed to the next generation. Additionally, the best solution found so far (symbolized as “bestEver”) is compared with the best individual of the new population (symbolized as “bestNext”). The GA main loop is repeated until a termination criterion is reached. The termination criterion is based on the maximum number of iterations (symbolized as “maxIter”).

4.2.4 Classification Based on Clustering

Considering the clustering result, there are three ways of building a classification model for the customer segmentation. The first option is to use the clustering result as the prediction model. This means that whenever we need to classify a new customer, the distances from new data point to each cluster center should be calculated and then classify the new data point into the cluster whose center is the nearest to it. The second option assumes that someone can focus on the most significant features of the dataset (due to the feature selection process) to group customers, observe the possible values of each significant feature for all clusters and create rules that should be followed when new customers come. A more efficient way to create classification models (third option) is to use the clustering result, which is a labeled dataset, for training supervised machine learning algorithms. After the clustering, each data point, which is associated with a set of independent variables (i.e., features), can be labeled as it belongs to specific data cluster. A label corresponds to specific cluster and may be an integer value, whereas it can be used as the dependent variable of the dataset. Depending on the number of clusters, binary (two labels) or multi-class (more than two labels) classifications problems can be defined. As a result, the main idea behind the third option is to use the labeled dataset to train supervised machine learning algorithms that can predict the cluster with which a new customer can be associated.

In this work, deep learning [100] and random forests [101] are suggested to be trained for prediction purposes. Deep learning models correspond to multilayer perceptrons or artificial neural networks concepts. Actually, the building blocks of neural networks including neurons, weights, and activation functions are combined

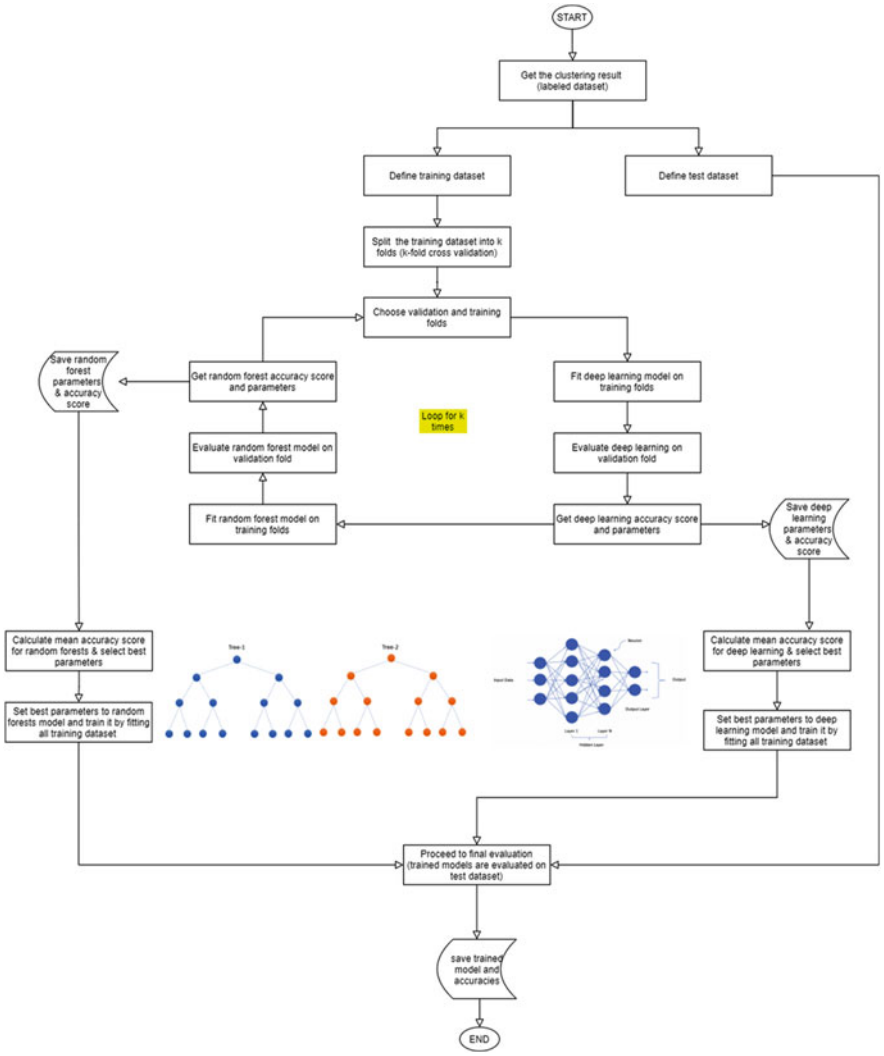


Fig. 4.7 Training pipeline

to create various network topologies that are able to learn from examples in the training dataset and predict the output variable. The random forest algorithm consists of many decision trees and establishes the outcome based on the predictions of the decision trees (i.e., it predicts by taking the average of the output from various decision trees). Figure 4.7 depicts the flowchart for the training process.

As it can be observed, we use a simple separation of labeled data (i.e., clustering result) into training and test datasets (e.g., 70% for training and 30% for testing). The training dataset is further divided into training and validation datasets to keep

track of how well a ML model is doing as it learns in the context of a k-fold cross validation process. Cross validation is also suggested to tune parameters of ML models (e.g., increase the number of trees for random forests or increase the number of hidden layers in a neural network). In particular, the training dataset is divided into k equal-sized parts, of which one part is used as a validation dataset, and $k - 1$ parts as training datasets. Then, the results of the k iterations are averaged. Finding the best parameters of a ML model, all the training dataset used to fit that model. Trained model is then evaluated on test dataset. The performance measure used for evaluating ML models is the classification accuracy (i.e., the fraction of predictions a ML model got right).

4.3 Computational Experiments and Results

4.3.1 Experiment Setup

The FS-ECEA algorithm was developed in the Python programming language and executed on a DELL personal computer with an Intel® Core™i7-1065G7, clocked at 1.30–1.50 GHz, a microprocessor with 8GB of RAM memory under the operating system Microsoft Windows 10 Enterprise.

This work comprises two set of experiments. The first set of experiments is conducted to verify the effectiveness of the FS-ECEA algorithm based on well-known benchmark datasets from UCI machine learning repository. The second set of experiments is related to a cluster analysis conducted on behalf of the EXUS financial solutions company, meeting both academic requirements and practical needs. EXUS is a software house that delivers debt collection software services via the EXUS EFS debt collection system. Two real-world datasets collected by EXUS from financial institutions in Greece. Data anonymized, explored and analyzed for customer segmentation purposes. After the clustering result of the second experiment, labeled datasets used to train and evaluate (in terms of accuracy) supervised ML algorithms that will be able to predict the cluster with which a new customer can be associated.

4.3.2 Considered Datasets

As mentioned above, the datasets used for experimentation in this work are real-world data collected by EXUS financial solutions company and well-known UCI datasets. The performance of the FS-ECEA algorithm was tested initially on 2 benchmark instances taken from UCI ML repository: Breast Cancer Wisconsin (Diagnostic) Data Set (breast-cancer), and Statlog (German Credit Data) Data Set (german). Table 4.3 gives details for the UCI datasets regarding the number of

Table 4.3 Related works using UCI dataset

Dataset	Observations	Data type	Num. of features	Num. of clusters
breast-cancer	569	Numerical	30	2
german	1000	Mixed (categorical and numerical data)	20 (61)	2

observations, number of features, number of clusters and data types. The german dataset consists of both categorical and numerical features. After quantifying the categorical features, we have increased the number of features from 20 to 61. As far as the real-world datasets are concerned, they were anonymized and collected from EXUS EFS debt collection database. At this point, we describe some basic concepts in the debt collections domain and give some details about the data we were used.

Generally, after a customer defaults on his/her repayment obligations, collectors of unsecured credit debt proceed to a number of actions (e.g., telephone call, e-mail, sms, formal letter, etc.) to secure some repayment on the debt. If these actions fail, collectors could seek legal proceedings. In the context of an early debt collection process, the focus is placed on collecting payments within 0–90 days from customers before the legal procedures start. The success of the collection process is measured by the recovery rate, that is the percentage of the defaulted amount that is recovered during that process. EXUS EFS is a system that allows decision makers (collection managers, collection supervisors and agents) to monitor collection processes, as well as design and apply treatment plan strategies per specific group of customers with respect to measures such as total past due amounts, total recovery rate, etc. An EXUS EFS database consists of monthly account records (e.g., bucket, past due amounts, days past due, balances, etc.), daily transaction records (e.g., payments, collector-debtor interaction-based information, interest rates, etc.), behavioral scoring variables (e.g., risk levels, collection phases, etc.), demographic variables (age, marital status, etc.), as well as product-based details (e.g., specific type of consumer loan, business loan, credit card, etc.) and occupation-based information. In this work, a cluster analysis conducted on behalf of the EXUS financial solutions company to group customers into clusters from a risk management point of view. In particular, EXUS was interested in grouping customers into two main groups with respect to their payment behavior.

It is worth mentioning that after cleaning and exploring in depth EXUS datasets, we applied feature engineering techniques to create historical features (e.g., average number of calls, maximum pay amount ever, minimum pay amount 3 months ago, maximum days past due ever, sum of total balances last 3 months, number of times a customer refused to pay last 3 months, and so on), increasing thus the data dimensionality. Furthermore, categorical variables were quantified by applying one hot encoding. In the same way we handled categorical variables of german UCI dataset. For instance, assume that there is a categorical variable for the product loan (product_loan) with four possible values: credit card, consumer loan, business loan and mortgage loan. One hot encoding creates four new

Table 4.4 EXUS datasets

Dataset	Observations	Data type	Number of features
XYZ-1 business dataset	79,729	Numerical	369
XYZ-1 retail dataset	191,186	Numerical	393
XYZ-2 business dataset	27, 074	Numerical	410
XYZ-2 retail dataset	61,168	Numerical	390

binary variables and remove the initial categorical variable from the dataset: product_loan_credit_card (0/1), product_loan_consumer (0/1), product_loan_business (0/1) and product_loan_mortgage (0/1). In such a way the data dimensionality is further increased.

Two EXUS datasets were collected from two financial institutions in Greece: XYZ-1 and XYZ-2. The two datasets were divided into four datasets with respect to the product type (business or retail): one business dataset and one retail dataset for XYZ-1 financial institution; one business dataset and one retail dataset for XYZ-2 financial institution. Table 4.4 gives some details regarding EXUS datasets.

4.3.3 Parameter Settings

The parameters settings for FS-ECEA algorithm were selected after thorough empirical testing. The number of individuals in the populations is set equal to 40, two crossover percentage values are evaluated (0.70 and 0.80), the mutation percentage value is set to 0.10, two mutation rates are evaluated (0.05 and 0.10), the number of generations is set to 100, the desired number of clusters is set to 2 (UCI datasets depict binary classifications problems, whereas EXUS preferred to investigate initially two main group of customers), the number of time the K-means algorithm will be run with different centroid seeds is set to 200, and the maximum number of iterations of the K-means algorithm for a single run is set to 1000.

4.3.4 Results and Conversation

4.3.4.1 Experiment 1

The objective of this set of experiments is to show the performance of the proposed algorithm in searching for an optimal subset of features that can cluster high-dimensional data. It should be mentioned that the main purpose of this paper is to describe the methodology that has been developed and its methodological parts based on machine learning and metaheuristic algorithms. Our goal is not to improve the performance of classical machine learning algorithms as it happens in many studies, but to facilitate the hybridization of machine learning with metaheuristics.

Although different datasets could be potential, the selected datasets (german and breast-cancer) which consists more than 10 features (at least) are used as an illustrative example since they are publicly available and commonly usable in the literature.

Table 4.5 gives numerical results found by the FS-ECEA algorithm after 10 independent execution runs per dataset. For each dataset, we give the total number of available features, observations in the dataset, the number of selected features, the number of correct clustered samples corresponding to the best solution found, as well as average values of correct clustered samples with respect to 10 independent runs of the FS-ECEA algorithm per dataset.

As it can be observed, the proposed algorithm has achieved a very good performance concerning the clustered samples. The percentage of corrected clustered samples for the breast-cancer dataset is 86.64%, while for the german dataset is 94.80%. The significance of the solutions obtained in terms of selected features using the FS-ECEA algorithm, proved also by the fact that best solutions obtained using small number of features (8 features out of 30 for the breast-cancer problem and 6 features out of 61 for the german problem) to group data points. This is very significant for high-dimensional-data clustering concepts. In addition to this, Fig. 4.8 illustrates typical graphs of the fitness value in the population as a function of the generation. It actually presents snapshots of the convergence of fitness values for each UCI dataset (70 generations out of 200). Fluctuations can be observed during convergence. However, the entirely direction of evolution indicates improvement. For both UCI datasets, it appears that the best candidate solution will continue to improve considering additional generations.

4.3.4.2 Experiment 2

As mentioned in Sect. 4.3.2, four real-world datasets were processed and prepared for the customer segmentation experiment. Two datasets correspond to the XYZ-1 financial institution and the rest datasets correspond to the XYZ-2 financial institution. For each financial institution the first dataset is associated with business clients (depicting similar business operations within similar markets), whereas the second dataset is related to the retail part of the portfolio (depicting individuals with similar needs in terms of selected products). As a result, four customer-segmentation-oriented experiments took place.

Clustering Results

In this stage, we applied the FS-ECEA algorithm to the data processed. The dataset was standardized to have mean 0 and standard deviation 1. Tables 4.6 and 4.7 present the clustering result per each financial institution, mentioning the number of selected features, as well as the distribution of customers to two clusters (Cluster-1; Cluster-2).

Table 4.5 FS-ECEA on breast-cancer and german datasets

Dataset	Features	Observations	Selected features	Correct clustered samples	Average of correct clustered samples
breast-cancer	30	569	8	493 (86.64%)	432.14 (75.95%)
german	61	1000	6	948 (94.80%)	866.86 (86.69%)

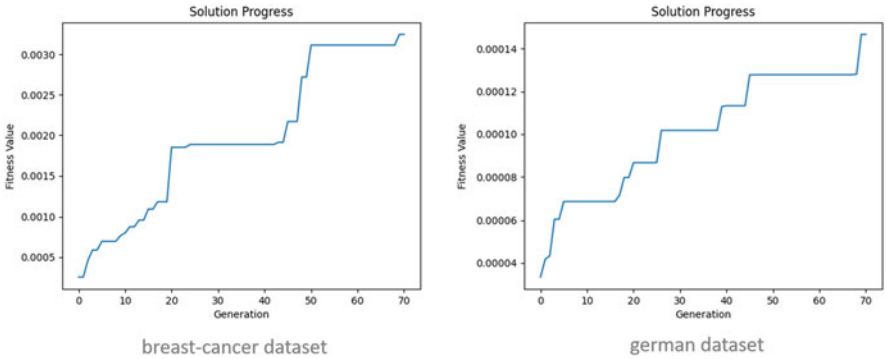


Fig. 4.8 Convergences of fitness values

As far as XYZ-1 business customers are concerned, only 12.19% (45) of available features (369) used for grouping them into two clusters consisting of 51,444 and 28,285 customers, respectively. In addition to this, XYZ-1 retail customers grouped into two clusters consisting of 65.40% (125,036) and 34.60% (66,150) of the retail portfolio, respectively. In this case, only 14.25% (56) of available features (393) used for grouping XYZ-1 retail customers.

Regarding XYZ-2 business customers, only 6.10% (25) of available features (410) used for grouping them into two clusters consisting of 3642 and 23,432 customers, respectively. Additionally, the XYZ-2 retail customers grouped into two clusters consisting of 10.44% (6388) and 89.56% (54,780) of the retail portfolio, respectively. In this case, only 9.23% (36) of available features (390) used for grouping the XYZ-1 retail customers.

Having carefully reviewed the clustering results we found common patterns to all datasets. The two groups per financial institution are defined as follows:

1. **Lazy Payers:** 55,086 business customers (i.e., 5144 XYZ-1 customers and 3642 XYZ-2 customers), as well as 131,424 retail customers (i.e., 125,036 XYZ-1 customers and 6388 XYZ-2 customers) who have been clustered as "Cluster-1", could be considered as lazy payers. Lazy payers consist of the group of customers that despite having the financial capacity, they do not usually give strong attention to the billing date of their payment obligation. Hence, they may exceed the payment date, but always pay totally the obligation. Considering the available datasets, found that the maximum days past due ever for the majority of the customers do not exceed 13–18 days. By identifying lazy payers, collection managers are able to allocate better the limited resources so as to maximize recovery rates related to more risky customer segments, and minimize simultaneously communication costs.
2. **Delinquent Customers/Defaulters:** 51,717 business customers (i.e., 28,285 XYZ-1 customers and 23,432 XYZ-2 customers), as well as 120,930 retail customers (i.e., 66,150 XYZ-1 customers and 54,780 XYZ-2 customers) who have

Table 4.6 XYZ-1 Customer segmentation

Dataset	Observations	Data type	Number of features	Number of selected features	Number of customers in cluster-1	Number of customers in cluster-2
XYZ-1 business dataset	79,729	Numerical	369	45 (12.19%)	51,444 (64.52%)	28,285 (35.48%)
XYZ-1 retail dataset	191,186	Numerical	393	56 (14.25%)	125,036 (65.40%)	66,150 (34.60%)

Table 4.7 XYZ-2 Customer segmentation

Dataset	Observations	Data type	Number of features	Number of selected features	Number of customers in cluster-1	Number of customers in cluster-2
XYZ-2 business dataset	27,074	Numerical	410	25 (6.10%)	3642 (13.45%)	23,432 (86.55%)
XYZ-2 retail dataset	61,168	Numerical	390	36 (9.23%)	6388 (10.44%)	54,780 (89.56%)

been clustered as "Cluster-2", could be considered as delinquent customers and defaulters. Delinquent customers are actually customers who fail to keep up with ongoing payment obligations, making thus insufficient payments (i.e., we have missed payments over a period). In general, financial institutions categorized delinquent customers into buckets that represent number of missed payments on an account. Thus, delinquent customers do not pay (some times) or pay partially (very often) their payment obligations. Additionally, defaulters are customers who fail to fulfill an obligation. Considering the available datasets, found that the maximum days past due ever for the majority of the customers exceed 18 days. Since delinquent customers and defaulters represent different degrees of the same problem (i.e., missed payments) further segmentation is needed. Thus, identifying delinquent customers and defaulters is very crucial for designing appropriate collection strategies and applying personalized treatment plans. For this reason, these customers were used for training purposes. Associating them with a probability, useful rules can be extracted.

From a feature selection point of view, it is worth mentioning that only a small number of features used to group customers, addressing in such a way high-dimensional data challenges. Some of the most significant features used to group customers into lazy payers and delinquent customers/defaulters are the following: maximum days past due ever; tenure of a customer, current total balance, average pay amount, number of promise to pays last 3 months, number of active business/retail products, maximum bucket last 6 months, number of agent actions (i.e., collector's effort to communicate with customers), days since last collector-debtor interactions, days since promise to pay, number of previous phone calls, days since last phone call, feature indicating whether a settlement plan is in place or not and so on. Feature selection approach is a crucial step towards an explainable and trustworthy ML since the most significant features and their corresponding values per cluster facilitated decision makers to extract data-driven rules for the segmentation and understand the clustering result with non-technical terms. For instance, in the experiment, lazy payers seem to not participate in any settlement plan, maximum bucket reached last 6 months is not greater than 1 (the bucket 1 depicts a range of 1–30 days), whereas maximum days past due ever is less than 17–18 days.

Classification Results

In this work, clustering results used to produce eight labeled datasets: 4 datasets (XYZ-1 business, XYZ-1 retail, XYZ-2 business and XYZ-2 retail) consisting of lazy payer and non-lazy payers, as well as 4 datasets (XYZ-1 business, XYZ-1 retail, XYZ-2 business and XYZ-2 retail) consisting of low-risk defaulters (consisting of delinquent customers who sometimes do not pay and very often partially pay their payment obligations) and high-risk defaulters (who fail to fulfill an obligation). The main idea is to (1) use lazy-payer-based labeled datasets for

Table 4.8 Classification results

Prediction tool	Classification accuracy on test sets (deep learning)	Classification accuracy on test sets (random forests)
XYZ-1_BLP-PT	97.39%	96.38%
XYZ-1_RLP-PT	98.10%	96.45%
XYZ-2_BLP-PT	96.43%	94.28%
XYZ-2_RLP-PT	97.05%	95.33%
XYZ-1_BD-PT	98.71%	98.17%
XYZ-1_RD-PT	98.85%	98.38%
XYZ-2_BD-PT	96.08%	95.37%
XYZ-2_RD-PT	94.99%	93.29%

building prediction models for lazy payers and (2) group initially the cluster of delinquent customers/defaulters found, with respect to experts' knowledge, into two groups (low-risk and high-risk defaulters) for building prediction models for high-risk defaulters. For this purpose, labeled datasets used to train deep learning and random forests for classification purposes. The training dataset is divided into training and test sets with an 70–30 split (i.e., 70% of the dataset to train a machine learning model and 30% to test the trained model). Furthermore, a tenfold cross validation used to test the performance of supervised machine learning algorithms. Table 4.8 presents the classification accuracy scores on tests sets for the following eight prediction models:

1. XYZ-1 business lazy payers prediction tool (XYZ-1_BLP-PT)
2. XYZ-1 retail lazy payers prediction tool (XYZ-1_RLP-PT)
3. XYZ-2 business lazy payers prediction tool (XYZ-2_BLP-PT)
4. XYZ-2 retail lazy payers prediction tool (XYZ-2_RLP-PT)
5. XYZ-1 business defaulters prediction tool (XYZ-1_BD-PT)
6. XYZ-1 retail lazy defaulters prediction tool (XYZ-1_RD-PT)
7. XYZ-2 business defaulters prediction tool (XYZ-2_BD-PT)
8. XYZ-2 retail defaulters prediction tool (XYZ-2_RD-PT)

The results indicate that the separations among the two clusters are represented very well by our models. We can see that generally the deep learning model performs better that random forests model, whereas the average accuracy of models on test datasets is more than 90%.

4.4 Conclusion and Scope for the Future Work

Financial institutions have realized that large-scale data-driven accurate customer segmentation is of fundamental importance. In this work, a feature selection based evolutionary clustering ensemble (FS-ECEA) algorithm for customer segmentation proposed. The suggested method uses machine learning algorithms in hybrid

synthesis with metaheuristics to solve large-scale data clustering problems. The performance of the solution approach tested on UCI datasets and real-world datasets. Experiments indicate that the proposed algorithm tends to use small number of features with respect to the feature space to cluster data points and increase the percentage of the correct clustered samples. A training pipeline also introduced for using labeled datasets (i.e., the clustering result) for prediction purposes. Deep learning and random forests models used, obtaining high classification accuracy scores.

There are several further lines of investigation. Future research is intended to be focused on using different algorithms to (1) the feature selection phase (e.g., particle swarm optimization, ant colony optimization, greedy randomized adaptive search procedure, tabu search), (2) the clustering phase (e.g., density based and spectral clustering approaches), and (3) the classification phase (e.g., gradient boosting approaches such as XGBoost).

Acknowledgments The authors would like to thank EXUS financial solutions company for its support with respect to the work described here.

References

1. Monge, M., Quesada-López, Martínez, A., & Jenkins, M. (2021). Data mining and machine learning techniques for bank customers segmentation: A systematic mapping study. In: K. Arai, S. Kapoor, & R. Bhatia (Eds.), *Intelligent systems and applications: Proceedings of the 2020 intelligent systems conference (IntelliSys) Volume 2* (pp. 666–684). Springer. https://doi.org/10.1007/978-3-030-55187-2_48
2. Hong Kong Institute for Monetary and Financial Research (2020). *Artificial intelligence in banking: The changing landscape in compliance and supervision*. HKIMR Applied Research Report No. 2/2020.
3. Chawla, D., & Joshi, H. (2021). Segmenting mobile banking users based on the usage of mobile banking services. *Global Business Review, 22*(3), 68–704.
4. Bijak, K., & Thomas, L. (2012). Does segmentation always improve model performance in credit scoring? *Expert Systems with Applications, 39*, 2433–2442.
5. Thomas, L., Edelman, D., & Crook, J. (2002). *Credit scoring and its applications*. SIAM.
6. Baesens, B., Rösch, D., & Scheule, H. (2016). *Credit risk analytics: Measurement techniques, applications, and examples in SAS®*. Wiley.
7. Bequé, A., Coussement, K., Gayler, R., & Lessmann, S. (2017). Approaches for credit scorecard calibration: An empirical analysis. *Knowledge-based Systems, 134*, 213–227.
8. Lappas, P. Z., & Yannacopoulos, A. N. (2021). Credit scoring: A constrained optimization framework with hybrid evolutionary feature selection. In: B. Christiansen, & T. Škrinjarić (Eds.), *Handbook of research on applied AI for international business and marketing applications* (pp. 580–605). IGI Global. https://doi.org/10.4018/978-1-7998-5077-9.ch028
9. Hsieh, N.-C. (2004). An integrated data mining and behavioral scoring model for analyzing bank customers. *Expert Systems with Applications, 27*, 623–633.
10. Bizhani, M., & Tarokh, M.-J. (2011). Behavioral rules of bank's point-of-sale for segments description and scoring prediction. *International Journal of Industrial Engineering Computations, 2*, 337–350.
11. Bahrami, M., Bozkaya, B., & Balcisoy, S. (2020). Using behavioral analytics to predict customer invoice payment. *Big Data, 8*(1), 25–37.

12. Liao, S.-H., Chu, P.-H., & Hsiao, P.-Y. (2012). Data mining techniques and applications. *Expert Systems with Applications, 39*, 11303–11311.
13. Mirza, S., Mittal, S., & Zaman, M. (2016). A review of data mining literature. *International Journal of Computer Science and Information Security, 14*(11), 437–442.
14. Aggarwal, C., & Reddy, C. (2014). *Data clustering: Algorithms and applications*. CRC Press.
15. Bandyopadhyay, S., & Saha, S. (2013). *Unsupervised classification: Similarity measures, classical and metaheuristic approaches, and applications*. Springer.
16. Bizhani, M., & Tarokh, M.-J. (2010). Behavioral segmentation of bank's point-of-sales using RF*M* approach. *Proceedings of the 2010 IEEE 6th International Conference on Intelligent Computer Communication and Processing* (pp. 81–86). https://doi.org/10.1109/ICCP.2010.5606461
17. Rezaeinia, S.-M., Keramati, A., & Albadvi, A. (2012). An integrated AHP-RFM method to banking customer segmentation. *International Journal of Electronic Customer Relationship Management, 6*(2), 153–168.
18. Barman, D., & Chowdhury, N. (2019). A novel approach for the customer segmentation using clustering through self-organizing map. *International Journal of Business Analytics, 6*(2), 23–45.
19. Bach, M.-P., Juković, S., Dumičić, K., & Šarlija, N. (2013). Business client segmentation in banking using self-organizing maps. *South East European Journal of Economics and Business, 8*(2), 32–41.
20. Seret, A., Bejinaru, A., & Baesens, B. (2015). Domain knowledge based segmentation of online banking customers. *Intelligent Data Analysis, 19*, S163–S184.
21. Wang, G., Li, F., Zhang, P., Tian, Y., & Shi, Y. (2009). Data mining for customer segmentation in personal financial market. In: Y. Shi, S. Wang, J. Li, & Y. Zeng (Eds.), *Cutting-edge research topics on multiple criteria decision making* (pp. 614–621). Springer. https://doi.org/10.1007/978-3-642-02298-2_90
22. Sivasankar, E., & Vijaya, J. (2017). Customer segmentation by various clustering approaches and building an effective hybrid learning system on churn prediction dataset. In: H. Behera, & D. Mohapatra (Eds), *Computational Intelligence in Data Mining* (pp. 181–191). Springer. https://doi.org/10.1007/978-981-10-3874-7_18
23. Barman, D., Shaw, K.-K., Tudu, A., & Chowdhury, N. (2016). Classification of bank direct marketing data using subsets of training data. In: S. Satapathy, J. Mandal, S. Udgata, & V. Bhateja (Eds.), *Information systems design and intelligent applications* (pp. 143–151). Springer. https://doi.org/10.1007/978-81-322-2757-1_16
24. Rashinkar, P., & Krushnasamy, V. S. (2017). An overview of data fusion techniques. *Proceedings of the 2017 International Conference on Innovative Mechanisms for Industry Applications (ICIMIA)*, Bangalore (pp. 694–697).
25. Meng, T., Jing, X., Yan, Z., & Pedrycz, W. (2020). A survey on machine learning for data fusion. *Information Fusion, 57*, 115–129.
26. Oliveira, G., Coutinho, F., Campello, G., & Naldi, M. (2017). Improving k-means through distributed scalable metaheuristics. *Neurocomputing, 24*, 45–57.
27. Jamel, A., & Akay, B. (2019). A survey and systematic categorization of parallel K-means and Fuzzy-c-Means algorithms. *International Journal of Computer Systems Science and Engineering, 5*, 259–281.
28. Tsai, C.-W., Liu, S.-J., & Wang Y.-C. (2018). A parallel metaheuristic data clustering framework for cloud. *Journal of Parallel and Distributed Computing, 116*, 39–49.
29. Hossain, M., Sebestyen, M., Mayank, D., Ardakanian, O., & Khazaei, H. (2020). Large-scale data-driven segmentation of banking customers. *Proceedings of the 2020 IEEE International Conference on Big Data* (pp. 4392–4401). https://doi.org/10.1109/BigData50022.2020.9378483
30. Motevali, M., Shanghooshabad, A., Aram, R., & Keshavarz, H. (2019). WHO: A new evolutionary algorithm bio-inspired of wildebeests with a case study on bank customer segmentation. *International Journal of Pattern Recognition and Artificial Intelligence, 33*(5), 1959017.

31. Dhaenens, C., & Jourdan, L. (2016). *Metaheuristics for Big Data*. Wiley.
32. Talbi, E.-G. (2009). *Metaheuristics: From design to implementation*. Wiley
33. José-García, A., & Gómez-Flores, W. (2016). Automatic clustering using nature-inspired metaheuristics: A survey. *Applied Soft Computing Journal, 41*, 192–213.
34. Ezugwu, A., Shukla, A., Agbaje, M., Oyelade, O., José-García, A., & Agushaka, J. (2020). Automatic clustering algorithms: a systematic review and bibliometric analysis of relevant literature. Neural Computing and Applications, 33, 6247–6306. https://doi.org/10.1007/s00521-020-05395-4
35. Ezugwu, A. (2020). Nature-inspired metaheuristic techniques for automatic clustering: A survey and performance study. *SN Applied Sciences, 2*, 273.
36. Mehrmolaei, S., Keyvanpour, M., & Savargiv, M. (2020). Metaheuristics on time series clustering problem: Theoretical and empirical evaluation. *Evolutionary Intelligence*. https://doi.org/10.1007/s12065-020-00511-8
37. Mohanty, P., Nayak, S., Mohapatra, U., & Mishra, D. (2019). A survey on partitional clustering using single-objective metaheuristic approach. *International Journal of Innovative Computing and Applications, 10(3–4)*, 207–226.
38. Nanda, S., & Panda, G. (2014). A survey on nature inspired metaheuristic algorithms for partitional clustering. *Swarm and Evolutionary Computation, 16*, 1–18.
39. Nguyen, Q., & Rayward-Smith, V. J. (2011). CLAM: Clustering large applications using metaheuristics. *Journal of Mathematical Modelling and Algorithms, 10*(1), 57–78.
40. Gribel, D., & Vidal, T. (2019). HG-Means: A scalable hybrid genetic algorithm for minimum sum-of-squared clustering. *Pattern Recognition, 88*, 569–583.
41. Naik, B., Mahaptra, S., Nayak, J., & Behera, H. (2017). Fuzzy clustering with improved swarm optimization and genetic algorithm: Hybrid approach. In: H. Behera, & D. Mohapatra (Eds.), *Computational Intelligence in Data Mining* (pp. 237–247). Springer. https://doi.org/10.1007/978-981-10-3874-7_23
42. Nayak, J., Nanda, M., Nayak, K., Naik, B., & Behera, H. (2014). An improved firefly fuzzy c-means (FAFCM) algorithm for clustering real world data sets. In: M. Kumar, D. Mohapatra, A. Konar, & A. Chakraborty (Eds.), *Advanced Computing, Networking and Informatics* (pp. 339–348). Springer. https://doi.org/10.1007/978-3-319-07353-8_40
43. Valdez, F., Castillo, O., & Melin, P. (2021). Bio-inspired algorithms and its applications for optimization in fuzzy clustering. *Algorithms, 14*(4), 122.
44. Kuo, R., Zheng, Y., & Nguyen, T. P. Q. (2021). Metaheuristic-based possibilistic fuzzy k-modes algorithms for categorical data clustering. *Information Sciences, 557*, 1–15.
45. Consoli, S., Korst, J., Pauws, S., & Geleijnse, G. (2020). Improved metaheuristics for the quartet method of hierarchical clustering. *Journal of Global Optimization, 78*, 241–270.
46. Pacheco, J. (2005). A scatter search approach for the minimum sum-of-squares clustering problem. *Computers and Operations Research, 32*, 1325–1335.
47. Nayak, S., Rout, P., & Jagadev, A. (2019). Multi-objective clustering: a kernel based approach using differential evolution. *Connection Science, 31*(3), 294–321.
48. Hu, K.-C., Tsai, C.-W., & Chiang, M.-C. (2020). A multiple-search multi-start framework for metaheuristics for clustering problems. *IEEE Access, 8*, 96173–96183.
49. Belacel, N., Hansen, P., & Mladenović, N. (2002). Fuzzy J-Means: A new heuristic for fuzzy clustering. *Pattern Recognition, 35*, 2193–2200.
50. Senthilnath, J., Kulkarni, S., Suresh, S., Yang, X., & Benediktsson, J. (2019). FPA clust: evaluation of the flower pollination algorithm for data clustering. *Evolutionary Intelligence, 14*, 1189–1199. https://doi.org/10.1007/s12065-019-00254-1
51. Hansen, P., & Mladenović, N. (2001). J-Means: A new local search heuristic for minimum sum of squared clustering. *Pattern Recognition, 34*, 405–413.
52. Kumar, V., Chhabra, J., & Kumar, D. (2017). Grey wolf algorithm-based clustering technique. *Journal of Intelligent Systems, 26*(1), 153–168.
53. Bonab, M., Hashim, S., Haurt, T., & Kheng, G. (2019). A new swarm-based simulated annealing hyper-heuristic algorithm for clustering problem. *Procedia Computer Science, 163*, 228–236.

54. González-Almagro, G., Luengo, J., Cano, J.-R., & García, S. (2020). DILS: Constrained clustering through dual iterative local search. *Computers and Operations Research, 121*, 104979.
55. Liu, Y., Wang, L., & Chen, K. (2005). A tabu search based method for minimum sum of squares clustering. In: S. Singh, M. Singh, C. Apte, & P. Perner (Eds.), *Pattern Recognition and Data Mining* (pp. 248–256). Springer. https://doi.org/10.1007/11551188_27
56. Dowlatshahi, M. B., & Nezamabadi-pour, H. (2014). GGSA: A grouping gravitational search algorithm for data clustering. *Engineering Applications of Artificial Intelligence, 36*, 114–121.
57. Gyamfi, K., Brusey, J., & Hunt, A. (2017). K-means clustering using tabu search with quantized means. arXiv:1703.08440v1 [cs.LG]
58. Boushaki, S., Kamel, N., & Bendjeghaba, O. (2018). A new quantum chaotic cuckoo search algorithm for data clustering. *Expert Systems with Applications, 96*, 358–372.
59. Kuo, R., & Zulvia, F. (2020). Multi-objective cluster analysis using a gradient evolution algorithm. *Soft Computing, 24*, 11545–11559.
60. Liu, Y., & Shen, Y. (2010). Data clustering with cat swarm optimization. *Journal of Convergence Information Technology, 5*(8), 21–28.
61. Kamel, N., & Boucheta, R. (2014). A new clustering algorithm based on chameleon army strategy. In: S. Boonkrong, H. Unger, & P. Meesad (Eds.), *Recent Advances in Information and Communication Technology* (pp. 23–32). Springer. https://doi.org/10.1007/978-3-319-06538-0_3
62. Harifi, S., Khalilian, M., Mohammadzadeh, J., & Ebrahimnejad, S. (2020). New generation of metaheuristics by inspiration from ancient. *Proceedings of the 2020 10th International Conference on Computer and Knowledge Engineering* (pp. 256–261). https://doi.org/10.1109/ICCKE50421.2020.9303653
63. Kaur, A., & Kumar, Y. (2021). A new metaheuristic algorithm based on water wave optimization for data clustering. *Evolutionary Intelligence*. https://doi.org/10.1007/s12065-020-00562-x
64. Irsalinda, N., Yanto, I., Chiroma, H., & Herawan, T. (2017). A framework of clustering based on chicken swarm optimization. In: T. Herawan, R. Ghazali, N. Nawi, & M. Deris (Eds.), *Recent Advances on Soft Computing and Data Mining* (pp. 336–343). Springer. https://doi.org/10.1007/978-3-319-51281-5_34
65. Komarasamy, G., & Wahi, A. (2012). An optimized K-means clustering technique using bat algorithm. *European Journal of Scientific Research, 84*(2), 263–273.
66. Alshamiri, A. K., Singh, A., & Surampudi, R. B. (2016). Artificial bee colony algorithm for clustering: An extreme learning approach. *Soft Computing, 20*, 3163–3176.
67. Kumar, S., Datta, D., & Singh, S. (2015). Black hole algorithm and its applications. In: A. Azar, & S. Vaidyanathan (Eds.), *Computational Intelligence Applications in Modeling and Contro* (pp. 147–170). Springer. https://doi.org/10.1007/978-3-319-11017-2_7
68. Mageshkumar, C., Karthik, S., & Arunachalam, V. (2019). Hybrid metaheuristic algorithm for improving the efficiency of data clustering. *Cluster Computing, 22*, S435–S442.
69. Silva-Filho, T., Pimentel, B., Souza, R., & Oliveira, A. (2015). Hybrid methods for fuzzy clustering based on fuzzy c-means and improved particle swarm optimization. *Expert Systems with Applications, 42*, 6315–6328.
70. Sharma, M., & Chhabra, J. (2019). An efficient hybrid PSO polygamous crossover based clustering algorithm. *Evolutionary Intelligence, 14*, 1213–1231. https://doi.org/10.1007/s12065-019-00235-4
71. Zheng, L., Chao, F., Parthaláin, N., Zhang, D., & Shen, Q. (2021). Feature grouping and selection: A graph-based approach. *Information Sciences, 546*, 1256–1272.
72. Niño-Adan, I., Manjarres, D., Landa-Torres, I., & Portillo, E. (2021). Feature weighting methods: A review. *Expert Systems with Applications, 184*, 115424.
73. Wang, L., Wang, Y., & Chang, Q. (2016). Feature selection methods for big data bioinformatics: A survey from the search perspective. *Methods, 111*, 21–31.
74. Moshki, M., Kabiri, P., & Mohebaljojeh, A. (2015). Scalable feature selection in high-dimensional data based on grasp. *Applied Artificial Intelligence, 29*, 283–296.

75. Sarhani, M., & Vob, S. (2021). Chunking and cooperation in particle swarm optimization for feature selection. *Annals of Mathematics and Artificial Intelligence*. https://doi.org/10.1007/s10472-021-09752-4
76. Ji, B., Lu, X., Sun, G., Zhang, W., Li, J., & Xiao, Y. (2020). Bio-inspired feature selection: An improved binary particle swarm optimization approach. *IEEE Access, 8*, 85989–86002. https://doi.org/10.1109/ACCESS.2020.2992752
77. Lappas, P. Z., & Yannacopoulos, A. N. (2021). A machine learning approach combining expert knowledge with genetic algorithms in feature selection for credit risk assessment. *Applied Soft Computing Journal, 107*, 107391.
78. Kumar, V., & Kumar, D. (2019). Automatic clustering and feature selection using gravitational search algorithm and its application to microarray data analysis. *Neural Computing and Applications, 31*, 3647–3663.
79. Prakash, J., & Singh, P. (2019). Gravitational search algorithm and K-means for simultaneous feature selection and data clustering: A multi-objective approach. *Soft Computing, 23*, 2083–2100.
80. Gunning, D., Stefik, M., Choi, J., Miller, T., Stumpf, S., & Yang, G.-Z. (2019). XAI – Explainable artificial intelligence. *Science Robotics, 4*(37), 1–2.
81. Roselli, D., Matthews, J., & Talagala, N. (2019). Managing bias in AI. In L. Liu, & R. White (Eds.), *WWW '19: Companion Proceedings of the 2019 World Wide Web Conference* (pp. 539–544). San Francisco, USA.
82. Gaikwad, M. R., Umbarkar, A. J., & Bamane, S. S. (2020). Large-scale data clustering using improved artificial bee colony algorithm. In: M. Tuba, S. Akashe, & A. Joshi (Eds.), *ICT Systems and Sustainability* (pp. 467–475). Springer. https://doi.org/10.1007/978-981-15-0936-0_50
83. Marinakis, Y., Marinaki, M., Matsatsinis, N., & Zopounidis, C. (2008). A memetic-grasp algorithm for clustering. *Proceedings of the 10th International Conference on Enterprise Information Systems – AIDSS* (pp. 36–43). https://doi.org/10.5220/0001694700360043
84. Kowalski, P., Łukasik, S., Charytanowicz, M., & Kulczycki, P. (2019). Nature inspired clustering – Use cases of krill herd algorithm and flower pollination algorithm. In: L. Kóczy, J. Medina-Moreno, & E. Ramírez-Poussa (Eds), *Interactions between Computational Intelligence and Mathematics* (pp. 83–98). Springer. https://doi.org/10.1007/978-3-030-01632-6_6
85. Marinakis, Y., Marinaki, M., & Matsatsinis, N. (2007). A hybrid particle swarm optimization algorithm for clustering analysis. In: I. Y. Song, J. Eder, & T. M. Nguyen (Eds.), *Data Warehousing and Knowledge Discovery* (pp. 241–250). Springer. https://doi.org/10.1007/978-3-540-74553-2_22
86. Marinakis, Y., Marinaki, M., & Matsatsinis, N. (2008). A stochastic nature inspired metaheuristic for cluster analysis. *International Journal of Business Intelligence and Data Mining, 3*(1), 30–44.
87. Marinakis, Y., Marinaki, M., Matsatsinis, N. (2008). A hybrid clustering algorithm based on multi-swarm constriction PSO and GRASP. In: I.-Y. Song, J. Eder, & T. M. Nguyen (Eds.), Data Warehousing and Knowledge Discovery (pp. 186–195). Springer.
88. Marinakis, Y., Marinaki, M., & Matsatsinis, N. (2009). A hybrid bumble bees mating optimization – GRASP algorithm for clustering. In: E. Corchado, X. Wu, E. Oja, Á. Herrero, & B. Baruque (Eds.), *Hybrid Artificial Intelligence Systems* (pp. 549–556). Springer.
89. Marinakis, Y., Marinaki, M., Doumpos, M., Matsatsinis, N., & Zopounidis, C. (2011). A hybrid ACO-GRASP algorithm for clustering analysis. *Annals of Operations Research, 188*(1), 343–358.
90. Saida, I., Nadjet, K., & Omar, B. (2014). A new algorithm for data clustering based in cuckoo search optimization. In: J. S. Pan, P. Krömer, & V. Snášel (Eds.), *Genetic and Evolutionary Computing* (pp. 55–64). Springer. https://doi.org/10.1007/978-3-319-01796-9_6
91. Singh, T., Saxena, N., Khurana, M., Singh, D., & Abdalla, M. (2021). Data clustering using moth-flame optimization algorithm. Sensors, 21, 4086.

92. Tian, Z., Fong, S., Wong, R., & Millham, R. (2016). Elephant search algorithm on data clustering. *Proceedings of 2016 12th International Conference on Natural Computation, Fuzzy Systems and Knowledge Discovery* (pp. 787–793). https://doi.org/10.1109/FSKD.2016.7603276
93. Cho, P. P. W., & Nyunt, T. T. S. (2020). Data clustering based on differential evolution with modified mutation strategy. *Proceedings of the 2020 17th International Conference on Electrical Engineering/Electronics, Computer, Telecommunications and Information Technology* (pp. 222–225). https://doi.org/10.1109/ECTI-CON49241.2020.9158243
94. Kuo, R. J., Amornnikun, P., & Nguyen, T. P. Q. (2020). Metaheuristic-based possibilistic multivariate fuzzy weighted c-means algorithms for market segmentation. *Applied Soft Computing Journal, 96*, 106639.
95. Eskandari, S., & Javidi, M. M. (2019). A novel hybrid bat algorithm with a fast clustering-based hybridization. *Evolutionary Intelligence*. https://doi.org/10.1007/s12065-019-00307-5
96. Agbaje, M., Ezugwu, A., & Els, E. (2019). Automatic data clustering using hybrid firefly particle swarm optimization algorithm. *IEEE Access, 7*, 184963–184984. https://doi.org/10.1109/ACCESS.2019.2960925
97. Wu, Z.-X., Huang, K.-W., Chen, J.-L., & Yang, C.-S. (2019). A memetic fuzzy whale optimization algorithm for data clustering. *Proceedings of 2019 IEEE Congress on Evolutionary Computation* (pp. 1446–1452). https://doi.org/10.1109/CEC.2019.8790044
98. Hatamlou, A., & Hatamlou, M. (2013). PSOHS: An efficient two-stage approach for data clustering. *Memetic Computing, 5*, 155–161.
99. Mitchell, M. (1998). *An introduction to genetic algorithms*. MIT Press.
100. Kelleher, J. D. (2019). *Deep learning*. MIT Press.
101. Pavlov, Y. (2000). *Random forests*. VSP.

Chapter 5
Evolving Machine Learning-Based Classifiers by Metaheuristic Approach for Underwater Sonar Target Detection and Recognition

M. Khishe, H. Javdanfar, M. Kazemirad, and H. Mohammadi

Abstract Nowadays, considering their acoustic radiated signal, the classification of underwater sonar targets encompasses a broad spectrum of targets and techniques. However, classifying various sonar targets, such as maritime vessels, is difficult for academics, who must consider both military and commercial considerations. Support Vector Machines (SVMs) are the most operational type of machine learning model for creating classifiers. The most difficult aspect of SVM networks is the learning algorithms.Therefore, in this chapter, in order to have a reliable and accurate underwater sonar target classifier, we investigate the performance of ten metaheuristic algorithms, including slime mould algorithm (SMA), marine predator algorithm (MPA), Kalman filter (KF), Harris Hawks optimization (HHO), genetic algorithm (GA), particle swarm optimization (PSO), Henry gas solubility optimization (HGSO), chimp optimization algorithm (ChOA), gray wolf optimizer (GWO), and whale optimization algorithm (WOA), in underwater sonar target classification problem.

5.1 Introduction

There has been a lot of interest in underwater sonar target identification and recognition recently because of the great complexity of the issue, which requires the use of advanced algorithms [65, 68, 69]. Background clutter and targets can be differentiated based on the use of backscattered echoes [26]. The following features further alter the parameters of the backscattered echoes [61]: time dispersions, ambient sounds, and time-varying propagation pathways. Researchers have suggested two basic ways to detect and eliminate false alarms and targets, stochastic

M. Khishe (✉) · H. Javdanfar · M. Kazemirad · H. Mohammadi
Department of Marine Electronics and Communication Engineering, Imam Khomeini Marine Science University, Nowshahr, Iran
e-mail: m_khishe@alumni.iust.ac.ir

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

M. Eddaly et al. (eds.), *Metaheuristics for Machine Learning*,
Computational Intelligence Methods and Applications,
https://doi.org/10.1007/978-981-19-3888-7_5

and deterministic [29, 56]. Statistical methods, sonar models, and oceanography are central to deterministic methodologies [37]. Even if deterministic methods have been shown to be flawed, there are still arguments for using them. These networks involve high costs in terms of personnel and equipment. In addition, they cannot provide the robustness and comprehensiveness of other testing fields [5]. However, stochastic approaches are more affordable and less difficult. They are valuable for real-time surveillance systems because of their high accuracy [52], versatility [51], and parallel structure [36]. Artificial neural networks (ANNs) are known for their strength, robustness, and remarkable fault tolerance, and they can store extensive, accurate non-linear input-output mappings by training on a vast quantity of input-output data without disclosing the underlying model. ANNs have been put to use in underwater sonar target detection and recognition systems, which is why they are gaining in popularity. According to Neruda et al. [49], conventional ANNs face shortcomings in convergence speed and the ease of reversion to local optima, which makes their performance need to be improved. SVMs address ANNs' shortcomings as an alternative approach. Also, SVM can be used in conjunction with other intelligent algorithms to produce false-alarm samples with an accuracy rate comparable to recognition accuracy. A survey of NP-hard tasks, such as underwater sonar target detection and recognition systems, indicated that support vector machines (SVMs) are among the most widely employed models [9]. Even though SVMs are used in a variety of contexts, their primary asset is their capacity to learn. RBF kernel for SVM training relies on two sequential phases. The first step they take is to utilize unsupervised clustering methods to select the best hidden layer centers and widths [55]. The hidden and output layers are trained using linear weights [49]. Implementing standard strategies to train the SVM network leads to the network getting stuck in local minima and an overall poor detection accuracy and convergence rate. The restrictions on multimodal search spaces like underwater security systems are especially significant. Moreover, setting the initial parameters in the most common training methods is a time-consuming challenge [47]. Researchers were driven to find new meta-heuristic techniques to replace SVM training in light of these inadequacies [10]. It is well known that stochastic meta-heuristic algorithms excel in solving complex, multidimensional search spaces [4, 67]. These features include simple structure, robust and reliable optimization, flexible implementation, the solution to major problems, free of gradients, and more effectiveness in the search for a comprehensive solution [62]. Several meta-heuristic technique that are used to train SVMs under various training schemes are particle swarm optimizer (PSO) [34], ant colony optimization (ACO) algorithm [50], genetic algorithm (GA) [23], ant lion optimizer (ALO) [59], biogeography-based optimization (BBO) and artificial bee colony (ABC) [1], differential evolution (DE) [58], evolutionary programming (EP) [64], moth-flame optimizer (MFO) [12], whale optimization algorithm (WOA) [19], firefly algorithm (FA) [18], gray wolf optimizer (GWO) [35], and chimp optimization algorithm (ChOA) [27, 28]. A novel method of training SVMs starts with looking for all the parameters they need in a single phase. Although it appears simple, this training program can prove to be hard. A wide search space and the difficulty of finding optimal solutions are

two primary challenges in underwater surveillance and security systems. This is why approaches like GA, which are based on evaluation, are slow and require more generations to converge [14]. In other words, the NFL theorem indicates that no optimization technique can produce the optimal answer for every optimization process. Therefore, some algorithms are better for some optimization problems than others [33]. Due to this, researchers still find this field interesting. Research studies [2, 20, 25, 30–32, 42–46, 48, 54, 57, 60] show that nature-inspired algorithms and metaheuristics have the potential to be an algorithm that avoids trapping in local minima in NP-hard situations. To make a better target detector, we employed metaheuristics to train an SVM neural network. It should be noted that our training technique is based on optimizing all network parameters simultaneously, i. e. the connection weights and kernel parameters (centers, and widths). For this reason, we studied the accuracy and reliability of several metaheuristic algorithms in underwater sonar target categorization, including SMA [33], MPA [11], KF [22], HHO [17], GA [38], PSO [24], HGSO [16], ChOA [27], GWO [40], and WOA [39]. The metaheuristic detectors are then evaluated using three datasets: (1) a laboratory cavitation tunnel dataset, (2) a real-world underwater experimental dataset, and (3) a benchmark common dataset 2015 (CDS2015). The following sections outline the remainder of the paper. Section 5.2 covers some key terms and approaches. Section 5.3 introduces the method proposed in this study. Section 5.4 describes the outcomes of the experiment, and Sect. 5.5 concludes the research.

5.2 Theory of SVM

In the late 1990s, Vapnik et al. [7] developed an innovative learning algorithm called SVM that was based on statistical theory. SVM theory dictates that problems that are difficult to solve using linear methods can be mapped to a higher-dimensional feature space by a non-linear mapping function. As seen in Fig. 5.1, two types of data (grey circles and white circles) are classified in feature space. The SVM attempts to locate a hyperplane separating the two types of data while classifying. The two kinds' hyper-plane and closest data point are maximized in the distance between them for better classification accuracy. The dotted circles in Fig. 5.1 represent the location of the margin, which is defined by the support vectors. The midpoint of the two margins is where the hyper-plane is located. The support vectors have adequate information to establish the hyperplane. Once the support vectors are established, the other data points can permanently be deleted.

An SVM can have great classification accuracy using little original data. The hyperplane in high-dimensional feature space is defined as follows:

$$\varphi(x) = w.\psi(x) + b \tag{5.1}$$

This non-linear mapping function takes data and maps it to the high-dimensional feature space. Equation (5.1) defines a separation hyperplane between the two data

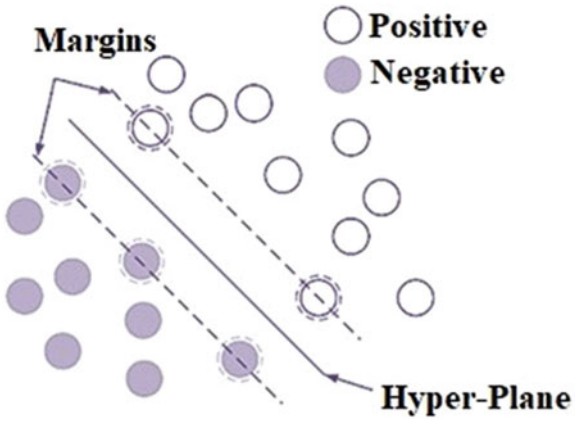


Fig. 5.1 SVM principle

types using a weight vector, a bias term. In Eq. (5.2), the following data set is presented:

$$F = \{x_i, y_i\}_{i=1}^{m} \tag{5.2}$$

For the SVM training data, the x_i is used for the samples, $y_i \in \{-1, 1\}$ identifies the various classes, and m denotes the number of samples. The hyperplane to separate the support vector classifier (SVC) features should be confined under the following constraints:

$$y_i = (w_i.\psi(x_i) + b) \geqslant 1 - \vartheta_i; \;\; i = 1, 2, \ldots, N \tag{5.3}$$

A positive loose coefficient is necessary to accurately measure the range between the margin and the vector X, as this lies on the wrong side. Additionally, the error penalty ε establishes an exchange between both the confidence interval and empirical risk. Finally, an optimization problem related to the matching constraint can be turned into a minimization problem:

$$\begin{cases} min : 0.5\|w\|^2 + \varepsilon \sum_i^N \vartheta_i \\ s.t \quad y_i(w^T.\psi(x_i) + b) \geqslant 1 - \vartheta_i \end{cases} \tag{5.4}$$

In Eq. (5.5), the standard convex quadratic programming issue arises. The Kuhn-Tucker criterion states that the calculation can be simplified by using the Lagrangian multiplier ζ_i:

$$min : L(w, b, \zeta) : 0.5\|w\|^2 + \sum_{i}^{N} \zeta_i + \sum_{i}^{N} \zeta_i.y_i(w_i.\psi(x_i) + b) \tag{5.5}$$

The partial derivative of L concerning w and b is obtained to solve the dual issue. Equation (5.6) can be found because the partial derivative is 0 at the minimum points:

$$w = \sum_{i}^{N} \zeta_i . y_i (w_i . \psi(x_i), \qquad \sum_{i}^{N} \zeta_i . y_i = 0 \tag{5.6}$$

A final optimization equation may be deduced by merging Eqs. (5.5) and (5.6):

$$\begin{cases} min : \partial(\vartheta) = \sum_{i}^{N} \vartheta_i - 0.5 \sum_{i=1}^{N} \sum_{j=1}^{N} \vartheta_i \vartheta_j y_i y_j \psi(x_i) . \psi(x_j) \\ \sum_{i}^{N} \zeta_i . y_i = 0 \\ \eta(x_i, x_j) = \psi(x_i) . \psi(x_j) \end{cases} \tag{5.7}$$

$\eta(x_i, x_j)$ denotes the input space kernel function that was introduced; therefore, the classifier can be described as follows:

$$\varphi(x) = sign(\sum_{i=1}^{N} \sum_{j=1}^{N} \vartheta_i y_i \eta(x_i, x_j) + b) \tag{5.8}$$

Previous research [13] indicates that SVM established by the Gaussian Radial Basis Function (RBF) has a remarkable capacity to classify non-linear relationships. Because of this, the Gaussian RBF [21] is employed to perform the computation in the original input space using Eq. (5.9):

$$\eta(x_i, x_j) = exp(-\frac{(x_i - x_j)^2}{2 \times \delta^2}) \tag{5.9}$$

5.3 Problem Definition

Metaheuristic algorithms are generally used to train SVM algorithms with three general approaches. The first approach uses meta-heuristic techniques to determine the hidden layer centers, connection weights, and bias parameters of an SVM neural network to reduce error values. Another technique of solving this problem uses algorithms to find the SVM fitting structure. The final method uses meta-heuristics to look at parameters such as learning rate and step size of the gradient training algorithm. We use metaheuristics in a support vector machine (SVM) using the first strategy in our current study. To create a proper training procedure for the SVMs, we must define certain parameters in the optimization algorithm. The hidden layer center, output node bias, hidden layer propagating parameters, and connecting

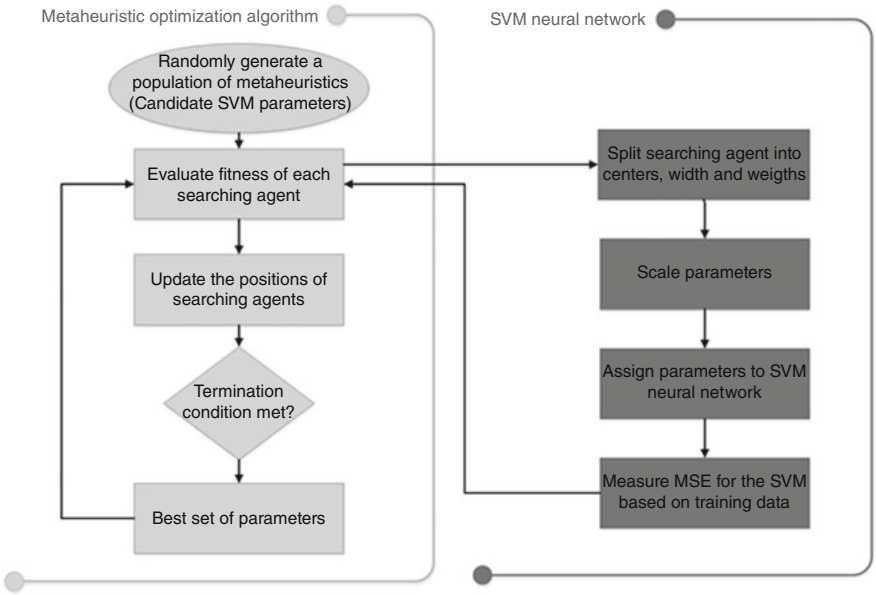


Fig. 5.2 Metaheuristic training of an SVM neural network

Table 5.1 The SVM neural network's optimization parameters with RBF kernel

No	Parameters	Representation
1	w	The connection weights among the output layer and the hidden layers
3	c	The hidden layer's center vectors
3	α	The basic function's propagation parameters of the hidden layer
4	β	The bias parameters

weights are all components of these parameters. Our teaching process can be shown in Fig. 5.2, a general diagram.

Binary state, matrix-based, and vector-based all provide vectors of potential solutions [3, 63]. SVM training requires the knowledge of the SVM's center points, propagation parameters, weights, and biases. As we are working with simple SVMs, we employ vector representation. As we have already discussed, SVM training can find the best variable values, as illustrated in Table 5.1.

Estimating the number of hidden layer neurons in a neural network system is a crucial and challenging undertaking. Using more neurons could lead to overlearning and longer execution times. Studies were undertaken by Aljarah et al. [3], and others have found that hidden layer neurons number 10. The network's performance does not change considerably when the number of neurons in the hidden layer rises, but the network's structural complexity is greatly increased. Equation (5.10) shows one

possible solution to the metaheuristic detectors:

$$P_i = [\vec{w} \;\; \vec{c} \;\; \vec{\alpha} \;\; \vec{\beta}] \tag{5.10}$$

In the current research, the fitness function of the metaheuristic-based detectors can be calculated by the Mean Squared Error (MSE) in Eq. (5.11):

$$MSE = \frac{1}{k}\sum_{i=1}^{k}(y - \hat{y})^2 \tag{5.11}$$

This equation represented the target and evaluated the network's output values, so the core objective is to optimize the possible solutions (Pi) to minimize the MSE value, as stated in Eq. (5.11).

5.4 Experiments and Results

We conducted our tests in three datasets: (1) a laboratory cavitation tunnel dataset (As shown in Fig. 5.3), (2) a real-world underwater experimental dataset, and (3) a benchmark common dataset 2015 (CDS2015). The tests are run on Windows 10 using Matlab R2019a on a PC with the following specifications: Intel Core i7-7800X, 3.8 GHz, 16GB RAM. The AVE and STD results for the tests are presented in the results tables, which include an average of the findings over 30 runs. The best achievements are highlighted in bold font. In order to further improve metaheuristic evaluation, according to [66], non-parametric statistical tests should be used in addition to the statistical tests. In order to see if metaheuristic detectors held any advantages over pairwise comparison, Wilcoxon's sign-rank test [53] was employed. If the p-value from the pairwise comparison is less than 0.05, it signifies that the method is statistically better than others. More importantly, it demonstrates that the two contestants' differences are not statistically significant.

SVM training is performed using metaheuristic algorithms such as SMA, MPA, KF, HHO, GA, PSO, HGSO, ChOA, GWO, and WOA. This complete comparison is made possible by using precision-recall curves and receiver operating characteristics (ROC). Table 5.2 reports the initial settings and the current benchmarks.

In what follows, two experimental underwater target datasets are developed, and a benchmark underwater dataset is used to evaluate the merits of metaheuristic-based detectors.

Table 5.2 Initial values and parameters of the trainers

Algorithm	Parameters	Value
SMA	$\vec{v}b$	$[-\alpha, \alpha]$
	α	$arctanh(-\frac{t}{t_{max}} + 1)$
	r	Random [0, 1]
HHO	v, u	Random (0, 1)
	B	1.5
KF	The covariance matrix, w_k and v_k	40I
MPA	P	0.5
	FADs	0.2
$HGSO$	l_1, l_2, l_3	5E-02, 100, 1E-02
PSO	c_1, c_2	2
WOA	a	Linearly decreased from 2 to 0
$ChOA$	r_1, r_2	Random
	m	Chaotic
GA	Selection	Roulette wheel
	Cross-over	Single-point
GWO	Mutation rate	0.01
	a	Linearly decreased from 2 to 0

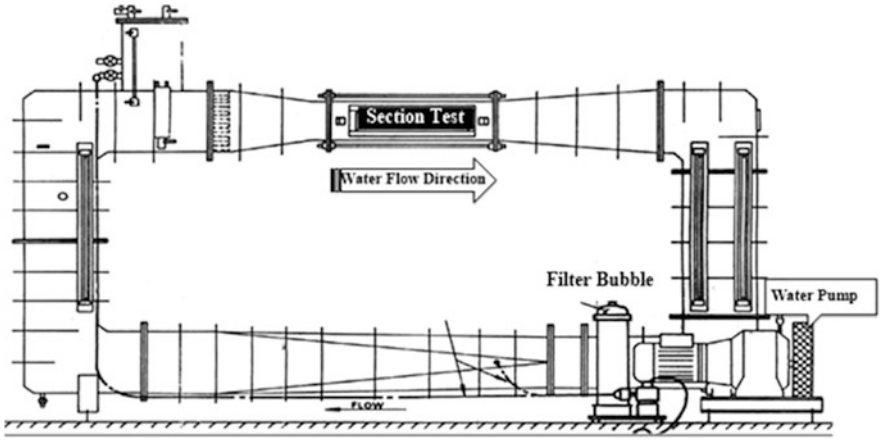


Fig. 5.3 NA-10 England Closed water circuit cavitation tunnel

5.4.1 Laboratory Dataset

Three model propellers with somewhat varied sizes are examined for the test. Table 5.3 summarizes the major characteristics and technical design data for each propeller. The propeller revolutions at 1800 RPM, and the flow velocity is 4 m/s in this experiment. Two B&K 8103 hydrophones were used to measure radiated sound.

Table 5.3 The main characteristics and detailed design parameters for each propeller

Name	Number of blades	Full-scale diameter (m)	AE/AO	P/D	T_0	J
A	5	119 mm	0.849	0.9	1.39	0.89
B	4	89 mm	0.649	0.89	1.39	0.69
C	3	69 mm	0.491	0.49	1.39	0.72

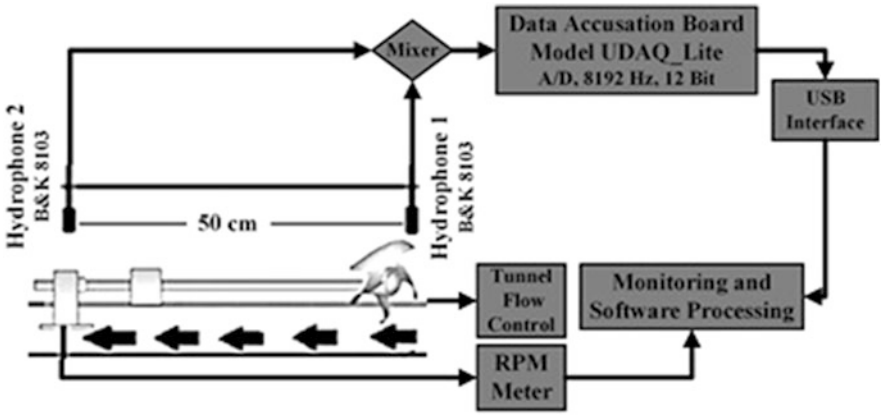


Fig. 5.4 The test scenario and hydrophones configuration

Hydrophones are used in conjunction with the UDAQ Lite data acquisition board model. The data accusation board's specifications may be found in [31]. Figure 5.4 depicts the test situation and hydrophone arrangement.

Where, P/D stands for the pitch-diameter ratio, AE/AO denotes the blade area ratio, T_0 denotes thrust in open water, and J is the advanced coefficient.

5.4.1.1 Noises and Reflections Removal Stages

Assume that $x(t)$ and $y(t)$ represent the reverberant-noisy signals from hydrophones 1 and 2. Additionally, $s(t)$ is the propeller's undistorted source. The following two reverberant signals are mathematically modeled:

$$x(t) = \int_{-\infty}^{t} h(t-\tau)s(\tau) \tag{5.12}$$

$$y(t) = \int_{-\infty}^{t} g(t-\tau)s(\tau) \tag{5.13}$$

Where $g(t)$ and $h(t)$ denotes the square testing section-response functions. It is presumed that these functions are unknown. Nonetheless, g and h have uncorrelated

"tails". It is evident that the initial few echoes do not arrive simultaneously at the two hydrophones.

Underwater acoustics standards [8] specify the sound pressure level (SPL) in dB; re 1Pa, but the output of the hydrophone is according to $\frac{\mu v}{Pa}$. As a result, the output of the hydrophones should be multiplied by 10^6. The frequency-domain SPL will be obtained through the use of hamming windows and following FFTs. It is customary in the literature to decrease the observed SPL values in each 1/3 Octave band to 1 Hz bandwidth using Eq. (5.14) [8].

$$SPL_1 = SPL_m - 10log\Delta f \tag{5.14}$$

SPL_m is measured in dB; also, Δf is the bandwidth for each 1/3 Octave band filter, and SPL_1 is the decreased SPL to 1 Hz bandwidth in dB; re 1 μPa.

In order to discriminate between propeller and background noise, the background noise measurements are adjusted to account for total noise levels. The background noise is captured with dummy hubs used instead of constructed propellers [6]. This adjustment is applied using the procedure described in the reference, considering the extent of the disparity [8]. According to this approach, the difference should be greater than 3 dB before any results are accepted. The results are altered using Eq. (5.15) when the gap is between 3 and 10 dB in a related context. When the discrepancy is more than 10 dB, no correction is made. To summarize, the signal-to-noise ratio determines whether the signal is used or not (i.e., less than 3 dB).

$$SPL_P = 10log(10^{\frac{SPL_T}{10}} - 10^{\frac{SPL_B}{10}}) \tag{5.15}$$

Where B denotes the background, P represents the propeller, and T stands for the total noise. In the next step, SPLs must be modified to a standard measuring distance of 1 m as shown in Eq. (5.16):

$$SPL = SPL_1 + 20log(r) \tag{5.16}$$

Where r denotes the distance between the noise source's center (the propeller disk's center) and the hydrophone, Fig. 5.5 illustrates the whole preprocessing stage of background noise removal.

In Fig. 5.6, just three representative examples of recorded propeller noise are shown, along with their Fourier transform and power spectral density function. As an illustration, Fig. 5.6c illustrates the frequency characteristics of physical processes in detail. However, these reasons can be extrapolated to different propellers and their operating conditions to understand noise measurements better. The fundamental frequency of propeller C (three blades) is 90 Hz, while the other tonal components are the fundamental frequency and revolution rate product. On the other hand, the effects of cavitation noise are shown by increasing values in the continuous region of the spectrum. As a result, it is challenging to detect separate spectrum components of propeller noise.

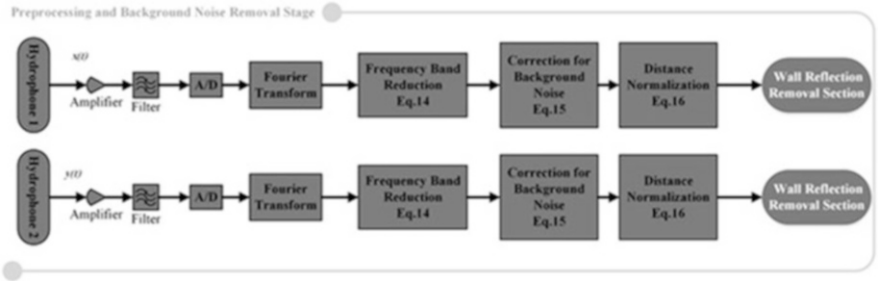


Fig. 5.5 The background noise removal and preprocessing block diagram

Thus, propeller cavitation is typically defined by distinct random components that contribute to the continuous portion of the spectrum. SPL, on the other hand, reduces little as frequency increases. To summarize, the continuous portion of the spectrum represents the broadband noise produced by suction side cavitation and tip vortex.

5.4.2 Common Dataset 2015 (CDS2015)

The Common CDS-2015 Dataset is the second dataset used to evaluate the proposed SVM-metaheuristic detectors [15]. This dataset is a collection of backscatter and bathymetric data obtained using modern shallow water survey technologies to promote comparisons and examine the relative merits of various strategies. The Plymouth Sound and Wembury Bay area was chosen for this purpose because it offers a high degree of variation in terms of depth, subsea conditions, and seabed kinds. The word "shallow water" refers to water depths less than 200 m. Nonetheless, this collection's depths seldom exceed 40 m. Due to the navigational safety standards for sub-40-m surveying, a very high level is required to do this work and acquire data of such high quality. This shallow-water dataset provides a new window into Kongsberg Maritime's current technical capabilities and deep-water surveys advancements. The data shown in Fig. 5.7 are all raw data from. As a result, CDS2015 is raw data without any statistical or manual filtration. Additional details about the experiment can be found in [15].

5.4.3 Experimental Real-World Underwater Dataset

The following paragraphs present a summary of the experimental underwater imagery data set gathered by the authors. This data was acquired during tests conducted on the shallow waters of the Caspian Sea, which are found at depths

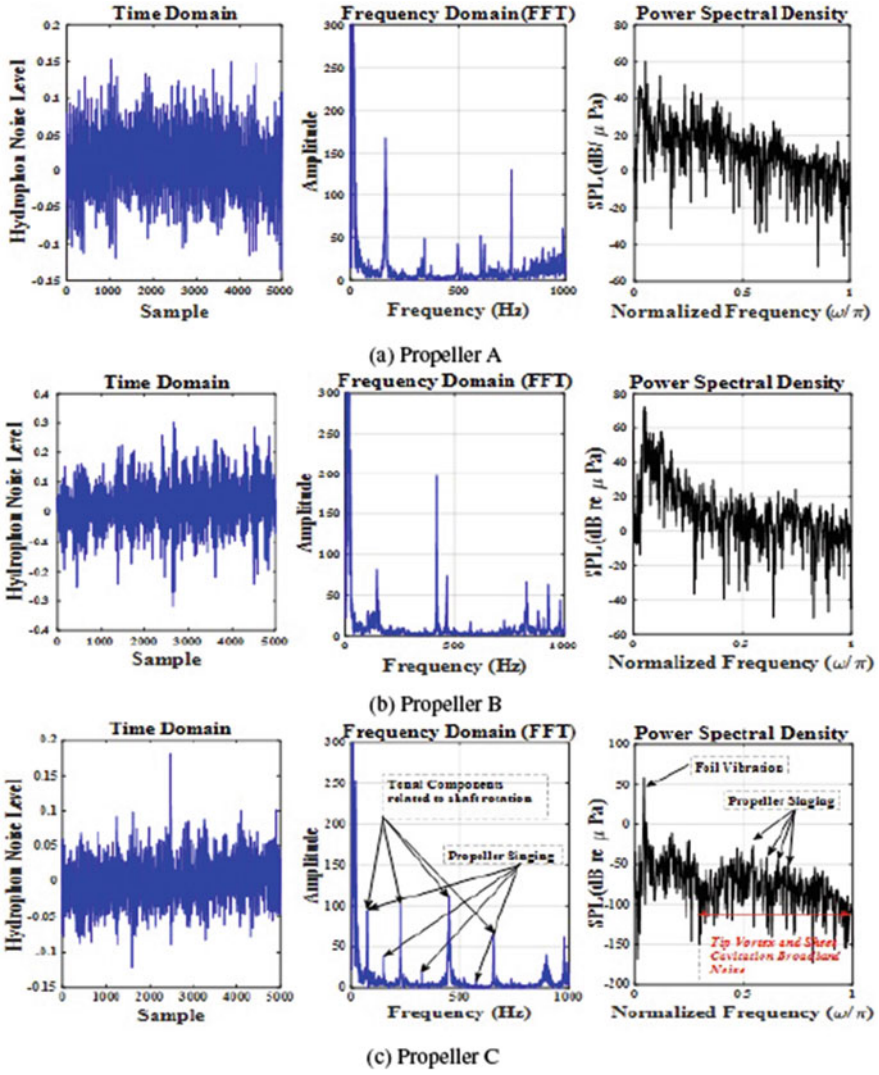
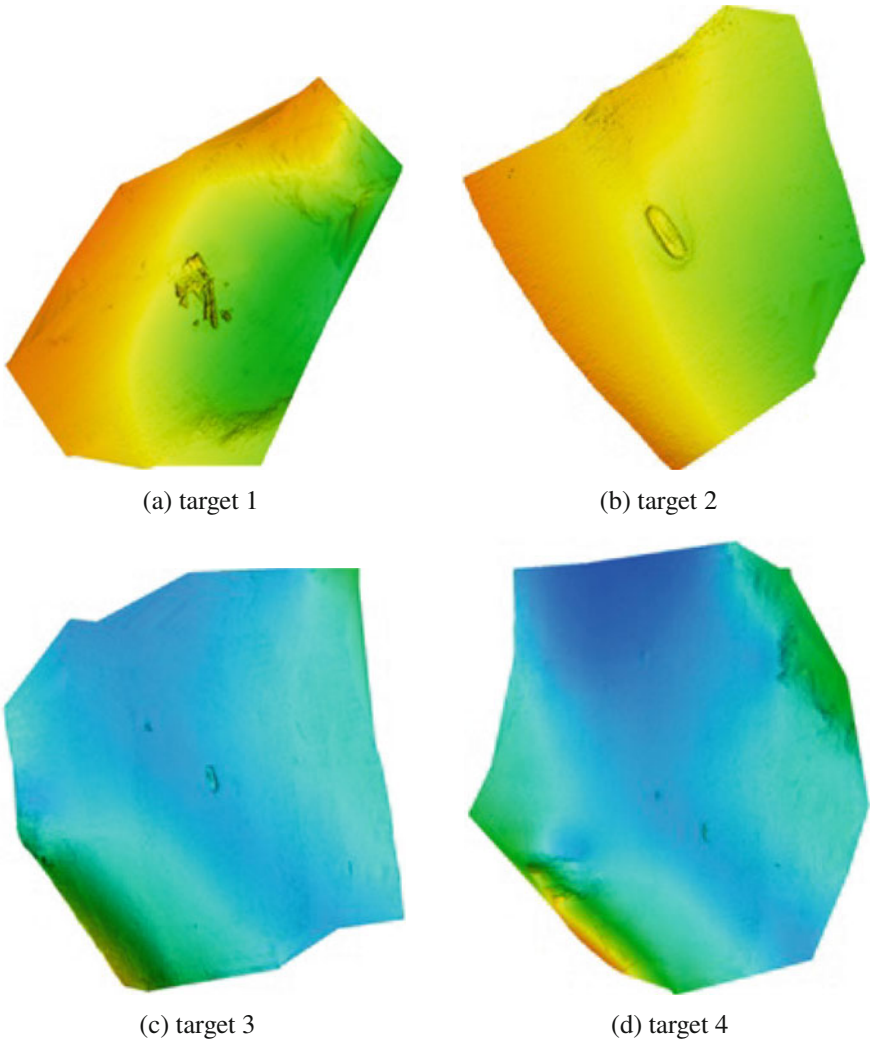


Fig. 5.6 The standard presentation of the propeller noise recorded, including its Fourier transform and power spectral density function. (**a**) Propeller A. (**b**) Propeller B. (**c**) Propeller C

of 40–100 m. Moreover, data on environmental parameters such as type of seabed, wind velocity, depth of water, salinity, and temperature were obtained from the PMO buoy [41]. In our experiment, we put four items in a sand-covered seafloor: two targets and two non-targets. A wideband, frequency-modulated ping with a 5–110 Hz frequency range is broadcast. Given the high computational burden that emerges from the vast amount of raw data in the previous phases, it is required to find the most likely targets in the data and lower the computation complexity of

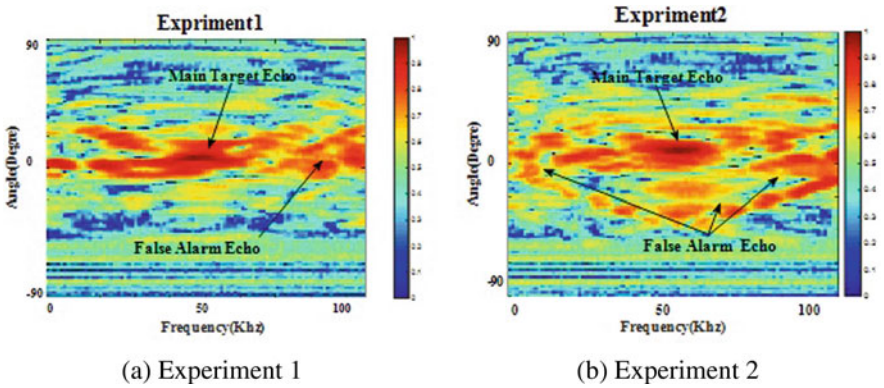


(a) target 1 (b) target 2

(c) target 3 (d) target 4

Fig. 5.7 Four typical targets in CDS2015. (**a**) Target 1. (**b**) Target 2. (**c**) Target 3. (**d**) Target 4

SVM-metaheuristic detectors. For this purpose, we employ the signal's intensity. Once the process of inverse filtering is complete, we can recover the original reflected signal. Time-domain filtering is more complicated than matched filter domain filtering. As discussed in [41], the whole preprocessing stage is broken into four parts. A demonstration of the received signals (both target and non-target) is shown in Fig. 5.8 as a result of the preprocessing phase.



(a) Experiment 1 (b) Experiment 2

Fig. 5.8 Sample backscatters returned from a target and non-target. (**a**) Experiment 1. (**b**) Experiment 2

Table 5.4 Laboratory cavitation tunnel dataset's results

Algorithms	MSE ($AVE \pm STD$)	P-Values	Recognition percentage
SVM-ChOA	0.0012 ± 0.0233	**N/A**	**91.9251**
SVM-MPA	0.0012 ± 0.0335	0.033	90.001
SVM-HHO	0.0012 ± 0.1001	0.047	90.025
SVM-KF	0.922 ± 0.0775	0.021	85.22
SVM-HGSO	0.0012 ± 0.1632	0.021	92.531
SVM-GA	0.2212 ± 0.0452	0.0334	86.442
SVM-PSO	0.2112 ± 0.1521	0.0563	85.752
SVM-GWO	0.1257 ± 0.0745	0.021	86.852
SVM-WOA	0.1212 ± 0.1521	0.021	87.632
SVM-SMA	0.0295 ± 0.1421	0.033	88.521

The best results are highlighted in bold

5.4.4 Experimental Results and Discussion

After preprocessing, the sonar recursive backscatters have been normalized to a dataset with a range of (0, 1) with dimensions of 150 × 34 (150 samples and 34 features) for the laboratory cavitation tunnel dataset, 625 × 144 for the Common CDS-2015, and 400 × 128 for the generated dataset. The findings are displayed in a sequential manner from Tables 5.4, 5.5, and 5.6. These tables illustrate that SVM-ChOA detectors exhibit the most successful outcomes with an average recognition accuracy of 92.1476%. In contrast, SVM-KF, with an average recognition accuracy of 80.7779%, exhibits the lowest performance due to its excessive local minima and fluctuating nature. The disappointing KF results are further proof of this notion. Unlike the other algorithms, the SVM-ChOA and SVM-MPA algorithms are better than other algorithms, thanks to the simultaneous exploration and exploitation abilities.

Table 5.5 Common CDS-2015 dataset's results

Algorithms	MSE ($AVE \pm STD$)	P-Values	Recognition percentage
SVM-ChOA	0.0062 ± 0.0191	**N/A**	**93.0002**
SVM-MPA	0.0063 ± 0.0412	0.047	92.0011
SVM-HHO	0.0085 ± 0.1215	0.002	91.9147
SVM-KF	0.2433 ± 0.0933	0.0044	85.0669
SVM-HGSO	0.0082 ± 0.1799	0.0101	90.9147
SVM-GA	0.2233 ± 0.0211	0.0052	86.0099
SVM-PSO	0.2111 ± 0.1855	0.0021	87.9177
SVM-GWO	0.1549 ± 0.0977	0.0033	88.0133
SVM-WOA	0.1277 ± 0.1521	0.0224	90.9188
SVM-SMA	0.0189 ± 0.1133	0.0251	90.0097

The best results are highlighted in bold

Table 5.6 The developed dataset's results

Algorithms	MSE ($AVE \pm STD$)	P-Values	Recognition percentage
SVM-ChOA	0.0425 ± 0.0009	N/A	90.0101
SVM-MPA	0.0451 ± 0.0009	0.0473	89.0001
SVM-HHO	0.0824 ± 0.0085	0.0024	87.8991
SVM-KF	0.3151 ± 0.0533	0.0054	73.1337
SVM-HGSO	0.0933 ± 0.0444	0.0012	86.9334
SVM-GA	0.2433 ± 0.0561	0.0055	75.0418
SVM-PSO	0.2306 ± 0.0556	0.0025	77.0011
SVM-GWO	0.1125 ± 0.0355	0.0032	85.1354
SVM-WOA	0.1255 ± 0.0477	0.0001	86.9334
SVM-SMA	0.0985 ± 0.0033	0.0412	88.9361

Researchers have suggested that a powerful algorithm is necessary for the exploration phase because underwater security and surveillance systems cover the whole research area, and metaheuristic detectors yield results comparable to or better than previous meta-heuristic algorithms in this domain. It is impossible to get any useful information about the efficacy of a detector by just comparing the recognition rates of different algorithms because varied thresholds lead to varying recognition rates. The precision-recall curve can be utilized to compare all levels of the cut-off threshold when a full comparison is required among various networks. Figures 5.9, 5.10, and 5.11 illustrate the precision-recall graphs of the 10 benchmark metaheuristic algorithms. The receiver operating characteristic (ROC) graph, which measures the true positive rate (TPR) in terms of false-positive rate (FPR), is a relevant tool as well. Based on these data, the ROC curves of the metaheuristic algorithms are also displayed in Figs. 5.9, 5.10, and 5.11.

The ROC curves show that the SVM-ChOA detector is substantially better than the other benchmark algorithms in the three datasets. It is worth mentioning that the AUC values in the ROC curves may not appropriately show the model's effectiveness because they may be high for highly imbalanced test sets.

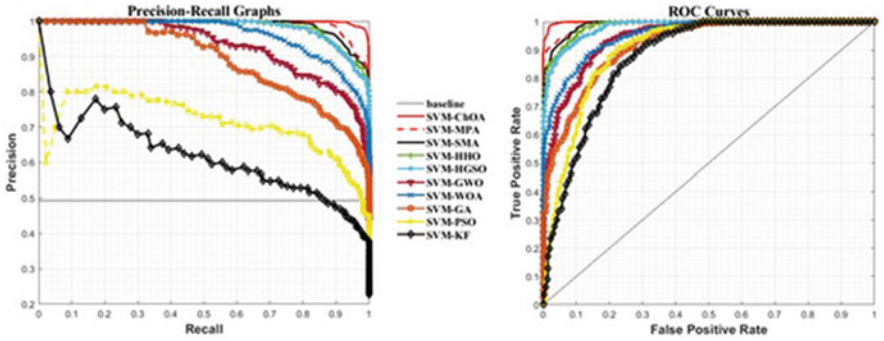


Fig. 5.9 ROC and Precision-recall curves for laboratory cavitation tunnel dataset

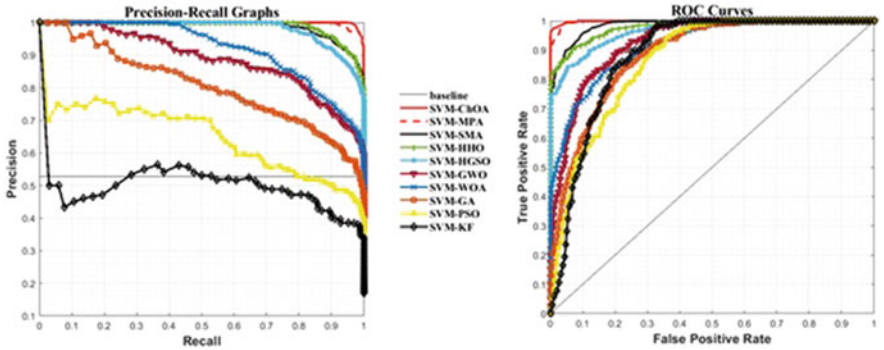


Fig. 5.10 ROC and precision-recall curves for the CDS2015 dataset

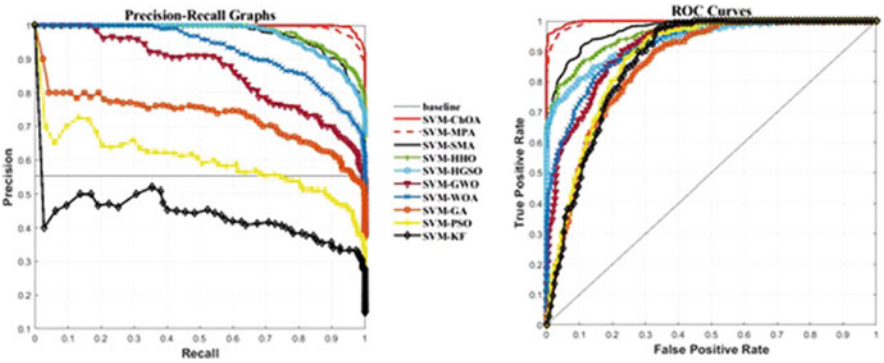


Fig. 5.11 ROC and precision-recall curves for the developed dataset

5.5 Conclusion

In this chapter, in order to investigate the reliability and accurateness of metaheuristic-based underwater sonar target classifier, the performance of various metaheuristic algorithms, including slime mould algorithm (SMA), marine predator algorithm (MPA), Kalman filter (KF), Harris Hawks optimization (HHO), genetic algorithm (GA), particle swarm optimization (PSO), Henry gas solubility optimization (HGSO), chimp optimization algorithm (ChOA), gray wolf optimizer (GWO), and whale optimization algorithm (WOA) in underwater sonar target classification problem was evaluated. In this regard, tests were conducted in three datasets: (1) a laboratory cavitation tunnel dataset, (2) a real-world underwater experimental dataset, and (3) a benchmark common dataset 2015 (CDS2015). The simulation results showed that SVM-ChOA followed by SVM-MPA produces better convergence rates and detection accuracies than the benchmark models. For future research directions, a simpler classifier, such as MLP and KNN, can be utilized to reduce the model's complexity. In addition, a new feature extraction method, such as an enhanced version of the wavelet technique, can be used to improve feature extraction.

Availability of Data and Material

The resource datasets can be downloaded using the following links:

http://archive.ics.uci.edu/ml/datasets/connectionist+bench+(sonar,+mines+vs.+rocks)
https://www.kaggle.com/mattcarter865/sonar-mines-vs-rocks
https://data.mendeley.com/datasets/fyxjjwzphf/1

Code Availability

The source code of the models can be available by request.

References

1. Rostami, O., Kaveh, M.: Optimal feature selection for SAR image classification using biogeography-based optimization (BBO), artificial bee colony (ABC) and support vector machine (SVM): a combined approach of optimization and machine learning. Computational Geosciences **25**(3), 911–930. Springer (2021)
2. Afrakhteh, S., Mosavi, M.R., Khishe, M., Ayatollahi, A.: Accurate classification of eeg signals using neural networks trained by hybrid population-physic-based algorithm. International Journal of Automation and Computing **17**(1), 108–122 (2020)

3. Aljarah, I., Faris, H., Mirjalili, S., Al-Madi, N.: Training radial basis function networks using biogeography-based optimizer. Neural Computing and Applications **29**(7), 529–553 (2018)
4. Amirkhani, A., Kolahdoozi, M., Papageorgiou, E.I., Mosavi, M.R.: Classifying mammography images by using fuzzy cognitive maps and a new segmentation algorithm. In: Advanced data analytics in health, pp. 99–116. Springer (2018)
5. Amirkhani, A., Mosavi, M.R., Mohammadizadeh, F., Shokouhi, S.B.: Classification of intraductal breast lesions based on the fuzzy cognitive map. Arabian Journal for Science and Engineering **39**(5), 3723–3732 (2014)
6. Bertschneider, H., Bosschers, J., Choi, G.H., Ciappi, E., Farabee, T., Kawakita, C., Tang, D.: Specialist committee on hydrodynamic noise. Final report and recommendations to the 27th ITTC. Copenhagen, Sweden **45** (2014)
7. Cortes, C., Vapnik, V.: Support-vector networks. Machine learning **20**(3), 273–297 (1995)
8. Day, S., Penesis, I., Babarit, A., Fontaine, A., He, Y., Kraskowski, M., Murai, M., Salvatore, F., Shin, H., et al.: Ittc recommended guidelines: Model tests for current turbines (7.5-02-07-03.9). In: 27th International Towing Tank Conference. pp. 1–17 (2014)
9. Du, K.L., Swamy, M.: Radial basis function networks. Neural networks in a softcomputing framework pp. 251–294 (2006)
10. Du, K.L., Swamy, M.N.: Neural networks and statistical learning. Springer Science & Business Media (2013)
11. Faramarzi, A., Heidarinejad, M., Mirjalili, S., Gandomi, A.H.: Marine predators algorithm: A nature-inspired metaheuristic. Expert Systems with Applications **152**, 113377 (2020)
12. Faris, H., Aljarah, I., Mirjalili, S.: Evolving radial basis function networks using moth–flame optimizer. In: Handbook of neural computation, pp. 537–550. Elsevier (2017)
13. Fei, S.w., Zhang, X.b.: Fault diagnosis of power transformer based on support vector machine with genetic algorithm. Expert Systems with Applications **36**(8), 11352–11357 (2009)
14. Gan, M., Peng, H., ping Dong, X.: A hybrid algorithm to optimize RBF network architecture and parameters for nonlinear time series prediction. Applied Mathematical Modelling **36**(7), 2911–2919 (2012). https://doi.org/https://doi.org/10.1016/j.apm.2011.09.066, https://www.sciencedirect.com/science/article/pii/S0307904X11006251
15. Gutiérrez, F., Parada, M.A.: Numerical modeling of time-dependent fluid dynamics and differentiation of a shallow basaltic magma chamber. Journal of Petrology **51**(3), 731–762 (2010)
16. Hashim, F.A., Houssein, E.H., Mabrouk, M.S., Al-Atabany, W., Mirjalili, S.: Henry gas solubility optimization: A novel physics-based algorithm. Future Generation Computer Systems **101**, 646–667 (2019)
17. Heidari, A.A., Mirjalili, S., Faris, H., Aljarah, I., Mafarja, M., Chen, H.: Harris hawks optimization: Algorithm and applications. Future generation computer systems **97**, 849–872 (2019)
18. Horng, M.H., Lee, Y.X., Lee, M.C., Liou, R.J.: Firefly metaheuristic algorithm for training the radial basis function network for data classification and disease diagnosis. Theory and new applications of swarm intelligence **4**(7), 115–132 (2012)
19. Houssein, E.H., Hamad, A., Hassanien, A.E., Fahmy, A.A.: Epileptic detection based on whale optimization enhanced support vector machine. Journal of Information and Optimization Sciences **40**(3), 699–723 (2019)
20. Hu, T., Khishe, M., Mohammadi, M., Parvizi, G.R., Karim, S.H.T., Rashid, T.A.: Real-time COVID-19 diagnosis from X-ray images using deep CNN and extreme learning machines stabilized by chimp optimization algorithm. Biomedical Signal Processing and Control **68**, 102764 (2021)
21. Huang, J., Hu, X., Yang, F.: Support vector machine with genetic algorithm for machinery fault diagnosis of high voltage circuit breaker. Measurement **44**(6), 1018–1027 (2011)
22. Ibrahim, Z., Aziz, N.A., Aziz, N.A.A., Razali, S., Shapiai, M.I., Nawawi, S., Mohamad, M.: A Kalman filter approach for solving unimodal optimization problems. ICIC Express Lett **9**(12), 3415–3422 (2015)

23. Ilhan, I., Tezel, G.: A genetic algorithm–support vector machine method with parameter optimization for selecting the tag SNPs. Journal of biomedical informatics **46**(2), 328–340 (2013)
24. Kennedy, J., Eberhart, R.: Particle swarm optimization in: Proceedings of ICNN'95-international conference on neural networks, 1942–1948. IEEE, Perth, WA, Australia (1995)
25. Kesavan, V., Kamalakannan, R., Sudhakarapandian, R., Sivakumar, P.: Heuristic and meta-heuristic algorithms for solving medium and large scale sized cellular manufacturing system np-hard problems: A comprehensive review. Materials today: proceedings **21**, 66–72 (2020)
26. Khishe, M., Aghababaee, M., Mohammadzadeh, F.: Active sonar clutter control by using array beamforming. Iranian Journal of Marine Science and Technology **68**(1), 1–6 (2014)
27. Khishe, M., Mosavi, M.R.: Chimp optimization algorithm. Expert systems with applications **149**, 113338 (2020)
28. Khishe, M., Mosavi, M.: Classification of underwater acoustical dataset using neural network trained by chimp optimization algorithm. Applied Acoustics **157**, 107005 (2020)
29. Khishe, M., Mosavi, M., Kaveh, M.: Improved migration models of biogeography-based optimization for sonar dataset classification by using neural network. Applied Acoustics **118**, 15–29 (2017)
30. Khishe, M., Mosavi, M., Moridi, A.: Chaotic fractal walk trainer for sonar data set classification using multi-layer perceptron neural network and its hardware implementation. Applied Acoustics **137**, 121–139 (2018)
31. Khishe, M., Mohammadi, H.: Passive sonar target classification using multi-layer perceptron trained by salp swarm algorithm. Ocean Engineering **181**, 98–108 (2019)
32. Khishe, M., Safari, A.: Classification of sonar targets using an MLP neural network trained by dragonfly algorithm. Wireless Personal Communications **108**(4), 2241–2260 (2019)
33. Li, S., Chen, H., Wang, M., Heidari, A.A., Mirjalili, S.: Slime mould algorithm: A new method for stochastic optimization. Future Generation Computer Systems **111**, 300–323 (2020). https://doi.org/https://doi.org/10.1016/j.future.2020.03.055, https://www.sciencedirect.com/science/article/pii/S0167739X19320941
34. Li, X., Wu, S., Li, X., Yuan, H., Zhao, D.: Particle swarm optimization-support vector machine model for machinery fault diagnoses in high-voltage circuit breakers. Chinese Journal of Mechanical Engineering **33**(1), 1–10 (2020)
35. Liu, D., Li, M., Ji, Y., Fu, Q., Li, M., Faiz, M.A., Ali, S., Li, T., Cui, S., Khan, M.I.: Spatial-temporal characteristics analysis of water resource system resilience in irrigation areas based on a support vector machine model optimized by the modified gray wolf algorithm. Journal of Hydrology **597**, 125758 (2021)
36. Liu, M., Xue, Z., Zhang, H., Li, Y.: Dual-channel membrane capacitive deionization based on asymmetric ion adsorption for continuous water desalination. Electrochemistry Communications **125**, 106974 (2021)
37. Luo-Theilen, X., Rung, T.: Numerical analysis of the installation procedures of offshore structures. Ocean Engineering **179**, 116–127 (2019)
38. Mirjalili, S., Dong, J.S., Sadiq, A.S., Faris, H.: Genetic algorithm: Theory, literature review, and application in image reconstruction. Nature-inspired optimizers pp. 69–85 (2020)
39. Mirjalili, S., Lewis, A.: The whale optimization algorithm. Advances in engineering software **95**, 51–67 (2016)
40. Mirjalili, S., Mirjalili, S.M., Lewis, A.: Grey wolf optimizer. Advances in engineering software **69**, 46–61 (2014)
41. Mosavi, M.R., Khishe, M.: Training a feed-forward neural network using particle swarm optimizer with autonomous groups for sonar target classification. Journal of Circuits, Systems and Computers **26**(11), 1750185 (2017)
42. Mosavi, M.R., Khishe, M., Akbarisani, M.: Neural network trained by biogeography-based optimizer with chaos for sonar data set classification. Wireless Personal Communications **95**(4), 4623–4642 (2017)
43. Mosavi, M.R., Khishe, M., Ebrahimi, E.: Classification of sonar targets using OMKC, genetic algorithms and statistical moments (2016)

44. Mosavi, M.R., Khishe, M., Ghamgosar, A.: Classification of sonar data set using neural network trained by gray wolf optimization. Neural Network World **26**(4), 393 (2016)
45. Mosavi, M.R., Khishe, M., Naseri, M.J., Parvizi, G.R., Ayat, M.: Multi-layer perceptron neural network utilizing adaptive best-mass gravitational search algorithm to classify sonar dataset. Archives of Acoustics **44** (2019)
46. Mosavi, M., Khishe, M., Moridi, A.: Classification of sonar target using hybrid particle swarm and gravitational search. Iranian journal of Marine technology **3**(1), 1–13 (2016)
47. Mosavi, M., Khishe, M., Hatam Khani, Y., Shabani, M.: Training radial basis function neural network using stochastic fractal search algorithm to classify sonar dataset. Iranian Journal of Electrical and Electronic Engineering **13**(1), 100–111 (2017)
48. Mousavi, M., Aghababaie, M., Naseri, M., Khishe, M.: Compression of respiratory signals using linear predictive coding method based on optimized algorithm of humpback whales to transfer by Sonobouy (2020)
49. Neruda, R., Kudovà, P.: Learning methods for radial basis function networks. Future Generation Computer Systems **21**(7), 1131–1142 (2005)
50. Pan, M., Li, C., Gao, R., Huang, Y., You, H., Gu, T., Qin, F.: Photovoltaic power forecasting based on a support vector machine with improved ant colony optimization. Journal of Cleaner Production **277**, 123948 (2020)
51. Qiao, W., Khishe, M., Ravakhah, S.: Underwater targets classification using local wavelet acoustic pattern and multi-layer perceptron neural network optimized by modified whale optimization algorithm. Ocean Engineering **219**, 108415 (2021)
52. Ravakhah, S., Khishe, M., Aghababaee, M., Hashemzadeh, E.: Sonar false alarm rate suppression using classification methods based on interior search algorithm. Int J Comput Sci Netw Secur **17**(7), 58–65 (2017)
53. Rey, D., Neuhäuser, M.: Wilcoxon-signed-rank test. (2011)
54. Saffari, A., Zahiri, S.H., Khishe, M., Mosavi, S.M.: Design of a fuzzy model of control parameters of chimp algorithm optimization for automatic sonar targets recognition. IJMT (2020)
55. Sug, H.: Using quick decision tree algorithm to find better RBF networks. In: Asian Conference on Intelligent Information and Database Systems. pp. 207–217. Springer (2011)
56. Sun, M., Yan, L., Zhang, L., Song, L., Guo, J., Zhang, H.: New insights into the rapid formation of initial membrane fouling after in-situ cleaning in a membrane bioreactor. Process biochemistry **78**, 108–113 (2019)
57. Taghavi, M., Khishe, M.: A modified grey wolf optimizer by individual best memory and penalty factor for sonar and radar dataset classification (2019)
58. Tao, W., Chen, J., Gui, Y., Kong, P.: Coking energy consumption radial basis function prediction model improved by differential evolution algorithm. Measurement and Control **52**(7–8), 1122–1130 (2019)
59. Tharwat, A., Hassanien, A.E.: Chaotic antlion algorithm for parameter optimization of support vector machine. Applied Intelligence **48**(3), 670–686 (2018)
60. Wu, C., Khishe, M., Mohammadi, M., Karim, S.H.T., Rashid, T.A.: Evolving deep convolutional neutral network by hybrid sine–cosine and extreme learning machine for real-time covid19 diagnosis from X-ray images. Soft Computing pp. 1–20 (2021)
61. Yang, M., Sowmya, A.: An underwater color image quality evaluation metric. IEEE Transactions on Image Processing **24**(12), 6062–6071 (2015)
62. Yang, Y., Hou, C., Lang, Y., Sakamoto, T., He, Y., Xiang, W.: Omnidirectional motion classification with monostatic radar system using micro-doppler signatures. IEEE Transactions on Geoscience and Remote Sensing **58**(5), 3574–3587 (2019)
63. Yang, Y., Tao, L., Yang, H., Iglauer, S., Wang, X., Askari, R., Yao, J., Zhang, K., Zhang, L., Sun, H.: Stress sensitivity of fractured and vuggy carbonate: an X-ray computed tomography analysis. Journal of Geophysical Research: Solid Earth **125**(3), e2019JB018759 (2020)
64. Yu, L.: An evolutionary programming based asymmetric weighted least squares support vector machine ensemble learning methodology for software repository mining. Information Sciences **191**, 31–46 (2012)

65. Zhang, H., Guan, W., Zhang, L., Guan, X., Wang, S.: Degradation of an organic dye by bisulfite catalytically activated with iron manganese oxides: the role of superoxide radicals. ACS omega **5**(29), 18007–18012 (2020)
66. Zhang, H., Sun, M., Song, L., Guo, J., Zhang, L.: Fate of naclo and membrane foulants during in-situ cleaning of membrane bioreactors: Combined effect on thermodynamic properties of sludge. Biochemical engineering journal **147**, 146–152 (2019)
67. Zhang, J., Shen, C.: Set-based obfuscation for strong PUFs against machine learning attacks. IEEE transactions on circuits and systems I: regular papers **68**(1), 288–300 (2020)
68. Zuo, C., Chen, Q., Tian, L., Waller, L., Asundi, A.: Transport of intensity phase retrieval and computational imaging for partially coherent fields: The phase space perspective. Optics and Lasers in Engineering **71**, 20–32 (2015)
69. Zuo, C., Sun, J., Li, J., Zhang, J., Asundi, A., Chen, Q.: High-resolution transport-of-intensity quantitative phase microscopy with annular illumination. Scientific reports **7**(1), 1–22 (2017)

Chapter 6
Solving the Quadratic Knapsack Problem Using GRASP



Raka Jovanovic and Stefan Voß

Abstract In this chapter, a greedy randomized adaptive search procedure (GRASP) approach for solving the Quadratic Knapsack Problem (QKP) is presented. In addition, a new local search has been developed that manages to explore a larger neighborhood of a solution while maintaining the same asymptotical computational cost as the commonly used ones. The second contributions of this paper is the introduction of a new set of test instances, which makes it possible to better evaluate the performance of algorithms designed for the QKP. The performed computational experiments show that the developed GRASP algorithm is highly competitive with existing ones on the standard test instances. Further, we have used the new set of test instances to evaluate the performance of the newly proposed local search against the one commonly used for the QKP. These tests have shown that although the proposed local search overall outperforms the existing one, they have a notably different behavior. Further, our tests have shown that the proposed instances are significantly harder to solve than the standard benchmark ones.

6.1 Introduction

The 0-1 knapsack problem (KP) is one of the standard combinatorial optimization problems. In it the objective is to select n items each having a profit p_i and a weight w_i having the maximal sum of profits while satisfying the constraint that the total weight of the items is less than the knapsack capacity c. Several different variations

R. Jovanovic (✉)
Qatar Environment and Energy Research Institute (QEERI), Hamad bin Khalifa University, Doha, Qatar
e-mail: rjovanovic@hbku.edu.qa

S. Voß
Institute of Information Systems, University of Hamburg, Hamburg, Germany

Escuela de Ingenieria Industrial, Pontificia Universidad Católica de Valparaíso, Valparaíso, Chile
e-mail: stefan.voss@uni-hamburg.de

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

M. Eddaly et al. (eds.), *Metaheuristics for Machine Learning*,
Computational Intelligence Methods and Applications,
https://doi.org/10.1007/978-981-19-3888-7_6

of the 0–1 KP have been addressed in literature like the multiple choice [28], with conflict graphs [25], etc. [18]. In this paper, we focus on the 0–1 Quadratic Knapsack Problem (QKP), for which an extensive survey can be found in [30]. In this version of the problem additional profits are added for pairs of elements. The interest for the QKP is due to the fact that it well represents a wide range of real-world problems or appears as a subproblem that needs to be solved. Some examples are applications in scheduling [19], facility location [20], finding optimal locations for airports and rail stations [32], compiler design [13], campaign budgeting [27] and many others. The QKP has also been used for solving clique problems and its variations [8, 22]. Recently, new variations of the QKP have been introduced like the inclusion of conflict graphs [33], multiple knapsacks [1], and fixing the number of items in the k-item QKP [21].

Due to its importance, a wide range of methods have been developed for solving the QKP. The first group are algorithms for finding exact solutions; some examples are the use of Lagrangian decomposition [5], linearization [3], reformulation [30] and others. One of the most successful methods of this type is based on the use of Lagrangian relaxation/decomposition and aggressive reduction [31]. Recently several effective approximation algorithms for the QKP have been developed [24, 26, 34]. Extensive research has also been done on finding upper bounds for the QKP using upper planes [22], Lagrangian relaxation [22] and decomposition [4].

Due to the NP-Hardness of the QKP, research has also been conducted in developing heuristic and metaheuristic algorithms. Hammer and Rader have presented a heuristic based on the best linear approximation for the QKP [11]. A basis for several later algorithms is the greedy constructive algorithm given in [6]. Peterson introduced an improvement method based on the flip-up and exchange operation that has later been extensively used in other algorithms for the QKP [23]. The combination of a heuristic approach with dynamic programming has proven to be very successful [10]. Recently there has been a growing interest for the applications of metaheuristics for this problem, like the use of genetic algorithms [17] and the mini-swarm method [35]. To the best of the authors knowledge, the best performing metaheuristics are based on the Greedy Randomized Adaptive Search Procedure (GRASP) [9]. Note that a method similar to the GRASP ones, based on mean field theory, has been presented in [2]. In the work by Yang et al. a hybridization of a memory-based GRASP with tabu search is presented [36]. The ideas from this paper are extended with the use of an iterated search of hyperplanes, which produced the best results when solution quality and computational speed are considered [7].

One of the disadvantages of the two best performing GRASP algorithms [7, 36] is the relative complexity of implementation due to the combination of the GRASP and tabu search. The second problem with these two methods is the need for tuning a large number of parameters. In this paper, we address these issues by proposing a new simplified GRASP algorithm focusing on the most important aspects of the algorithm. To achieve higher quality results, a new local search that considers multiple item swaps is introduced. It is important to note that the new local search

has the same asymptotically computational cost as the ones using the standard flip-up and exchange operations.

Another contribution of this paper is in developing a new set of test instances for the QKP. The standard ones used in [7, 10, 35, 36], are purely random. It has been observed in the work of Pisinger [29] for the KP, that in practice such instances are easy to solve. This can also be observed in case of the QKP where the standard instances have one or two thousand items and most of the state-of-the-art methods solve almost all of them to expected optimality. In the proposed set of test instances, we use the ideas from [29] for item related profits and extend them to the QKP. In addition, the proposed set of test instances has a wider range of distributions of item weights. The performed computational experiments show that the proposed simplified GRASP manages to find the best known solutions for all test instances having up to 2000 items. Further, it is shown that using the new local search can improve the performance of the GRASP compared to the standardly used one. Our tests also indicate that the new instances are much harder to solve.

The paper is organized as follows. In Sect. 6.2, the QKP is presented. Section 6.3 is dedicated to the greedy constructive algorithm for solving the QKP. In Sect. 6.4, details of the GRASP algorithm are presented. In this section, we also present the new local search for the problem of interest. In Sect. 6.5, the method for generating the new set of test instances is presented. In Sect. 6.6, the results of the performed computational experiments are analyzed. In the final section, we give some concluding remarks and potential directions of future research.

6.2 Quadratic Knapsack Problem

The QKP is defined in the following way. Let c, a positive integer value, be the capacity of the knapsack. Let $N = \{1, 2 \ldots, n\}$ be the set of items. Each item i has a corresponding weight w_i and item profit p_i, which are both positive integers. For each pair of items i and j ($1 \leqslant i < j \leqslant n$) there is a corresponding pair profit p_{ij}, a positive integer, which is added to the total profit if both items have been selected. The objective of the QKP is to maximize the profit that can be achieved by placing items in the knapsack. Formally, the QKP can be defined using the following formulation. Let x_i be a binary decision variable, defined for each $i = 1 \ldots n$ indicating if element i is selected. The objective is to maximize the following quadratic sum while satisfying the given constraints.

$$\text{Maximize} \qquad \sum_{i=1}^{n} p_i x_i + \sum_{i=1}^{n} \sum_{j=i+1}^{n} x_i x_j p_{ij} \tag{6.1}$$

$$\text{Subjected to} \qquad \sum_{i=1}^{n} x_i w_i \leqslant c \tag{6.2}$$

6.3 Greedy Algorithm

The basic idea of the standard constructive greedy algorithm is to iteratively expand a partial solution with the item that produces a maximal increase in profit with the minimal decrease of available weight capacity. Let us define the partial solution at step k as a set $S = \{e_1, \ldots e_k\}$. In relation let us define the set of candidates $C = N \setminus S$. The increase in profit when an element i is selected is given using the following formula

$$Con(S, i) = p_i + \sum_{j \in S} p_{ij} \tag{6.3}$$

The function $Con(S, i)$ is equal to the sum of the item profit p_i and all the related profit pairs p_{ij} ($j \in S$). The problem with the function $Con(S, i)$ is that it does not consider the weight of the element. Let us define the following density function.

$$Den(S, i) = \frac{Con(S, i)}{w_i} \tag{6.4}$$

The density function $Den(S, i)$ is equal to the contribution per one unit of weight. Let us define the functions $P(S)$ and $W(S)$ which are equal to the total profit and weight of the partial solution S. Now, we can define a greedy constructive algorithm which uses the function $Den(S, i)$ as the heuristic function, which can be seen in Algorithm 1.

Algorithm 1 Pseudocode for the constructive greedy algorithm for QKP

1: $S = \emptyset$
2: $C = N$
3: **repeat**
4: $\quad i = \arg\min_{j \in C} Den(S, j)$
5: $\quad S = S \cup \{i\}$
6: $\quad C = \{x \mid (x \in C) \wedge (x \neq i) \wedge (x + |S| \leqslant c)\}$
7: **until** $C = \emptyset$

The algorithm presented in Algorithm 1 iteratively expands the partial solution S with the element i having the maximal value of the heuristic function $Den(S, i)$. After each element i is added to the solution the list of candidates C is updated by removing i and all the items having a weight that can not fit in the knapsack.

6.4 GRASP

In this section, we extend the previously presented greedy algorithm to the GRASP metaheuristic. The basic idea of the GRASP is to randomize a greedy algorithm and use it to generate a large number of solutions and apply a local search on each of them. So, we need to include randomization in the greedy algorithm and develop a local search.

6.4.1 Randomization

For randomizing the greedy algorithm, the standard approach based on the restricted candidate list (RCL) is used. Let us define the RCL with R elements at step k, as the set of l elements that have the maximal value $Den(S, i)$, $i \in C$. In the randomized greedy algorithm the element for expanding the partial solution will be selected probabilistically from the RCL. The probability of selecting element i will be

$$prob(i) = \frac{R - rank(i) + 1}{\sum_{i=1}^{R} i} \tag{6.5}$$

Eq. (6.5) states that the probability of selecting element i is related to its rank ($rank(i)$) in the RCL; more precisely, it is inversely proportional to $R - rank(i)$. We would like to note that in [36] a more complex method for selecting elements from the RCL is presented. In our initial tests, we did not observe a significant improvement by using this approach in the GRASP algorithm. Since our goal was to maintain the simplicity of the method, we did not include the advanced method for selecting elements from the RCL.

6.4.2 Single Swap Local Search

In this section, we present the local searches for the QKP. The basic block of the existing and the proposed local searches is a swap operation. The swap operation consists in substituting an element inside the solution, $j \in S$, with an element outside of it, $i \in C$. The notation $i \rightarrow j$ will be used when item j is removed from solution S and item i is added. For the sake of simplicity of notation let us define the function $Swap(S, i \rightarrow j)$ which gives us as a result the solution after applying the swap. Let us also define the extension of the functions related to weight and profit of the solution as follows:

$$P(S, i \rightarrow j) = P(Swap(S, i \rightarrow j)) \tag{6.6}$$

$$W(S, i \rightarrow j) = W(Swap(S, i \rightarrow j)) \tag{6.7}$$

The standard local search for the QKP is based on two types of improvements. The first one checks if a new item can be added to the solutions. The second one checks if there is a swap that can improve the solution. Formally, we can define these two methods as follows:

- ADD: In case there is an item $i \in C$ such that $w(S) + w_i \leqslant c$ add i to S. Let us define $A(S)$ as the set of all such i.
- SWAP: In case there are items $i \in C$ and $j \in S$ such that $W(S, i \rightarrow j) \leqslant c$, and $P(S) < P(S, i \rightarrow j)$ remove item j from S and add i. Let us define $SW(S)$ as the set of all such swaps.

Using the improvements ADD and $SWAP$ a local search similar to the ones used in [7, 36] is defined. The idea of the local search is to repeatedly apply one of the improvements until this is not possible. The pseudocode for the local search is given in Algorithm 2.

Algorithm 2 Pseudocode for the greedy algorithm

```
1: repeat
2:     if A(S) ≠ ∅ then
3:         Select random i ∈ A
4:         Add(S, i)
5:     end if
6:     if SW(S) ≠ ∅ then
7:         Select random i → j ∈ SW(S)
8:         Swap(S, i → j)
9:     end if
10: until No Improvement
```

The local search given in Algorithm 2 is applied on a solution S. The solution is iteratively improved. In the main loop, it is first evaluated if there are items $i \in C$ that can be used to expand the solution S, or, in other words, if the set $A(S) \neq \emptyset$. If this is true a random element $i \in A(S)$ is selected for expansion. Next, it is checked if there are potential swap operations that can improve the quality of the solution S, or in other words if $SW(S) \neq \emptyset$. Note that it is possible to use the best improvement instead of a random one. Our tests have shown that this proves to be computationally more expensive and does not produce an improvement in the GRASP. Another variation is to randomly select which of the two improvements will be used at each step of the main loop.

6.4.3 Multi-Swap Local Search

The computational cost of an iteration of the presented local search is proportional to n^2. The main cost is related to finding a swap pair $i \rightarrow j$, since all the pairs $i \in C$ and $j \in S$ need to be tested. A speedup can be achieved if the first found

swap is applied, like in [7], but this does not change the asymptotic computational cost. A standard method for creating better local searches of this type is to increase the number of items that can be swapped. In case there is an exchange of k items the asymptotic cost will be n^{2k}. Due to the fact that interesting instances for the QKP have a large number of items n, this is not a realistic option.

The basic idea of the proposed local search is to evaluate the effect of applying multiple swap operations without evaluating all the potential ones and their combinations. In the previously presented local search n^2 potential swaps are evaluated to see if they produce an improvement. In this way a set I of improving swaps $i \rightarrow j$ is found. Our goal is to find a swap set $I' \subset I$ which produces the maximal improvement when applied to solution S. It is evident that this cannot be done by brute force since there are a potential $2^{|I|}$ subsets that need to be evaluated. In the further text, we will show how the size of the potential swap sets can be restricted in a way that this procedure becomes efficient. More precisely, several constraints are used for selecting swaps and for their combinations (set of swaps) that are tested:

- Only swaps that improve the solution will be considered.
- A limit m on the number of swaps in the test set is given.
- A limit T is given on the total number of combinations of swaps that will be tested.

The first issue that arises for a set of swaps is that it may not be possible to perform all of them. This can be due to the possibility that a swap operation may try to remove/add an item that is not inside/outside the solution. This can be avoided if no item appears in more than one swap in the set. Formally, the set of swaps $I = \{i_1 \rightarrow j_1, \ldots\ i_k \rightarrow j_k\}$, satisfies $i_a \neq i_b$ and $j_a \neq j_b$ for $a \neq b$; and $i_a \neq j_b$ for $a, b = 1..k$. An additional advantage of using this constraint is that the order in which the swaps are performed becomes irrelevant if capacity constraints are not considered. In the further text, we will assume all the sets of swaps satisfy this constraint, and call such sets conflict free.

Let us define the function $Swap(S, M)$, which has as parameters a set of items (solution) S and a set of swaps M. The result of the function is the set of elements S' after all the swap operations in M are applied to S. For the sake of simplicity let us define the following two functions

$$P(S, M) = P(Swap(S, M)) \tag{6.8}$$

$$W(S, M) = P(Swap(S, M)) \tag{6.9}$$

In Eqs. (6.8)/(6.9) function $P(S, M)/W(S, M)$ gives us the profit/weight of the set of items acquired by applying all the swap operations in M on S.

Let us define for a set of swaps M, the set $\mathcal{V}(S, M) \subset \mathcal{P}(M)$ of all valid combinations of swaps. The notation $\mathcal{P}(M)$ is used for the partition set of M. Let us extend this definition to the set $\mathcal{V}^m(S, M)$ which has an additional constraint on the

number of elements. These two sets can formally be defined as follows:

$$\mathcal{V}(S, M) = \{G \mid G \subset M \wedge W(S, G) \leqslant c\} \tag{6.10}$$

$$\mathcal{V}^m(S, M) = \{G \mid G \subset \mathcal{V}(S, M) \wedge |G| \leqslant m\} \tag{6.11}$$

Eq. (6.10) simply states that valid sets of swaps must satisfy capacity constraints. Equation (6.11) extends the function $\mathcal{V}(S, M)$ to have an additional constraint on the maximal number of elements of a valid swap set.

The final constraint that we need to implement is the limit T on the number of swap sets that will be tested. This can be done by iteratively expanding the set of swaps M by randomly selecting a swap $i \rightarrow j$ until $|\mathcal{V}^m(S, M)| \geqslant T$. In practice the entire set of combinations $\mathcal{V}^m(S, M)$ is not used. This is due to the fact that it can extremely grow after a new swap $i \rightarrow j$ is added to M. Instead, we use the set $\mathcal{V}^m_T(S, M)$ which is equal to the first valid T swap sets that are generated. In case $T << n^2$ there is no significant change in the computational cost compared to the previously presented local search and the one given in (6.12).

Now, we can define a new heuristic function using the set $\mathcal{V}^m_T(S, M)$ that instead of selecting a single swap selects a set of them. This can be done as follows.

$$Best(S, m, T) = \underset{H \in \mathcal{V}^m_T(S,M)}{\arg\max} \; P(S, H) \tag{6.12}$$

The heuristic function given in Eq. (6.12) selects the set of swaps that produces the largest improvement to the solution.

Using the heuristic function $Best(S, m, T)$ the local search is defined. As in the case of the single swap local search the solution S is iteratively improved until no improvement can be achieved. In our initial tests, we have observed that if all the swaps in $K = Best(S, m, T)$ are applied to S the method tends to quickly get trapped in local optima. It is expected that this is related to the fact that after applying a swap $i \rightarrow j$ to solution S it may be possible to find a better set of swaps than the initially found one. Because of this only a single swap $i \rightarrow j \in K$ is applied to S resulting in a new solution S'. At the next step a new set of best swaps is found $K' = Best(S', m, T)$ and its improvement is compared to the one achieved by the set of swaps $K \setminus \{i \rightarrow j\}$. This procedure is better understood by observing the pseudocode for the multi-swap local search given in Algorithm 3.

In Algorithm 3 the first step is initializing the set of best swaps ($BestSwaps$) to an empty set and the best improvement ($BestValue$) to zero. In the main loop, we iteratively improve the solution until no improvement can be achieved. This is done by first finding the set of best swaps $Swaps$ for the current solution S using the heuristic function $Best(S, m, T)$. Next, it is checked if the improvement achieved by applying $Swaps$ is better than the last found best improvement $BestValue$. If this is true the set of swaps $BestSwaps$ is set to Swaps. Next, a random valid swap is selected from $BestSwaps$ and applied to S. The term *valid* is used since it is possible that we cannot apply some swap due to capacity constraints. Finally, the

Algorithm 3 Pseudocode for the multi-swap local search

```
1: BestValue = 0
2: BestSwaps = ∅
3: repeat
4:     Swaps = Best(S, m, T)
5:     if BestValue < P(S, Swaps) then
6:         BestSwaps = Swaps
7:     end if
8:     Select valid random i → j in BestSwaps
9:     S = Swap(S, i → j)
10:    BestSwaps = BestSwaps \ {i → j}
11:    BestValue = P(S, BestSwaps)
12: until No Improvement
```

swap $i \rightarrow j$ is removed from $BestSwaps$ and the $BestValue$ is recalculated for the new solution.

6.4.4 Implementation

The proposed randomized greedy algorithm combined with the presented local searches can be directly extended to the GRASP metaheuristic in the standard way. From our computational experiments, we have observed several things in regard to the presented local searches. Firstly, in case of the single swap local search the addition of new items to the solution happens very rarely. The second observation is that the use of swap operations is very efficient in finding a high quality or optimal solution of the same size as the initial solution generated by the greedy randomized algorithm. Because of this in the proposed method, the GRASP algorithm is slightly adapted to explore solutions having a different numbers of items. As it is discussed in [7], it is frequently the case that a higher number of items in a solution correlates to a higher value of the objective function or, in other words, the profit.

In the proposed method, we use these two observations to effect the exploitation/exploration of the method in the following way. During the algorithm, we track the number of generated solutions having k elements since the last improvement of the best solution for that size is found. In case a new solution $|S| = l$ is generated and for the solution size $l + 1$ there has been a lower number of generated solution with no improvement, the generated solution will be force expanded to have an additional element satisfying the capacity constraints. This can be achieved by performing a set of swaps that increase the available capacity and adding a new element. Note that the new solution generally has a lower profit. On the new solution, the local search is applied. This procedure is repeated iteratively until a maximal allowed number of items is added to the solution. Note that it is easy to find the upper bound on the number of items for a solution [7].

In our initial computational experiments, it has been observed that the behavior of the two proposed local searches is different. To be more precise, although the multi-swap local search has a significantly better performance than the single swap one, the two local searches tend to get trapped in different local optima. This is a common property of local searches and is a basis for the variable neighborhood search (VNS) [12], where the different local searches are applied one after another to improve performance. Due to the fact that local optima for the single swap and multi-swap local searches are the same, this cannot be done. Instead, we simply change the used local search at each iteration, or, in other words, on each solution generated using the greedy algorithm, of the GRASP algorithm. The pseudocode for the proposed algorithm can be seen in Algorithm 4.

Algorithm 4 Pseudocode for the GRASP for the QKP

```
1: while Not stopping criteria satisfied do
2:    Generate solution S using RGA
3:    Dist = 0
4:    repeat
5:       Apply local search to S
6:       Check new best
7:       Update Solution Holder
8:       Dist = Dist + 1
9:       if ShouldExpand(S) then
10:         ForceExpand(S)
11:      end if
12:   until NotExpanded(S) ∨ (Dist > MaxDist)
13: end while
```

As it can be seen in Algorithm 4, in the main loop a new solutions S is generated at each iteration using the randomized greedy algorithm (RGA). In the inner loop, the first step is applying the selected local search to S. Next, the solution holder, used to track the number of generated solutions of a specific size, since the last improvement of the best solution for that size, is updated for the new solution. Next, based on the state of the solution holder, we check if S should be expanded. If this is true and the maximal number of allowed elements has not been added to S, the inner loop is reentered.

With the intention of having an efficient implementation of the greedy algorithm and the local searches the values of the function $Con(S, i)$ are stored at each iteration and updated as new elements are added to S. The calculation of the function $P(S, i \rightarrow j)$ can be efficiently done based on the values of $Con(S, i)$ [7]. In the multi-swap local search due to the choice of $T << n^2$ the computational cost of calculating the values of $P(S, H)$ for a swap set $H \subset M$ did not prove to be very significant.

Although the asymptotical computational cost of the single swap and multi-swap local searches is the same in case of large problem instances, we have experienced a significant difference. In our initial implementation, in case of hard instances having

2000 items the multi-swap local search was even ten times slower. The reason for this is that the time needed to find a single improving swap for solution S was drastically lower than the one needed to find multiple ones in the multi-swap local search. The reason for this is that the search is terminated as soon as the needed number of improving swaps has been found.

The brute force evaluation of all swaps, as used in our initial implementation, is a double loop, where the outer one is related to the elements inside the solution (S) and the inner one to items outside the solution (C). The randomization has been achieved by randomizing the order in which the items are visited in each of these sets. The number of tested swaps can be drastically decreased if the elements in the inner loop are visited in an ascending order by weight. This is due to the fact that if at some iteration of the outer loop an item $j \in S$ is tested for a potential improving swap, as soon as an item $i \in C$ for which $W(S, i \rightarrow j) > c$ is encountered, the inner loop can be exited. A similar approach can be used if the elements of C are used in the outer loop and the ones in S are used in the inner loop, with the difference that the order is now descending. In practice, this has a minor negative effect on the GRASP since randomness in the local search is reduced. Our computational tests have shown that by using this improvement the computational cost of the multi-swap search becomes only double the one of the single swap search in case of problem instances having 2000 items.

6.5 Instance Generation

In this section, we present the method used for generating a new set of test instances. In case of the knapsack problem, it has been shown that randomly selecting weights and profits results in easy-to-solve instances [29]. On the other hand, in case of the use of the KP and the QKP for modeling real-world problems, there is often some correlation between these values. In generating the new set of test instances, we extend the ideas from [29] for the KP to the QKP. In generating the test instances, the focus is on the method for generating weights, profits related to individual items and pairs. Our goal was to use reasonable assumptions in adding correlation between the weights and profits.

6.5.1 Weight

In case of weights in the KP there is a significant difference in hardness of instances depending on the distribution of weights, more precisely, if there is a large difference between the minimal and maximal weight of an item. Because of this the objective was to evaluate the performance of the algorithms for different ranges. The item weights are randomly selected from the range $[R_{min}, R_{max}]$. The value of R_{max} has

been generated based on the following formula

$$R_{max} = R_{min} + 2^i R_{min} \tag{6.13}$$

The value of i is an integer uniformly selected from the range -4 to 6. In all the instances, we have used the value $R_{min} = 100$.

6.5.2 Item Profit

In case of item profits, we follow the ideas from [29]. To be more precise, the item profits have been generated based on weights using the following approaches, with rounding to the closest integer value:

- *Random:* The profit p_i for item i is randomly selected from the range $[R_{min}, R_{max}]$.
- *Strongly Correlated:* The profit p_i is directly specified based on the weight. The idea, as in [29], is that the profit is directly connected to the weight with some additional benefits. This is done using the following formula

$$p_i = w_i + \frac{R_{max}}{10} \tag{6.14}$$

- *Weakly correlated:* The profit p_i is correlated with the w_i but with a small level of randomization. In the formula used for specifying this relation, R^* is a random variable selected using a uniform distribution from the range $[1, R_{max}]$.

$$p_i = w_i + \frac{R^*}{20} - \frac{R_{max}}{10} \tag{6.15}$$

6.5.3 Pair Profit

In this subsection the methods for generating profits for item pairs are presented. The following three methods are used, with rounding to the closest integer value.

- *Random* The profit p_{ij} for item pair i, j is randomly selected from the range $[R_{min}, R_{max}]$ with a uniform distribution.
- *Geometric mean* The profit p_{ij} is directly specified based on the weights of the two related items. More precisely, the rounded value of the geometric mean is used

$$p_{ij} = \sqrt{w_i w_j} \tag{6.16}$$

The idea behind this method is that there is a relation between the possible profit of a pair of items and their weights. The basic idea is that it is related to the total weight of both items. The choice for the geometric means has been used to penalize large differences in weights of the two items.

- *Minimum* The p_{ij} is equal to the minimum value of w_i and w_j. The inspiration for this method comes from factory production; to be more precise, it is only possible to acquire additional profit by combining the items "produced" by each of the two factories.

In case of paired profits, as in the instances given in [7, 10, 35, 36], density is also included. Here we use the term density for the percentage of paired profit values p_{ij} that is not equal to zero. In the generated instances, we have used densities of 25%, 50%, 75% and 100%. The pairs are randomly selected using a uniform probability distribution.

6.6 Computational Results

In this section, we present the results of the computational experiments. The tests are divided into three main groups. In the first group a comparison is done to the state-of-the-art methods [7, 10, 36] to verify the effectiveness of the proposed GRASP methods. The objective of the second group of tests is to evaluate the effectiveness of the GRASP algorithm using the multi-swaps local search (GR-M) against a GRASP using the standard single swap local search (GR-S). In the last group of tests, the hardness of the newly generated test instances is evaluated. The newly generated instances can be downloaded from [14].

Both the GRASP algorithms have been implemented in C# using Microsoft Visual Studio 2017. The calculations have been done on a machine with Intel(R) Xeon(R) CPU E5-2643 3.30 GHz (2 processors), 96 GB RAM, running on Microsoft Windows 7 Home Premium 64-bit. The parameters used for the GRASP algorithms are the following. The size of the RCL was $R = 10$. In case of the GR-M, the limit on the size of a tested swap set was $m = 6$. The maximal number of tested swap sets was $T = 1000$. The maximal number of items that would be used for force expanding a solution was $MaxDist = 3$. The speedup for finding improving swaps has only been used in case of tests on the newly generated instances having 1500 and 2000 items.

6.6.1 Comparison to Existing Methods

In this subsection, we compare the performance of the proposed GRASP algorithm to the GRASP algorithm hybridized with tabu search (GR-T), the iterated "hyperplane exploration" approach (IHEA) and the dynamic programming heuristic

method (DE) [10]. In the presented results, we did not include the tests for instances from 100 to 300 items used in papers [7, 36] since all the GRASP-based methods (GR-T, IHEA, GR-S and GR-M) manage to find the known best solution at a very low computational cost. The results for the methods used in the comparison are taken from [7]. The used test instances have 1000 or 2000 items and the density of 25%, 50%, 75% and 100%. For each combination of instance size and density, 10 different random instances exist, in total there are 80 of them.

As it will be shown in the following section although GR-M has a better performance than GR-S they tend to get trapped in different local optima. Because of this, we used the combined method GR-C where one half of the solutions are generated using GR-S and the other using GR-M. The solutions generated using the two local searches are generated interchangeably.

The experimental setup has been slightly changed from the ones used in previous published work where for each instance each method has been run 100 times. In our tests, we have only conducted a single run of GR-C and the termination criterion was that 2000 solutions have been generated. The main reason for this is that we did not consider that there is a significant increase of information acquired with the large number of independent runs, while this would greatly increase in computational time. The second reason was that our focus was on showing that the proposed method manages to find the best known solutions. The results of the performed computational experiments are given in Table 6.1. In it, we give the aggregated results for the instances for each pair of instance size and density. Detailed results containing the best known solutions for each instance can be found in [7]. In Table 6.1, the number of found best-known solutions for each method and average computational time are given. For IHEA and GR-C, the average time for finding the best solution are also presented. For all the competing methods, the best found solution over 100 runs for each instance is used. Note, that this gives an advantage to the competing methods.

From these results, it can be observed that the GRASP-based methods have the best performance. Out of them only GR-T does not manage to find all the best known solutions. We wish to mention, although the results are not included in Table 6.1, GR-S and GR-M did not manage to find two and one of the best-known solutions, respectively. This indicates that force expanding a solution greatly improves the performance of the basic GRASP method, like the one used in GR-T, with a very small increase in implementation complexity. When we observe the computational cost, it is evident that the IHEA outperforms all the other methods. On the other hand GR-C had a lower computational cost than GR-T, but compared to IHEA it is relatively similar. This is expected since the basic structure of both methods is similar. Note that in the IHEA, an important component of the algorithm is selecting a part of the solution that will be fixed in the use of hyperplanes. This is a relatively simple task in case of completely random instances but we expect it would be much more complex in case of instances containing some correlation like in the newly generated ones. In addition, the concept of hyperplanes could be used to hybridize GR-C. One thing that we have observed for the computational time for finding the best solution for GR-C is that for a vast majority of instances it is very

Table 6.1 Computational results indicating the number of best solutions found and average time needed

	Number of found best				Avg. time				Avg. time best	
Items(density)	DE	GR-T	GR-C	IHEA	DE	GR-T	IHEA	GR-C	IHEA	GR-C
1000(25%)	3	10	10	10	2441.35	21.87	5.49	19.20	0.14	2.15
1000(50%)	1	10	10	10	2862.22	31.17	6.02	22.07	0.39	0.76
1000(75%)	1	9	10	10	3143.04	26.28	6.39	37.86	0.16	12.56
1000(100%)	2	9	10	10	3224.19	32.53	6.12	28.69	0.13	0.38
2000(25%)	0	10	10	10	50,528.39	295.86	22.24	148.68	0.75	38.80
2000(50%)	2	9	10	10	50,427.95	352.22	23.24	186.87	0.53	42.38
2000(75%)	1	8	10	10	52,334.30	287.10	22.72	216.16	0.60	0.96
2000(100%)	1	9	10	10	53,492.38	383.42	22.72	182.72	1.06	1.46

fast since it finds it in the first few iterations. On the other hand, for the remaining instances this is much more time consuming because it takes a very large number of iterations to find the best solution, and this drastically effects the average time.

6.6.2 Comparison of Local Searches

In this subsection the computational experiments used for evaluating the performance of the two proposed local searches are presented. With the intention of having a more extensive evaluation, the tests have been done on a wide range of newly generated instances. To be more precise, instances have been generated having between 100 and 2000 items. For each problem size a total of 36 random instances have been generated. Each of the instances would correspond to a triplet (i_p, p_p, d) related to the generating methods, where i_p corresponds to the method for generating item profits, p_p to the method for generating item pair profits and d is the density. For each such triplet, the value of the integer parameter i used to specify the weight distribution has been randomly selected with a uniform distribution from the range $[-4, 6]$. In total there are 324 instances.

The main objective of these tests is to see the quality of solutions that can be found by GR-S and GR-M. The stopping criterion for executing each of the algorithms was that there was no improvement in the last 2000/4000 generated solutions for instances having up to/above 500 instances. The reason for choosing a relatively high number of stagnant iterations is to make it highly unlikely that a method would find a higher quality solution. The second criterion was that computational time was 600 s. In Table 6.2 the aggregated values for each problem size are given. The selected properties are the sum of profits, the number of times GR-S or GR-M found a better solution than the other method, the average computational time and the average time for finding the best solution.

From these results it is evident that GR-M has a significantly better performance than GR-S when solution quality is considered. In total GR-M has found better/worse solutions than GR-S for 85/40 problem instances. GR-M managed to have a higher total profit than GR-S for all the tested problem sizes. The average computational time for both GR-S and GR-M are very similar, as in the case of the time for finding the best solutions. It is important to point out that for problems having 1500 and 2000 items, for both GR-S and GR-M, for a majority of instances the stopping criteria was that 600 s have passed. This indicates that both methods had the potential of finding better solutions. It is noticeable that the computational time for finding the best solution for the newly generated instances is significantly greater than for the standard instances, for problems having 1000 or 2000 items.

Table 6.2 Comparison of GRASP-S and GRASP-M on generated instances

	Total profit		Num. better		Avg. time		Avg. time best	
Items	GR-S	GR-M	GR-S	GR-M	GR-S	GR-M	GR-S	GR-M
100	48,211,193	48,212,693	0	4	8.8	8.8	0.79	1.61
200	115,358,886	115,359,619	0	3	26.2	23.3	3.64	3.16
300	216,445,966	216,449,956	1	12	33.3	42.6	2.72	7.25
400	620,735,360	620,739,193	5	11	73.7	65.8	14.3	10.71
500	793,535,829	793,529,527	6	7	85.1	78.1	14.59	12.85
750	1,213,590,665	1,213,616,321	4	10	331.53	348.4	50.83	51.82
1000	4,975,415,246	4,975,416,082	7	9	455.8	472.4	77.50	100.17
1500	7,663,759,287	7,663,773,948	6	14	536.7	526.1	133.24	134.074
2000	10,808,221,963	10,808,282,096	11	15	527.6	528.9	107.63	144.60

6.6.3 Evaluation of Test Instances

In this section, we analyze the properties of the problem instances generated using different methods presented in Sect. 6.5. Secondly, the relation between the performance of the GR-S and GR-M and instance properties are also evaluated. To achieve this, the goal was to solve a large number of instances with a fixed size in a reasonable time. Because of this, we have chosen to generate instances having 300 items. For each combination of generation methods corresponding to a 4-tuple (i, i_p, p_p, d), 5 different random instances are generated. As before i corresponds to the integer parameter used to specify the weight distribution, i_p corresponds to the method for generating item profits, p_p corresponds to the method for generating item pair profits and d to densities. The evaluation has been done on a total of $11 \times 3 \times 3 \times 4 \times 5 = 1980$ different random instances.

Each of the generated instances has been solved using GR-S and GR-M, using the same settings as in the previous subsection. To be able to assess the hardness of the instances several properties of executing the two GRASP algorithms are observed. To be exact, the average time and number of iterations needed to find the best solution; the average number of swaps that has been applied; and the number of times GR-S or GR-M managed to find a better solution than the other method. Theses aggregated values are analyzed for each of the components of the generation method (weight distribution, item profit, pair profit, density), separately. This is done by dividing all the generated test instances into n disjunctive sets for each component of the generation algorithm, where n is the number of different options for that component. Each of these sets has $\frac{1980}{n}$ elements. The group for option o consists of all instance that have used this option in there generation. The related results can be seen in Table 6.3.

The first thing that can be observed from the values in Table 6.3 is that the method for generating profits related to items does not significantly impact the computational cost, hardness and the performance of the local searches. We expect that this is due to the fact that this value has a relative small impact on the total profit of an instance.

The effect of the method for generating the profit related to item pairs is more significant. The GR-M managed to find a significantly higher number of better solutions in case of using the geometric mean and minimum method than GR-S. Around 85% of the instances for which GR-C and GR-M did not find the same solution are of these two types. In case of using the random approach for generating item pair profits GR-S and GR-M had a very similar performance, each method being better than the other in around 5% of the total number of instance. Both methods managed to find the best solution for instances of this type generating a much lower number of solutions (iterations of the GRASP) and computational cost, than for the ones having some correlation. Only in case of using this generation method there was a significant difference in computational time. To be specific, GR-S was two times faster than GR-M.

Table 6.3 Comparison of GRASP-S and GRASP-M on generated instances

	Num. better		Average					
			Time best [s]		Iteration best		Number of swaps	
Generation method	GR-S	GR-M	GR-S	GR-M	GR-S	GR-M	GR-S	GR-M
Item profit								
Random	45	121	6.2	7.0	531.8	449.3	28.1	17.2
Strongly correlated	52	136	8.5	6.3	554.5	430.3	26.0	16.0
Weakly correlated	40	135	7.7	7.3	578.0	501.1	27.7	17.1
Pair profit								
Random	26	29	1.4	3.3	216.0	215.3	29.5	18.3
Geometric mean	75	211	12.2	9.8	838.8	681.2	26.0	15.7
Minimum	36	152	8.8	7.5	609.5	484.2	26.4	16.2
Density								
25%	56	80	4.9	8.1	618.2	614.1	31.1	19.2
50%	38	66	3.2	5.0	454.1	414.0	27.7	16.9
75%	32	58	3.0	5.9	411.7	381.3	27.5	16.5
100%	11	188	18.8	8.4	735.0	431.4	22.7	14.3
Weight distribution								
$i = -4$	21	47	7.7	14.8	700.0	599.5	72.7	43.1
$i = -3$	21	54	9.9	14.5	756.0	727.9	57.9	34.5
$i = -2$	27	64	10.3	13.2	889.9	816.1	40.8	24.9
$i = -1$	21	56	9.3	11.8	808.4	822.0	28.4	17.6
$i = 0$	12	48	11.4	6.6	746.0	574.4	21.3	13.2
$i = 1$	13	26	4.3	3.2	446.2	384.5	16.2	10.3
$i = 2$	6	26	9.5	3.1	453.2	302.5	13.8	8.8
$i = 3$	6	16	5.4	2.8	358.1	238.4	12.8	8.3
$i = 4$	2	22	3.8	1.3	284.3	167.4	12.1	7.8
$i = 5$	3	21	3.9	2.2	325.0	218.0	12.3	8.0
$i = 6$	5	12	6.7	2.1	335.5	211.5	11.6	7.7

In case of densities GR-M produces the highest improvement for the density of 100%, finding a better solution for almost 40% of the instances of this type. For highly dense instances, GR-M had a significantly lower computational cost and number of generated solutions than GR-S. In case of lower densities GR-M manages to find a higher number of better solutions than GR-S but not as significant as in case of high densities. In case of lower densities the two GRASP algorithms performed a similar number of iterations but the computational cost of GR-M was 1.5–2 times that of GR-S. It is interesting to point out that the computational cost for solving instances was higher for the two extremes (25% and 100%).

In case of different distribution of item weights, there is a very significant difference in the computational cost for solving the instances using GR-M. The time needed for solving instances with the largest difference between the minimal and maximal weight was 7 times lower than in the case of the smallest one. This

type of behavior does not exist for GR-S. The number of instances for which GR-M found better solutions than GR-S was generally around 3 times. For both methods the number of generated solutions and applied swaps was much lower for instances with a wider range of item weights. The instances having a lower difference in item weights had a higher number of ones where the two GRASP algorithms found different solutions.

This indicates that these instances are harder to solve, at least in case of using GRASP-based approaches. In case of the standardly used test instances practically all the state-of-the-art methods manage to find the same best-known solutions (expected optimal) for instances having less than 1000 items. This was also the case for the methods GR-S and GR-M. In case of the proposed new test instances, this is not the case, where these two methods often find different best solutions for the same instance indicating that at least one of them did not find the best possible solution. One of the reasons for this is that the standard instances for the QKP have a wide range of item weights which is one of the reasons they are easier to solve.

6.7 Conclusions

In this paper we have presented a GRASP method for solving the QKP. The proposed algorithm is simple to implement and is not dependent on a large number of parameters. In the design of the proposed method the focus was only on incorporating essential components of existing similar methods. In addition, a new type of local search has been introduced that considers removing and adding multiple elements in the solutions instead of a single one as in the commonly used local search. Although the proposed local search considers multiple swap operations it has the same asymptotic computational cost as in case only a single one is used.

Another contribution of this paper is the generation of a new set of hard test instances for the QKP, which have been extensively analyzed. It has been shown that the new instances are significantly harder to solve than the standard ones. The new instances have been used to evaluate the GRASP algorithms based on the two local searches. The computational experiments have shown that the local search based on multiple swaps has an overall better performance than the one based on single swaps. On the other hand the GRASP algorithms based on the two local searches tend to get trapped in different local optima. As a consequence the best performance can be achieved by a GRASP algorithm that uses both local searches interchangeably. Our computational results have shown that the proposed method manages to find all known best solutions for instances having up to 2000 items. Further, an extensive analysis has been done on the performance of the two local searches on problem instances having different properties.

In the future we plan to extend the proposed algorithm to the fixed set search metaheuristic [15, 16] which adds a learning mechanism to GRASP. Another potential extension of the proposed work is adapting the multi-swap local search to other variations of the KP like the knapsack problem with conflict graphs.

References

1. Avci, M., Topaloglu, S.: A multi-start iterated local search algorithm for the generalized quadratic multiple knapsack problem. Computers & Operations Research **83**, 54–65 (2017)
2. Banda, J., Velasco, J., Berrones, A.: A hybrid heuristic algorithm based on mean-field theory with a simple local search for the quadratic knapsack problem. In: 2017 IEEE Congress on Evolutionary Computation (CEC). pp. 2559–2565 (June 2017)
3. Billionnet, A., Calmels, F.: Linear programming for the 0–1 quadratic knapsack problem. European Journal of Operational Research **92**(2), 310–325 (1996)
4. Billionnet, A., Faye, A., Soutif, É.: A new upper bound for the 0–1 quadratic knapsack problem. European Journal of Operational Research **112**(3), 664–672 (1999)
5. Billionnet, A., Soutif, É.: An exact method based on Lagrangian decomposition for the 0–1 quadratic knapsack problem. European Journal of Operational Research **157**(3), 565–575 (2004)
6. Chaillou, P., Hansen, P., Mahieu, Y.: Best network flow bounds for the quadratic knapsack problem. In: Combinatorial Optimization, pp. 225–235. Springer (1989)
7. Chen, Y., Hao, J.K.: An iterated "hyperplane exploration approach" for the quadratic knapsack problem. Computers & Operations Research **77**, 226–239 (2017)
8. Dijkhuizen, G., Faigle, U.: A cutting-plane approach to the edge-weighted maximal clique problem. European Journal of Operational Research **69**(1), 121–130 (1993)
9. Feo, T.A., Resende, M.G.: Greedy randomized adaptive search procedures. Journal of Global Optimization **6**(2), 109–133 (1995)
10. Fomeni, F.D., Letchford, A.N.: A dynamic programming heuristic for the quadratic knapsack problem. INFORMS Journal on Computing **26**(1), 173–182 (2013)
11. Hammer, P., Rader Jr, D.J.: Efficient methods for solving quadratic 0–1 knapsack problems. INFOR: Information Systems and Operational Research **35**(3), 170–182 (1997)
12. Hansen, P., Mladenović, N.: Variable neighborhood search: Principles and applications. European Journal of Operational Research **130**(3), 449–467 (2001)
13. Johnson, E.L., Mehrotra, A., Nemhauser, G.L.: Min-cut clustering. Mathematical Programming **62**(1–3), 133–151 (1993)
14. Jovanovic, R.: QKPLIB. https://doi.org/10.17632/82pxy6yv49.1, Mendeley Data
15. Jovanovic, R., Voss, S.: The fixed set search applied to the power dominating set problem. Expert Systems **37**(6), e12559 (2020)
16. Jovanovic, R., Voß, S.: Fixed set search application for minimizing the makespan on unrelated parallel machines with sequence-dependent setup times. Applied Soft Computing **110**, 107521 (2021)
17. Julstrom, B.A.: Greedy, genetic, and greedy genetic algorithms for the quadratic knapsack problem. In: Proceedings of the 7th annual conference on Genetic and evolutionary computation. pp. 607–614. ACM (2005)
18. Kellerer, H., Pferschy, U., Pisinger, D.: Introduction to NP-completeness of knapsack problems. In: Knapsack problems, pp. 483–493. Springer (2004)
19. Kellerer, H., Strusevich, V.A.: Fully polynomial approximation schemes for a symmetric quadratic knapsack problem and its scheduling applications. Algorithmica **57**(4), 769–795 (2010)
20. Krarup, J., Pisinger, D., Plastria, F.: Discrete location problems with push–pull objectives. Discrete Applied Mathematics **123**(1–3), 363–378 (2002)
21. Létocart, L., Plateau, M.C., Plateau, G.: An efficient hybrid heuristic method for the 0–1 exact k-item quadratic knapsack problem. Pesquisa Operacional **34**(1), 49–72 (2014)
22. Park, K., Lee, K., Park, S.: An extended formulation approach to the edge-weighted maximal clique problem. European Journal of Operational Research **95**(3), 671–682 (1996)
23. Petersen, C.C.: A capital budgeting heuristic algorithm using exchange operations. AIIE Transactions **6**(2), 143–150 (1974)

24. Pferschy, U., Schauer, J.: Approximation of the quadratic knapsack problem. INFORMS Journal on Computing **28**(2), 308–318 (2016)
25. Pferschy, U., Schauer, J.: The knapsack problem with conflict graphs. J. Graph Algorithms Appl. **13**(2), 233–249 (2009)
26. Pferschy, U., Schauer, J.: Approximating the quadratic knapsack problem on special graph classes. In: Kaklamanis, C., Pruhs, K. (eds.) Approximation and Online Algorithms. pp. 61–72. Springer International Publishing, Cham (2014)
27. Pferschy, U., Schauer, J., Maier, G.: A quadratic knapsack model for optimizing the media mix of a promotional campaign. In: Pinson, E., Valente, F., Vitoriano, B. (eds.) Operations Research and Enterprise Systems. pp. 251–264. Springer International Publishing, Cham (2015)
28. Pisinger, D.: A minimal algorithm for the multiple-choice knapsack problem. European Journal of Operational Research **83**(2), 394–410 (1995)
29. Pisinger, D.: Where are the hard knapsack problems? Computers & Operations Research **32**(9), 2271–2284 (2005)
30. Pisinger, D.: The quadratic knapsack problem—a survey. Discrete Applied Mathematics **155**(5), 623–648 (2007)
31. Pisinger, W.D., Rasmussen, A.B., Sandvik, R.: Solution of large quadratic knapsack problems through aggressive reduction. INFORMS Journal on Computing **19**(2), 280–290 (2007)
32. Rhys, J.: A selection problem of shared fixed costs and network flows. Management Science **17**(3), 200–207 (1970)
33. Shi, X., Wu, L., Meng, X.: A new optimization model for the sustainable development: Quadratic knapsack problem with conflict graphs. Sustainability **9**(2), 236 (2017)
34. Taylor, R.: Approximation of the quadratic knapsack problem. Operations Research Letters **44**(4), 495–497 (2016)
35. Xie, X.F., Liu, J.: A mini-swarm for the quadratic knapsack problem. In: Swarm Intelligence Symposium, 2007. pp. 190–197. IEEE (2007)
36. Yang, Z., Wang, G., Chu, F.: An effective grasp and tabu search for the 0–1 quadratic knapsack problem. Computers & Operations Research **40**(5), 1176–1185 (2013)

Chapter 7
Algorithm vs Processing Manipulation to Scale Genetic Programming to Big Data Mining



S. Ben Hamida and H. Hmida

Abstract In the era of petabyte, robust machine learning tools are needed to cope with the volume and high dimensionality of data to min. Evolutionary Algorithms (EA), such as Genetic Programming (GP), are powerful machine learning techniques with great potential to deal with big data challenges. To better exploit their capacities, additional manipulations can help the EA to alleviate the computation cost and then better look insight the large data sets. This chapter summarizes some solutions and trends to address difficulties when training EA/GP on big data sets and proposes a taxonomy to classify these solutions on three categories: Processing manipulation, algorithm manipulation and data manipulation. Two approaches are then presented and discussed. The first one, from the processing manipulation category, parallelizes GP over Spark. The second one, from the algorithm manipulation category, extends GP with active learning using dynamic and adaptive sampling. For each approach, some guidelines of implementation into the GP loop over an EA Python framework are given. A combination of the two approaches is also presented. The efficiency of these solutions is then discussed according to some published experimental studies.

S. Ben Hamida (✉)
LAMSADE, Paris Dauphine University, PSL Research University, Paris, France
e-mail: Sana.mrabet@dauphine.psl.eu

H. Hmida
Institut Supérieur des Études Technologiques de Bizerte, Menzel Abderrahmane, Tunisia

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

M. Eddaly et al. (eds.), *Metaheuristics for Machine Learning*,
Computational Intelligence Methods and Applications,
https://doi.org/10.1007/978-981-19-3888-7_7

7.1 Introduction

In the era of 'Big Data', the amount of digitized data has reached a huge volume and is announced to grow up to 175 ZB[1] by 2025 according to the IDC report sponsored by Seagate [37]. This abundance of data provides a wealth of knowledge for the field of machine learning. In this context, several publications covered this topic regardless of the machine learning approach in [1, 3, 8] and particularly for evolutionary algorithms in [6]. These different challenges are related to the 'Big Data'attributes *Volume, Variety and Velocity*. In the field of Machine Learning (ML), mining big data is a main challenge for ML tools. Only powerful knowledge discovery techniques are able to deal with the related challenges.

Recent theoretical and experimental studies in Machine Learning have shown the potential of the evolutionary computation techniques to deal with large scale data mining. Genetic Programming (GP), as a powerful evolutionary computation technique, has a great potential to deal with Big Data analysis. It aspires to be a promising tool to look insight massive data thanks to its global search in the solution space and its natural parallel implementation. In this chapter, we explain how GP, with some extensions such as adding a proper learning paradigm or an efficient enhancement technique, could be an opportunity to deal with Big Data mining challenges.

The solutions to deal with the Big Data challenges with GP are multiple and can be applied at different steps of the learning pipeline. They can belong to the data manipulation, process manipulation or algorithm manipulation category (Fig. 7.2). These solutions are summarized in the next section with a taxonomy for each category. In the rest of the chapter, we are mainly interested on three approaches. The first concerns the horizontal parallelization in the category of process manipulation. Its goal is to port GP over a Big Data ecosystem. The second approach consists on adding a proper learning paradigm to GP and it is part of the algorithm manipulation category. The last approach combines the algorithm and processing manipulations in the same GP engine.

Parallelizing GP in a distributed environment is the oldest and most studied approach to scale up GP to large size problems such as Big Data mining. GP could be distributed either with vertical parallelization (scaling up) paradigm or with horizontal parallelization (scaling out) paradigm [29]. These paradigms are summarized in the next section. We then present in details a scaling out solution, that aims to parallelize GP in a distributed ecosystem by operating existing bookstores based on Apache Spark.[2] A Python implementation is proposed to port a known Python GP framework to the Spark context [22]. The complete source code of this model is available at GitHub.[3]

[1] ZB: Zettabyte = 10^{21} bytes.

[2] https://spark.apache.org.

[3] https://github.com/hhmida/gp-spark.

The first paradigm that has been well studied in the context of large scale data mining is the Active Learning. This paradigm can be easily integrated into the GP loop and has proven to be very effective in improving the GP performance and reducing its computational cost [19, 20]. It is implemented mainly by dynamic sampling, but a more advanced version has been proposed in [7, 23] with an adaptive sampling taking into account the evolution of the learning process. An example of Python code to implement the Active Learning paradigm within a GP loop is given in Sect. 7.4.1. Section 7.3 discuss how both solutions with processing and algorithm manipulations could be combined and integrated simultaneously in the GP engine. We then discuss the efficiency of these different approaches based on different experiments to solve classification problems on large and very large data sets (Sect. 7.6).

7.2 Scaling GP to Big Data Mining

Solutions to deal with Big Data challenges in the ML field are numerous. The most important contributions concern the Big Data frameworks such as Hadoop and Map/Reduce. However, the majority of the proposed approaches are inspired from traditional techniques already used in ML and have been adapted to the big data context. For example, some paradigms that have been defined to improve learning from complex and/or imbalanced data, or to avoid over-fitting, such as Active Learning and Ensemble Learning, have been improved to be reused in the Big Data context. Data pre-processing techniques are also improved to deal with big data characteristics such as volume and velocity.

All Big Data solutions, either new approaches or improved old approaches, can be classified on three categories through the ML pipeline [29]: data manipulation based solutions, algorithm manipulation based solutions and processing manipulation based solutions. Figure 7.1 illustrates these three categories according to the application step on the ML pipeline.

Data manipulation is applied in the pre-processing phase before administering the data to the learning process. Processing manipulation aims to handle the additional computational cost by vertical or horizontal scaling. Algorithm manipulation extends any ML algorithm with additional ML paradigms such as Active Learning

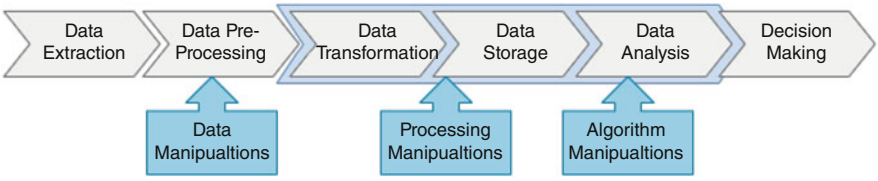


Fig. 7.1 Manipulation categories through the Machine Learning pipeline [29]

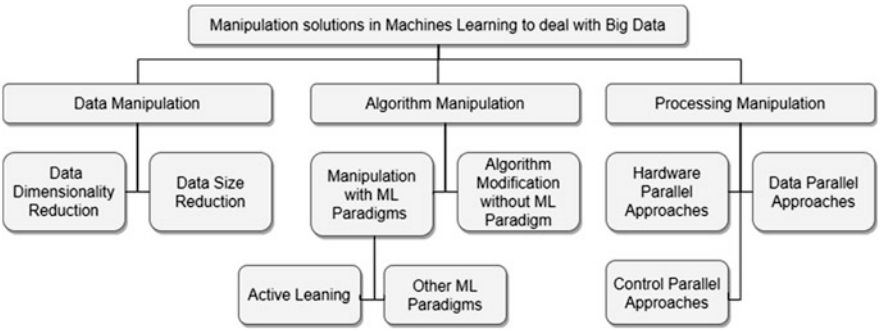


Fig. 7.2 Solutions' categories to scale EA/GP to Big Data

or Ensemble Learning. We summarize below some solutions in each category and how it could be introduced to the GP engine. Solutions discussed in this section are representative and in no way provide a comprehensive list of solutions.

7.2.1 Data Manipulation

The goal of data manipulation is to reduce the size or the dimensionality of the training database (Fig. 7.2). Dimensionality reduction aims to reduce the number of attributes in the data set by either selecting the most significant ones or by creating new ones. Size reduction aims to reduce the number of exemplars in the database by using some selection techniques. Instance selection, in this case, is independent of the learning process. Dimensionality and size reduction could be applied simultaneously on the data set and they are independent of the learning step. Therefore, there are no solutions specific to GP. However, GP is less affected by this step than other learning techniques needing some linear relation between variables, such as logistic regression or neural networks. Data cleaning, that is a very important step to deal with Big Data challenges related to the variety, is not discussed in this chapter.

- Dimensionality reduction: The dimensionality defines the number of features present in the data set. For several scientific domains, such as biology and physics, data is collected using sophisticated instruments. Thus, the resulting database might have very high dimensionality. However, as the number of features increases, the performance of machine learning algorithms decreases. Dimensionality reduction is a major issue in machine learning and in usually associated to the Feature Engineering domain.
- Size reduction: It refers to instance selection from Big Data in order to create a representative subset of the initial database having less number of exemplars. The simplest technique for instance selection is the *'Holdout'* method where a subset is selected randomly and administrated to the ML pipeline. More sophisticated

techniques could be used in the case of Big Data to conserve some characteristics or to remedy the class imbalance problem. As for the dimensionality reduction, techniques for instance selection are generic and there are not specific to EA/GP.

7.2.2 Processing Manipulation

Processing manipulation includes parallelization approaches using the parallel nature of some population based algorithms such as EA/GP. Parallelization can be done with a hardware acceleration or a distribution mechanism.

- Hardware Parallel Approaches: Hardware parallelization includes all classic methods using multiple computing nodes. They are based on increasing resources for a single node. For example, hardware acceleration based on graphics processors called General Purpose Graphics Processing Units (GPGPU) and parallelization using multicore CPUs [18, 27, 31].
- Control Parallel Approaches: In the case of population based algorithms such EAs, the control-parallelization is implemented by distributing the population. Two schemas of distribution are possible: coarse-grained and fine-grained. With coarse-grained schema, the population is divided on a few number of sub-populations where each sub-population is associated with a single processor. For a ML task, each processor computes the fitness of its sub-population according to the whole training set. Coarse-grained EAs are also called "island model" EAs [9]. When the number of sub-populations and genetic operators are limited to the small neighborhood, the distribution is fine-grained. Fine-Grained Parallel EAs are also known as "cellular" EAs [9]. Control-parallelization with EAs is not yet well explored for Big Data mining tasks.
- Data Parallel Approaches: Data parallelization, also known as Data intensive computing, includes parallel scenarios where the computing cores are multiple, heterogeneous and distributed [6, 31]. This is the most common form of distribution with Big Data problems. It becomes very popular since the appearance of the Map-Reduce data analysis methodology of Google and its open-source implementation, Hadoop [13]. Several frameworks have been put in place for two types of parallelization: by batch or flow-oriented. The most used are Hadoop/MapReduce [13] and Apache Spark [40]. Further discussion about these frameworks is given in Sect. 7.3.

7.2.3 Algorithm Manipulation

Methods in this category relies on algorithm manipulations in order to optimize its running time or to improve the learning quality. It involves the modification of the

machine learning algorithms either by the introduction of some ML paradigms or some optimization strategies.

- Algorithm modification by an optimization strategy: In the cases of EAs, many optimization strategies could be experimented to better min Big Data. For example, min a large data set could be modified as a multi-objective optimization problem where the training data is sampled in multiple subsets and each subset is associated to a single objective. Otherwise, hybridization with local optimization is frequently implemented with EAs and could help to better look insight large data sets.
- Algorithm manipulation by a machine learning paradigm: a variety of ML paradigms exists in the field of machine learning and some of these paradigms are well suited to address some challenges in the context of Big Data. The first paradigm proved to be efficient to deal with very large data sets is the active learning paradigm. It is able to alleviate the computational cost of the learning process, help EAs to better min the available data, and it could be easily integrated in the EA loop [7, 20]. This paradigm is discussed in details in Sect. 7.4. Ensemble Learning is another paradigm frequently implemented with EA, and recently used in the context of Big Data [14]. Other paradigms could be promising approaches to improve the training capacities of EA in the context of Big Data such Online Learning, Local Learning or Transfer learning. Heureux et al. discussed in their paper [29] how these paradigms can deal with Big Data challenges.

7.3 Scaling GP by Processing Manipulation: Horizontal Parallelization

7.3.1 Spark vs MapReduce

The evolution of the Big Data ecosystem has allowed the development of new approaches and tools such as Hadoop MapReduce[4] and Apache Spark[5] that implement a new programming model over a distributed storage architecture. MapReduce is a parallel programming model introduced by Dean et al. in [13] for Google. It was made popular with its Apache Hadoop implementation.The main idea of this model is that moving computation is cheaper than moving data. On a cluster, data is distributed over nodes and processing instructions are distributed and performed by nodes on local data. This phase is known as the Map phase. Finally, at the Reduce phase, some aggregation is performed to produce the final output. Hadoop Distributed Files System (HDFS) ensures data partitioning and replication

[4] https://hadoop.apache.org.

[5] https://spark.apache.org.

for MapReduce. However, it needs many serialization operations and disk accesses. Thus, it is not suited to iterative algorithms such as EAs that are penalized by the large number of I/Os and network latency.

Apache Spark is one of many frameworks intended to neutralize the limitations of MapReduce while keeping its scalability, data locality and fault tolerance. The keystone of Spark is the Resilient Distributed Datasets (RDD) [40].

RDD is a typical, immutable parallel data structure that can be cached. RDDs support two types of operations: transformations (*map. filter, join,* ...) that produce a modified RDD and actions (*count, reduce, reduceByKey,* ...) that generate non-RDD-type results (an integer, a table, etc.) that require all the RDD partitions to be performed. Spark is up to 100 times faster than MapReduce with iterative algorithms such as EA/GP. Spark computes an optimized Directed Acyclic Graph exploiting lazy evaluation and data locality. It is compatible with different cluster managers (Built-in Standalone, Hadoop YARN, Mesos and Kubernetes) [26].

7.3.2 Parallelizing GP over a Distributed Environment

For an evolutionary machine learning algorithm, the most computationally expensive procedure is the fitness evaluation, thus it is the first step introduced in a distributed environment to parallelize GP. The cost of the population reproduction step is in general negligible according to the fitness evaluation, but it can also be parallelized. For example, Rong-Zhi Qi et al. [36] and Padaruru et al. [33] proposed solutions for parallelizing the entire evolutionary process (fitness evaluation and genetic operators) with Spark. This solution has been applied to automatic software test game generation. However, the majority of published works proposed to distribute only the fitness evaluation. For example, Chávez et al. [10] modified a Java EA framework in order to use MapReduce for fitness evaluations. This tool is tested to resolve a face recognition problem over around 50MB of data. Peralta et al. [35] applied the MapReduce model to implement an EA for the pre-processing of big datasets (up to 67 million records and attributes from 631 to 2000). It's a feature selection application where each map creates a vector of attributes on disjoint subsets of the original database. The Reduce phase aggregates all the vectors obtained.

Hmida et al. introduced in [22] an implementation inspired by the works of Chavez et al. [10] and Peralta et al. [35]. It is a transformation of an existing tool DEAP (Distributed Evolutionary Algorithms in Python)[6] [15] using a different underlying infrastructure which is Spark engine and handles a big dataset to assess the advantages of distributing the GP evaluation. The modified evaluation is illustrated in the flowchart of Fig. 7.3.

[6] https://github.com/deap/deap.

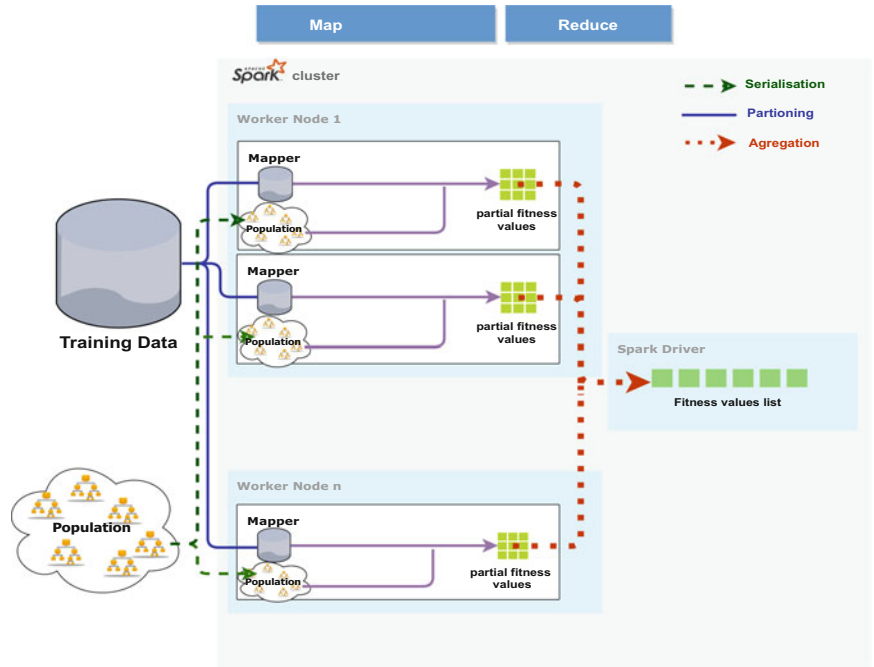


Fig. 7.3 Flowchart of the distributed GP fitness evaluation over SPark

7.3.3 Example of Implementation

DEAP is an EA Python framework presented as a rapid prototyping and testing framework that supports parallelization and multiprocessing [15]. It is structured in a way that facilitates distribution of computing tasks. Figure 7.3 illustrates an example to implement a horizontal parallel implementation within the DEAP GP engine introduced in [22]. It is a solution for the distribution of the training data base using the Spark engine to parallelize the GP evaluation step. The GP loop with the different distribution steps is illustrated by the pseudo-code given in Fig. 7.4. Lines 3, 8, 9, 11 and 12 concern the distribution of the training data set for the population fitness computation. First, a RDD containing the training set is created (line 3). Then, at each generation, a map transformation (lines 8 and 9) is performed by sending individuals code to be executed on RDD and then get results to compute each individual fitness (lines 11 and 12). Afterwards, GP pursues its standard evolutionary steps.

```
from pyspark import SparkContext
sc = SparkContext(appName="DEAPSPARK")
TrainingRDD = sc.textFile("training.csv").cache() #parallelize
training set initGP()
while(generation<maxGeneration): # GP loop
    # serialize population and map it on training subset
    popFunctions = [toolbox.compile(ind) for ind in population]
    fitnessRDD = TrainingRDD.map(lambda line:\
        [getPrediction(func,line) for func in popFunctions])
    # compute final fitness using reduce
    fitnessValues =\
        fitnessRDD.reduce(lambda v1,v2:list(map(operator.add,
        v1,v2)))
    updatePopulationFitnesses(population, fitnessValues)
    # Select the next generation individuals
    offspring = select(population)
    # Apply genetic operators
    offspring = evolve(offspring, crossoverProb, mutationProb)
    population[:] = offspring
```

Fig. 7.4 Modified DEAP GP loop

7.4 Scaling GP by Algorithm Manipulation: Learning Paradigm

Several machine learning paradigms are introduced to address difficulties for complex or special machine learning problems such as Transfer Learning, Active Learning, Local Learning, etc. Learning paradigms could also be a solution in the Big Data context. For example, Active Learning and Ensemble Learning paradigms help reducing the number of data instances used for the training step. The present section discusses briefly the Ensemble Learning paradigm and presents in details the Active Learning paradigm and how it could be an efficient solution for training GP on big data sets.

Active Learning is a learning paradigm based on active sampling techniques. Sampling training datasets has been first used to boost the learning process and to avoid over-fitting [25, 30, 34]. Later, it was introduced in machine learning as a strategy for handling large input databases. Data sampling is a very suitable technique to deal with this issue. With the increasing size of available training datasets, this practice is widely used and it belongs to the Data manipulation category (Fig. 7.1).

A wide variety of sampling methods were investigated in population based machine learning methods such GP, and demonstrated a success comparing to the use of the entire training database. With EAs for ML, it is possible to train a population of learners on a single subset $\mathcal{S}$. The subset $\mathcal{S}$ is then used to evaluate all individuals throughout an evolutionary run. It is the *run-wise* sampling frequency [16]. This sampling approach is also known as the *static sampling* (Fig. 7.5). To

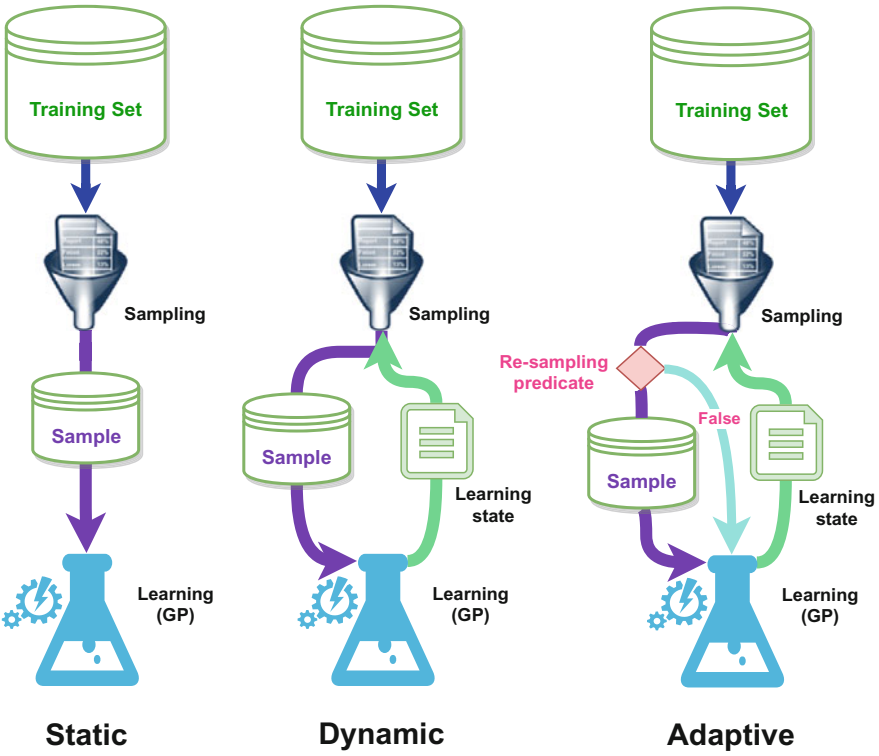


Fig. 7.5 Static vs Dynamic vs Adaptive sampling

deal with large data sets, static sampling is usually used with the Ensemble Learning paradigm, where several training runs are applied on different subsets independently or with a parallel mechanism. The obtained models are then combined either by aggregation or by a voting mechanism. For example, the bagging, (or bootstrap aggregating) and boosting techniques were introduced to GP in [25] not as a speed-up method but rather to improve GP quality and solve the problem of weak learners. The proposed algorithms BagGP and BoostGP divide the population into sub-populations where each sub-population is independently evolvable on a unique training sub-sample. Thus, the sampling technique is called only once before the evolution.

When a sampling technique is called by the learner to change the training subset along the evolution, then it is the *η-generation-wise* sampling that is applied. In this case, each η generations, a new subset $\mathcal{S}$ is extracted and used for the evaluation step, where $\eta \geqslant 1$. When $\eta = 1$, the population is evaluated on a different data subset each generation and the sampling approach is called *generation-wise* sampling. Methods in this category are known as *dynamic sampling* techniques. Dynamic sampling is the main purpose of Active Learning paradigm where the learner has some control over the inputs on which it trains [5, 11]. Several sampling

techniques with different approaches can be applied to select a training subset. Some of these techniques are summarized in the following paragraph. The training subset could be selected from the initial database or from an intermediate sample created dynamically on a precedent step. The second case is known as multi-level sampling.

A recent improvement of the traditional active sampling strategy is proposed in [7, 23] to better consider the state of the learning process. Indeed, with *generation-wise* strategy, individuals do not have enough time to extract the hidden knowledge in the current training set. An alternative approach proposed in [7, 23] is to use some information about the learning state to adapt the frequency of the training data change. It is the *adaptive sampling*. It is based on an adaptive scheme using some feed-backs from the learning state such as the proportion of solved fitness cases in the current sample or the improvement rate of best/average fitness. This information is used to design a predicate that controls the re-sampling decision. Then, the sampling frequency f can increase or decrease by a varying amount according to the general training performance.

7.4.1 Active Learning with One-Level/Multi-Level Dynamic Sampling

To select a training subset $\mathcal{S}$ from a database $\mathcal{B}$, the simplest technique is to select randomly $TargetSize$ records from $\mathcal{B}$ with a uniform probability. It is the basic approach for the Random Subset Selection method [17] (**RSS**) and some variants such as the Stochastic Sampling [32] or the Fixed Random Selection [41].

Several other techniques use more sophisticated criteria in order to address some learning difficulties like over-fitting or imbalanced data. For example, weighted sampling techniques use information about the current state of the training data that measures how much an exemplar is worthy and can help to sharpen the population quality. The well known method in this category is the *Dynamic Subset Selection* (**DSS**) [17] for classification problems intended to preserve training set consistency while alleviating its size by keeping only the difficult cases through the evolution. Each record in the data set is assigned a difficulty degree and an age updated at every generation. The difficulty is incremented for each individual misclassification and reset to 0 if the fitness case is solved. The age is equal to the number of generations since last selection, so it is incremented when the fitness case has not been selected and reset to 0 otherwise. In the same category, Lazarczyk [28] suggests a *fitness case Topology Based Sampling* **TBS** in which the relationship between fitness cases in the training set is represented by an undirected weighted graph. Edges in the graph have weights measuring a similarity or a distance induced from individuals' performance. Then, cases having a tight relationship cannot be selected together in the same subset assuming that they have an equivalent difficulty for the population.

Balanced Sampling is a third category of dynamic sampling that aims to overcome imbalance in the original data sets. The well known techniques in this category are those proposed by Hun et al. [24] which the purpose is to improve classifiers accuracy by correcting the original data set imbalance within majority and minority class instances. Some of these methods are based on the minority class size and thus reduce the number of instances. For example, *Static Balanced Sampling (SBS)* selects cases with uniform probability from each class without replacement until obtaining a balanced subset. This subset contains an equal number of majority and minority class instances of the desired size and is renewed every generation. *Basic Under-Sampling Selection* ***(BUSS)*** is a sampling technique quite similar to SBS that selects all minority class instances and then an equal number from majority class randomly. Other variants of SBS and BUSS are proposed in [24].

Weighted sampling (DSS) and Topology Based Sampling (TBS) are efficient sampling techniques but they have a high computational cost and they are not suited for Big Data. R. Curry et al. [12, 38] and Hmida et al. [21] proposed solutions to benefit from the advantages of these methods by incorporating them in multi-level or hierarchical sampling, where sampling techniques are applied in different levels (Fig. 7.6). The first implementation of hierarchical sampling proposed in [38] experiments a two level sampling using RSS and DSS to solve the KDD Intrusion Detection Problem (KDD-IPD) [4]. In [12], a three level sampling is proposed to solve the same problem, where a balanced sampling is applied in the first level. The aim is to handle classes' imbalance by generating balanced blocks from the starting database. These blocks are then used as input for a two-level sampling based on RSS and DSS methods. A complete comparative study on the same problem (KDD-IPD) published in [20] shows how the hierarchical sampling can reduce significantly the computational cost, which is an important issue when dealing with big training sets.

Otherwise, Hmida et al. [21] demonstrated that the multi-level sampling could also provide a trade-off between speed up and generalization ability, specially for complex dynamic sampling techniques such as DSS and TBS. The authors experimented a three level hierarchical sampling BB-RSS-DSS to solve the Higgs Boson classification problem with several millions instances (Sect. 7.6). The corresponding

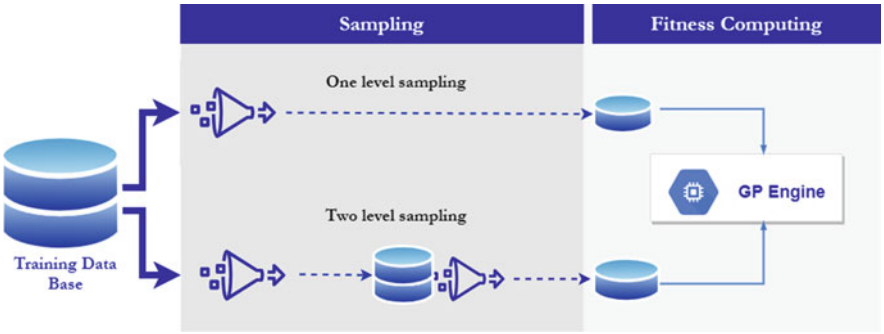


Fig. 7.6 One-level vs Two-level sampling for active learning with GP

```
import pandas as pd
TrainingSet = pd.read_csv("training.csv") #load training set
initGP()
while(generation<maxGeneration): # GP loop
    if generation(\% frequency)==0 : #sampling frequency
        current_sample = newSample(trainingSet,targetSize)
    # compute population fitness
    popFunctions = [toolbox.compile(ind) for ind in population]
    fitnessValues =  [getPrediction(func,current_sample) for
    func in popFunctions])
    updatePopulationFitnesses(population, fitnessValues)
    # Select the next generation individuals
    offspring = select(population)
    # Apply genetic operators
    offspring = evolve(offspring, crossoverProb, mutationProb)
    population[:] = offspring
```

Fig. 7.7 DEAP GP loop implementing a one-level *η-generation-wise* sampling where η is the number of generations designing the sampling frequency and *newSample* is a sampling method to select a new current sample for the evaluation step

experimental study proves that hierarchical sampling could be an efficient solution to scale EA/GP to Big Data mining.

7.4.1.1 Example of Implementation over DEAP

Figure 7.7 gives an example of how to implement one-level active sampling in the GP loop over DEAP. First, the whole train set is loaded in a pandas data frame that is used as an input argument for any active sampling technique designed as *newSample*. This method needs as argument the target subset size that should be small enough to reduce the computational cost and great enough to be representative of the initial set.

7.4.2 Active Learning with Adaptive Sampling

With the adaptive sampling, the sampling frequency f is adjusted according to the general training performance to accommodate the current state of the learning process.

Two adaptive models were proposed in [7, 23]. The first one, named *'Average Fitness'*, uses a threshold for the population mean fitness to detect whether the population is making improvements or not. In the case of very little or no improvement, a new sample is generated since the old one is fully exploited. For example, if the threshold to the best fitness improvement rate is set to 0.002, then

GP will continue to use the same sample if the best fitness of the current generation is better than the previous generation with 0.2% or more. Otherwise, a new sample must be created. The second model, named *'Min Resolved'*, is based on measuring the mean number of individuals (learners) that have resolved each record in the training sample. When this value reaches an expected threshold, then new records are selected in a new sample.

Experimented on the KDD-IDP data set, both *'Average Fitness'* and *'Min Resolved'* controlling techniques allowed GP to improve its accuracy metric up to 12% according to the generation-wise dynamic sampling using RSS and DSS methods, but it isn't the case with the False Positive Rate metric. However, the main interesting result in the case of Big data is the decrease of the computational cost of some complex and time consuming dynamic sampling techniques such as DSS and TBS. This due to the fact that the application of the sampling technique is less frequent since the corresponding frequency is computed and adjusted with the ongoing evolutionary process. An extended comparative study in [7] shows how the adaptive sampling can accomplish the same performance compared to the hierarchical one and do not need the step of hand tuning of the parameter η when using the η-generation-wise dynamic sampling.

7.5 Combining Horizontal Parallelization and Active Learning

GP is a costly algorithm, and this cost is intensified with big datasets. For clusters with a few number of nodes, running GP with a very large training set remains of high-cost. Furthermore, with big data sets, redundancy is inescapable. Thus, extending the parallelization mechanism with the active sampling paradigm was also explored in [22] as an efficient solution for Big Data mining. This extension, might help to handle some additional learning difficulties such as countering over-fitting, enhancing learning quality and allowing larger populations.

7.5.1 Dynamic Sampling in Distributed GP

While integrating static sampling in GP over a distributed environment is seamless, it is challenging when it comes to dynamic sampling. Static sampling is conducted off the GP run, and thus they are run independently. Whereas dynamic sampling algorithms are entangled with GP process at different levels with the respect to their complexity and their behaviour to learning state.

In fact, in order to implement a dynamic sampling method in this context, we need to consider the following elements:

- The distribution strategy adopted to parallelize GP steps as discussed in Sect. 7.3.
- Sampling algorithm dependence to the population (or populations): does the algorithm need information about individuals or not?
- Sampling algorithm dependence to the used dataset: does the algorithm rely on data about records or not?
- The distributed processing framework. Integrating sampling into MapReduce or Spark shall not be treated indifferently and must be adapted to underlying architecture and make use of optimized services, objects and primitives.

An efficient implementation of the dynamic sampling algorithm is tightly related to, first, when it is performed and second, if it needs additional data storage and computation. For the first point, in the case of GP, this means to schedule sampling among the steps of population evaluation and genetic operations (selection, crossover, mutation). Otherwise, the implementation must consider the additional data storage and the computation generated by the sampling method. Indeed, most of the dynamic sampling algorithms need to compute specific parameters to be used for the dataset record selection. For example, DSS computes a record age and difficulty depending on evaluation results. Besides computing these values, the storage is also an important issue. It is possible to add new columns to the dataset schema or to use a global variable.

The following paragraph gives an example of the combination between horizontal GP parallelization using Spark framework and dynamic sampling.

7.5.2 *Example of Dynamic-Sampling Implementation with Parallelized GP*

In [22], a simple and efficient sampling which is RSS is applied on Spark RDDs while running GP on a cluster. RDDs have two methods *sample* and *takeSample*.[7] They select instances of the data in a random way by respecting a given proportion and with or without replacement. The sample size is around the requested proportion. This is the principle of the RSS algorithm. In order to exploit the in-memory processing of Spark, a sample transformation step is added that therefore returns the sample as an RDD. *takeSample* is an action and cannot be optimized by the Spark scheduler. The sample is renewed by generation before the population evaluation. Figure 7.8 illustrates this sampling integration. Spark provides another method that can be used for sampling called *filter*,it relies on a function to decide whether to select a record or not. Nevertheless, sampling algorithms that need additional data

[7] https://spark.apache.org/docs/latest/api/python/reference/pyspark.html.

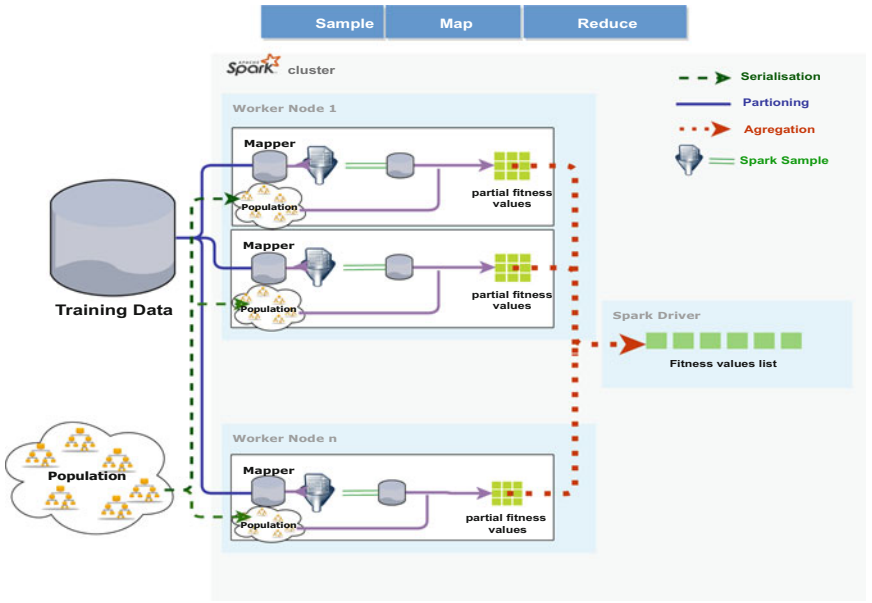


Fig. 7.8 Sampling with parallelized evaluation

(such as DSS, which calculates the age and difficulty of each instance) are more difficult to implement in Spark.

7.6 Discussion

In [20], multiple one-level and multi-level active sampling techniques are implemented in the GP loop and experimented to solve the KDD intrusion detection problem with 500 thousand instances in the training set [4]. The experimental study demonstrated how active sampling is able to improve the GP efficiency according to the performance metrics and the computational time. For example, the best accuracy measures vary between 91% and 93% with one-level sampling methods and reach 94% with hierarchical sampling (Fig. 7.9a), but with lower mean accuracy. Experiments results show how active learning improves GP generalisation ability and helps to produce more fitted classification models. Otherwise, each method has a different additional contribution in the Big Data context. For example, BUSS helps to deal with the class imbalance problem in Big Data, DSS helps to better train on difficult cases and TBS helps to boost the training with a better exploration of the fitness space topology. To take advantage of the benefits of different methods simultaneously, it is possible to apply these techniques on different levels with the hierarchical sampling (Sect. 7.4). Hmida et al. experimented

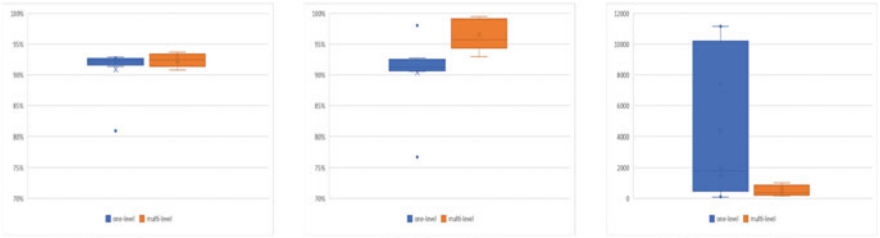


Fig. 7.9 Average best performances of the one-level and multi-level sampling techniques for the KDD intrusion detection problem. (**a**) Best accuracy. (**b**) Best TPR. (**c**) Best run time

different combinations such that RSS-DSS, BUSS-DSS, BUSS-TBS or BUSS-RSS-TBS. Defining the better hierarchical order sampling techniques depends tightly on the data characteristics. For example, applying BUSS at the first level helps to better deal with imbalanced data. Otherwise, DSS applied at the last level helps to better look insight difficult fitness cases with lower cost.

With the hierarchical sampling, GP is able to accomplish nearly the same performance as with the on-level sampling. Otherwise, the computational cost is reduced significantly, which is an important issue when dealing with big training sets. However, the number of hyperparameters naturally increases resulting from the inherent sampling methods used at each level. An extended experimental study published in [7] for solving the same problem showed that the same performance according to the classification metrics and the computing time could be reached with the Adaptive Sampling.

Hmida et al. published two experimental studies to solve a hard large scale classification problem with a hybrid approach combining both dynamic sampling [21] and parallelization approaches [22] described in Sect. 7.3. The classification problem concerns the Higgs Bosons detection [2] with 11 million observations. This set do not fit in the memory and GP can't tackle this problem without scaling solutions. First, the problem is handled with GP extended with an active learning implemented with two-level sampling using RSS at level one applied each 50 generations and DSS at level two. The best accuracy reached 0.65, which is a satisfactory result. It is important to note that this implementation was the first attempt to min the whole data set of 11 millions records, and the previous solutions were able to handle until 2.6 millions instances, such as Logistic Regression (best accuracy about 0.6) and Linear SVM (best accuracy about 0.6). Active learning make possible to look insight very large data sets and simultaneously to improve the learning performance.

Processing manipulation with parallelization over Spark was the second solution proposed to tackle the Higgs Bosons classification problem [22]. GP is parallelized over Spark with 4 nodes respecting the implementation summarized in Fig. 7.3. The experimental study demonstrates how the parallelization over Spark speeds up the population evaluation over nine times according to a serial execution of GP on a single node. However, the computational cost increases linearly with

Table 7.1 Best accuracy obtained for the Higgs Bosons classification problem published in different experimental studies [21, 22, 39]

Best results with modified GP on 11 millions instances				Best results with other methods on 2.6 millions instances
GP+RSS	GP+RSS-DSS	GP+SPark	GP+RSS+Spark	
0.638	0.65	0.6	0.607	0.67

respect to the population size and the generations' number. Injecting random sampling in the GP engine with the parallelization mechanism respecting the implementation described in Sect. 7.3 reduces this dependency and allows using larger populations without additional nodes, and then reaching better performances. The best accuracy obtained with this configuration is about 0.67 with random samples having 10,000 instances. Table 7.1 gives the best accuracy obtained with the different configurations experimented with the Higgs Bosons classification problem.

It becomes evident that active sampling and the horizontal parallelization over Spark are efficient scaling solutions for knowledge extraction from Big Data. They not only reduce execution time but also improve GP performance to produce better classification models. However, their basic implementations might be insufficient when dealing with very large data sets. In this case, extending these solutions with hierarchical implementation in the case of active learning and with hybridization using dynamic sampling in the case of Spark are promising solutions to go further and to better deal with the increasing size of the available data.

7.7 Conclusion

This chapter has summarized the broad range of approaches that could be applied to improve and adapt Evolutionary Machine Learning techniques to cope with large-scale datasets. These approaches are classified on three categories: data manipulation approaches, algorithm manipulation approaches and processing manipulation approaches. Two approaches from the two last categories are presented: active learning with dynamic/adaptive sampling and horizontal parallelization over Spark. For both approaches, an example of implementation on a Python EA framework is given, with some guidelines for hybridization.

Based on the different experimental studies published in [7, 19–23] and discussed above, we can state that data parallelization approach and active learning approach are efficient solutions to scale GP to Big Data mining. They significantly sped up the fitness analysis of big datasets and allowed for the processing of data sets that did not fit into a single machine's memory. GPs extended with an hybrid parallel implementation over Spark and dynamic sampling provides new possibilities toward solving hard problems related to big data mining.

The efficiency of the approaches summarized in this chapter is not limited to EA/GP. However, other approaches are not yet exploited with evolutionary algorithms, and could be promising opportunities in the Big Data context, such as Online Learning, Transfer Learning or algorithm modification with multi-objective and local optimization.

References

1. ACM (ed.): Genetic and Evolutionary Computation Conference, Berlin, Germany, July 15–19, 2017, Companion Material Proceedings. ACM (2017)
2. Adam-Bourdarios, C., Cowan, G., Germain, C., Guyon, I., Kegl, B., Rousseau, D.: Learning to discover: the higgs boson machine learning challenge (2014), http://higgsml.lal.in2p3.fr/documentation
3. Alves, A.: Stacking machine learning classifiers to identify higgs bosons at the LHC. Journal of Instrumentation **12**(05), T05005 (2017)
4. Archive, U.K.: Kdd cup: http://kdd.ics.uci.edu/databases/kddcup99/ (1999), http://archive.ics.uci.edu/ml/machine-learning-databases/kddcup99-mld/kddcup99.html
5. Atlas, L.E., Cohn, D., Ladner, R.: Training connectionist networks with queries and selective sampling. In: Touretzky, D. (ed.) Advances in Neural Information Processing Systems 2, pp. 566–573. Morgan-Kaufmann (1990)
6. Bacardit, J., Llorà, X.: Large-scale data mining using genetics-based machine learning. Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery **3**(1), 37–61 (2013)
7. Ben Hamida, S., Hmida, H., Borgi, A., Rukoz, M.: Adaptive sampling for active learning with genetic programming. Cognitive Systems Research **65**, 23–39 (2021). https://doi.org/10.1016/j.cogsys.2020.08.008, https://www.sciencedirect.com/science/article/pii/S1389041720300541
8. Bhatnagar, R.: Unleashing machine learning onto big data: Issues, challenges and trends. In: Machine Learning Paradigms: Theory and Application, pp. 271–286. Springer (2019)
9. Cantu-Paz, E.: Efficient and accurate parallel genetic algorithms, vol. 1. Springer Science & Business Media (2000)
10. Chávez, F., Fernández, F., Benavides, C., Lanza, D., Villegas-Cortez, J., Trujillo, L., Olague, G., Román, G.: ECJ+HADOOP: an easy way to deploy massive runs of evolutionary algorithms. In: Applications of Evolutionary Computation, EvoApplications 2016, March 30 - April 1, Proceedings, Part II. Lecture Notes in Computer Science, vol. 9598, pp. 91–106. Springer (2016)
11. Cohn, D., Atlas, L.E., Ladner, R., Waibel, A.: Improving generalization with active learning. In: Machine Learning. pp. 201–221 (1994)
12. Curry, R., Lichodzijewski, P., Heywood, M.I.: Scaling genetic programming to large datasets using hierarchical dynamic subset selection. IEEE Transactions on Systems, Man, and Cybernetics: Part B - Cybernetics **37**(4), 1065–1073 (2007), https://doi.org/10.1109/TSMCB.2007.896406
13. Dean, J., Ghemawat, S.: Mapreduce: Simplified data processing on large clusters. In: Brewer, E.A., Chen, P. (eds.) 6th Symposium on Operating System Design and Implementation (OSDI 2004), San Francisco, California, USA, December 6–8, 2004. pp. 137–150. USENIX Association (2004)
14. Dushatskiy, A., Alderliesten, T., Bosman, P.A.: A novel surrogate-assisted evolutionary algorithm applied to partition-based ensemble learning. arXiv preprint arXiv:2104.08048 (2021)
15. Fortin, F.A., De Rainville, F.M., Gardner, M.A., Parizeau, M., Gagné, C.: DEAP: Evolutionary algorithms made easy. Journal of Machine Learning Research **13**, 2171–2175 (Jul 2012)

16. Freitas, A.A.: Data mining and knowledge discovery with evolutionary algorithms. Springer Science & Business Media (2018)
17. Gathercole, C., Ross, P.: Dynamic training subset selection for supervised learning in genetic programming. In: Parallel Problem Solving from Nature - PPSN III. Lecture Notes in Computer Science, vol. 866, pp. 312–321. Springer (1994)
18. Harding, S., Banzhaf, W.: Implementing cartesian genetic programming classifiers on graphics processing units using gpu. net. In: Proceedings of the 13th annual conference companion on Genetic and evolutionary computation. pp. 463–470 (2011)
19. Hmida, H., Ben Hamida, S., Borgi, A., Rukoz, M.: Hierarchical data topology based selection for large scale learning. In: Ubiquitous Intelligence & Computing, Advanced and Trusted Computing, Scalable Computing and Communications, Cloud and Big Data Computing, Internet of People, and Smart World Congress, 2016 Intl IEEE Conferences. pp. 1221–1226. IEEE (2016)
20. Hmida, H., Ben Hamida, S., Borgi, A., Rukoz, M.: Sampling methods in genetic programming learners from large datasets: A comparative study. In: Angelov, P., Manolopoulos, Y., Iliadis, L.S., Roy, A., Vellasco, M.M.B.R. (eds.) Advances in Big Data - Proceedings of the 2nd INNS Conference on Big Data, October 23–25, 2016, Thessaloniki, Greece. Advances in Intelligent Systems and Computing, vol. 529, pp. 50–60 (2016). https://doi.org/10.1007/978-3-319-47898-2_6
21. Hmida, H., Ben Hamida, S., Borgi, A., Rukoz, M.: Scale genetic programming for large data sets: Case of higgs bosons classification. Procedia Computer Science **126**, 302–311 (2018), the 22nd International Conference, KES-2018
22. Hmida, H., Ben Hamida, S., Borgi, A., Rukoz, M.: Genetic programming over spark for higgs boson classification. In: International Conference on Business Information Systems. pp. 300–312. Springer (2019)
23. Hmida, H., Ben Hamida, S.B., Borgi, A., Rukoz, M.: A new adaptive sampling approach for genetic programming. In: 2019 Third International Conference on Intelligent Computing in Data Sciences (ICDS). pp. 1–8 (2019). https://doi.org/10.1109/ICDS47004.2019.8942353
24. Hunt, R., Johnston, M., Browne, W.N., Zhang, M.: Sampling methods in genetic programming for classification with unbalanced data. In: Li, J. (ed.) Australasian Conference on Artificial Intelligence. Lecture Notes in Computer Science, vol. 6464, pp. 273–282. Springer (2010)
25. Iba, H.: Bagging, boosting, and bloating in genetic programming. In: Banzhaf, W., Daida, J., Eiben, A.E., Garzon, M.H., Honavar, V., Jakiela, M., Smith, R.E. (eds.) Proc. of the Genetic and Evolutionary Computation Conf. GECCO-99. pp. 1053–1060. Morgan Kaufmann, San Francisco, CA (1999)
26. Kienzler, R.: Mastering Apache Spark 2.x. Packt Publishing (2017)
27. Langdon, W.B.: Graphics processing units and genetic programming: an overview. Soft Computing **15**(8), 1657–1669 (2011)
28. Lasarczyk, C.W.G., Dittrich, P., Banzhaf, W.: Dynamic subset selection based on a fitness case topology. Evolutionary Computation **12**(2), 223–242 (2004), https://doi.org/10.1162/106365604773955157
29. L'Heureux, A., Grolinger, K., ElYamany, H.F., Capretz, M.A.M.: Machine learning with big data: Challenges and approaches. IEEE Access **5**, 7776–7797 (2017). https://doi.org/10.1109/ACCESS.2017.2696365
30. Liu, Y., Khoshgoftaar, T.M.: Reducing overfitting in genetic programming models for software quality classification. In: 8th IEEE International Symposium on High-Assurance Systems Engineering (HASE 2004), 25–26 March 2004, Tampa, FL, USA. pp. 56–65 (2004). https://doi.org/10.1109/HASE.2004.1281730
31. Maitre, O.: Genetic programming on GPGPU cards using EASEA. In: Massively Parallel Evolutionary Computation on GPGPUs, pp. 227–248. Springer (2013)
32. Nordin, P., Banzhaf, W.: An on-line method to evolve behavior and to control a miniature robot in real time with genetic programming. Adaptive Behaviour **5**(2), 107–140 (1997). https://doi.org/10.1177/105971239700500201

33. Paduraru, C., Melemciuc, M., Stefanescu, A.: A distributed implementation using apache spark of a genetic algorithm applied to test data generation. In: ACM [1], pp. 1857–1863
34. Paris, G., Robilliard, D., Fonlupt, C.: Exploring overfitting in genetic programming. In: Artificial Evolution, 6th International Conference, Evolution Artificielle, EA 2003, Marseille, France, October 27–30, 2003. pp. 267–277 (2003)
35. Peralta, D., del Río, S., Ramírez-Gallego, S., Triguero, I., Benitez, J.M., Herrera, F.: Evolutionary Feature Selection for Big Data Classification: A MapReduce Approach. Mathematical Problems in Engineering **2015**, 11 (2015)
36. Qi, R., Wang, Z., Li, S.: A parallel genetic algorithm based on spark for pairwise test suite generation. J. Comput. Sci. Technol. **31**(2), 417–427 (2016)
37. Reinsel, D., Gantz, J., Rydning, J.: The digitization of the world from edge to core. Tech. Rep. US44413318, International Data Corporation (November 2018), https://www.seagate.com/files/www-content/our-story/trends/files/idc-seagate-dataage-whitepaper.pdf
38. Robert Curry, M.H.: Towards efficient training on large datasets for genetic programming. Lecture Notes in Computer Science **866** (Advances in Artificial Intelligence), 161–174 (2004)
39. Shashidhara, B.M., Jain, S., Rao, V.D., Patil, N., Raghavendra, G.S.: Evaluation of machine learning frameworks on bank marketing and higgs datasets. In: 2nd International Conference on Advances in Computing and Communication Engineering. pp. 551–555 (2015)
40. Zaharia, M., Chowdhury, M., Das, T., Dave, A., Ma, J., McCauly, M., Franklin, M.J., Shenker, S., Stoica, I.: Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing. In: Proceedings of the 9th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2012, April 25–27. pp. 15–28. USENIX Association (2012)
41. Zhang, B.T., Joung, J.G.: Genetic programming with incremental data inheritance. In: Proceedings of the Genetic and Evolutionary Computation Conference. vol. 2, pp. 1217–1224. Morgan Kaufmann, Orlando, Florida, USA (13–17 July 1999), http://www.cs.bham.ac.uk/~wbl/biblio/gecco1999/GP-460.pdf

Chapter 8
Dynamic Assignment Problem of Parking Slots



M. Ratli, A. Ait El Cadi, B. Jarboui, and M. Eddaly

Abstract The parking problem is nowadays one of the major issues in urban transportation planning and traffic management research. The present chapter deals with the dynamic assignment problem of the parking slots. The objectives are to provide a global satisfaction of all customers and maximize the parking lots occupancy. A dynamic assignment problem consists of solving a sequence of assignment problems over time. At each time period, decisions must be made as to which resources and tasks will or will not be assigned to each other. Assignments which are made at earlier time periods affect which assignments can be made during later time periods, and information about the future is often uncertain. In this chapter, we propose a MIP formulation with a time partition, throw a set of decision points, to handle the dynamic aspect. To solve this problem, we propose a hybrid approach using Munkres' Assignment Algorithm, a Local search and an Estimation of Distribution Algorithm (EDA) with a reinforcement learning. We tested our approach with and without the learning effect. Our approach is efficient; we were able to manage a set of 10 parking lots over 120 days (problems with up to 7000 parking slots and 13,000 requests per day). The saving is up to 80% and the results show, also, the benefit of the learning effect.

M. Ratli · A. Ait El Cadi (✉)
Univ. Polytechnique Hauts-de-France, LAMIH, CNRS, UMR 8201, Valenciennes, France

INSA Hauts-de-France, Valenciennes, France
e-mail: mustapha.ratli@uphf.fr; abdessamad.AitElcadi@uphf.fr

B. Jarboui
Higher Colleges of Technology HCT, Abu Dhabi, UAE

M. Eddaly
Department of Management Information Systems and Production Management, College of Business and Economics, Qassim University, Buraydah, Saudi Arabia

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

M. Eddaly et al. (eds.), *Metaheuristics for Machine Learning*,
Computational Intelligence Methods and Applications,
https://doi.org/10.1007/978-981-19-3888-7_8

8.1 Introduction

A dynamic assignment problem consists of solving a sequence of assignment problems over time. At each time period, decisions must be made in what concerns which resources and tasks to assign to each other. Assignments that are made at earlier time periods affect the assignments that are made during later time periods, and information about the future is often uncertain. Some examples of dynamic assignment problems include dispatching truck drivers to deliver loads, pairing locomotives with trains and assigning company employees to jobs [1].

Geng and Cassandras [2] propose a "smart parking" system for an urban environment based on a dynamic resource allocation approach. The goal of the system is to provide an optimal assignment of users (drivers) to the parking slots, respecting the overall parking capacity. The quality of an assignment is measured using a function that combines proximity to destination and parking cost. At each decision point, the system considers two queues of users: WAIT queue (consists of users who wait to be assigned to resources) and RESERVE queue (consists of users who have already been assigned and have reserved a resource in some earlier decision point). An optimal allocation of all users in both WAIT queue and RESERVE queue at each decision point is determined by solving a mixed integer linear programming problem. Simulation results show that the "smart parking" approach provides near-optimal resource utilization and significant improvement compared to classical guidance-based systems.

In [3], the authors present an approach for assigning parking slots using a semi-centralized structure that uses Parking Coordinators (PC). They propose a new approach to guide drivers to parkings. It aims to ensure driver satisfaction and improve the occupancy balance between parking areas. They propose two variants: in the first variant, each PC affects a parking independently of the others; in the second variant, the PCs interact with their near neighbors, to assign a location with the constraint of balancing the load of each car park. The idea behind this approach is to use PCs to gather vehicles (drivers) queries during a certain time window and assign a parking slot according to these vehicles preferences. Both variants of the solution were simulated. In the first one, the PCs take parking decisions regardless of their environment and neighbors. In the second variant, these computers (i.e. PCs) exchange information with their close neighbors. The simulation results showed that the second variant outperformed the first one, specially when the number of conflicting demands between PCs increased. The results for the second variant are very close to the case with a decision-centralized system, while being more scalable. These preliminary results showed that cooperation between the parking coordinators is strongly recommended.

8.2 Mixed Integer Programming Formulation

A driver represents a user and a parking slot represents a resource. A driver $i \in I = \{1, 2, \ldots, n\}$, aiming to visit a given destination, starts looking for a parking slot by launching a request at a non-deterministic moment. These moments are not known in advance and are discovered during the assignment process. A parking $j \in J = \{1, 2, \ldots m\}$ has a certain number of available slots that change during the time. Both drivers requests and available parking slots appear in a non-deterministic way and can change their state at any moment. We assume that each resource, each driver and each destination has a known location associated to it in a two-dimensional Euclidean space.

The density of drivers requests is not uniform, it varies from a time period to another during the same day. For example, the requests density for a parking slot near to a hospital rises in the visiting hours and declines at other times. In the same way, the requests density of a parking slot around public administration buildings depends on the working hours. Therefore, to take into account this feature, each day of the time horizon is subdivided into a fixed number T of equally spaced time periods. Periods that represent peak hours, normal hours, ... off-peak hours.

The objective of assigning vehicles to parking slots is to provide a global satisfaction of all customers. For instance, when the decision is performed request per request independently using a FIFO (First In First Out) rule, it may not be satisfactory because it can have a negative impact on the future situations. The following example, presented in Fig. 8.1, illustrates the impact of the starting choice on the future decision when using a FIFO rule. We assume that we have one available parking slot in parking p_1 and another in parking p_2. Moreover, we suppose that vehicle v_1 launches a request for a parking slot near to destination d_1 and vehicle v_2 looks for a place near to destination d_2. In addition, it is assumed that vehicle v_1 arrives before vehicle v_2 into the system. If we aim to minimize the distance between the parking and the destination, the FIFO rule will suggest

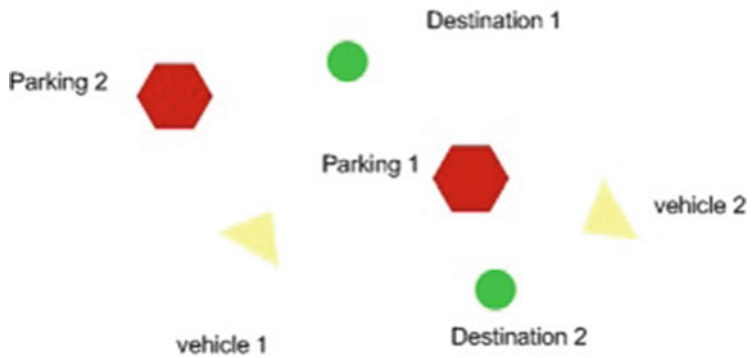


Fig. 8.1 Illustrative example

to assign vehicle v_1 to parking p_1 and vehicle v_2 to parking p_2. As the vehicle v_1 request is handled first, regardless of the vehicle v_2 request. The result is "good" for deriver v_1 but "bad" for driver v_2 and bad for the global system. This is due to the fact that the future information concerning the second driver is excluded. However, if vehicle v_1 is assigned to parking p_2 and vehicle v_2 to parking p_1, driver v_1 preserves his satisfaction degree as the distance from parking p_1 to destination d_1 is similar to the distance from parking p_2 to destination d_1, but the degree of satisfaction of driver v_2 is increased.

To escape from this drawback, two alternatives may be presented: the first one consists in collecting a subset of the requests in a given moment and solving the associated problem. The second one consists in establishing a forecast process to model the future information. In this section, we will discuss the first alternative and in a later section we will propose a forecast approach. The operation of collecting requests needs a collecting time period. We define a partition of time to gather requests and to determine the number of available slots in each parking created by the outgoing vehicles. Each time period, $t = 1, 2, \ldots, T$, of a day is partitioned into fixed K equally spaced sub-periods. We denote by k the index of the kth sub-period. The process of assignment is performed in a discrete given moment, at the end of each sub-period called "decision point". Each decision point has also an index denoted by k.

At each decision point k, we denote by N_k the set of new requests gathered during the sub-period k. Due to the limited capacity of the parking, some requests may not be satisfied at each decision point k. We denote by R_k the set of yet unassigned vehicles at the decision point k. Therefore, the set of vehicles that have to be assigned at the next decision point $k + 1$ is $E_{k+1} = N_{k+1} \cup R_k$, the sum of new requests N_{k+1} and the non-handled requests up-to-now R_k.

To formally define the dynamic assignment problem, we need the following:

1. Parameters:

- $dest_i^k$: the location of the destination of vehicle i from the set E_k in the two-dimensional Euclidean space.
- loc_i^k: the location of vehicle i from the set E_k in the two-dimensional Euclidean space.
- $d_{i,j}^k$: the distance between the location of the vehicle i from the set E_k and the parking j at the decision point k.
- $d_{[i],j}^k$: the walking distance, i.e. distance between the destination $dest_i^k$ of the vehicle i from the set E_k and the parking j at the decision point k.
- V: the velocity of vehicles. We assume that all vehicles have the same velocity.
- V': the walking velocity of a driver needed to reach the destination after parking the vehicle. We assume that this velocity is the same for all drivers.
- w_i^k: the waiting time of a vehicle i between the launch of the request and the decision point k.
- C_j^k: the capacity of the parking j at the decision point k. This parameter designates the number of available slots in the parking j at the kth decision point.

2. Variables: We introduce the binary decision variables defined as follows:

$$x_{i,j}^{k} = \begin{cases} 1 & \text{if the vehicle } i \text{ is assigned to parking } j \\ & \text{at the decision point } k, \\ 0 & \text{otherwise.} \end{cases} \tag{8.1}$$

3. Criteria: As the occupancy time of a parking slot begins from the assignment of the requests, which is not necessary the arrival time to the parking, the objective of the decision maker is to minimize the total distance between all vehicles and their assigned parking slots. This maximizes the occupancy and satisfies the parking manager. However, to guarantee a good quality of service to the customers, it is also necessary to minimize two elements:

   - the distance traveled between the assigned parking slot and the customer final destination. This could be done by choosing the closest possible available parking slot;
   - the waiting time that separates the launch of the request and the decision to assign the vehicle to a parking slot. This could be done by introducing a queuing factor to provide a priority for the vehicles having longer waiting times.

   These objectives are aggregated into a single weighted objective as follows:

$$Minf^{k}(x) = \sum_{i \in I} \sum_{j \in J} x_{i,j}^{k} (\lambda_1 \frac{d_{i,j}^{k}}{V} + \lambda_2 \frac{d_{[i],j}^{k}}{V'} - \lambda_3 w_i^{k}) \tag{8.2}$$

   where λ_1, λ_2 and λ_3 are non-negative values denoting the weights of each criterion. The non-positive coefficient associated to the third criterion is used as a priority factor.

4. Constraints: The main constraints in the considered problem at each decision point deal with the limited capacity of each parking and the satisfaction of the drivers requests. However, as the total number of available parking slots at each decision point is not necessarily equal to the total requests to be assigned, the constraints will be different. In this subsection, the constraints of the problem are expressed according to the availability of the parking slots and the number of launched requests at the considered decision point. Three possible cases will be considered.

   - **Case 1:** The number of available slots is larger than the number of requests at the kth decision point:

$$\sum_{i \in I} x_{i,j}^{k} \leqslant C_j^{k} \qquad \forall j \in J \tag{8.3}$$

$$\sum_{j \in J} x_{i,j}^{k} = 1 \qquad \forall i \in I \tag{8.4}$$

Constraints (8.3) guarantee that, at each decision point k, the number of assigned cars in the parking j cannot exceed its capacity. Constraints (8.4) ensure that each driver i is assigned to one and only one parking slot.

- **Case 2:** The number of available slots is less than the number of requests at the kth decision point:

$$\sum_{i \in I} x_{i,j}^{k} = C_j^k \qquad \forall j \in J \tag{8.5}$$

$$\sum_{j \in J} x_{i,j}^{k} \leqslant 1 \qquad \forall i \in I \tag{8.6}$$

In this case, we formulate the constraints such that all the slots in the parking j will be occupied and some drivers may not be assigned to any parking slot.

- **Case 3:** The number of available slots is equal to the number of requests at the kth decision point:

$$\sum_{i \in I} x_{i,j}^{k} = C_j^k \qquad \forall j \in J \tag{8.7}$$

$$\sum_{j \in J} x_{i,j}^{k} = 1 \qquad \forall i \in I \tag{8.8}$$

In this last case, the offer of the parking slots is equal to the demand and thus each parking slot should be occupied and each driver should find a parking slot.

For the ease of use, we propose to formulate our problem as a simple assignment problem, where the capacity of each agent is equal to one. This is done by disaggregating the parking slots. Instead of considering the slots through the capacity of each parking we consider the available slots individually. Therefore, the problem becomes an assignment of drivers to parking slots instead of an assignment of drivers to parking. Let J_j^k be the set of available slots in a parking j in the set J at the decision point k. The binary decision variables will be defined, from now on, as follows:

$$x_{i,h}^{k} = \begin{cases} 1 & \text{if the vehicle } i \text{ is assigned to parking} \\ & \text{slot } h \text{ at the decision point } k, \\ 0 & \text{otherwise.} \end{cases} \tag{8.9}$$

The new proposed formulation of our problem can be presented as follows:

$$Minf^k(x) = \sum_{i=1}^{n} \sum_{j \in J} \sum_{h \in J_j^k} x_{i,h}^k (\lambda_1 \frac{d_{i,h}^k}{V} + \lambda_2 \frac{d_{[i],h}^k}{V'} - \lambda_3 w_i^k), \tag{8.10}$$

subject to the following constraints:

- **Case 1:** The number of available slots is larger than the number of requests at the kth decision point:

$$\sum_{i \in I} x_{i,h}^k \leqslant 1 \qquad \forall j \in J, \forall h \in J_j^k \tag{8.11}$$

$$\sum_{j \in J} \sum_{h \in J_j^k} x_{i,h}^k = 1 \qquad \forall i \in I \tag{8.12}$$

- **Case 2:** The number of available slots is less than the number of requests at the kth decision point:

$$\sum_{i \in I} x_{i,h}^k = 1 \qquad \forall j \in J, \forall h \in J_j^k \tag{8.13}$$

$$\sum_{j \in J} \sum_{h \in J_j^k} x_{i,h}^k \leqslant 1 \qquad \forall i \in I \tag{8.14}$$

- **Case 3:** The number of available slots is equal to the number of requests at the kth decision point:

$$\sum_{i \in I} x_{i,h}^k = 1 \qquad \forall j \in J, \forall h \in J_j^k \tag{8.15}$$

$$\sum_{j \in J} \sum_{h \in J_j^k} x_{i,h}^k = 1 \qquad \forall i \in I \tag{8.16}$$

The considered problem is dynamic because, during a given day, the demand for parking slots is not uniform and the number of requests launched by the drivers changes from one period $t \in T$ of the day to another. Therefore, assignments that are made at earlier periods affect which assignments can be made during later periods. Moreover, the information about the future periods is uncertain. Our goal is to efficiently manage the requests in the time to deal with this uncertainty. To do so, we propose to establish a forecasting process based on a learning effect. We introduce a penalty term in the objective function that determines

for each parking and each period of the day if the current assignment has or not an impact on the future ones. In other words, if the future demand around a parking j is going be higher, the current penalty should be big, in order to leave more available slots in this parking for the future period. Otherwise, the penalty will be small, in order to make the slots of this parking more attractive for the current assignment. The value of these penalties are calibrated through a learning process.

Let p_j^t denote the penalty term associated to parking j at the period of time t ($t = 1, 2, \ldots, T$). It should be noted that for each slot in the parking j, the penalty is equal to the parking penalty p_j^t. In addition, between two consecutive time periods t_1 and t_2, such that $t_1 < t_2$ and for a decision points k such that $t_1 \leqslant k < t_2$, we set $p_j^k = p_j^{t_1}$.

The new objective function can be written as follows:

$$\begin{aligned} Minf^k(x) = \sum_{i=1}^{n} \sum_{j \in J} \sum_{h \in J_j^k} \\ x_{i,h}^k (\lambda_1 \frac{d_{i,h}^k}{V} + \lambda_2 \frac{d_{[i],h}^k}{V'} - \lambda_3 w_i^k + p_j^k) \end{aligned} \tag{8.17}$$

8.3 Estimation of Distribution Algorithms

According to the framework of the classical Genetic Algorithm (GA), the process of recombination occurs during meiosis, resulting from crossovers between parental chromosomes. Through this process, the offspring inherit different combinations of genes from their parents regardless the link between them. Moreover, the tuning of the parameters (population size, probabilities of crossover and mutation, etc) and the prediction of the movements of the populations are difficult tasks to perform in a GA. These drawbacks motivated the development of the Estimation of Distribution Algorithm (EDA).

The EDA was introduced to estimate the correlation between genes and uses this information during the search process. In an EDA there are neither crossover nor mutation operators. It was first introduced by Mühlenbein and Paass [4], and it is a stochastic optimization technique that explores the space of potential solutions by building and sampling explicit probabilistic models of promising candidate solutions. Starting with a population of individuals (candidate solutions), generally randomly generated, this algorithm selects good individuals with respect to their fitness. Then, a new distribution of probability is estimated from the selected candidates. Next, new offspring are generated from the estimated distribution. The process is repeated until the termination criterion is met.

This model-based approach to optimization has allowed EDAs to solve successfully many large and complex problems such as the quadratic assignment problem [5], the 0–1 knapsack problem [6], the n-queen problem [7], the traveling salesman problem [8].

EDA typically works with a population of individuals generated at random. At each generation, a subset of the most promising solutions is selected by a selection operator with respect to a fitness function. The main phase of the algorithm is to estimate, from information contained in the selected individuals, a probability distribution. Then, a new individual is generated from the constructed probability model. The new solution may be incorporated into the previous population if it satisfies a replacement criterion. Otherwise, it will be rejected. Finally, the process is reiterated until a stopping criterion is satisfied. The main steps of a basic EDA are summarize in Algorithm 1.

Algorithm 1: Basic EDA

Generate an initial population of P individuals
repeat
  Select a set of Q parents from the current population P with a selection method;
  Build a probabilistic model for the set of selected parents;
  Create a new offspring to P according to the estimated probability distribution;
  Replace some individuals in the current population P with new individuals;
until *a stopping criterion is met*;

The choice of the probabilistic model is not a trivial task. Many works have focused on the way to establish the distribution of probability that allows capturing the features of the promising solutions. Three classes of EDA may be presented according to the chosen probabilistic model and the degree of dependency between the variables of the problem. The first class called *univariate model*, assumes no dependencies between variables of candidate solutions, which is to say that all variables are independent. The second one, called *bivariate model*, assumes only pairwise dependencies between these variables and the last class, called *multivariate model*, assumes multiple dependencies between variables. In the next sections, we describe the main principles of those methods.

8.3.1 Univariate Models

In this section, we discuss the simplest approaches of EDA where it is assumed that the problem variables are independent. Under this assumption, the probability distribution of any individual variable should not depend on the values of any other variables. The common characteristic between all the models belonging to this category is to consider that, for each generation g, the n-dimensional joint probability distribution decomposes the probability of a candidate solution into a product of n univariate and independent probability distributions such that $p^g(x) = \prod_{i=1}^{n} p^g(x_i)$.

8.3.1.1 Population Based Incremental Learning

It was introduced by Baluja [9] and considers binary variables. Here, at each generation g, a candidate solution $x \in \{0, 1\}^n$ of a current population P is encoded by a vector of probability $p^g(x) = (p^g(x_1), p^g(x_2), \ldots, p^g(x_n))$, where $p^g(x_i)$ denotes the probability of the component x_i to take value "1". Initially, all positions are equiprobable and all probabilities are set to 0.5. Then, based on Q selected individuals, the probability vector is updated according to the following expression

$$p^{g+1}(x) = (1 - \alpha)p^g(x) + \frac{\alpha}{Q}\sum_{k=1}^{Q} x_k^g \tag{8.18}$$

where x_k^g is the kth best individual in the population at the gth generation and α is the learning rate. It is easy to observe that in Eq. (8.18) each component is evaluated independently of others and thus no interaction is considered. In [10], the authors proposed an adaptation of the population based incremental learning to continuous domain. Each element of the mean vector is estimated, at generation $g + 1$, by the following equation

$$\hat{\mu}_k^{g+1} = (1 - \alpha)\hat{\mu}_k^g + \alpha(x_{1*}^g + x_{2*}^g - x_w^g) \tag{8.19}$$

where solutions x_{1*}^g and x_{2*}^g are the two best solutions and solution $x^g{*}_w$ is the worst solution discovered in the current generation. Moreover, the authors proposed some heuristics to estimate the variance vector.

8.3.1.2 Stochastic Hill Climbing with Learning by Vectors of Normal Distributions

It was developed by Rudlof and Köppen [11] specifically for the continuous domain. The parameters of the density function, the mean vector $\hat{\mu}$ and the variance vector $\hat{\sigma}$ are estimated using:

$$\hat{\mu}^{g+1} = \hat{\mu}^g + \alpha(b^g - \hat{\mu}^g) \tag{8.20}$$

$$\hat{\sigma}^{g+1} = \beta \times \hat{\sigma}^g \tag{8.21}$$

where α denotes the learning factor, b^g denotes the barycenter of the B best individuals in the gth generation and $0 < \beta < 1$ denotes a fixed constant.

8.3.1.3 Univariate Marginal Distribution Algorithm

This algorithm, proposed by Mühlenbein [12], behaves differently from the two previous algorithms. It estimates the joint probability distribution $p^g(x)$ of the selected individuals at each generation. In the case of binary variables, the probability vector is estimated from marginal frequencies and $p^g(x_i)$ is set by counting the number of occurrences of "1", $f_k(x_i = 1)$ for $k = 1, 2, \ldots, Q$, in the set of selected individuals. In order to generate new individuals, each variable is generated according to $p^g(x_i)$ as follows

$$p^g(x_i) = \frac{1}{Q} \sum_{k=1}^{Q} f_k(x_i = 1) \tag{8.22}$$

For continuous domains, the Univariate Marginal Distribution Algorithm is designed, through statistical tests, to find the density function that best fits the variables. Then, using the maximum likelihood estimates, the evaluation of the parameters is performed.

8.3.2 Bivariate Models

In order to make the interactions between variables more realistic, this class of models takes into account pairwise dependencies. In this class of EDA, we focus only on the Mutual Information Maximization for Input Clustering (MIMIC) proposed by Bonet et al. [13] as it is used for both continuous and discrete domains. In MIMIC, the conditional dependencies of $p^g(x)$ are defined by a Markovian chain in which each variable is conditioned by the previous one. Therefore, in each generation, the MIMIC uses a permutation framework of ordered pairwise conditional probabilities, which can be written as follows:

$$\begin{aligned} p^g_\pi(x) = {} & p^g(x_{i_1}|x_{i_2}) p^g(x_{i_2}|x_{i_3}) \\ & \ldots p^g(x_{i_{n-1}}|x_{i_n}) \end{aligned} \tag{8.23}$$

where $\pi = \{i_1, i_2, \ldots, i_n\}$ is a permutation of the indexes $1, 2, \ldots, n$. The objective is to find the best $p^g_\pi(x)$ as closely as possible to the complete joint probability:

$$\begin{aligned} p^g(x) = {} & p^g(x_1|x_2, \ldots, x_n) \\ & p^g(x_2|x_3, \ldots, x_n) \ldots p^g(x_{n-1}|x_n) p^g(x_n) \end{aligned} \tag{8.24}$$

The degree of similarity between $p^g_\pi(x)$ and $p^g(x)$ is measured by using the Kullback-Leibler distance. The same idea was used by Larrañaga et al. [14] to extend this algorithm to the continuous space.

8.3.3 Multivariate Models

This section discusses the models that do not impose any restriction about the dependencies among variables.

8.3.3.1 Estimation of Multivariate Normal Density Algorithms (EMNA)

It was developed by Larrañaga et al. [15]. At each generation g, the multivariate normal density function is estimated. Therefore, the vector of mean $\hat{\mu}^g$ and the variance-covariance matrix $\hat{\sigma}^g$ are estimated using their maximum likelihood estimates:

$$\hat{\mu}_k^g = \frac{1}{Q}\sum_{r=1}^{Q} x_{k,r}^g \qquad k = 1, 2, \ldots, Q \tag{8.25}$$

$$(\hat{\sigma}_k^g)^2 = \frac{1}{Q}\sum_{r=1}^{Q}(x_{k,r}^g - \hat{\mu}_k^g)^2 \qquad k = 1, 2, \ldots, Q \tag{8.26}$$

$$(\hat{\sigma}_{j,k}^g)^2 = \tfrac{1}{Q}\textstyle\sum_{r=1}^{Q}(x_{j,r}^g - \hat{\mu}_j^g)(x_{k,r}^g - \hat{\mu}_k^g) \qquad j \neq k = 1, 2, \ldots, Q. \tag{8.27}$$

Finally, the new individuals are generated following the estimated function. An adaptive version of this algorithm was also developed by Larrañaga et al. [15]. In this algorithm, the first model is estimated according to the multivariate normal density function. Next, one individual $x_{current}^g$ is generated from the current density function. Depending on the fitness of this individual, it will be kept for the next population or not. If the answer is yes, the new individual is introduced in the population, and it is necessary to update the parameters of the multivariate normal density function as follows:

$$\hat{\mu}^{g+1} = \hat{\mu}^g + \frac{1}{Q}(x_{current}^g - x_Q^g) \tag{8.28}$$

$$\begin{aligned}(\hat{\sigma}_{j,k}^{g+1})^2 = {} & (\hat{\sigma}_{j,k}^g)^2 \\ & -\tfrac{1}{Q^2}(x_{k,current}^g - x_{k,Q}^g)\textstyle\sum_{r=1}^{Q}(x_{j,r}^g - \hat{\mu}_j^g) \\ & -\tfrac{1}{Q^2}(x_{j,current}^g - x_{j,Q}^g)\textstyle\sum_{r=1}^{Q}(x_{k,r}^g - \hat{\mu}_k^g) \\ & +\tfrac{1}{Q^2}(x_{k,current}^g - x_{k,Q}^g)(x_{j,current}^g - x_{j,Q}^g)\end{aligned} \tag{8.29}$$

$$-\frac{1}{Q}(x^g_{k,Q} - \hat{\mu}^{g+1}_k)(x^g_{j,Q} - \hat{\mu}^{g+1}_j)$$
$$+\frac{1}{Q}(x^g_{k,current} - \hat{\mu}^{g+1}_k)(x^g_{j,current} - \hat{\mu}^{g+1}_j)$$

Moreover, the authors proposed an incremental version of EMNA. The main differences comparing to the previous one are that each generated individual is added to the population regardless of its fitness and the update rules are given by:

$$\hat{\mu}^{g+1} = \frac{Q^g}{Q^g + 1}\hat{\mu}^g + \frac{1}{Q^g + 1}x^g_{current} \tag{8.30}$$

$$(\hat{\sigma}^{g+1}_{j,k})^2 = \frac{Q_g}{Q_g+1}\hat{\sigma}^g_{j,k} +$$
$$\frac{1}{Q_g+1}(x^g_{k,current} - \hat{\mu}^g_k)(x^g_{j,current} - \hat{\mu}^g_j) \tag{8.31}$$

It should be noted that the size of the population increases as the algorithm evolves.

8.3.3.2 Estimation of Gaussian Network Algorithms

This algorithm was developed by Larrañaga et al. [14]. The first step is to induce the Gaussian network from the data. The authors present three different induction models: edge-exclusion tests, Bayesian score + search and penalized maximum likelihood + search. Once the induction is done, a new individual is created according to the scheme of the learned network.

8.3.3.3 Iterative Density-Estimation Evolutionary Algorithm

It was proposed by Bosman and Thierens [16]. It uses the Bayesian factorization and mixture distributions for learning probabilistic models. Moreover, the iterative density-estimation evolutionary algorithm uses the truncated distribution for sampling the new individuals and only part of the population is replaced in each generation.

8.4 Estimation of Distribution Algorithm with Reinforcement Learning

Generally speaking, the estimation of distribution algorithm proceeds as follows. First, an initial population of candidate solutions is generated randomly. Then, a subset of solutions is selected from the initial population. At this moment comes the main step of the algorithm, consisting of building a probability model based on the

Fig. 8.2 Encoding solution

$$\pi = \begin{pmatrix} p_1^1 & p_1^2 & \cdots & p_1^T \\ p_2^1 & p_2^2 & \cdots & p_2^T \\ \vdots & \vdots & \cdots & \vdots \\ \vdots & \vdots & \cdots & \vdots \\ p_m^1 & p_m^2 & \cdots & p_m^T \end{pmatrix}$$

selected solutions to create a new "good" solution. Finally, the step of replacement decides if the new solution should be kept or not. The algorithm continues until it reaches a stopping criterion. In our context, we propose to apply the EDA to find good values for the penalties p_j^k.

Therefore, the problem considered in this section consists of finding the best values for the matrix of the penalties p_j^k to be used in our dynamic assignment problem. This current problem will be referred as the Penalties Calibration Problem (PCP). Our proposed algorithm follows these steps:

1. Encoding solution: A solution of the PCP problem is encoded by a matrix π where the rows represent the parking $j \in J$ and the columns correspond to the time periods $t = 1, 2 \ldots, T$. The intersection between each row j and each column t represents the penalty term p_j^t (Fig. 8.2).
2. Evaluation and forecasting: Each solution of the PCP problem is evaluated according to the objective function described in (8.17). The evaluation of any given solution of the PCP is the total assignment cost, over a day, for the dynamic assignment problem with the corresponding penalties. That is to say, a solution of the PCP is a set of penalties. These penalties are used in Eq. (8.17); the problem is solved for all the decision points, the total cost of period t is given by $f^t(x) = \sum_{k=1}^{K} f^k(x)$ and the total cost of a day d is $f^d(x) = \sum_{t=1}^{T} f^t(x)$. After that, a smoothing technique is used to take into account the forecasting of the demand. At the end, the evaluation of any solution π of PCP is given by the following equation:

$$F(x) = \sum_{q=0}^{\delta} \alpha(1-\alpha)^q f^{d-q}(x), \tag{8.32}$$

   where $0 < \alpha < 1$ is the smoothing factor which represents the weight of the previous observations. Therefore, the EDA consists to minimize $F(x)$.
3. Initial population: The initial population of P solutions is randomly generated. This means that we generate P matrices π such that each penalty $p_{j,r}^t$ (penalty for parking j during period t in the PCP solution r) of matrix π_r is generated according to a uniform distribution.
4. Selection: From the initial population we propose to select Q solutions according to the ranking of the objective functions $F(x)$ defined in Eq. (8.32).
5. Probabilistic model: In order to generate new candidate solutions, in our proposition we use the probabilistic model of the univariate marginal distribution

algorithm for Gaussian models [17] where the parameters mean and standard deviation of a solution are extracted from population information during the optimization process. The two parameters to be estimated at each generation for each variable are the mean $\hat{\mu}_j$ and the standard deviation $\hat{\sigma}_j$. Their respective maximum likelihood estimates are:

$$\hat{\mu}_j^t = \bar{p_j^t} = \frac{1}{Q} \sum_{r=1}^{Q} p_{j,r}^t \tag{8.33}$$

$$\hat{\sigma}_j^t = \sqrt{\frac{1}{Q} \sum_{r=1}^{Q} (p_{j,r}^t - \bar{p_j^t})^2} \tag{8.34}$$

6. Replacement: We compare the new solution with the worst solution in the current population. If the new solution is best than this solution, then the worst solution is removed from the population and it is replaced with the new one.
7. Stopping criterion: The stopping criterion indicates when the search finishes. We set a maximum number of iterations and a maximal computational time in our algorithm.
8. Local search procedure for dynamic assignment problem: The local search procedure was proposed to improve the performance of the algorithm through problem decomposition. Instead of tackling the whole complex assignment problem at the same time, the problem is divided into a set of smaller sub-problems, each of which can be solved easily in terms of computational time. The purpose of the decomposition scheme is to break down a large problem into smaller ones. Practically, for small assignment problems or when we have enough time, we use an adaptation of the Munkres' Assignment Algorithm [18]; But when the size is huge a local search procedure is used.

   In fact, if the number of vehicles that appear in the system and the number of considered parking are large, then the number of variables and constraints taken into account for solving the whole problem at each decision point may be huge. The idea is to decompose the area of the system (city) into a set of regions and solve an assignment problem for each one of them. At each iteration, a region is selected randomly, we record the set of requests Ω from vehicles in this region and the set of parking lots L existing in the same region.Then, the problem formulated by the associated variables is solved while taking into account the assignments of the remaining regions.

   Therefore, we set a threshold on the number of vehicles n_{max} from which the local search procedure is applied. The procedure starts from an initial solution randomly generated. Then, we select at random some decision variables to be fixed and optimize the remaining sub-problem, according to the objective function. The process is repeated until a given stopping criterion is reached (Algorithm 2).

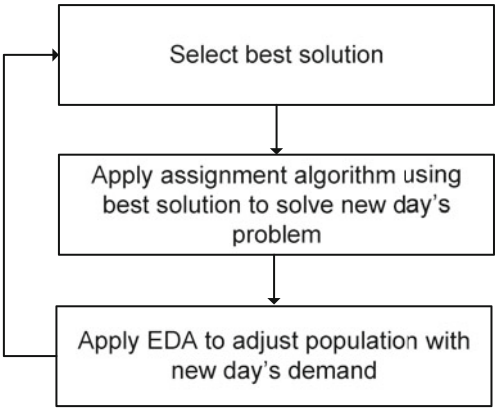


Fig. 8.3 The proposed algorithm

The framework of the proposed algorithm with forecast is given in Fig. 8.3 and Algorithm 3. It should be noted that if we set π_{best} to 0, we obtain the assignment algorithm without forecasting process. We denote by AA_{EDA} and AA the assignment algorithm with and without EDA, respectively.

Algorithm 2: Pseudo-code of the local search procedure

repeat
  Select a set L of parking slots at random at the decision point k;
  Find the set Ω associated to those spaces at the decision point k;
  Solve the assignment problem of (Ω, L, π_{best});
until *A stopping criterion is met*;

Algorithm 3: Pseudo-code of the assignment algorithm with forecast process based on EDA

$R_0 = \emptyset$;
for $k = 1, 2, ..., (K \times T \times D)$ **do**
  Find N_k, R_k and loc_i^k;
  $E_k = \{N_k \bigcup R_{k-1}\}$;
  if $|E_k| < n_{max}$ **then**
    (A_k, R_k)=Apply assignment algorithm $\left(E_k, J_j^k, \pi_{best}\right)$;
  else
    (A_k, R_k)=Apply local search procedure $\left(E_k, J_j^k, \pi_{best}\right)$;
  Update loc_i^k of R_k

8.5 Computational Results

In our experiments, we developed a simulation environment using the C++ programming language to reproduce the features of a real world problem. The simulation tests are generated in the two dimensional Euclidean space with different number of parkings. We assume that we have the map of the locations of the parking. Figures 8.4 and 8.5 present two examples of these maps with 5 and 10 parking, respectively. Each figure represents a problem instance. The parking are denoted by red circles. Then, each parking is located in a given region as in the figures. The regions are denoted by the rectangles in the figures. These regions define the density of the parking slots at each period $t = 1, 2, \ldots, 6$ at each day. The map size is 100 by 100 units of distance.

In the simulation, we consider that the parking opens at 07:00 a.m. and closes at 07:00 p.m. That is to say, each day consists of 6 periods of 2 h and the time horizon of the simulations lasts for 100 or 120 days. It is assumed that all the parking, in a given instance, have the same capacity. For example, in the case of 5 parkings the common capacity is of 1400 slots. Moreover, the number of occupied slots in each parking, at the beginning of each day, is generated according to the uniform distribution. Initially, the parkings are partially occupied as some vehicles can stay overnight in the parking slots. We assume that all parking slots can be used by any vehicle without any time limit. If a vehicle is assigned to a parking, the system selects any available slot in that parking. The frequencies of enter/exit of each parking for each region are randomly generated for each period according to the Poisson process Scheme, with the rates provided in Table 8.1 for the case of 5 parkings and in Table 8.2 for the case of 10 parkings. These frequencies define the number of requests associated to each region (parking neighborhood)

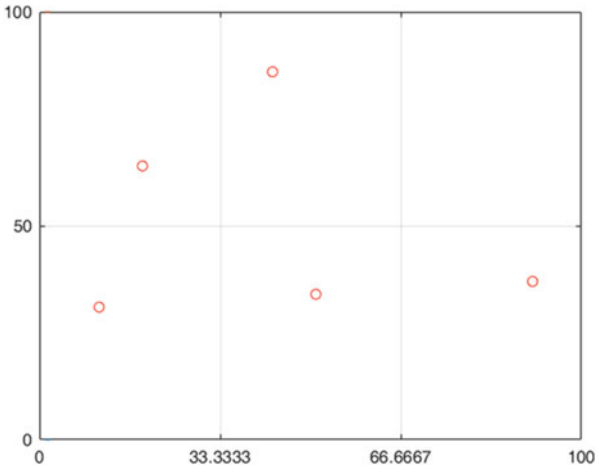


Fig. 8.4 Map with 5 parkings

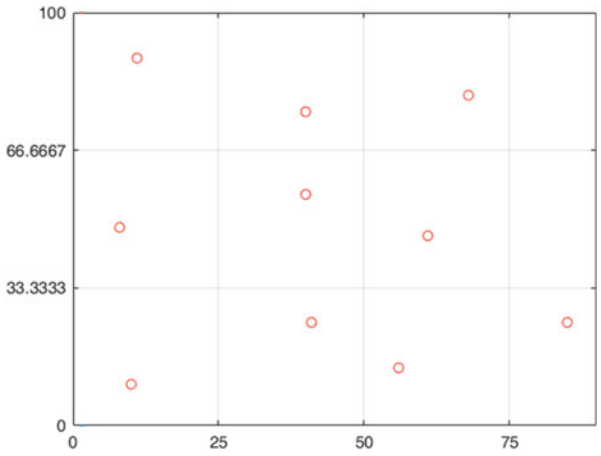


Fig. 8.5 Map with 10 parkings

Table 8.1 Instance problem with 5 parkings

Instance		t_1	t_2	t_3	t_4	t_5	t_6
Parking 1	Enter	0	0	1	3	6	15
	Exit	8	7	2	1	0	0
Parking 2	Enter	15	7	2	1	0	0
	Exit	0	0	1	1	3	8
Parking 3	Enter	0	0	0	2	7	19
	Exit	11	4	2	0	0	0
Parking 4	Enter	11	4	3	0	0	0
	Exit	0	0	0	3	3	11
Parking 5	Enter	8	4	2	0	0	0
	Exit	0	0	1	1	7	8

and to each period of the day. It should be noted that these parameters were set experimentally and the same distribution is used for all days. Each time period t, is partitioned into 120 equally spaced decision points where the decisions take place, i.e. each minute the problem will be solved taking into account the frequencies of enter/exit of each parking. The same distribution is used for all decision points $k = 1, 2, \ldots, K = 120$ between two consecutive time periods. For each generated request, a vehicle and its destination appear on the map. The location of the vehicle can be generated anywhere and the location of the destination must be generated within the considered region. We note that more than one vehicle may have the same destination.

In Fig. 8.4, we illustrate an example of a vehicle and associated destinations points with the yellow triangle and the green star respectively. Therefore, for each region, for each decision point, the total number of requests is computed by adding the number of new generated requests and the number of requests not assigned at the previous decision point. The vehicles not assigned in the previous period are

Table 8.2 Instance problem with 10 parkings

Instance		t_1	t_2	t_3	t_4	t_5	t_6
Parking 1	Enter	0	0	1	3	6	15
	Exit	9	7	3	2	0	0
Parking 2	Enter	15	7	2	1	0	0
	Exit	0	0	1	2	5	9
Parking 3	Enter	0	0	0	2	7	19
	Exit	10	3	2	1	0	0
Parking 4	Enter	11	4	3	0	0	0
	Exit	0	0	0	2	3	10
Parking 5	Enter	8	4	2	0	0	0
	Exit	0	0	1	2	6	9
Parking 6	Enter	0	0	2	4	7	12
	Exit	8	6	3	1	0	0
Parking 7	Enter	13	6	3	2	1	0
	Exit	0	0	1	2	3	10
Parking 8	Enter	0	0	1	3	8	16
	Exit	10	5	2	1	0	0
Parking 9	Enter	10	5	2	1	0	0
	Exit	0	0	0	2	3	12
Parking 10	Enter	9	4	3	0	0	0
	Exit	0	0	1	1	8	9

assumed to move in the direction of their destination, as denoted by the black line in Fig. 8.4. Thus, their distances $d^k_{i,j}$ are recomputed.

The performance measure employed in our numerical study was the average relative percentage deviation in terms of the objective functions at each day d:

$$\Delta_d = 100 \times \frac{f^d(AA) - f^d(AA_{EDA})}{f^d(AA)},$$

where AA denotes the proposed algorithm without the learning factor and AA_{EDA} the proposed algorithm with learning factor. As mentioned above, the total assignment cost of period t in a given day is, $f^t(x) = \sum_{k=1}^{K} f^k(x)$ and the total cost of each day d, $f^d(x) = \sum_{k=1}^{6} f^t(x)$. It is assumed that all objectives have the same weights.

Figures 8.6 and 8.7 show the evolution of Δ_d during, respectively, the 100 and 120 days of the simulation. It can be clearly seen that the values of Δ_d are positive (up to 80% for the case of 5 parkings and up to 80% for the case of 10 parkings). Therefore, the learning effect by forecasting the requests has improved the total assignment costs. Moreover, we observe that the curves reach the steady state rapidly. This shows that the learning speed of AA_{EDA} is very fast.

Furthermore, we notice that the savings are not the same for the two cases. In fact the distributions of the entries and exits during the day are different. The available

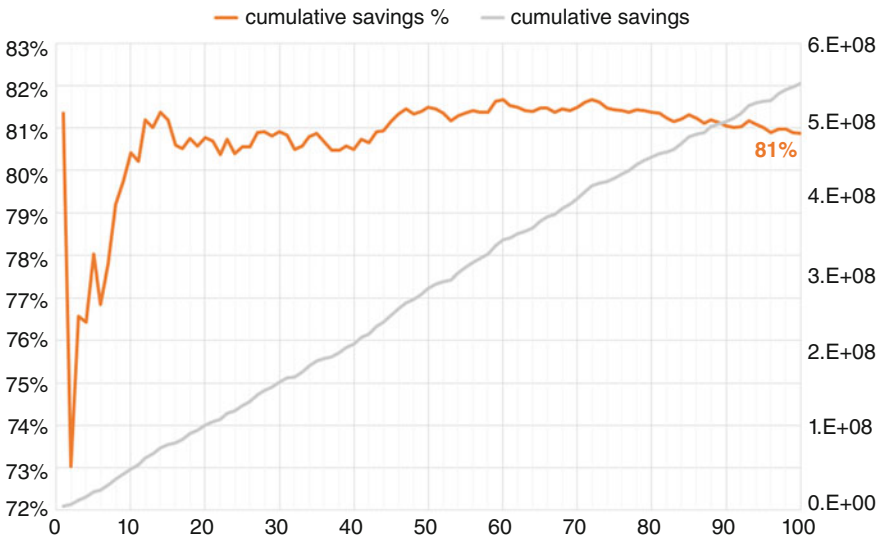


Fig. 8.6 Computational results for the instance with 5 parkings

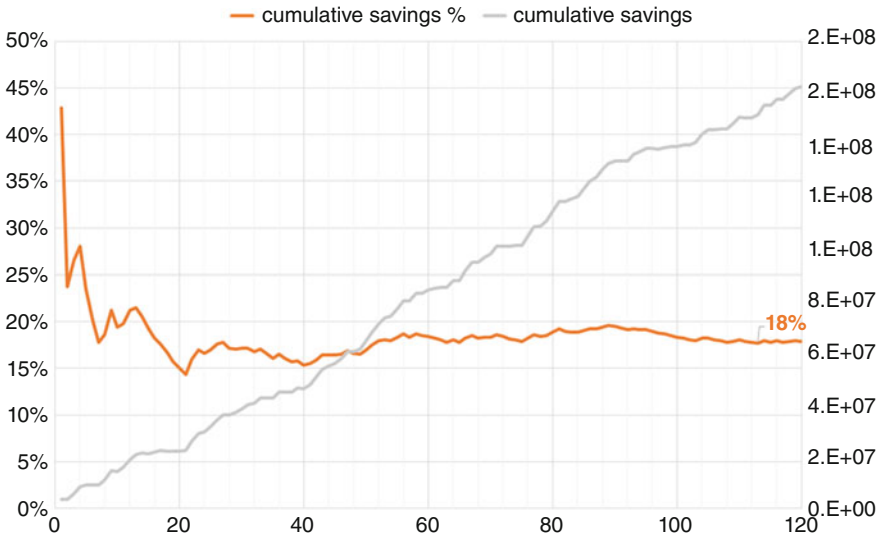


Fig. 8.7 Computational results for the instance with 10 parkings

Table 8.3 Mean equality unilateral test (99%): paired observations—Case of 5 parkings

	Cost without LF	Cost with LF
Mean	6,780,689.68	1,296,354.42
Variance	4.99 E+12	2.39 E+08
Observations	100	100
Degree of freedom	99	
Statistic t	24.66	
P-value	**2,35 E-44**	

Table 8.4 Mean equality unilateral test (99%): paired observations—Case of 10 parkings

	Cost without LF	Cost with LF
Mean	7,566,800.01	6,213,522.54
Variance	7.99 E+11	8.38 E+11
Observations	120	120
Degree of freedom	119	
Statistic t	11.58	
P-value	**1,56 E-21**	

capacity of the parkings at any period of the day and the dynamicity of the requests have an impact on the learning process and its efficiency.

In order to compare the performance of our algorithms we used the unilateral paired *t-test* procedure (Montgomery, 2001) at the 99% significance level. This procedure consists of comparing the means of two samples coming from paired observations. Let μ_A and μ_B denote the average of the evaluation of the fitness function for respectively, algorithm A algorithm B. The tested hypotheses are:

$$H_0 : \mu_A - \mu_B = 0$$

$$H_1 : \mu_A - \mu_B < 0$$

H_0 implies that the average relative percent deviations of the two algorithms are similar while H_1 implies that the average relative percent deviations of algorithm A are less than the ones of the algorithm B.

In our case, the algorithms A and B denote AA (without learning effect) and AA_{EDA} (with learning effect) respectively. The statistical tests prove that the negative difference between AA_{EDA} and AA is meaningful at the 99% confidence level (Tables 8.3 and 8.4).

8.6 Conclusion

We approached the problem of dynamic assignment for parking slots. A driver, aiming to visit a given destination, starts looking for a parking slot by launching a request at a non-deterministic moment. The parking lot manager has to fulfill

these requests by assigning available slots to vehicles. The objectives are to provide a global satisfaction to all customers and to maximize the parking lots occupancy. The problem is dynamic, the requests and the parking lots change over the time. First, the problem is modeled as a sequence of consecutive assignment problems, over the time. These problems are inter-related. At each decision point (a small time window) a static assignment problem is solved, we assign non-handled requests up-to-this point to the current available slots. Second, as the assignments that are made at earlier periods affect which assignments can be made during later periods, we propose to establish a forecasting process based on a learning effect. We introduce penalty terms in the objective function. The values of these penalties are calibrated through a learning process using the Estimation of Distribution Algorithm (EDA). We notice that solving each assignment problem at every decision point can be time consuming, depending on the number of the concerned requests and available slots. A local search procedure is proposed to improve the performance of our approach through problem decomposition and, hence, to reduce the solving time. Instead of tackling the whole complex assignment problem at the same time, the problem is divided into a set of smaller sub-problems. We tested our approach with and without the learning effect. Our approach is efficient since we were able to manage a set of parking lots, of up-to 10 parking lots, during a horizon of 120 days, which corresponds to assignment problems with up-to 7000 parking slots to manage and 13,000 requests per day to handle. The results also show the benefit of the learning effect. The total cost of the solutions with learning effect is less than the cost of the solutions without learning effect (a student test is used to prove the difference between these two methods).

References

1. M. Z. Spivey, W. B. Powell, Some fixed-point results for the dynamic assignment problem., Annals OR 124 (1–4) (2003) 15–33.
2. Y. Geng, C. G. Cassandras, Dynamic resource allocation in urban settings: A “smart parking” approach, in: Computer-Aided Control System Design (CACSD), 2011 IEEE International Symposium on, IEEE, 2011, pp. 1–6.
3. N. Mejri, M. Ayari, F. Kamoun, An efficient cooperative parking slot assignment solution, in: UBICOMM 2013, The Seventh International Conference on Mobile Ubiquitous Computing, Systems, Services and Technologies, 2013, pp. 119–125.
4. H. Mühlenbein, G. Paass, From recombination of genes to the estimation of distributions I. binary parameters, in: Proceedings of the 4th International Conference on Parallel Problem Solving from Nature, PPSN IV, Springer-Verlag, London, UK, UK, 1996, pp. 178–187. URL http://dl.acm.org/citation.cfm?id=645823.670694
5. Q. Zhang, J. Sun, E. Tsang, J. Ford, Estimation of distribution algorithm with 2-opt local search for the quadratic assignment problem, in: Towards a New Evolutionary Computation. Advances in Estimation of Distribution Algorithm, Springer-Verlag, 2006, pp. 281–292.
6. H. Li, Q. Zhang, E. Tsang, J. Ford, Hybrid Estimation of Distribution Algorithm for Multi-objective Knapsack Problem, in: Evolutionary Computation in Combinatorial Optimization, 2004, pp. 145–154.

7. T. Paul, H. Iba, Linear and combinatorial optimizations by estimation of distribution algorithms (2002).
8. V. Robles, P. de Miguel, P. Larrañaga, Solving the traveling salesman problem with edas, in: Estimation of Distribution Algorithms. A New Tool for Evolutionary Computation, 2002, pp. 211–229.
9. S. Baluja, Population-based incremental learning: A method for integrating genetic search based function optimization and competitive learning, Tech. rep. (1994).
10. M. Sebag, A. Ducoulombier, Extending population-based incremental learning to continuous search spaces, in: Proceedings of the 5th International Conference on Parallel Problem Solving from Nature, PPSN V, Springer-Verlag, London, UK, UK, 1998, pp. 418–427.
11. S. Rudlof, M. Köppen, Stochastic hill climbing with learning by vectors of normal distributions, 1996, pp. 60–70.
12. H. Mühlenbein, The equation for response to selection and its use for prediction, Evol. Comput. 5 (3) (1997) 303–346.
13. J. S. D. Bonet, C. L. Isbell, Jr., P. Viola, Mimic: Finding optima by estimating probability densities, in: ADVANCES IN NEURAL INFORMATION PROCESSING SYSTEMS, The MIT Press, 1996, p. 424.
14. P. Larrañaga, R. Etxeberria, J. A. Lozano, J. M. Peña, Optimization in continuous domains by learning and simulation of Gaussian networks, in: A. S. Wu (Ed.), Proceedings of the 2000 Genetic and Evolutionary Computation Conference, 2000, pp. 201–204.
15. P. Larrañaga, J. A. Lozano, E. Bengoetxea, Estimation of distribution algorithms based on multivariate normal distributions and Gaussian networks, Tech. rep., Dept. of Computer Science and Artificial Intelligence, University of Basque Country (2001).
16. P. A. Bosman, D. Thierens, Linkage information processing in distribution estimation algorithms, in: W. Banzhaf, J. Daida, A. E. Eiben, M. H. Garzon, V. Honavar, M. Jakiela, R. E. Smith (Eds.), Proceedings of the Genetic and Evolutionary Computation Conference GECCO-1999, Vol. I, Morgan Kaufmann Publishers, San Francisco, CA, 1999, pp. 60–67.
17. P. Larrañaga, J. Lozano, Estimation of Distribution Algorithms: A New Tool for Evolutionary Computation, Kluwer, Boston, MA, 2002.
18. J. Munkres, Algorithms for the assignment and transportation problems, Journal of the society for industrial and applied mathematics 5 (1) (1957) 32–38.